

Would You Please — Design Conversation History

A dated record of product and design decisions, from project inception through the present

Compiled August 20, 2026 · covers June 28, 2026 – August 20, 2026

Contents

About this document

1

About this document

This document compiles the “Would You Please” project’s decisions log — a running, dated record of substantive product, design, and asset decisions made over the course of the project — into a single chronological history, ordered oldest first (project inception through the most recent entry).

Each dated section below corresponds to one entry in the log as originally written. Entries capture the decision made, the reasoning and alternatives considered, and any follow-ups — in the words used at the time, not summarized or condensed for this compilation.

Images and screenshots that were pasted into the original design conversations are not reproduced here; only the written decisions and reasoning that resulted from them are included.

2026-06-28 — Logo asset finalization (PRD v11.1 / UI spec v2.2)

Decisions taken

1. **Horizontal lockup completed in matched light- and dark-background variants.** Two SVG files: `wyp\_\_\_\_\_\_\_logo\_\_\_\_\_\_\_horizontal\_\_\_\_\_\_\_light\_\_\_\_\_\_\_bg.svg` (mark + brand-blue wordmark + grey tagline) and `wyp\_\_\_\_\_\_\_logo\_\_\_\_\_\_\_horizontal\_\_\_\_\_\_\_dark\_\_\_\_\_\_\_bg.svg` (mark + white wordmark + light-blue tagline). Proportions match the main-screen header rendering in `WYP\_\_\_\_\_\_\_main\_\_\_\_\_\_\_screen\_\_\_\_\_\_\_content\_\_\_\_\_\_\_only.html` (40-px mark height in the live mockup); the lockup viewBox is 820×220 with the mark seated at x=0 and the text block left-aligned at x=240.
2. **Stacked lockup format dropped.** Previously listed in PRD §14.3 and UI spec §11.2 as “to be regenerated.” No longer needed; the horizontal lockup covers the use cases the stacked format was reserved for.
3. **Tagline canonical wording confirmed: “Tracking Requests and ToDos.”** This matches the existing usage in PRD §12.1, UI spec §6.8, and the main-screen header mockup. Intermediate variants that surfaced during design exploration (“For Request Tracking and ToDos” in `wyp\_\_\_\_\_\_\_logo\_\_\_\_\_\_\_final\_\_\_\_\_\_\_overview.svg`, “Request and ToDo Tracking” considered briefly during horizontal lockup design) are explicitly non-canonical and should not be reintroduced. Capitalization is “ToDos” (capital T, capital D) wherever the term appears.
4. **`wyp\_\_\_\_\_\_\_icon\_\_\_\_\_\_\_only.svg` removed from the canonical inventory.** The prior speech-bubble mark was carried as “superseded” in PRD §14.3 since v7.2. It is no longer part of the asset bundle and is not maintained. If a future project needs the prior mark for historical context, it can be retrieved from git history of the asset source file.
5. **Naming convention for logo variants:** `wyp\_\_\_\_\_\_\_logo\_\_\_\_\_\_\_<format>\_\_\_\_\_\_\_<background>.svg`, where `<background>` is either `light\_\_\_\_\_\_\_bg` or `dark\_\_\_\_\_\_\_bg`. The existing `wyp\_\_\_\_\_\_\_logo\_\_\_\_\_\_\_checked\_\_\_\_\_\_\_request\_\_\_\_\_\_\_light\_\_\_\_\_\_\_bg.svg` / `wyp\_\_\_\_\_\_\_logo\_\_\_\_\_\_\_checked\_\_\_\_\_\_\_request\_\_\_\_\_\_\_dark\_\_\_\_\_\_\_bg.svg` pair already followed this; the new horizontal lockup files extend it. Future lockup formats, if any, should follow the same pattern.

Asset bundle layout

The canonical assets now live in two project-knowledge files:

- `wyp\_\_\_\_\_\_\_assets\_\_\_\_\_\_\_reference.pdf` — visual reference with every icon and logo rendered, including dark-background variants shown on illustrative brand-blue and navy panels.
- `wyp\_\_\_\_\_\_\_assets\_\_\_\_\_\_\_source.md` — markdown file with the raw SVG source for every asset in fenced code blocks. SVG is plain XML text, so any asset can be recovered by copying the relevant code block into a `.svg` file.

This pattern exists because the project knowledge base does not accept direct `.svg` uploads. The PDF and markdown together replace what would otherwise be a folder of nine standalone SVG files.

Documents updated

- PRD v11.0 → v11.1 (title page, version history, §14.3 asset inventory, page footer).
- UI Design Specification v2.1 → v2.2 (title page, version history, §6.3 logo narrative, §11.2 asset inventory, page footer corrected from a stale v1.0 to v2.2).
- Project instructions: “Canonical sources” block replaced to reference the new asset bundle files and the updated 7-icon / 4-logo inventory.

Alternatives considered and rejected

- *Single horizontal lockup file with no light/dark distinction.* Rejected because the dark variant needs different fill on the bubble (white instead of outlined) and different text colors (white wordmark, light-blue tagline). Trying to express both in one SVG would have required either inline CSS that responds to the host context (fragile across renderers) or runtime swapping (adds complexity for no clear gain). Two files is cleaner and matches the precedent set by the mark-only pair.

- *Keeping `wyp\_\_\_\_\_\_logo\_\_\_\_\_\_horizontal.svg` as a third lockup file alongside the light- and dark-background variants.* Rejected because there was no defined use case for an “unspecified background” lockup that wasn’t already covered by one of the two background-specific variants.
- *Retaining `wyp\_\_\_\_\_\_icon\_\_\_\_\_\_only.svg` in the inventory as a historical reference.* Rejected because the canonical inventory should reflect current assets only. Historical references belong in the decisions log (this file) or git history, not in the active asset list.
- *Producing partial doc revisions per change rather than waiting for a version cut.* Rejected at the time and reversed by the explicit ask for tracked-changes docs. Decided that for substantive asset changes that affect multiple sections across both docs, doing a clean v11.1 / v2.2 cut with tracked changes is preferable to partial in-place revisions that would have left a v11.0 / v2.1 document inconsistent with the new canonical sources block in the project instructions.

Follow-ups

- After accepting tracked changes in both docs, regenerate the PDF exports if any external readers receive them.
- The main-screen mockup (PRD Figure 12.1, UI spec Figure 9.1) currently renders the mark plus separately-positioned wordmark and tagline text rather than the horizontal lockup SVG. No change required — the mockup composition still matches the design intent — but if Figure 12.1 / 9.1 is ever regenerated, consider whether dropping in `wyp\_\_\_\_\_\_logo\_\_\_\_\_\_horizontal\_\_\_\_\_\_light\_\_\_\_\_\_bg.svg` as a single asset would simplify the markup.
- The Sales One-Pager PDF (in project knowledge) is presumed to use the dark-background mark in its header/footer bands. Confirm at the next sales-collateral refresh.

2026-07-05 — Add/Edit Contact screen design finalized (UI spec v2.2 → v2.3 cut pending)

Decisions taken

1. **Notes field retained** per PRD §3.8: multi-line, optional, 500-character limit consistent with the app-wide text convention.
2. **Minimum-required rule** per owner mockup: either First or Last Name, and either Email or Phone.
3. **Field order:** First Name, Last Name, Email, Phone, Notes (First Name first — matches North American entry habit and browser autofill order).
4. **Phone field uses a country-code selector** that defaults to a *visible* “+1” from browser locale — never blank. Stored values are E.164, required for SMS invitations via SNS/Twilio; Canada is also +1, so the default is not US-specific.
5. **Duplicate handling on Save:** if the entered email or phone matches an existing contact, a modal offers “This email matches your existing contact [Name].” with actions Update that contact / Save as new / Cancel. Resolves the open duplicate question in UI spec §9.8.
6. **No-contact interception flow:** tapping Create Request with zero contacts routes to Add a Contact with an acknowledge-only modal (“To create a Request, a Contact is needed. Click anywhere to proceed. After you click Save Contact, you’ll be returned to Create a Request.”). Save returns to Create Request with the new contact selected; Cancel returns to the main screen with toast “Without adding at least one Contact, Requests cannot be created.”
7. **New design-system items:** Scrim token (Ink #1F2933 at 45% opacity, §3.1); Text input (§6.10, including the phone country-code variant); Secondary button (§6.11 — white fill, 1.5-px Brand Blue outline); Modal dialog (§6.12); Toast (§6.13).
8. **Resting inputs are white with a Rule border;** Field Yellow marks only the focused field, preserving the yellow-means-active convention established by the search bar.
9. **Header bottom rule removed on all screens;** the first Field-Yellow heading band sits flush beneath the header with no top margin or top hairline (recovers 12 px). Subsequent heading bands on the same screen keep the 11-px top margin and hairline. Implemented in all three HTML mockups via `.hdr + .band{margin-top:0;border-top:none;}`.

Alternatives considered and rejected

- *First-Name-required rule* — stricter than the owner’s stated minimum; rejected in favor of either-name.
- *Cancel returning to Create Request* — would immediately re-trigger the interception modal; main screen chosen instead.
- *“OK”-buttoned interception dialog* — click-anywhere dismissal is faster and matches the owner mockup.
- *Second filled-blue Cancel button* — equal visual weight for opposite actions; secondary style introduced instead.
- *Blank country-code display for the US default* — reads as unset/broken, hides that international entry is possible, and E.164 storage needs the code regardless.
- *Bottom-of-form button placement* — top placement matches the owner mockup and the established heading-band-plus-button pattern, and keeps Save visible without scrolling.
- *Disabling Save until the minimum is met* — hides why the button won’t work; always-enabled with inline errors is more discoverable and screen-reader friendly.
- *Keeping the first band’s top hairline at the header junction* — would visually reproduce the removed header rule.

Mockup files

- WYP__add__contact__screen.html — new; intended as Figure 9.9.1 in UI spec v2.3.
- WYP__add__contact__no__contact__dialog.html — new; intended as Figure 9.9.2.
- WYP__main__screen__content__only.html — updated (header rule removed, Requests band raised); **supersedes the copy in project knowledge**, which must be replaced or Figure 9.1 will show a line the spec no longer describes.

Documents to update (pending)

- UI spec v2.2 → v2.3 with tracked changes: §3.1 Scrim token; §6.8 header note (“no bottom rule; first heading band sits flush”); new §6.10–6.13; §9.8 duplicate cross-reference; §9.9 placeholder replaced with the full design; title page, version history, and footer. **Blocked on upload of the true v2.2 .docx** — project knowledge holds a text extraction, not the Word container.
- Project instructions “Canonical sources” block is stale (cites PRD v11, Excel UI spec, 6 icons, 3 logos, \$11.99/year); refresh to PRD v12.1.1, UI spec v2.3 once cut, 7 icons, 4 logos, \$17.95/year at the same time.

Follow-ups

- Toast microcopy sentence-case convention rolls into the existing §12 open question.
- Consider listing the three HTML mockup filenames in the canonical-sources block so they are formally part of the asset inventory.

2026-07-05 v12.1 — PRD restyled to v11.x template after v12.0 formatting loss; title-page tagline corrected to “Tracking Requests and ToDos”; file format flag updated so Word 365 no longer prompts to upgrade on save.”

2026-07-02 — Section 6.2 cost/revenue model corrections and monetization direction (PRD v11.1 → next revision)
Decisions taken

1. Long-term volumes corrected to linear per-user scaling. The prior Long-Term figures (5M items/day, 3M Dialog/day at 1M users) implied a 10x jump in per-user activity versus Pilot/Year 1. Per-user rates are now held constant (~0.5 items, ~0.3 Dialog/day), giving 500,000 items and 300,000 Dialog entries/day at 1M users. 500,000/day is a planning figure; owner expects volumes may ramp well beyond it. Section 7.2 updated to match. Long-term annualized infrastructure drops from \$410K–\$660K to ~\$100K–\$180K.
2. Ad revenue now applies only to monthly-active users, at a 50–70% active share (owner selected the higher band). Long-term ad revenue \$215K–\$605K. No revenue from ad-driven purchases is assumed — owner has no basis for such an assumption.

3. Usage split assumption: ~50/50 personal/business, with most users doing both. Consequently, no personal/business flag at signup — users won't self-classify cleanly. Business-oriented monetization instead comes from optional paid add-on features drawn from the Section 9 roadmap.
4. Pricing held at \$17.95/year, annual billing only, through the pilot. Monthly billing rejected: Stripe's \$0.30 fixed fee is ~23% of a ~\$1.50 monthly charge. Quarterly identified as the lowest practical recurring period (~9% fee load) but deferred — it adds churn/dunning complexity for unproven demand. Corrected cost base no longer forces a price increase; revisit after pilot conversion data. Alternatives rejected: personal/business signup flag; immediate price increase; quarterly SKU at launch; modeling ad-to-purchase uplift. Follow-ups: add-on packaging/pricing added to PRD §15.2 open items; pilot analytics should track ad RPM and paid conversion as first-class metrics (already in §16).

2026-07-06 — Per-contact Request delivery channel, subscription gating, Subscription banner, ad slot, Settings icon (UI spec v2.3 → v2.4; PRD v12.1.1 → v12.2 pending)

Decisions taken

1. **Per-contact delivery channel on Add / Edit Contact.** A “Send Requests by” field — a two-chip single-select row (Email / Text) in the §6.2 chip anatomy on a Strip surface — sits directly after Phone, before Notes (Notes stays last, preserving the 2026-07-05 field order). Email is the default; until the user taps a chip explicitly, the selection auto-follows data entry (filling Phone while Email is empty switches to Text, subscribers only, and vice versa); once tapped, the choice sticks.
2. **Text delivery is a subscriber feature.** For free users, tapping Text does not change the selection; an inline Ink Soft notice appears beneath the chip row — “Sending Requests by Text is available with a subscription — see the banner below.” — and the Subscription banner washes Field Yellow for ~2 s (no wash under reduced motion). Rationale: Twilio/SNS per-message SMS cost is then incurred only by revenue-generating accounts, consistent with the 2026-07-02 decision to monetize via optional paid add-on features.
3. **Validation follows the selection.** On Save the field matching the delivery selection is required. Because a free user's selection is always Email, free-tier contacts effectively require an Email; Phone remains optional “for later subscription use — or just for reference” (owner wording). This narrows the 2026-07-05 either-Email-or-Phone minimum for the free tier; confirmed by the owner. Free-tier caption: “Minimum required — Either First or Last Name, and an Email. Phone is optional and can be used for Text delivery with a subscription.” Subscriber caption: “...and the Email or Phone that matches your Send Requests by choice.”
4. **Subscription banner (§6.14), free tier only, all screens.** Full-width, min 40-px, white, 1-px Rule top hairline, centered 13-px/700 Brand Blue label “See Subscription Options and Features”; the whole strip opens Subscription/Upgrade (§9.14). Subscribers never see it (owner decision this date, superseding the earlier all-users intent) and manage their plan through Settings instead.
5. **Settings icon and search-bar segment.** Eighth icon `wyp\\\\\\\\\\\\_icon\\\\\\\\\\\\_settings.svg` (gear, house style: 48 viewBox, #2A5FC8, 2.5-px strokes, round caps). Lives in a new leading Strip-tinted segment of the search bar (§6.6): full bar height, 46-px wide, 1-px Rule right hairline. Strip chosen over the owner mockup's darker-yellow segment — Strip already means “interactive chrome surface,” avoids a new §3.1 token, and a darker yellow risks reading as a focused field. Icon renders at 24 px matching Voice Search and Search (initial 22-px draft corrected after owner review; glyph extent also enlarged to match optical weight). Settings is main-screen-only; task screens carry no Settings entry.
6. **AdSense shares the bottom edge (free tier).** Reserved fixed 50-px slot (320×50 unit) at the true bottom edge, below the banner (§6.15). Stack order search bar → banner → ad keeps the banner as a buffer against accidental ad taps. Free-tier bottom chrome ≈ 148 px on the main screen; subscribers carry only the ~56-px search bar. 320×100 rejected (≈30% of a small-phone viewport).
7. **Figures regenerated.** Figures 9.9.1/9.9.2 re-rendered from the updated mockups (free-tier state, gating notice shown for documentation); Figure 9.1 re-rendered from the updated main-screen mockup at the original 480×1391 aspect — this also clears the v2.3 flag that Figure 9.1 still showed the removed header rule.

Alternatives considered and rejected

- *Owner-mockup presentation* — two wide verb-labeled pills (“Send Requests by Email” / “Send Requests by Text”) below Notes: the selected pill is visually identical to the Secondary button (§6.11), verb-led labels read as immediate actions, and the control sits away from the fields it validates. Replaced by a labeled noun-chip pair after Phone.
- *New darker background token* for the chip strip and settings segment — Strip (#E5ECF7) already serves the purpose; §3.3 restricts new tinted backgrounds.
- *Hiding the Text chip from free users* — invisible features don’t upsell.
- *Modal upsell on tapping Text* — heavier than the moment warrants; inline notice + banner wash chosen.
- *Toast for the gating notice* — auto-dismisses before the user finds the banner.
- *Radio buttons / dropdown* for the channel choice — radios retired in v2.1; a dropdown hides a two-option choice.
- *No control (auto email-else-SMS)* — the pre-existing PRD §7.3 rule; defeats the stated intent.
- *Header-band Settings placement* — owner judged it visually distracting; the band was deliberately stripped in v2.0.
- *Dismissible banner for subscribers* — mooted once the banner became free-tier-only.
- *320×100 ad unit* — excessive fixed chrome on small phones.

Mockup and asset files

- WYP_add_contact_screen.html, WYP_add_contact_no_con — chip pair, gating notice, updated minimum-required caption, banner + ad slot. Supersede prior copies.
- WYP_main_screen_content_only.html — settings segment, banner, ad slot. Supersedes prior copy.
- wyp_icon_settings.svg — new; source block added to wyp_assets_source.md (icon table now 8 entries). Add to wyp_assets_reference.pdf at next regeneration.

Documents updated / pending

- UI spec v2.3 → v2.4 cut with tracked changes: title page, version history, §3.3, §5 intro (icon count corrected Six → Eight — “Six” was already stale in v2.3, predating the v2.1 Contract icon), §5.1 table row, §6.2 Notes, §6.6 anatomy and notes, new §6.14 Subscription banner and §6.15 Reserved ad slot, §9.9 (composition, behavior, captions, open question on a per-Request channel override), §9.13/§9.14 entry points, §11.1, footers; Figures 9.1/9.9.1/9.9.2 images replaced.
- PRD v12.1.1 → v12.2 **blocked on upload of the true .docx** — project knowledge holds a text extraction, not the Word container (same situation as the UI spec before 2026-07-05). Required edits (exact wording drafted in chat): §3.8 contact delivery-method field; §6 feature table (“Send Requests by Text” paid; banner + ad stack free); §6.2 SMS-cost note; §7.3 pilot note reword (free senders deliver by email; SMS for subscriber senders per contact setting, and as fallback for non-registered recipients without a known email); §14.3 icon inventory 7 → 8.
- Project instructions “Canonical sources” block refreshed this date (see chat) — replaces the stale block citing PRD v11, the Excel UI spec, 6 icons, 3 logos, and \$11.99/year.

Follow-ups

- Upload true PRD v12.1.1 .docx; cut v12.2 with tracked changes.
- Accept v2.4 tracked changes; replace UI spec v2.3 with v2.4 in project knowledge; replace the three HTML mockups and wyp_assets_source.md; regenerate wyp_assets_reference.pdf to include the settings icon.
- Open question carried in §9.9: per-Request one-off override of the contact’s delivery channel (Phase 2 candidate).
- Pilot analytics: track gating-notice impressions → subscription conversions as a first-class metric alongside the §16 set.

2026-07-15 — Palette 1 (“modernize around Brand Blue”) adopted; entry-field visibility resolved; floating-label exploration continued (UI spec v2.4 → v2.5 cut pending)

Recovery note: the three palette options were designed in the “Color palette design for screen...” conversation, which hit the per-conversation length limit and could no longer accept input. The option text was recovered by copy-paste and is recorded here in full so the reasoning is not lost. Lesson applied: palette-level decisions now land in this log the day they are made.

Decisions taken

1. **Palette option 1 adopted.** Keep Brand Blue #2A5FC8 exactly as is (protects the nine logo/icon SVGs, the sales one-pager, and the domain’s identity) and modernize everything around it: primary buttons go flat (solid Brand Blue; hover/pressed at Blue Pressed #1E4AA0; no gradient, no 2-px bevel shadow); Row Tint moves from blue #EFF4FC to neutral near-white #F6F7F9; Field Yellow retires as a fill; the focus state becomes a soft blue tint #EDF2FD under a 1-px Brand Blue border and the proposed Focus Halo; red becomes exclusively a status color. Owner confirmed after reviewing the applied main-screen mockup.
2. **Heading bands survive, in neutral.** The Requests / ToDos heading bands keep their structural job (quick visual reference to the two major screen regions — the original purpose of the yellow) but the fill moves to a new neutral **Band** token #EFF1F4 and the group labels move from Alert Red to Ink. This resolves palette option 1’s own open question (“bands neutral, or keep a yellow accent?”) toward neutral.
3. **Strip stays blue.** Palette 1 retired the blue Row Tint canvas, not Strip #E5ECF7. The Sent/Received/ToDos sub-heading and chips strips (and the search bar’s Settings segment) keep Strip, preserving the second-level cue — blue-tinted strip = section chrome — and Strip becomes the only blue-tinted surface, which sharpens the cue.
4. **Sort Highlight #FFE34D is the single surviving yellow,** used for the active-sort pill only. Field Yellow’s heading-band job passes to Band (decision 2); its focus-fill job passes to #EDF2FD (decision 1). The Field Yellow token is retired from §3.1.
5. **New Input Border token #7E8A9A.** With near-white surfaces, a white field with the 1-px Rule border (#E2E6EC, ≈1.3:1 on white) effectively disappears — the owner’s concern for Add a Contact and all future entry screens, confirmed by contrast math. Text inputs, the scope pull-down, the search field, and the country-code selector take a 1-px **Input Border #7E8A9A** boundary (≈3.5:1 on white, clearing the WCAG 2.1 §14.11 3:1 non-text minimum). Rule remains for hairlines, dividers, and card borders only. This settles field visibility once at the token level.
6. **Focus system unifies on one color family.** Focused fields: #EDF2FD fill, 1-px Brand Blue border, 3-px Focus Halo (proposed token --focus-halo: 0 0 0 3px rgba(42,95,200,.22)). The yellow-means-active convention (2026-07-05 decision 8) is superseded: blue-tint-means-active, matching the Brand Blue focus rings already in §10.3.
7. **Alert Red is status-only.** Overdue rows, the Overdue chip, and inline error messages keep red; group labels lose it (decision 2). This strengthens the PRD’s existing rule that red = Overdue by making red mean exactly one thing app-wide.
8. **Floating-label input pattern applied to Add a Contact for review (not yet adopted).** Per the recovered exploration: label floats inside the field, rising on focus or when a value is present; field minimum height 50 px (up from 40 px) to seat the risen label; label transition suppressed under prefers-reduced-motion per §8.9. Under palette 1 the floating pattern’s focus fill is #EDF2FD (the exploration predated the palette decision and used Field Yellow). The two questions remain separable and open: halo yes/no, floating label yes/no.

Palette options recorded (recovered text, condensed)

- *Option 1 — adopted (above).* “Cheapest change with the biggest perceived-age effect”; touches no PRD design constants except the Field Yellow references in the UI spec.
- *Option 2 — deep teal.* Suits the courteous product personality; rejected as second choice: full asset repaint, abandons blue equity, and teal pairs poorly with a red status color at equal prominence.
- *Option 3 — iris violet.* Current productivity-app vernacular (Linear, Notion accents); rejected: fashionable in the way the 2012 gradient buttons were — most likely to date soonest.
- *Rejected outright:* warm cream + terracotta (this year’s most overused AI-generated look); dark-mode-first with

a neon accent (poor for a data-dense list app whose web-respond view opens for unregistered recipients in default light contexts); any green accent (collides with the natural semantic for Done).

Alternatives considered and rejected (application decisions)

- *Dropping the heading bands entirely* — fails the squint-test grouping of the two major regions, the bands’ stated purpose.
- *Keeping the bands yellow as “the single yellow accent”* — competes with the Sort pill and keeps two yellows alive.
- *Blue-tinted (Strip) heading bands* — collapses the two-level hierarchy; group band and section strip would read as the same surface.
- *Grey-filled resting fields instead of a stronger border* — conflicts with white-at-rest (2026-07-05 decision 8’s structure, which survives) and grey fills read as disabled.
- *Keeping Rule as the input border* — fails 3:1 on the new near-white surfaces; Rule was already $\approx 1.3:1$ on white and was carried by the old tinted form background.

Mockup files

- WYP__main__screen__palette1.html — new; main screen with palette 1 applied (neutral bands, Ink group labels, flat buttons, #F6F7F9 zebra, Input Border on the search bar). Candidate Figure 9.1 for v2.5.
- WYP__add__contact__palette1__floating.html — new; Add a Contact with palette 1 plus the floating-label pattern (Email pre-filled to show the risen-label state). Review pending — floating label and halo are separate open decisions.
- The three existing v2.4 mockups remain canonical until v2.5 is cut.

Documents to update (pending — UI spec v2.4 → v2.5 with tracked changes)

- §3.1: Row Tint hex → #F6F7F9; Field Yellow row retired; new rows Band #EFF1F4, Input Border #7E8A9A, and (if adopted) Focus Halo; Strip usage note extended.
- §3.2: “Brand Blue on Field Yellow” verification mooted; re-verify Alert Red on the new Row Tint (passes — lighter surface, contrast improves); add Input Border 3:1 non-text verification.
- §3.3: tinted-background list updated (Band replaces Field Yellow; focus tint #EDF2FD noted as a state fill, not a surface).
- §6.1 Primary button: flat anatomy, pressed = Blue Pressed fill.
- §6.3 sort pill note: Sort Highlight is the sole yellow.
- §6.6 Search bar: field border → Input Border; bar surface → Band.
- §6.7 Group label: Ink on Band.
- §6.10 Text input: rest = white + Input Border; focus = #EDF2FD + Brand Blue border (+ halo if adopted); floating-label anatomy and 50-px height if adopted.
- §9.1 / §9.9 figures regenerated from the new mockups.
- PRD: Figure 12.1 regeneration; §12.1 key-decisions bullets touching yellow bands and red labels.

Follow-ups

- Owner review of WYP__add__contact__palette1__floating.html: decide halo yes/no and floating label yes/no (separable).
- Decide whether the Todos chips strip keeps Strip blue or goes neutral to extend the color-coding logic (Sent/Received = Requests family).
- Confirm the Band grey #EFF1F4 after on-device review (warmer/cooler/darker).
- Cut UI spec v2.5 with tracked changes per the list above; then replace mockups in project knowledge and update the Canonical sources block.
- Web-respond emails already in flight render with the old palette; no action needed, but note for support if recipients report a mismatch during rollout.

2026-07-16 — Floating labels, Focus Halo, and blue focus tint adopted; Band darkened to #E7E7E7

Decisions taken

1. **Floating-label input pattern adopted** (closes the 2026-07-15 open question). §6.10 anatomy becomes: label inside the field at rest (14 px/500, Ink Soft), rising above the value on focus or when a value is present (≈ 11 px/600; Brand Blue while focused, Ink otherwise); field minimum height 50 px; transition suppressed under prefers-reduced-motion per §8.9.
2. **Focus Halo adopted.** `--focus-halo: 0 0 0 3px rgba(42,95,200,.22)` is promoted from proposed to canonical (§3.1). Full focus state for text inputs: #EDF2FD fill, 1-px Brand Blue border, 3-px halo.
3. **Blue focus tint #EDF2FD confirmed** as the merged behavior of the floating-label exploration (which predated palette 1 and used Field Yellow) and palette 1's retirement of yellow fills.
4. **Band darkened: #EFF1F4 → #E7E7E7.** The lighter grey read too weak, most visibly on Add a Contact where the band sits directly against the #F6F7F9 form surface with no white gap; owner judged it too light even on the main screen where white space intervenes. #E7E7E7 was the owner's darkness reference and is adopted exactly: a *pure neutral* grey, deliberately not a cool grey, so the group-level Band is unmistakably distinct from the blue-family Strip #E5ECF7 (one hue step per hierarchy level). Owner notes the hex is approximate to the intended darkness; confirm on device.
5. **Compressed "Send Requests by" row adopted** (owner mockup, this date). The chip pair and the gating notice share one full-width Strip row (chips left, notice right, vertically centered, ~ 44 -px min height), replacing the v2.4 fit-content Strip island with the notice below. Rationale: the fit-content tint existed only to keep the white unselected chip visible and read as arbitrary; full-width, Strip reads as an intentional control row, consistent with the search bar's Strip-tinted Settings segment. The notice sits adjacent to the Text chip it explains, and its appearance can never reflow the form.
6. **Gating notice is persistent, both tiers** — supersedes 2026-07-06 decision 2's reactive behavior (notice appeared only when a free user tapped Text). The reactive design existed to avoid vertical cost, which the side-by-side layout eliminates; a persistent notice also upsells before the failed tap and keeps the row identical across tiers. On narrow screens the notice may wrap to three lines and the row grows — accepted by owner; no stack-back breakpoint.
7. **Notice text: Ink Soft #5A6675 at 12 px** ($\approx 4.9:1$ on Strip, clears AA 4.5:1). Owner's mockup grey was lighter; no new grey token introduced. Escalate to Ink if faint on device.
8. **Tier-differentiated notice wording adopted.** Free user: "Texting a Request is available with a subscription — see the banner below." Subscriber: "Your subscription includes Request Texting — see the banner below for other options." Line breaks fall naturally after "subscription" (free) and "Texting" (subscriber) at the standard 480-px width. *Unresolved conflict, flagged:* §6.14 hides the subscription banner for subscribers, so the subscriber string's "see the banner below" currently points at nothing — resolve before v2.5 (reword, or change §6.14 to show subscribers an options banner).
9. **Banner strategy revised: the bottom banner shows for both tiers** (owner mockups, this date), with tier wording — free user: "See Subscription Options and Features"; subscriber: "Account Options and Features". This resolves decision 8's flagged conflict: the subscriber notice's "banner below" reference now points at a real surface. §6.14's "subscribers: hidden" rule is superseded. The ad slot remains free-tier only per the PRD freemium constant (paid = ad-free); the subscriber mockup showed the ad slot, treated as a mockup artifact pending owner confirmation.
10. **Settings gear removed from the main-screen search bar;** remaining search-bar content (scope pull-down, field, voice, search icons) shifts left. Settings/account functions are assumed to live behind the banner destination for both tiers. §6.6 "part 0" Settings segment is retired. The `wyp_icon` set is unaffected (the gear was not one of the six canonical icons).
11. **"Request Texting" locked as the feature's proper name** — add to the project terminology list. Both notice strings now use it (decision 12), replacing the mixed "Texting a Request"/"Request Texting" phrasing.
12. **Notice strings revised (supersede decision 8's strings).** Free user: "Request Texting is available with a subscription — see the banner below." Subscriber: "Request Texting is included in your subscription — see the

Account banner below for other options.”

Alternatives considered and rejected

- *Cool grey at the same darkness* ($\approx$ #E3E7EC) — sits within 1–2 steps of both Rule #E2E6EC and Strip #E5ECF7; the band would read as a wide hairline or a faded Strip, re-collapsing the two-level cue the palette work just sharpened.
- *Keeping #EFF1F4 and adding a white gap on Add a Contact instead* — treats the symptom on one screen; owner confirmed the band was too light even with the gap.

Consequence to note

- The band’s 1-px Rule hairlines (#E2E6EC on #E7E7E7) are now effectively invisible. Harmless — the fill self-defines the band edge against white above and #F6F7F9 below — but v2.5 should either drop the band hairlines or leave them documented as vestigial. Recommendation at cut time: drop.

Mockup files

- WYP______main______screen______palette1.html, WYP______add______contact______palette1______ — both updated in place with Band #E7E7E7. The Add a Contact variant is now the approved reference for §6.10.
- *Dropping the Strip and giving unselected chips the Input Border* — compresses equally but bordered white chips read as buttons/fields rather than a selector, and discards Strip-means-control-surface.

Follow-ups

- Confirm the ad slot stays free-tier only (subscriber mockup showed it; PRD says paid = ad-free).
- Confirm where free users reach non-subscription settings now that the gear is gone (assumed: the banner destination page carries account options for both tiers).
- Add “Request Texting” to the terminology list in the Project instructions block.
- On-device check of #E7E7E7 before the v2.5 cut (owner flagged the hex as approximate).
- Still open from 2026-07-15: ToDos chips strip — Strip blue vs neutral.
- Cut UI spec v2.5 with tracked changes (list in the 2026-07-15 entry, plus: §3.1 Focus Halo row and Band #E7E7E7; §6.10 floating-label anatomy and 50-px height; band hairline decision; §6.2/§9.9 compressed send-by row and persistent notice with decision-12 strings; §6.6 gear removal; §6.14 both-tier banner with tier wording; §9.1 figure regenerated without the gear).

2026-07-16 (later) — v2.5 / v12.3 redlines cut

UI spec v2.5 and PRD v12.3 produced as tracked-changes documents (author “Claude”): WouldYouPlease_UI_Design_Specification_v2_5_tracked.docx, WouldYouPlease_PRD_v12_3_tracked.docx, per the change manifest (WYP_v2.5_v12.3_change_manifest.md). Owner approved the three flagged items before the cut: band hairlines dropped, banner attention wash moved to Focus Tint, and the stale “radio buttons” bullet in PRD §12.1 corrected to chips. Figure 12.1 / 9.1 image re-embeds remain pending (requires rendering the HTML mockups to images). After review/accept: replace the canonical files in project knowledge, update the Canonical sources block version numbers, add “Request Texting” to the terminology list, and correct the instructions-block price to \$17.95/year (PRD wins per precedence rule).

2026-07-17 — Figures embedded, §12 restructure, banner label finalized

Decisions taken

1. **Screen layouts live only in the UI Specification** (owner decision). PRD §12 is restructured to a pointer: the embedded main-screen figure, its caption, and the §12.1 design-decision bullets are removed; §12 now states that all screen designs, renders, and design commentary are specified only in the companion spec, with functional

behavior remaining in PRD §3. Before deletion, every §12.1 rule was verified as already present elsewhere (filter independence PRD §3.3; sort defaults and tap-to-reverse §3.2/§3.4; overdue-red rows §3.2; print Dialog prompt §3.6; truncation §2.4) — nothing orphaned. The §14.2 cross-reference to Figure 12.1 now points to spec Figure 9.1. Rationale: recurring update hiccups and the stale “radio buttons” bullet showed duplicated design detail rots in place; the documents’ own separation-of-concerns statement argues for single-homing.

2. **Free-tier banner label finalized: “See Subscription Features and Other Options”** (renamed from “See Subscription Options and Features”), making non-subscription settings discoverable behind the free-tier banner and resolving the open question logged 2026-07-16. Subscriber label unchanged. Updated in both redlines, both mockups, and the §12 open-questions list (question removed).
3. **Figures embedded in the UI spec** from the canonical mockups at 480-px width (2x): Figure 9.1 from WYP_main_screen_palette1.html; Figure 9.9.1 from WYP_add_contact_palette1_floating.html; Figure 9.9.2 from the new WYP_add_contact_no_contact_dialog_palette1.html (palette-1 form under the §6.12 acknowledge-only dialog and Scrim), which replaces WYP_add_contact_no_contact_dialog.html as the canonical dialog mockup. Image swaps are not trackable as Word revisions; captions carry the tracked source references.
4. **UI spec §11.1:** the `wyp_icon_settings.svg` bullet is tracked-deleted (retirement was already recorded in §5.1 and the version histories).

Follow-ups

- Owner: replace canonical files in project knowledge (both redlines after accept, three mockups, this log); update the Canonical sources block (UI spec v2.5, PRD v12.3, mockup filenames); add “Request Texting” to the terminology list; correct the instructions-block price to \$17.95/year; add the new maintenance sentence (screen renders and design detail belong to the UI spec only).
- Still open: on-device check of Band #E7E7E7; ToDos chips strip color.

2026-07-17 (later) — Readability restructure of both documents

Both redlines reorganized per owner request: (1) a Contents section (two levels, Word TOC field over Heading 1/2) inserted at the front of each document — the field populates on open in Word (auto-update enabled) or via F9; (2) the version history moved out of the front matter into a new “Schedule A — Revision History” at the end of each document, split into one entry per version for readability (UI spec: 7 entries; PRD: 18). A schedule was chosen over a numbered section before the open-questions/issues sections to avoid renumbering §12/§13+ and breaking cross-references. Title-page and footer version strings bumped to v2.5 / v12.3 (footer edits are outside Word’s tracked-changes scope; all body changes remain tracked). Both documents revalidated against their originals.

2026-07-20 — Create a Request fully designed; lookup/picker glyph convention; Due Time added; §9.8 retired (UI spec v2.6 → v2.7; PRD v12.3 → v12.4)

Decisions taken

1. **Create a Request fully designed** from the owner mockup (`Create_a_Request_-_new_format.png`) and specified as UI spec §9.2, replacing its placeholder. Layout follows the Add a Contact pattern: “Create a Request” group label on the Band heading band with Send (Primary) and Cancel (Secondary) right-aligned; floating-label fields on Row Tint; Subscription banner and free-tier ad slot at bottom. Screen title standardized as **“Create a Request”** — sending is part of creating, and “Create & Send” makes the label too long. Canonical mockup: **WYP_create_request_palette1.html** (Figure 9.2.1 embedded from the 480-px render, 2x).
2. **Label-affordance glyph convention adopted (new component §6.16; §6.17 placeholder for the pickers).** Type-ahead and picker fields carry a leading glyph inside the resting floating label; the glyph disappears when the label floats. Glyphs render at **≈120% of the resting label font size (≈17 px against the 14-px label), vertically centered on the label text** (owner sizing decision, this date; centering chosen over baseline alignment because the three shapes have different visual weights). Component numbering note: drafted as §6.15/§6.16, renumbered to §6.16/§6.17 because v2.6 already used §6.15 for the Reserved ad slot.

3. **Lookup glyph final design (rev. c, two owner mockups this date):** faded-grey magnifier in **Input Border #7E8A9A** overlaid by a **solid Ink #1F2933 down-pointer**, with a **white knockout halo** breaking the lens stroke around the pointer — the same white-behind-dark technique as the logo’s check mark — so the two elements read as an overlay, not one shape. Iteration history: (a) stroked down-caret beside the lens — rejected, caret too small to recognize; (b) solid Ink pointer protruding at the lens’s right — rejected, read as an extension of the magnifier rather than an overlay. Calendar and clock glyphs remain Ink Soft #5A6675. The plain magnifier (wyp_icon_search.svg) retains its single meaning: execute a search. Assets: wyp_icon_lookup.svg, wyp_icon_calendar.svg, wyp_icon_clock.svg (knockout assumes a light field background).
4. **Recipient picker is inline type-ahead.** First Name field lists/sorts “First Last”; Last Name lists/sorts “Last, First”; left-to-right prefix match with the matched prefix highlighted; selecting from either list fills both fields. No-match state offers “Add ‘{typed}’ as a new contact...” → §9.9 pre-filled. After selection, a caption shows the delivery channel (“Will be sent by Email” / “Will be sent by Request Texting”).
5. **§9.8 Find / Select / Add Contact retired** (heading retained for numbering; duplicate detection stays in §9.9.4; the Phase 2 contact-import consent question moves to PRD §9.3).
6. **Due Time (optional) added to the Request data model** (PRD §2.1; new §2.7). A Request with a Due Time becomes **Overdue at the stated time**; date-only Requests become Overdue at end of the due date (11:59 PM), **anchored to the sender’s time zone**; times are entered in the sender’s time zone and displayed in each viewer’s local time zone. On the main screen, **rows carrying a Due Time gain a third line: the time renders beneath the date in the Due column**, inheriting the row’s bold/red treatment (PRD §3.2; UI spec §6.4.1); description truncation (2 lines) is unaffected. Todos unchanged: no Due Time, Overdue still does not apply.
7. **Add Category confirmed as a one-field dialog** (§6.12) that creates the category (20-cap enforced inline) and selects it; rename/delete remain Settings-only. The compose Category field accepts list selection or dialog creation, not raw free text.
8. **Request Type selector deliberately omitted** from the pilot screen; it ships with the Phase 2 Custom Data Fields bundle.
9. **Compose behavior details:** required fields are recipient, Due Date, Description (optional fields explicitly labeled); Send is never disabled, failures marked inline; confirm-discard dialog on Cancel when the form is dirty; description counter appears past 400 of 500 characters; attachments panel is a fixed-height list with per-row remove and a vertical elevator; Add Attachment disabled at the 10-file cap; Due Time inert until a Due Date is set; month-forward calendar / now-forward clock (visual design deferred to §6.17; pilot uses browser-native pickers styled to palette 1).
10. **v2.7 / v12.4 cut and accepted.** Tracked redlines (author “Claude”) were produced against the uploaded v2.6 / v12.3 originals and validated (schema-valid; every change tracked); clean versions were then generated with the post-redline wording agreed this date (glyph sizing/coloring, Due Time third line) already applied: WouldYouPlease_UI_Design_Specification_v2_7.docx, WouldYouPlease_PRD_v12_4.docx. Remaining manual steps in Word: refresh the PRD table of contents (F9) so §2.7 appears, and embed Figure 9.2.1 from the browser render of the canonical mockup.

Alternatives considered and rejected

- *Reusing wyp_icon_search.svg for lookup fields* — one glyph, two behaviors (executes a search vs. announces focus behavior) collides with the documented search-bar meaning.
- *Unicode character (e.g. U+2315) instead of an SVG* — glyph coverage and rendering weight vary across Windows/iOS/Android/web fonts; an SVG is the only controllable rendering.
- *Trailing chevron only* — standard combobox cue but reads as a plain pull-down and undersells the type-ahead.
- *“Create & Send Request” title* — sending is inherent to creating; label too long.
- *Full-width third line for the Due Time* — breaks four-column scanning; time beneath the date in the Due column keeps the grid aligned.
- *Pure #000 pointer* — Ink #1F2933 is visually identical at size and stays inside the palette.

Follow-ups

- Owner: replace canonical files in project knowledge (PRD v12.4, UI spec v2.7,

WYP_create_request_palette1.html, the three glyph SVGs, this log); update the Canonical sources block per wyp_project_instructions_update.md (PRD v12.4, UI spec v2.7 docx, nine icons, retire the Excel sheet/row/column preference).

- Owner: on-device legibility check of the three glyphs at 17 px inside resting labels; F9 the PRD TOC; visual pass over Figures 5.1, 9.1, 9.2.1, 9.9.1, 9.9.2 (image references were rebuilt after an accept-step fault mispaired pictures, and Figure 9.2.1 was rendered in-environment rather than in Chrome).
- Next design task (owner-queued): §6.17 calendar and clock picker visual design.
- Still open from prior entries: ToDos chips strip color.

2026-07-21 — Respond to Request fully designed; Category made sender-private; §6.16 picker exception (UI spec v2.7 → v2.8; PRD v12.4 → v12.5 pending)

Decisions taken

1. **Respond to Request fully designed** from the owner mockups (Rerspond_to_Request.png, Rerspond_to_Request_v2.png) and specified as UI spec §9.3, replacing its placeholder. Layout follows the Create a Request pattern: “Respond to Request” group label on the Band heading band with Send (Primary) and Cancel (Secondary) right-aligned; read-only request header; form fields on Row Tint; Subscription banner and free-tier ad slot at bottom. Canonical mockup: **WYP_respond_to_request_palette1.html** (Figure 9.3.1 embedded from the 480-px render, 2x).
2. **No dedicated free-text response field.** The recipient’s reply is carried by three existing channels — the Done state, the Dialog thread, and response attachments. Rationale: a separate response box duplicates the Comment type in Dialog and splits the conversation across two places with different notification and audit behavior; Dialog entries are already immutable and threaded (PRD §4).
3. **Category is private to the sender.** On a Request the Category is a sender-side organizing label only. It is never shown to the recipient: not on Received rows, not on Respond to Request, not on the recipient’s Detailed Item view, not in the non-registered web response view, and not in the notification email or its .ics attachment. The sender keeps Category display, search, and filtering on Sent. ToDos are unaffected (no recipient). Consequence: the Create a Request field label becomes **“Private Category (optional)”** so the sender knows the label is not transmitted.
4. **Done Date and Done Time are selected, not auto-filled.** Opening the screen leaves both empty; the recipient sets each with its picker. Rationale: auto-filling today’s date on arrival would mark items Done merely by opening them, and would make “respond without completing” impossible to express. Empty Done fields at rest are therefore the resting state, and Send never marks an item Done implicitly.
5. **Done Date renders in the locale-aware weekday-long form** (e.g. “Monday, October 19, 2026”), matching the Due line so the two read as a matched pair; the Done Date field is widened and **Done Time narrowed** to accommodate it.
6. **Done Time is conditional:** shown only while the selected Done Date equals the Request’s Due Date *and* the Request carries a Due Time; hidden on any other Done Date and on date-only Requests. Evaluated live as the Done Date changes. Rationale: time of day changes the on-time / Overdue outcome only on the due date itself (PRD §2.7), so on every other day the field is noise.
7. **§6.16 gains a picker-field exception.** The Calendar and Clock affordance glyphs **persist** beside the floated label whenever the field holds a value, because a picker field has no text to retype and the glyph is the only visible cue that the picker can be re-opened. Type-ahead **Lookup** fields keep the default rule (glyph hidden once a value is present) since they are edited by typing — preserving the Category-filled-without-glyph state shown in Figure 9.2.1. The exception also applies to Due Date / Due Time on Create a Request (§9.2); the figure is unaffected because those fields are shown empty. *Reasoning correction logged:* the exception was briefly assessed as moot once auto-fill was dropped (empty fields already show the glyph under the default rule); that was wrong — the exception governs the **filled** state, which is exactly the state the owner’s v1 mockup and the “editable with the respective icons” note called for.
8. **Request attachments are read-only but downloadable; no overwrite.** The recipient can download the sender’s

files, edit them locally, and re-upload the edited copy as a *new* response attachment; the sender's original is never replaced or removed. Request attachment rows therefore carry no remove (×) control — the filename itself is the download affordance in Brand Blue — while the recipient's own uploads render as removable rows. The 10-attachment cap (PRD §2.1) applies to the combined count.

9. **Read-only detail field labels are app (Brand) blue**, with Ink values. §3.1's Brand Blue usage cell is extended accordingly. Applies to the Date / From / Due header and the panel labels on this screen.
10. **Add to Calendar** is a Primary button beside the request header, downloading an .ics for the due date and, where present, the Due Time (PRD §7.3).

Alternatives considered and rejected

- *A dedicated Response text field* — duplicates the Dialog Comment type and fragments the thread; rejected per decision 2.
- *Auto-filling Done Date/Time on open, editable afterwards* — silently completes items and removes the participate-without-completing path; rejected per decision 4.
- *Showing Done Time always* — presents a control that cannot affect the outcome on any day but the due date; rejected per decision 6.
- *Dropping the §6.16 exception once auto-fill was dropped* — conflates the empty and filled states; see the correction in decision 7.
- *A dedicated download icon on attachment rows* — would require a twelfth icon asset and a new group in §5.1/§11.1 for a single screen; the filename-as-link affordance carries the same meaning with no asset change. Revisit if download appears on other screens.
- *Showing the Category read-only to the recipient* — rejected as decision 3; a sender's private filing label ("Chase later", "Cheap vendor") is not recipient-facing content.

Process note — document regeneration

The v2.8 cut was first regenerated from the markdown extraction held in project knowledge. That file is a text extraction, not the Word master, so the rebuilt document lost the type ramp, heading colors, table borders and header shading, and **all sixteen embedded images** (the four figures plus the §3.1 swatch cells and §5.1 icon cells). Corrected by editing the uploaded v2.7 master's XML in place, which preserves every unedited byte of formatting. **Rule going forward: cuts are made against the uploaded Word master, never regenerated from the project-knowledge extraction.** The PRD cut is held for the same reason until the v12.4 master is uploaded.

Follow-ups

- Owner: replace canonical files in project knowledge (UI spec v2.8, WYP_respond_to_request_palette1.html, updated WYP_create_request_palette1.html, this log); update the Canonical sources block to UI spec v2.8.
- Owner: upload WouldYouPlease_PRD_v12_4.docx (the Word master) so PRD v12.5 can be cut — §2.3 Category-privacy bullet, §5 and §14.1 pointers to §9.3, Schedule A entry, and a correction to the stale §5 line that still calls Category "a free-text field with autocomplete" (contradicts the v12.4 selection-or-Add-Category rule).
- Owner: open v2.8 in Word and press F9 once so the TOC picks up §9.3.1–§9.3.5 (the field is flagged dirty and should refresh on open).
- Confirm the three items built to recommended defaults: Done fields shown filled in Figure 9.3.1 (illustrative, as Category is on Figure 9.2.1) rather than empty; "(optional)" kept lowercase to match Due Time and Attachments; panel labels Brand Blue on Respond where Create a Request uses Ink.
- Confirm the assumption that the header's "Date:" is the Request's **sent** date.
- Next design task (still queued): §6.17 calendar and clock picker visual design.
- Still open from prior entries: none outstanding.

2026-07-22 — Screen-name normalization; tier-scoped contact minimum; subscription-lapse policy; attachment storage warning strip and Storage Maintenance screen (UI spec v2.9; PRD v12.6 — both cut 2026-07-22)

Decisions taken

1. **Button label shortened to “Save”** on Add Contact, replacing “Save Contact.” Matches the verb-only convention already used for Send on Create Request and Respond to Request. Buttons remain padding-sized, so the Primary is now visibly narrower than the adjacent Cancel — the same relationship those two screens already show, so it reads as the established pattern rather than an anomaly.
2. **Screen name is “Add Contact”**, not “Add a Contact.” Side benefit: the recipient-row control on Create Request already read “Add Contact,” so the button and the screen it opens now carry one name.
3. **Screen name is “Create Request”**, not “Create a Request.” The article is dropped only where the phrase *names the screen*. Where the PRD describes the *action* — “Creating a Request when the user has zero contacts,” “to create a Request” — the lowercase phrasing stands unchanged. §9.9.5’s modal copy carries both forms in one sentence by design: it opens “To create a Request, a Contact is needed” (action) and closes “you’ll be returned to Create Request” (screen).
4. **“Respond to Request” is retained** over “Request Response.” It is a verb phrase that tells a first-time recipient — often a non-registered user in the secure web view — what to do; the noun form names an object and loses the instruction.
5. **Contact minimum is tier-scoped.** Name clause unchanged for both tiers: either a First or a Last Name. Reachability clause now depends on tier — the free tier requires an **Email**; a subscriber requires an **Email or a Phone**, with §9.9.3’s save-time channel validation unchanged, so the field matching the selected delivery channel must be populated. Rationale: subscribers can send by text and have no reason to hold an email address for a text-only contact, while free-tier delivery is email-only and a phone-only contact would be unsendable. This also resolves a latent conflict — §9.9.3’s auto-follow switches a subscriber’s delivery selection to Text when Phone is filled and Email is empty, a state an always-Email rule would have failed on Save. The mockup’s caption string is correct as written for the free tier; §9.9.2 and §9.9.3’s “either Email or Phone” text was the stale side of the divergence.
6. **Lapsed subscribers are prompted, not blocked.** A free-tier user (lapsed or never subscribed) opening an existing contact that has a Phone and no Email sees a persistent inline notice in Ink Soft beneath the Email field: “Add an email address to send Requests to this contact.” **Save still succeeds without one.** Rationale: a record valid when created must not fail validation later because the tier changed, and blocking Save would trap someone who opened the form to fix a phone number. Contacts whose delivery channel was Text revert to Email display at lapse, the free tier having only one channel.
7. **The phone-only count appears in two places, framed as incentive before lapse and as instruction after.** Pre-lapse renewal reminder: “Renewing keeps Request Texting for the 36 contacts you reach by text.” Post-lapse Account banner (§6.14), suppressed when the count is zero: “36 of your contacts have no email address. Requests are sent by email on the free tier, so you’ll need to add an address before you can send to them.” Both count contacts *lacking an email* rather than contacts “with only phone numbers” — identical sets today, but the first states the condition that actually gates sending.
8. **Subscription lapse carries a 30-day grace window.** Ads resume on day 31. Attachments are auto-deleted **oldest-first** down to the free 100 MB limit on day 31. Incremental email notices during the window carry the count, the megabytes over, and a **download link** — the user must be able to keep a file rather than only watch it expire.
9. **Storage warning strip (new UI spec §6.18)** — a full-width strip flush beneath the header band, above the first heading band; the whole strip is one control opening Storage Maintenance. Two severities, both from existing tokens: **Caution** at 20% or less remaining (Brand Blue on Focus Tint #EDF2FD with a Brand Blue bottom rule, main screen only) and **Critical** at 10% or less, or full (Alert Red on white between hairline Alert Red rules, every screen). The lapse countdown uses the Critical treatment.
10. **No amber token.** The owner’s mockup used an amber chip; adopting it would have required amending §3.1 (which names Sort Highlight #FFE34D “the sole yellow in the system from v2.5”), §3.2, and §3.3 (which states

Brand Blue and Alert Red are the only saturated colors used at scale, that additional accent colors are not part of the system, and which closes the list of permitted tinted backgrounds). Impending irreversible deletion is an error condition, so Alert Red carries it under a single extended clause in §3.1’s usage cell instead. Red stays off Row Tint, so it never enters the visual register that means Overdue.

11. **Strip placement is not a header chip.** PRD §3.1 records that the user name and the date/time were removed from the header in v10 because the clock overlapped the tagline on a Galaxy S24+; a chip beside the wordmark reoccupies exactly that reclaimed space and reintroduces the overflow. A full-width strip has no overflow failure mode and carries longer copy plus a right-aligned action.
12. **Warning copy states the condition, not the gesture.** “Click to fix” is mouse-specific in a touch-first product. Caution: “Attachment storage is 82% full.” Critical: “Attachment storage is full. New attachments will fail.” Lapse: “Your subscription ended. Attachments over 100 MB are removed in 12 days, on Wednesday, September 8.” Action text “Free up space ›” / “Review ›”. Sentence case per §6.13.
13. **Storage Maintenance screen (new UI spec §9.19)**, appended after §9.18 — §9.8’s retired number is preserved. Composition: header band; “Storage Maintenance” heading band with Close; summary block (used / limit / available with a proportional bar in Brand Blue that switches to Alert Red at the Critical threshold); sort strip; attachment rows; footer note; then the standard §6.14 banner and free-tier ad slot. Rows reuse the §9.2.2 attachments-panel anatomy — filename truncated mid-string, then size, type, and date — so no new component is introduced.
14. **Rows identify the attachment’s source:** the parent Request, Dialog entry, or ToDo, plus who added the file, rendered “Added by Roman Atley” or “Added by you.”
15. **Sort defaults to largest-first**, with an oldest-first option, and a caption noting that auto-deletion removes oldest first. The two orders differ deliberately: auto-deletion runs oldest-first, but manual cleanup is only efficient largest-first, and the screen optimizes for the user’s task rather than mirroring the machine’s rule.
16. **Each row carries a Download control.** Without it, the only way to comply with a storage warning is to lose the file permanently. Rendered as text in Brand Blue, not an icon — the icon family has ten members and no download glyph, and adding one would mean an eleventh asset plus a new §5.1 group. (Consistent with the 2026-07-21 rejection of a download icon on §9.3 attachment rows.)
17. **Storage is charged to the requestor**, and includes attachments added by the recipient. Stated on the screen in the summary note so it is not a surprise.
18. **Attachment caps: 25 MB per file and 50 MB per Request**, alongside the existing v7.0 count limit of ten per Request, per Response, and per Dialog entry. Necessary because decision 17 lets a third party consume a free-tier sender’s quota, and nothing previously capped an individual file — a single recipient video could have exceeded the sender’s entire allowance.
19. **Refusal to a blocked recipient upload is neutral:** “This Request can’t accept more attachments.” It does not name the cause, since the alternative discloses the sender’s account state and tier to someone who may be a stranger. Consistent with the Category-privacy reasoning of 2026-07-21.
20. **Close returns to the immediately prior screen; Enter does the same** when focus is not in a field. Stated generally in **§10.3 Keyboard navigation**, which already specifies Enter and Escape, rather than in §8 (which carries no keyboard content) or locally in §9.19: on a screen with no form to submit, Enter activates the heading band’s dismissive control — Close, or Cancel where no Close is present.
21. **Removed attachments leave a tombstone.** Because decision 17 means senders will routinely delete files their recipients uploaded, removal must not silently alter the recipient’s record. Rendered on the item (§9.6) as a single Ink Soft line — “1 attachment removed by Jim Kelley on Sep 8, 2027” — aggregated rather than one line per file, so clearing twenty files does not leave twenty tombstones on one Request.
22. **§4.3 gains a weekday exception for deadline dates within thirty days**, permitting “Wednesday, September 8” on the lapse strip. Narrow by construction: §4.3’s M-D-YY list rows and locale-aware long form elsewhere are otherwise unchanged. The year is dropped on the strip (the window is never more than 30 days out, so it carries no information and costs a line wrap at 360 px) and retained in the Storage Maintenance summary and the notice emails, which may be read weeks later.
23. **The countdown is stated as days remaining, on a named date** — “removed in 12 days, on Wednesday, September 8” — not as days elapsed since lapse. At day 18 of a 30-day window there are 12 days left, not 12 days since.
24. **A contextual subscription offer sits at the foot of the summary block:** “Subscribe for more attachment storage”

on the free tier, “Renew to keep these attachments” in the lapse state. Rendered as a tappable line in Brand Blue at 13 px / 700 with a 40 px target — the §6.14 banner’s treatment — on its own line. Deliberately *not* bolded inside the note, and deliberately not a verbatim quotation of the banner’s own label: emphasis without an affordance produces a phrase that reads as a control but isn’t, and the duplicated string would read as a repeated element rather than two placements. The persistent banner is retained; the two now say different things, the contextual one naming the specific benefit at the moment it is legible.

25. **No storage figure is promised.** PRD §6 gives the free tier as up to 100 MB but states the paid tier only as perpetual retention with add-ons available, and §15.2 lists add-on increments and pricing as an owner decision. Copy therefore says “more,” with no number, until cost estimates are run (owner, 2026-07-22). The missing paid-tier figure is now recorded as its own open item in PRD §15.2. *Correction to an earlier note in this session:* the subscription screen is **§9.14 Subscription / Upgrade**, not §9.17, which is the banners section.
26. **Screen names are set in plain title case; the convention is written down rather than marked up.** New §1.5: title case names a screen or component, lowercase names the action, a section reference accompanies the first mention of a screen within a section, and double quotes are reserved for verbatim product copy. Applied prospectively — sections conform as they are touched, not by a sweep of both documents.

Alternatives considered and rejected

- *Quotes, italics, bold, or color for screen names in the documents* — quotes already mean verbatim UI copy, so quoting a screen name makes “Create Request” ambiguous between the name and the button’s literal label, which is the confusion the markup was meant to remove; italics collide with figure captions, which are set fully in italic; bold collides with table lead-ins and adds noise to running prose; color fails in print and greyscale, is an accessibility problem on its own, and drifts the first time a paragraph is reformatted in Word. A written convention is the proportionate fix.
- *Requiring an Email of every contact regardless of tier* — taxes exactly the users who paid to avoid email; a subscriber reaching someone only by text would have to invent an address.
- *“Email or Phone” for both tiers* — reintroduces the problem the always-Email rule prevented: a free user saving a phone-only contact they can never send to.
- *Deferring the reachability check to send time* — surfaces the failure mid-compose, the worst moment, and leaves unusable records in the contact list.
- *Forcing a contact cleanup pass at lapse* — punishes the user at the moment of downgrade; the compose-time prompt of decision 6 catches the same problem at the point of use.
- *Adding a Warn Amber token* — costs amendments to §3.1, §3.2, and §3.3 to buy a convention users do not need once red is available; see decision 10.
- *Taking over the §6.14 banner for the storage warning* — that strip is the paid-conversion lever, and displacing it during a lapse is precisely backwards.
- *A modal on entry* — a recurring blocking dialog for a condition persisting for weeks trains people to dismiss it unread.
- *Making the offer sentence itself the link, inside the note* — buries the action in an explanatory paragraph where its tap target is an irregular multi-line shape.
- *A Primary button in the summary block* — competes with the heading band’s control and would be the screen’s only Primary button pointing away from the screen’s own task.
- *Dropping the bottom banner in favor of the contextual line* — loses the persistent lever on every other screen for no gain.

Corrections made during the cut

Four pre-existing errors were found in the masters while cutting and are recorded in the v2.9 Schedule A entry rather than fixed silently. §9.1.2 (both the Requests and the ToDos band) and §9.9.2 still described heading bands as Field-Yellow, a token retired in v2.5 and replaced by Band #E7E7E7 — the Figure 9.9.1 caption two paragraphs above already said Band, so the section contradicted its own figure. The §9.9.5 figure caption omitted the `\\\\\\\\\\\\_palette1` suffix from the mockup filename. The §3.2 note recording Field Yellow’s retirement, and the v2.0, v2.3, and v2.5 history entries that describe it as current at the time, stand as written.

Follow-ups

- Owner: replace canonical files in project knowledge with **WouldYouPlease_PRD_v12_6.docx** and **WouldYouPlease_UI_Design_Specification_v2_9.docx**, and update the Canonical sources block to v12.6 / v2.9. The mockups in project knowledge are current except **WYP_storage_maintenance_palette1.html**, which is one revision behind — the copy there predates the contextual offer line (decision 24).
- Owner: **WYP_main_screen_palette1.html is missing from project knowledge**, though the Canonical sources block lists it. It was present at the start of this session and absent by the end.
- Owner: open v2.9 and v12.6 in Word and press F9 once so each table of contents picks up the new sections (§1.5, §6.18, §9.19; PRD §6.3). The TOC fields are flagged dirty and should offer to refresh on open.
- Owner: run cost estimates, then set the paid-tier storage figure in PRD §6 and add-on increments and pricing in §11. Blocks both the §9.19 offer copy and the §9.17 subscription-features screen, neither of which can state a number until then.
- **Applied in v2.9:** §6.11.1, §9.9.1, §9.9.2, §9.9.3, §9.9.5 (Save / Add Contact); §9.2 heading and TOC line, §9.2.1 caption, §9.2.2, §6.16 Notes, §9.8 retirement note (Create Request); §9.9.2 and §9.9.3 (tier-scoped minimum); new §1.5, §6.18, §9.19; §3.1 Alert Red usage cell and Band token; §4.3 weekday exception; §10.3 Enter-key behavior; §9.6 tombstone paragraph; §9.2.3 and a new §9.2.5 edge case for the size caps; Schedule A v2.9. Figures 9.2.1, 9.9.1, and 9.9.2 re-rendered from the updated mockups; Figure 9.19.1 added. Version-history entries for v2.0, v2.3, v2.5, v2.7, and v2.8 stand as history.
- **Applied in v12.6:** §2.1 caps and the storage-accounting and neutral-refusal rules; §2.6 Email and Phone field definitions made tier-scoped, plus the closing sentence; six screen-name instances; new §6.3 Subscription Lapse; §14.1 and §14.2 entries; §15.2 paid-tier storage figure. The v12.4 history entry stands as history.
- Confirm the deletion hour, **written into §6.3 as end of day in the account holder's profile time zone** to match §2.7's Overdue anchor. Flagged rather than assumed: nothing in the PRD previously specified it.
- Confirm whether the 30-day clock also governs the retention reversion from perpetual to one year. Recorded as an open item inside §6.3 itself. If it does, day 31 removes far more than attachments and the notices need a corresponding line.
- Confirm whether editing a phone-only contact is the only prompt point, or whether selecting such a contact on Create Request should also prompt inline.
- Open: does the §9.19 list support multi-select for bulk removal? Ties to the bulk-actions question already open in §9.7.
- Open: destination for the contextual offer — §9.17 whole, or a storage section within it once that screen is structured.
- Open: §9.19's Close is a placeholder for whatever back affordance §9 eventually adopts; the spec defines none.
- ~~Confirm the §2.6 sentence.~~ **Confirmed by the owner, 2026-07-22.** Its closing paragraph reads “routes through Add Contact ... returned to Create Request,” renaming both screens, with the action phrasing “Creating a Request” left lowercase. An earlier instruction in this session said that paragraph needed no change; a later one directed the PRD renames generally, and renaming only one would have left the sentence mixing conventions. As shipped in v12.6. No further action.
- Still open from prior entries: §6.17 calendar and clock picker visual design (queued 2026-07-21); the 2026-07-21 confirmations, if not yet given.

2026-07-23 — Sales one-pager refreshed to \$17.95 pricing; callout panels auto-sized; collateral given a rebuildable source (no PRD or UI spec change)

Decisions taken

1. **Paid-tier price on the one-pager corrected to \$17.95 / year.** The approved 2026-06-11 PDF still carried **\$11.99**, a figure superseded when PRD v12.0 fixed annual-only billing at \$17.95 — the number used throughout PRD §6.2.3, §13, and §15.1. Recorded here as a stale-artifact correction rather than a pricing decision: nothing about the price changed, only the collateral that misstated it.

2. **The monthly framing is “Only \$1.50 a month — a fraction of team tools,”** replacing “Less than \$1 a month — a fraction of team tools.” $\$17.95 \div 12 = \1.496 , so \$1.50 is accurate to the cent under normal rounding. “Less than \$2” was rejected as technically true but read as rounding *up* to make a point. PRD §13’s “under a nickel a day” framing stays in the PRD only; on the one-pager this line does double duty as the per-month comparison against per-seat team tools, and a per-day figure loses the comparison.
3. **The two accent-bar callout panels are auto-sized from their content, with equal padding above and below.** Both the light-blue “Nothing to install” panel and the manilla “Coming soon” panel had been drawn at fixed heights that ended on the last text baseline: the light-blue panel’s final descenders crossed its bottom edge by roughly 0.4 pt, and the manilla panel cleared its own by 0.1 pt. The tint therefore read as cut off rather than as a panel containing text. Padding is now **9 pt top and bottom**, measured from the first line’s ascent and the last line’s descent rather than from baselines, so the text block is optically centered.
4. **The extra height is absorbed by shifting the lower half of the page down 9 pt**, not by reducing type size, leading, or copy. The light-blue panel grows from 46 pt to 54.7 pt and the manilla panel from 44 pt to 52.4 pt; everything from the “Everything you need to follow through” heading downward moves 9 pt lower. White space above the footer band falls from 64 pt to 47 pt, which is still generous. Column grid, card geometry, gutters, and every type size are unchanged.
5. **The solid blue pricing card and the header and footer bands are left alone.** Measured padding is 11.4 pt above and 13.2 pt below the text on the pricing cards, and 17.0 / 15.2 pt in the footer band — already balanced, and not what the eye was reading as clipped. Only the two panels that were genuinely clipped were touched, so the rest of the approved composition carries forward unmodified.
6. **The collateral now has a source of record: build_onepager.py (ReportLab).** The page was transcribed from the drawing operations of the approved PDF — identical coordinates, color values, Helvetica sizes, and vector checked-request mark — and verified two ways: a layout text diff showing only the two intended pricing lines changed, and a side-by-side render at 150 dpi. Copy, palette, and geometry are now editable in one place, and the callout helper re-centers automatically when a line of copy changes length. This mirrors the practice already established for screens, where the HTML mockups are the source and the figures are exports.
7. **Deliverable is WYP_Sales_OnePager_v2.pdf.** The “FinalLogo” qualifier is dropped: it marked the one-time asset milestone of 2026-06-28, not a document version, and reads as a version once a second version exists.

Alternatives considered and rejected

- *Patching the existing PDF content stream in place* — the price swap alone would have worked, since both replacement strings are left-aligned at a fixed origin. Re-centering the panels, though, means moving every coordinate below $y \approx 518$ in a 14 KB stream by hand, where a single arithmetic slip renders plausibly and silently wrong.
- *Rebuilding the one-pager as HTML to match the mockup family* — the screen mockups are HTML because they specify on-screen UI. Print collateral needs fixed page geometry and a PDF handoff, and browser print pipelines differ in margins, hyphenation, and color handling. ReportLab produces the artifact directly, with no render step to disagree about.
- *Adding fill below the last line only, as literally requested* — restores the missing tint but leaves the text block sitting high in the panel, which is the same defect a few points smaller. Centering costs the same edit and fixes the cause.
- *Enlarging the pricing card and the two bands to match* — change for symmetry’s sake on elements that already read correctly, and it would have cost another vertical shift on an already-full page.
- *Re-flowing or trimming copy to reclaim the vertical space instead of shifting content down* — the page carried 64 pt of unused white above the footer. Spending that is cheaper than re-breaking approved lines and re-reviewing the copy.

Confirmed

- The 2026-06-28 follow-up — “*The Sales One-Pager PDF (in project knowledge) is presumed to use the dark-background mark in its header/footer bands. Confirm at the next sales-collateral refresh.*” — is **confirmed and closed**. Both bands draw the dark-background variant: white bubble, pale-blue rules (#A7BCE8), navy check (#1A3A75) over a white keyline, at full size in the header and 0.533 scale in the footer.

Follow-ups

- Owner: replace **WYP_Sales_OnePager_FinalLogo.pdf** in project knowledge with **WYP_Sales_OnePager_v2.pdf**, add **build_onepager.py** alongside it as the collateral’s source of record, and update the Canonical sources block to list both. Note that the copy of the one-pager currently in project knowledge is **not a readable PDF** — it has no trailer dictionary and no readable cross-reference table — so it needs replacing on those grounds alone; this refresh worked from the owner-supplied upload.
- The free-tier card states 100 MB; the paid card still promises only “Keep your requests and files forever” with no storage figure. Consistent with the PRD §15.2 open item and the no-number rule adopted for §9.19 (2026-07-22, decision 25). When the paid-tier figure is set, the paid card gains a fourth line and the layout absorbs it without a manual re-measure.
- Sweep the other outward-facing surfaces for the stale \$11.99 — website copy, any deck or listing draft, and anything already sent to a reader outside the project. The one-pager was caught by inspection, not by a sweep, so it is unlikely to be the only place.
- At the next refresh, re-check “Now available in your web browser” and “Coming soon: native apps for iPhone, Android, and Windows” against the phase language in PRD §11.1, so the collateral’s promises and the roadmap’s phases stay in step.

2026-07-24 — Component system introduced (shared design-token and component stylesheets); WYP- identifier scheme adopted; token-name reconciliation; main-screen assets vectorized (no PRD or UI-spec content change; new Component System Specification v0.1)

The seven screen mockups were refactored from standalone HTML files — each carrying a full duplicated copy of the palette-1 CSS — into markup plus two shared stylesheets. No product requirement or design-system *semantic* changed this session; the single design-system-adjacent change is a token **name** reconciliation to the §3.1 label, with the value unchanged. Every collapsed screen was rendered and pixel-diffed against its pre-refactor original before delivery.

Decisions taken

1. **Design tokens centralized into one source of truth, `tokens.css`.** The palette-1 `:root` block — seventeen tokens — had been duplicated verbatim at the top of all seven mockups, so a token change meant seven edits. It now lives in one file every screen imports. The values written are the exact union of the seven files (verified equal across them). Treated as governed by the same change control as UI-spec §3.1, per the §3.3 rule that a new token requires a §3.1 revision.
2. **Shared component styles extracted into `components.css`.** The shell, header band (§6.8), group-label band (§6.7), primary and secondary buttons (§6.1/§6.11), base text input and floating-label behavior (§6.10), subscription banner and ad slot (§6.14/§6.15), and the modal frame (§6.12) are defined once and read their colors from `tokens.css`. Each screen now links, in order, Google Fonts → `tokens.css` → `components.css` → a small screen-specific `<style>`. Per-screen style blocks shrank sharply (e.g. Create Request 117 → 33 style lines). Verification: six screens render pixel-identical to their originals; the main screen differs only where its logo and icons were vectorized (decisions 7–8).
3. **Token-name reconciliation: `--field` → `--focus-tint`.** §3.1 names this token “Focus Tint” (#EDF2FD). Four form mockups defined it as `--field` while the two storage mockups already used `--focus-tint`; standardized on `--focus-tint` everywhere — the name that matches §3.1 and the majority of files. Pure rename:

the value #EDF2FD is unchanged, so nothing renders differently. Closes the latent drift first flagged when the mockups were consolidated.

4. **--scrim tokenization gap closed.** The modal scrim (Add Contact no-contact dialog, Storage Maintenance) hardcoded `rgba(31,41,51,.45)` inline although §3.1 defines a Scrim token; `components.css` now applies `var(--scrim)`, so the overlay is governed in one place like every other color.
5. **WYP- component-identifier scheme adopted (owner, 2026-07-24), specified in the new Component System Specification v0.1.** Stable identifiers — PascalCase under the WYP- prefix (e.g. WYP-Button, WYP-LookupField) — map one-to-one onto the eighteen UI-spec §6 components, so one name denotes the same part in the spec, in the eventual code, and in working conversation. The identifier is the durable handle; the §6 section number stays where the anatomy lives. Identifiers are set in monospace and never quoted, keeping a component name distinct from a verbatim on-screen label (which §1.5 reserves quotes for). Consequent to adoption, §6 entries carry their identifier as each is next revised (prospectively, per the §1.5 conformance rule), and the Component System Specification becomes a companion to the UI spec in the Canonical sources block.
6. **Self-contained copies produced for viewing.** Because the linked files reference `tokens.css/components.css` by relative path, they render unstyled in an isolated preview (no sibling files) and locally only when all three files share one folder with exact names. A parallel set with both stylesheets inlined into each `<head>` was produced; these render anywhere with no setup. Working model: edit the linked set (single source of truth), view and share the standalone set, regenerate the standalone set from the canonical CSS after any change.
7. **Main-screen logo converted from a base64 PNG to the canonical inline SVG.** The main screen alone embedded its logo as a raster data-URI (~13 KB); the other six draw it as inline SVG, and those six were verified byte-identical to one another. The main screen now carries that same SVG — crisper on high-DPI, lighter, and consistent. Only the logo pixels change; nothing else on the screen moves. Judged an iteration artifact (owner concurred), not an intentional divergence.
8. **Main-screen icons converted from base64 PNGs to inline SVG from the project icon files.** All seventeen raster icons (three Expand, three Print, Search, Voice Search, six Attachment glyphs, three Dialog glyphs) were replaced with the `wyp\_icon\*.svg` assets. Decorative inline SVGs are marked `aria-hidden`; the accessible name stays on each wrapping control (`role="button", aria-label`), so screen-reader behavior is unchanged. Sizing rules retargeted to the inline SVG (`.iconbtn svg, .sb .iconbtn svg, svg.iico`). The main-screen file dropped from ~157 KB to ~22 KB. `wyp\_icon\_contract.svg` is present in the asset set but unused in the pilot mockups — it belongs to the not-yet-mocked expanded-list view (the collapse counterpart to Expand).

Alternatives considered and rejected

- *Keeping the --field name, or minting a new token* — `--focus-tint` already matches §3.1 and was already used by two screens; renaming the four outliers is the smaller, drift-closing change.
- *Promoting the chip and sort-pill styles into components.css now* — their definitions genuinely diverge across screens (main uses `.sel/.over/.done`; Add Contact uses `.selected`; Storage uses simpler variants; the main and storage `.pill` differ in size). A shared definition would risk a visual change and presupposes a single canonical chip spec, which is a design decision, not a mechanical extraction. Left local; the main screen is now the reference implementation for §6.2/§6.3.
- *Leaving the raster logo and icons in place* — they read as iteration artifacts, not intent; the spec already names these as SVG assets (§5.1, §6.16). Vector is crisper, far lighter, and consistent with the other screens.
- *Combining all seven screens into one multi-screen file with a switcher* — offered as a viewing convenience; not adopted, left available.

Follow-ups — documents

- **Component System Specification** — completed to v1.0 (2026-07-24): all eighteen §6 components carry a full anatomy entry (Purpose / Anatomy / States / Composition / Reference implementation). Filed as `WYP_Component_System_Specification_v1_0.docx`, part of the canonical set and the Project Instruc-

tions Canonical sources block. Its token table still reads “Focus Tint (--field in mockups)” — update to --focus-tint when next touched.

- **UI spec §3.1:** when next revised, cite the CSS name --focus-tint alongside the “Focus Tint” label, and record tokens.css as the canonical token file (same change control as §3.1, per §3.3). No value or semantic change.
- **UI spec §6:** add each component’s WYP- identifier as each §6 entry is next touched — prospectively per §1.5, not as a sweep.
- **Canonical sources block (project instructions):** updated this session — adds tokens.css, components.css, and the Component System Specification, and records that the seven mockups now exist as a linked set (depending on the two stylesheets) plus a standalone inlined set for viewing.
- **Mockups in project knowledge:** the main-screen mockup is now updated (logo and icons vectorized) — this supplies the file the 2026-07-22 entry recorded as *missing* from project knowledge. Upload the current seven (both forms). The 2026-07-22 note that WYP_storage_maintenance_palette1.html is one revision behind still stands and is unaffected by this session.

Deferred / open

- **WYP-ChipRow and the sort-pill component:** unify the divergent chip/pill definitions into shared components once the single canonical spec is decided (§6.2/§6.3). Main screen is the reference.
- **Unused tokens --btn-top and --icon-grey:** defined but never referenced through var(); --icon-grey (#6B7280) is also misleading, since the information-only glyphs actually render in --ink-soft (#5A6675). Retire once confirmed unused in the build; carried in tokens.css under a “deprecated / unused” note meanwhile.
- **Icon components / sprite:** with the glyphs now defined once as files, the shared layer can absorb them (a small icon-component set, or a sprite) rather than pasting SVG per screen — do this alongside the chip/pill unification.
- **Spacing scale:** only --pad is tokenized; the full §7.2 scale is not yet in tokens.css. Add --space-* mirroring §7.2 when convenient (a design decision, so not invented here).

2026-07-28 — v1 scope cuts (attachments → paid; voice search deferred; .ics kept; monetization deferred); offering/messaging rework; local + free-tier build approach adopted; PRD finalized to v12.7 (sales one-pager v3; UI spec §6.18/§9.19 still to update)

A roadmap-and-sequencing session. No UI-spec component anatomy changed, but the free/paid feature split was reworked, two mockups gained a locked “paid feature” attachments state, and the build path was set to a local + free-tier stack rather than the PRD §8.2 AWS target. Reasoning, alternatives, and document impacts below.

Decisions taken

1. **Attachments deferred out of v1 and repositioned as a paid-subscription feature.** They arrive with monetization, not at free launch. This removes the entire storage subsystem from the v1 critical path (S3, quotas, retention enforcement, the §6.18 warning strip, and the §9.19 Storage Maintenance screen all become paid-tier surfaces). Rationale: attachments are not the product’s value proposition (which is send / track / follow through), and cutting them buys back the most build time of any single scope decision.
2. **In the Create Request and Respond mockups, the Attachments panel shows a locked “paid feature” state rather than being removed.** Owner decision, overriding the earlier recommendation to hide the panel entirely. The panel frame is kept; the functional file list and Add control are replaced with a lock glyph, an “A subscription feature” heading, and gating copy pointing to the subscription banner (§6.14) — mirroring the existing Request-Texting gating pattern. The Add Attachment button renders disabled. Rationale: the panel advertises the upcoming paid capability in place rather than hiding that attachments exist. Implemented as screen-local CSS (a new .locked treatment) reusing existing tokens; introduces no new token and no change to components.css.
3. **Voice search deferred from v1, to return later covering both the search bar and request-text composition.** The search bar ships without the voice glyph in v1; when voice returns it applies both to search entry and to dictating request text. The one-pager lists it under Coming soon.

4. **.ics / Add to Calendar is included in the initial offering.** Owner considers it a critical usage feature for request recipients — the due date drops onto the recipient’s calendar in one tap. It is application-side code (generate the calendar file from the request’s due date/time), with no infrastructure dependency, so it ships free in v1.
5. **Monetization deferred: the free tier alone is live at v1 launch.** The paid tier (\$17.95/yr) is introduced later and unlocks attachments, Request Texting (SMS), perpetual retention, and ad-free. Free-tier ads remain deferred to 200 registered users (prior decision). Consequently the paid tier and its features are messaged as “coming with a subscription” at launch, not as live options.
6. **Build approach: a local + free-tier development stack, not the PRD §8.2 AWS architecture.** §8.2 (React SPA, NestJS/Go, RDS, S3, Redis, SES, SNS/Twilio, Stripe, AdSense on AWS) is confirmed as a sound production target but explicitly not a prerequisite to building. The this-week stack has a zero-cost stand-in for every production piece: Next.js + TypeScript locally, Vercel free for host and CI/CD, Supabase free for Postgres + Auth (magic links) + storage, Resend free for email (avoiding the SES sandbox), Sentry free for error tracking. Genuinely deferred to launch: SES with SPF/DKIM/DMARC and domain warming, Stripe live (test mode for all development), Twilio with its 10DLC lead time, S3, Redis, and AdSense. A Week 1 setup guide (repo → Supabase → Add Contact wired to real data) was produced to make the “this week” column concrete.

Alternatives considered and rejected

- *Removing the Attachments panel entirely from the v1 mockups* — the recommended option; owner chose the locked “paid feature” state instead, to keep the capability visible and advertised rather than absent.
- *Keeping attachments as a free feature, or launching with monetization live* — rejected; deferring both is what makes the solo 2–3-month timeline reachable and lets a free product validate before any billing, storage, or ad infrastructure exists.
- *Building on AWS from day one (with a credits application)* — rejected as slower and costlier for a solo developer with no benefit until there are users; AWS is sequenced last, after a beta proves the product and reveals real scale (which also strengthens the credits application).

Follow-ups — documents

- **Sales one-pager: done.** Rewritten to the free / coming-with-a-subscription / coming-soon split as WYP_Sales_OnePager_v3.pdf (with a rebuildable HTML source, onepager.html), superseding v2. Attachments and Request Texting moved to the subscription column; voice search and native apps to Coming soon; Add to Calendar highlighted as a free feature. The v2 claims that broke — “with attachments” in the free description, “reply with answers and files,” voice search as a live feature, and “100 MB attachment storage” in the free tier — are removed.
- **PRD updated to v12.7 (done, 2026-07-28).** Sections changed: §6 Monetization Model — added a “Launch sequencing” note (free tier only at launch; paid tier and its features activate at Phase 0+; ads deferred to 200 users) and reconciled the free/paid matrix (File attachments = paid, not free; Advertising = deferred). §9 — added the base-subscription note (\$17.95/yr unlocks attachments + Request Texting + perpetual retention + ad-free), distinct from the future business add-ons. §10 — rewrote the Phase 0 scope (free tier; email; .ics; attachments locked; SMS/OAuth/voice deferred) and added a Phase 0+ — Monetization row. §11 — added a “Two build paths” note (team basis vs the solo/AI/free-tier path), revised the §11.1 work breakdown (magic-link auth, email-only notifications, .ics row added; Attachment/Ad/Subscription streams deferred → Phase 0+; free-tier subtotal ≈ 20–30 wks) and the §11.3 budget (cloud/third-party → free-tier ≈ \$0–\$2,000; security-audit scope narrowed; TOTAL \$118k–\$319k). §12 — UI-spec reference corrected v2.5 → v2.9. Title, footer, and Revision History bumped to v12.7. Old file WouldYouPlease_PRD_v12_6.docx superseded; Project Instructions Canonical-sources reference bumped. STILL OPEN: UI spec §6.18 (storage warning strip) and §9.19 (Storage Maintenance) become paid-tier-only surfaces — not yet edited in the UI spec.
- **UI spec §6.18 (storage warning strip) and §9.19 (Storage Maintenance):** these govern attachment storage, now a paid-tier surface — note that they do not appear for free-tier users. §9.2.2 (Create Request attachments) and §9.3.2 (Respond attachments) should record the v1 locked “paid feature” presentation.
- **Component System Specification:** the locked attachments treatment is a candidate shared component — provi-

sionally WYP-LockedFeature — once a second surface uses it (today it is screen-local in two mockups, consistent with the chip/pill deferral). Add it to the registry when promoted.

- **In-app gating copy:** the attachments panels now carry the same “available with a subscription” pattern as Request Texting; keep the wording consistent across both when the PRD copy is finalized.

Deferred / open

- **Recipient-facing attachment upsell:** a non-registered recipient responding via the secure web link sees the locked “subscription feature” attachments panel, but the upsell target is ambiguous for someone without an account. Revisit the recipient-facing copy (and whether the panel should appear at all in the recipient view) when the paid tier ships.
- **WYP-LockedFeature** shared component — promote from the two screen-local copies once a third surface needs it.
- **Voice search** return — search bar and request-text composition — tier and timing not yet set.
- **Security-review scope, recalibrated:** the owner’s law-firm CRM secure-landing-page product (a provided server mapped to a firm’s sensitive internal contact database via an iFrame on the firm’s public site, reachable from email links, for firms up to 300+ attorneys) is directly analogous to WYP’s token/link-mediated secure recipient-response flow. The earlier “single-user / trusted-environment” characterization was wrong. The pre-launch security review therefore narrows from “validate the whole model” to “confirm a known model is translated correctly into web-stack idioms” (Supabase RLS, signed URLs/JWTs, the browser threat model).

2026-08-05 — Housekeeping section added to Main Screen mockup (Contacts and Account Profile gain an entry point); new §6.23 row component

Neither Contacts nor Account Profile had a way in. §9.8 retired the standalone contact-browse screen in v2.7 (contacts are reached only via Add/Edit Contact or the Create Request type-ahead), and §6.14’s Subscription/Account banner opens Settings for subscribers but reads as an upsell strip, not an “edit your profile” entry point. Owner mocked up a “Housekeeping” section at the bottom of the Main Screen scroll, below Todos, to hold both.

Decisions taken

1. **Housekeeping is a new top-level section on the Main Screen**, structured identically to Requests and Todos: a `.band/.glabel` heading (“Housekeeping”, no button — nothing to create) followed by a `.subcard` with a `.subhead/.subname` label (“Tasks”).
2. **New row component, §6.23:** `.hkrows/.hkrow/.hktitle/.hknote`. Bold Brand-Blue title, em-dash, regular Ink description, 2-line clamp — the same visual language as the §6.5 ToDo row’s Priority/description pairing, in its own classes rather than reused `.pri/.tdd`, since the meaning here is navigation, not Priority. No chevron or other new affordance glyph: every list row in the app (Sent, Received, ToDo) already signals “tappable” by row context and convention alone, and Housekeeping stays consistent with that rather than introducing a new one.
3. **Two rows shipped in the mockup: “My Contacts — view and edit” and “Account Profile — view and edit”.** Both carry a description even though the title alone would likely read fine, so the pattern is established consistently from the first two items — future rows (Storage Maintenance is the obvious next candidate, and already has a screen with no entry point either) follow the same shape without a new styling decision.
4. **Punctuation is the em-dash from §6.5’s convention (“—”, not a hyphen)**, for consistency with the row style being reused.

Alternatives considered and rejected

- *Restoring the Settings gear to the search bar* — owner’s first instinct, self-rejected before proposing Housekeeping. §6.6.1 already states the gear was removed in v2.5 in favor of the bottom banner; reintroducing it would reopen a settled decision and bury the entry point behind an icon rather than plain-language text.
- *Underlined link-style text for each row* — owner’s original sketch. Reads as inline hypertext inside a paragraph, not a navigational list row, and doesn’t hold a two-line title-plus-description gracefully.

- *Stacked .btn/.btn-secondary per item* — rejected; buttons in this system are single-line, fixed-emphasis controls, would compete visually with the actual primary actions on the same screen (Create Request, Create ToDo), and have no room for a description line.
- *Reusing .checkrow, checkbox included* — rejected; there’s no on/off state being represented, and a checkbox implies one that isn’t there.

Follow-ups

- §9.13 Settings’ Purpose line still describes the retired gear-icon entry point rather than the bottom banner (§6.14) — stale, needs a correction pass independent of this change.
- Whether Account Profile via Housekeeping duplicates or supplements the existing banner→Settings path is intentionally left as two entry points to the same destination (owner confirmed comfortable with that), not resolved as a single canonical path.
- Not yet promoted to the UI spec proper (§6 fully occupied through §6.22 plus this §6.23) or converted to React — Main Screen remains Mockup status.

Confirmed — owner reviewed the rendered mockup (WYP_main_screen_palette1.html) and approved it as-is, 2026-08-05, including the .subname blue heading treatment on the Tasks subcard. §6.23 and the Housekeeping section are locked; further changes go through a new entry rather than editing this one.

2026-08-05 — Print icon (Create/Respond), Dialog order + color fix, Time Zone field added across three mockups

Print icon. Added a print icon (.iconbtn, inline SVG, 40×40 touch target) to the band cluster on Create Request and Respond to Request, first child before Send/Cancel, per owner’s rough mockups.

Dialog panel fix (Respond to Request). Two changes, both in WYP_respond_to_request_palette1.html:

1. Entries reordered to descending/newest-first (was ascending). The demo data has no real ordering field yet, so this is a manual reorder of the two mocked entries, flagged in an HTML comment for the real implementation.
2. .dlgdate color changed from var(--alert-red) to var(--brand-blue). Red is reserved for Overdue and other status/error use (§3.1); decorative use on Dialog dates was a mismatch. Dialog type label stays bold black, unchanged.

Follow-up — Dialog needs a real ordering field. Owner’s observation: a Question and Answer landing on the same date is ambiguous under date-only sort. If entries carry a sequentially-assigned internal ID (or a timestamp used only for ordering, not display), that resolves it without exposing time-of-day in the UI, which isn’t meant to matter. No dialog table exists yet (Week 2 plan doesn’t include it), so this is a schema note for whenever Dialog is designed, not an open bug: **the dialog table’s primary key or a created_at timestampz should double as the sort key**, not the display date.

Time Zone field. Added a §6.16 lookup/picker field labeled “Time Zone” to three mockups:

- WYP_add_contact_palette1_floating.html — after Phone, before Send Requests by. Empty (no default rendered in the mockup).
- WYP_add_contact_no_contact_dialog_palette1.html — same position, propagated to match (this file didn’t yet have .lglyph CSS; added it here).
- WYP_create_free_account_palette1.html — after the Display Name note, before Phone. Pre-filled with “Central Time (Chicago)” to match this screen’s convention of showing example data throughout.

Defaulting — answering owner’s question directly. These are two different fields with two different answers:

- **Create My Free Account’s Time Zone is the signed-in user’s own** — browser-detectable (Intl.DateTimeFormat().resolvedOptions().timeZone), so defaulting it is correct and is what the mockup’s pre-filled value implies. This also becomes profiles.time_zone, already an open item in WYP_Week2_Plan.md for §2.7 Overdue evaluation.

- **Add Contact's Time Zone is the *contact's*, not the *user's*.** Resolved 2026-08-06: default it to the owner's own profile time zone (`profiles.time_zone`) at Add Contact's initial render — most contacts share the owner's zone often enough that this is a reasonable starting guess, and the field stays editable so the owner can correct it when they know otherwise. This makes `profiles.time_zone` a dependency for populating the default (not just for Overdue evaluation), and it's the reason `contacts.time_zone` should default from the *client-side* value already on hand rather than a second lookup. Mockup comments in both Add Contact files updated to reflect this.

Schema follow-up — `contacts` needs the same column `profiles` needs. WYP_Week2_Plan.md's open-questions section already flags `profiles.time_zone` as undecided; this now needs a `contacts.time_zone` counterpart for the same reason (§2.7-adjacent: knowing a contact's zone matters for showing them times/dates meaningfully, even though Overdue itself is anchored to the sender). Not yet added to the migration 003 draft.

Live-component gap. `app/components/AddContactForm.tsx` and the `contacts` table (migration 002) do not have a Time Zone field or column — Add Contact is the one screen among those touched here that's already Converted/Live, so this mockup change is now ahead of the live component. Flagging rather than silently adding a column: the field needs the defaulting question above resolved first, and touching the live form/table for a still-open design question isn't the right order.

2026-08-06 — Create ToDo designed (§9.4 had no mockup); Dialog/Attachments held as client-side draft state rather than a staging table

Owner mocked up Create ToDo by hand and pasted it in — the first design pass this screen has ever had; §9.4 was a placeholder with no mockup at all (Week 2 plan, “three screens that don't exist yet”). Built as `design/screens/WYP_create_todo_palette1.html`, reusing Create Request's structure (band cluster, Category lookup, Attachments locked panel) rather than inventing new patterns.

Priority — caught before building, not after. The pasted mockup omitted Priority (ASAP/SOON/LATER), which is the one field that actually distinguishes a ToDo from a Request in the unified `requests` table (§2.5), and Main Screen's default ToDo sort is Priority ascending — without it, every ToDo would sort identically. Added as a one-of-three chip row, reusing the §6.2 chip-row pattern already established for Send Requests by (previously only ever used as a two-way choice).

Due Date — confirmed intentionally absent. PRD lists Due Date as optional for ToDos; owner confirmed omitting the field entirely from this screen (rather than including it as an optional picker) is deliberate, not an oversight.

Description and Dialog use the floating-label pattern (§6.10), not the fixed top-left `.p1abel` caption style Attachments uses. Owner's call, given both styles exist in the system and the pasted mockup was ambiguous between them.

Dialog and Attachments on an unsaved ToDo — client-side draft state, not a staging table. Owner raised a real concern from prior CRM work (law-firm systems where users were blocked by database sequencing from entering data in the order they expected) and described a pattern used there: a temporary table, one row per user per draft “kind,” holding entries not yet linked to a master record, reconciled to the real row on save.

Recommendation given and accepted: for Create ToDo specifically, Dialog entries (and, later, Attachments) are held as ordinary client-side form state — the same mechanism already used for every other field on this form — and written to the database together with the ToDo row itself, using its just-created id, in one Save. This avoids inventing a new persistent table for state that only needs to survive one browser tab, one sitting.

The staging-table pattern is *not* rejected outright — it's the right tool for a different screen. Once a Request exists and Dialog is added to it over time by two different people (owner and recipient, separate sessions, no single overarching Save action — the Detailed Item / Respond to Request case), there's no browser tab to hold that state in, and something server-side is required there. That's a decision for whenever Detailed Item (§9.6) gets designed, not for Create ToDo.

Follow-up — abandoned-form data loss. Owner noted that if a session times out or the tab is abandoned mid-entry, losing unsaved Dialog/Description work without warning is standard behavior across most software, but a `beforeunload` “you have unsaved changes” browser prompt would be a cheap, standard way to soften it. Not built yet — Supabase’s session in this app is long-lived (no idle-timeout mechanism exists currently), so the realistic failure mode is closing the tab or navigating away, which `beforeunload` does address. Flagged as a small, deferred enhancement rather than built unprompted, per CLAUDE.md’s scope-discipline convention.

Add Attachment alignment — top, not centered. Briefly changed to vertically centered on the panel, then reverted. Top-aligned is correct: the panel’s 118px height is an artifact of the locked v1 state, not real content, so a top-anchored button stays consistent once the panel becomes content-driven and its height changes; a centered button would drift with it. Matches Add Dialog’s existing top alignment, and matches Create Request’s Attachments row, which was already top-aligned and untouched throughout this back-and-forth.

2026-08-06 — Dialog added to Create Request (live); migration 004; chancap overlap bug fixed

Dialog field. Added to both the Create Request mockup and the live `CreateRequestForm.tsx`, matching Create ToDo’s field exactly (floating label, top-aligned Add Dialog, same `.frow.top` fix). Entries are staged as plain client-side draft state (an array of strings — no kind picker in this v1) and written to a new `dialog` table together with the Request when Send succeeds, using the new row’s id. Reuses `.attitem/.attname/.attremove` for the staged-entries list (new `.dlgstaged` wrapper, since the fixed-height `.attachpanel` frame doesn’t fit a list that grows with entries) rather than inventing a second list treatment.

Migration 004 — dialog table, not yet run. `request_id` covers both Requests and Todos uniformly, since Todos are rows in `requests` too (§2.5). `who` is a display-name **snapshot** taken at post time, not a live join to profiles — matches the immutable-fact reasoning already used for events (migration 002): a later name change shouldn’t rewrite what a past Dialog entry showed. `id` is a plain sequential bigint, deliberately, so it doubles as the sort key alongside `created_at` when two entries land on the same day — this is the mechanism the 2026-08-05 entry called for (“if internal IDs are sequentially assigned, they could be used for this sort”). No UPDATE or DELETE policy: append-only, same reasoning as events. Full SQL in `docs/Week2 - SQL history.txt`; **owner needs to run this in the Supabase SQL editor before Send will succeed on a Request with any Dialog entries** — until then, Send fails at the Dialog insert step with the Request already saved (handled as a distinct partial-failure error message, not a silent loss).

Attachments alignment carried over. Add Attachment stays top-aligned on Create Request too, for the same reasoning worked out on Create ToDo (2026-08-06 entry above) — it was already correct there and untouched by this change.

Bug found and fixed — .chancap overlap. The “Will be sent by Email” caption under Recipient used `margin: -8px 0 12px 2px`, copied verbatim from the mockup. The mockup never actually renders this caption (`display:none` always, since the mockup has no live contact-selection state), so the negative margin was never visually checked — live, it overlapped the bottom edge of the First/Last Name row. Fixed to a small positive top margin (4px, matching `.ferror`’s spacing convention) in both `globals.css` and the mockup’s duplicate rule. Worth remembering as a general risk: CSS ported from a mockup for a state the mockup itself never actually shows isn’t verified by the mockup looking right.

2026-08-07 — §6.24 lookup fields: browse a short list on focus, search a long one

Owner noticed Category’s lookup on Create Request shows nothing until a character is typed, which is unnecessary friction when the whole list would fit on screen. Rule adopted, meant to apply to every §6.24 lookup app-wide, not just this one: on focus, if the field’s full option list has fewer than 12 entries, show all of them immediately; at 12 or more, wait for typed input before showing anything, same as before. `LOOKUP_BROWSE_THRESHOLD = 12` in `CreateRequestForm.tsx`; applied to both Recipient (contacts) and Category (categories). Recorded against the

§6.24 component definition in `design/README.md` rather than only in code, so it isn't rediscovered per-field. Empty-list messaging was split too: "No contacts yet" when browsing an empty list vs. "No matching contact" when a typed query has no hits — different situations, shouldn't read the same.

2026-08-07 — Android layout fixes (Create Request, Respond to Request, Main Screen); optional-field shading previewed

Owner reviewed Create Request on an S24+ and found several issues that a desktop-width mockup review hadn't caught: the print icon crowded Send/Cancel and wrapped "Create a Request"; "(optional)" overflowed narrow fields (Category, Due Time); Dialog's "(Questions, Answers, Comments)" did too; the Attachments panel's fixed height clipped its own copy; and on Main Screen, Log Out's `margin-left:auto` pushed against the wordmark and wrapped "Would You Please."

Print icon moved to the header, right-aligned. Out of the band cluster (`.bandcluster`) and into `.hdr`, roughly where Log Out used to sit on Main Screen — the two moves are opposite directions for the same underlying reason (a right-aligned control competing with title text for width). Applied to Create Request and Respond to Request mockups, and to the live `CreateRequestForm.tsx`. `WypHeader.tsx` gained an optional `action` prop (a right-aligned slot) rather than duplicating header markup per screen — Respond to Request will use the same prop once it's converted.

Log Out moved out of Main Screen's header, into its own thin row below (`.logoutrow`). Opposite fix, same cause: freeing header width so the wordmark doesn't wrap.

.subnote — small parenthetical label text, PROPOSED, no § yet. "(optional)" (Category, Due Time) and Dialog's subtitle now render at 11px, matching the size a risen/floated label settles at (14px × the existing 0.79 scale ≈ 11px), so it reads as "the same label, just settled" rather than a new visual language. Dialog's subtitle also moves to its own line (`.flabel.twoline`, since the default `.flabel` is a flex row for the §6.16 glyph case and won't wrap on its own); its `textarea` gained `padding-top:36px` to clear the taller two-line label. Applied to Create Request (mockup + live) and Create ToDo (mockup). Not applied to Add Contact's "Notes (optional)" — not reported as a problem, left alone rather than changed speculatively.

Attachments panel: height → min-height. The fixed 118px height clipped `.locknote`'s copy on narrow widths or larger Android font-scale settings; letting it grow avoids the clip. `globals.css` plus both mockups that use the panel (Create Request, Create ToDo).

Optional-field shading — previewed, not adopted yet. Owner's own idea: tint optional fields so required ones (left plain white) are what the eye catches without an asterisk or a legend — and asked for a color, having tried a light green as a placeholder and ruled out light blue himself (already carries focus/chip-strip meaning). Recommended and applied as a preview on Create Request's mockup only: reuse `--row-tint` rather than introduce a new token. Reasoning: Row Tint is already the `.form` page background, so a Row-Tint-filled optional field visually recedes toward the page while a white required field pops — the exact effect being asked for — and it's one of the only three tinted backgrounds §3.3 permits (Band, Row Tint, Strip), so this needed no new color at all. A pale yellow was also considered and rejected: `--sort` is documented as "the sole yellow in the system," reserved for the active sort pill; a second, diluted yellow use would undercut that. Fields shaded: Due Time, Category, Dialog (the three actually-optional fields on this screen) — not Due Date, which is required, even though the owner's own rough mockup appeared to shade it too (read as an inconsistency in a quick draft, not a stated intent to make Due Date optional). Still switches to `--focus-tint` on focus, same as every field. **Open:** whether to adopt this app-wide, and whether Row Tint is the right final answer — flagged as a preview, not locked in.

2026-08-07 — Optional-field shading adopted app-wide; secondary buttons re-colored; Log Out relocated again; contacts gain Display Name (migration 005); Add Contact and Create Request’s Recipient collapse to a single Name field

Follow-on session, same day as the layout-fixes entry below. Owner confirmed the Row Tint preview worked and gave a full rule plus several more fixes in one message.

Optional-field shading adopted, with a rule: empty optional → Row Tint, has content → white, required → always white. Previously only previewed. `.opt` is now a real toggled class, not a static mockup treatment — `.finput.opt/.ftextarea.opt` in `globals.css` (Row Tint at rest, Focus Tint on focus, inserted so the focus rule still wins). In the live `CreateRequestForm.tsx`, Due Time, Category, and Dialog each compute `opt` from their own current value (`field.trim() === '' ? 'opt' : ''`) rather than being permanently classed — the background now genuinely tracks content, not just field identity. Applied the same class logic to Create ToDo’s mockup (Category, Dialog — that screen has no live component yet). **Consequence caught along the way:** both mockups had Category pre-filled with “Personal Fin” to demonstrate the risen-label state, which under the new rule means it should render *white*, not tinted — directly contradicts Jim’s ask to see Category Row-Tinted on Create ToDo. Emptied the field in both mockups instead of leaving the contradiction; the risen-label demo is a lesser loss than a mockup that visibly violates its own stated rule.

Respond to Request needed no Attachments/Dialog change. It doesn’t use `.attachpanel` at all — both its Dialog and Attachments panels are a separate `.panel{background:#fff;...}` component, already white, and both already carry example content. Matches the `has-content→white` rule with zero edits; confirmed by reading the file rather than assumed.

Log Out moved a second time, out of its own row (2026-08-07, previous entry) and into the Housekeeping band header itself, right-aligned via `.btn-quiet’s` existing `margin-left:auto` — the same slot Create/Send buttons occupy in every other band row. Chosen over showing it as the first Housekeeping task-list row: a band-header control reads as “available here,” not as a task, and Jim’s own framing (“not something the app expects end-users to do unless on a public computer”) fits a low-emphasis header control better than a list entry.

Attachments panel Row-Tinted everywhere it appears (`globals.css`, Create Request mockup, Create ToDo mockup) — it can never hold real content in this v1 locked state, so under the empty/filled rule it’s permanently in the “optional and empty” bucket, not a one-off exception.

Create ToDo: print icon moved to the header, out of the band cluster, matching Create Request and Respond to Request — same reasoning as the earlier print-icon moves (a right-aligned control competing with title text for width).

Secondary buttons (`.btn-secondary`) recolored from white to Strip. Once white started meaning “required, or optional-with-content” (the shading rule above), a white secondary button read as part of that same group and visually “popped” next to it. Rejected literally copying the chip’s `rgba(255,255,255,.5)` value: a chip only reads as light blue because it composites translucent white over the Strip-tinted `.sendrow` container; a `.btn-secondary` doesn’t sit on that container; the same `rgba` over Row Tint or white would barely register. Used `var(--strip)` as an opaque fill instead — no new token, Strip is already one of the three tinted surfaces §3.3 permits. Updated in `globals.css` and every mockup that duplicates the rule (Create Request, Create ToDo, Respond to Request, both Add Contact mockups, component-states reference, Your Account, both Storage mockups) — left the superseded `design/drafts/WYP_requests_partial_SUPERSEDED.html` untouched, since it’s a frozen draft, not a live surface.

Pull-downs/lookups sort alphabetically, not by creation order — owner’s rule, app-wide, with the Housekeeping task list’s Log Out entry as the sole named exception. Categories were already `.order('name')`; contacts previously had no `.order()` at all — added (now `.order('display_name')` after the Name-field merge below).

Contacts gain `display_name`; migration 005 written, not yet run. Resolves the open question Jim left me: whether to keep or drop `contacts.first_name/last_name`. **Kept**, matching Jim’s own more detailed proposal over the alternative he also offered (dropping them) — the deciding asymmetry is that a comment on column explaining “pre-served, not populated” costs nothing, while dropping and later wanting them back means re-adding the columns and

re-migrating. `display_name` is backfilled from `trim(concat_ws(' ', first_name, last_name))`, falling back to `email` for the handful of rows with neither name (see Week1 SQL history’s email-only test inserts), then constrained not-null and non-blank. This is a different object from `profiles/Create My Free Account`, which explicitly keeps its separate First Name / Last Name / Display Name three-field structure — Jim reaffirmed that design after initially proposing to unify it with contacts, then reconsidering. The two forms now diverge on purpose: a contact is entered by the request-sender about someone else and needs no legal-name precision; a profile is self-entered at signup and different rules can apply if a Request recipient signs up (their contact’s Display Name seeds the new profile’s Display Name field, then they must add a First or Last Name of their own).

Add Contact’s First/Last Name fields collapsed into one Name field, writing only `display_name` — `AddContactForm.tsx` and both Add Contact mockups (`WYP_add_contact_palette1_floating.html`, `WYP_add_contact_no_contact_dialog_palette1.html`). Validation changed from “First or Last Name” to a single non-blank Name. `first_name/last_name` are simply never written by this form going forward — the columns exist per migration 005 but sit inert.

Create Request’s Recipient row collapsed the same way, into a single Name lookup — `CreateRequestForm.tsx` (Contact type now `{ id, display_name, send_by }`; the old `firstName/lastName` split-field filter logic replaced with one substring match against `display_name`) and the Create Request mockup. Field `id` changed `fn/ln` → `rn`; label changed “First Name”/“Last Name” → “Recipient”.

Follow-up — the Week2 - SQL history.txt header comment on migration 004 still says “Not yet run”, though Jim confirmed running it 2026-08-06. Left as-is rather than rewritten silently; a documentation-accuracy nit, not a functional gap, flagged here so it isn’t lost.

2026-08-07 — Header wordmark shrunk to stop wrapping; four screen titles shortened; Dialog/Attachments Add buttons moved above their boxes (§6.26); required fields get an Ink border

Third round of Android-width fixes today, from an owner mockup pasted in against a live screenshot of Create Request. Two things were wrapping that hadn’t before: the “Would You Please” wordmark itself (not just the tagline, which was already expected to wrap) and the “Create a Request” screen title.

Wordmark shrunk 27px → 23px, tagline pulled up 3px → 1px margin-top. `.word/.tag` in `globals.css` plus all eleven live mockups (not the superseded draft). 23px was picked to match `.glabel` — the wordmark and a screen title now read at the same scale, which also reads as more consistent than an arbitrary in-between size. Applies everywhere at once since every live screen already reads these two classes through `WypHeader.tsx`; only the mockups needed a direct edit.

Four screen titles shortened, continuing the article-dropping convention already established 2026-07-22 (“Create Request,” not “Create a Request,” is the name; the shorter form applies where the phrase *names the screen*):

- “Create a Request” → “Create Request” — mockup and the live `CreateRequestForm.tsx` band label. (This was already the documented decision from 2026-07-22; the band label had just never actually been updated to match — a gap, not a new call.)
- “Create a ToDo” → “Create ToDo” — mockup only (ToDo isn’t converted yet).
- “Create my Free Account” → “Create Free Account” — mockup, including the quoted screen-name references on the Sign In mockup that point to it.
- “Start my Free Account” → “Start Free Account” — Sign In mockup’s band label, its JS mode-toggle, and the demo-switcher button label.

“Respond to Request” stays as-is — owner confirmed it would still wrap even as “Request Response,” so a rename buys nothing there; not touched.

Add Dialog / Add Attachment moved off to their own row, above a full-width box — new §6.26, `.fieldact` (Create Request/Create ToDo) and `.panelact+.panelfull` (Respond to Request). The previous side-by-side

layout (.frow.top, .attachrow, .panelrow) gave the Add button a fixed width and handed the Dialog textarea or the Attachments file list whatever was left, which starves a full-paragraph field on a narrow phone the same way two name fields did last round. Stacking costs one line of vertical space and returns most of the row’s width to the box that actually needs it. Recipient and Category keep their existing .frow — a single-line input doesn’t have this problem, so only Dialog and Attachments moved. Respond to Request has no .form/.fgroup wrapper the way the two Create screens do, so its version of the pattern (.panelact for the button, .panelfull for the panel) carries its own padding:0 var(--pad) instead of inheriting it; functionally the same idea, named for what it actually wraps.

Required fields get an Ink-colored border — .req, bundled into §6.26. Row Tint on optional fields (2026-08-07, earlier entry) didn’t make required fields “pop” the way intended, because a *filled* optional field is also white under that rule — required and optional-but-filled were visually identical. .req sets the resting border to var(--ink) instead of Input Border; still focuses to Brand Blue like every field. **Used the existing Ink token rather than literal black** — the system has no pure-black token, --alert-red is reserved for Overdue/error/impending-loss and wasn’t a fit, and #1F2933 reads as black at 1px, so this needed no new hex and no §3.1 change. Flagging the substitution here in case a literal #000 was actually wanted. .fgroup.is-invalid .finput{border-color:var(--alert-red)} still wins over .req on validation failure — its selector is more specific ((0,3,0) vs .finput.req’s (0,2,0)), so no conflict. Applied to Create Request’s Recipient, Due Date, and Description (the three fields validate() actually requires) and to Create ToDo’s Description (its only required field — Priority is chip-based, not bordered, and Due Date is intentionally absent). Not applied to Add Contact, which hasn’t adopted the Row Tint/.opt scheme at all yet (2026-08-07 note: “not applied to Add Contact’s Notes... left alone rather than changed speculatively”) — extending .req there without .opt first would create a required/optional distinction on a screen that doesn’t otherwise draw one, so left out rather than expanding scope unprompted. Not applicable to Respond to Request, which has no bordered required inputs.

2026-08-07 — Fourth round: Dialog label bug fixed, Phone gets Row Tint on three more screens, Storage Maintenance’s Close button corrected to Primary

Owner review of the §6.26 restructure (previous entry) turned up a real bug plus three smaller consistency fixes.

Bug — Dialog’s floating label rendered below the box, not overlaid. Root cause: .ffloat on Dialog was a , and a is inline by default. It worked fine before §6.26 because Dialog’s .ffloat was a flex child of .frow.top — flex children are blockified regardless of their own display value, which is what made the absolutely-positioned label sit correctly over the textarea. Moving Add Dialog into its own .fieldact row took the .ffloat out of any flex container, so the span reverted to genuinely inline, and the label’s containing block came out wrong. Every other .ffloat on Create Request still lives inside a .frow (Recipient, Due Date/Time, Category), so none of them showed this. Fixed by changing Dialog’s wrapper from to <div class="ffloat">, matching how Request Description (never in a .frow) was already written — CreateRequestForm.tsx, and the Create Request and Create ToDo mockups.

Dialog’s label collapsed back to one line, same font as “Dialog.” The two-line .subnote treatment existed to fit “(Questions, Answers, Comments)” in a box that used to share its row with the Add Dialog button. Now that the box is full-width, “Dialog (Questions, Answers, Comments)” fits on one line at the ordinary label size — owner confirmed by comparing it to the wordmark’s own width, now 23px, which is roughly the same. Reverted .ftextarea-dialog’s extra padding-top:36px to the base value (no longer needs clearance for a second line) and removed .flabel.twoline, which nothing else in the app used.

Phone gets Row Tint while it isn’t the send-by channel — Create Free Account, Your Account, and (retroactively, for consistency) Add Contact. Create Free Account and Your Account hadn’t adopted .opt at all yet; added the rule and applied it to their one optional field, Phone. **Add Contact’s case is state-driven, not static:** Phone is genuinely skippable today because Email is the only send_by that works (Text stays .is-locked), so AddContactForm.tsx now reads sendBy === 'email' ? 'opt' : '' rather than a hardcoded class — the day Text unlocks and a user actually selects it, Phone becomes the delivery channel and should read as required-in-practice (white, Input Border), which the existing §6.25 empty/filled logic already expresses without a new rule. Both Add Contact mockups

(`_floating` and the no-contact-dialog variant) show the current single legal state — Email selected, Phone Row-Tinted — same as Respond to Request’s tier demo shows only what today’s real states can be, not a speculative one.

Storage Maintenance’s Close button corrected from Secondary to Primary. It’s the only button on the band. `.btn-secondary` means “available, lower emphasis *than a Primary also present*” — Add Category’s Cancel next to Save is the paradigm case. With no Primary sibling to be lower emphasis than, the blue-outline Secondary treatment was simply wrong, not a deliberate de-emphasis; changed to `.btn`.

2026-08-07 — Fifth round: rule-consistency sweep — country-code selector Row-Tinted, required-field Ink borders filled in on four more screens, Done Date/Time on Respond to Request, Sign In’s Email; Main Screen’s search bar kept exempt on purpose

Owner audit against the §6.25 rule (Row Tint = optional/inactive, Ink border = required) turned up places it had been applied halfway — Phone got Row Tint in the last round but the *required* fields on the same screens never got their Ink border, an omission rather than a decision.

Country-code selector (`.ccode`) goes Row Tint. It’s a fixed, non-required value (one legal choice, US +1, no picker wired yet) sitting right next to Phone, which already reads as optional — leaving `.ccode` white made it look like the more important of the two. `.opt/.opt:focus` extended to include `.ccode` in `globals.css` and all four mockups that have one (Add Contact, its no-contact-dialog variant, Create Free Account, Your Account).

Required fields on those same four screens get the Ink border they were missing: Add Contact’s Name/Email/Time Zone (mockup and, where live, `AddContactForm.tsx`); Create Free Account’s First/Last/Display Name and Time Zone; Your Account’s First/Last/Display Name. **Not applied to any readonly Email field** (Add Contact has none; Create Free Account’s and Your Account’s sign-in-ID Email is `.fininput[readonly]`) — that field already carries its own locked/dashed treatment, a stronger and more specific signal than a border color, and layering `.req` on top would muddy two different meanings (“required” vs. “not editable here”).

Respond to Request’s Done Date/Done Time get the Ink border in the state the mockup shows. These aren’t `.fininputs` — they’re read-only `.panel/.fieldval` boxes — so a new `.panel.req{border-color:var(--ink)}` was added alongside the existing `.panel{border-color:var(--rule)}` rather than reusing `.fininput.req`. This is exactly the “conditionally required” case flagged as a future example in the first Row Tint entry (2026-08-07: “a Done Time if the Done Date is the same as the Due Date and a Due Time was part of the Request”) — the mockup’s demo data happens to be in precisely that state (Due: Monday, October 19, 2026, 9:00 AM; Done Date: the same Monday), so both panels are `.req` as shown. Flagged in a comment that once Done Date/Done Time become live pickers, `.req` needs to be conditional on that date/time comparison, not permanent.

Sign In’s Email field gets the Ink border. It’s the only field on the screen; `.fininput.req` added alongside the existing `.fgroup.is-invalid` rule (which still wins on validation failure — its selector is more specific, same as everywhere else this pattern is used).

Main Screen’s search bar (Search Type, Search text) stays white-on-Band — deliberate exception, not left half-finished. Owner’s own instinct, asked for a second opinion before locking it in. Agreed, for three reasons: (1) the whole point of the Row Tint / Ink Border pair is to let a required field visually win against optional siblings *within one form a person is completing* — the search bar has no Send/Save it’s gating, no required sibling to contrast against, and isn’t a form at all, so the rule has nothing to apply itself to. (2) Row Tint (`#F6F7F9`) sits close in lightness to the search bar’s own Band background (`#E7E7E7`) — tinting the fields would blur their edges against the bar they sit in, weakening the very affordance (“this is a tappable/typeable control”) that white currently provides, for a signal (“optional”) nobody’s asking. (3) The search bar already reads as its own component, not a form field in the §6.10 sense — precedent exists for controls sitting outside the required/optional binary entirely (`.is-locked`, `.btn-quiet`). Recorded as a scoped exception in the §6.25 entry in `design/README.md` rather than left implicit, so a future pass doesn’t “fix” it by mistake.

2026-08-07 — Sixth round, small: Add Contact’s Notes field was missed in the \$6.25 sweep

Owner caught one the previous round’s audit didn’t: Notes (optional) on Add Contact was still plain white — the decisions log’s own note from two entries back (“not applied to Add Contact’s Notes... left alone rather than changed speculatively”) turned out to be the gap, not a considered exception. Given `.opt` is now the established rule for every other optional field on this screen (Phone, country code), there was no remaining reason to leave Notes out. Added `.opt` to the textarea in `AddContactForm.tsx`, both Add Contact mockups, and extended each mockup’s local `.opt` rule to cover `.ftextarea` (it previously only listed `.finput/.ccode` — `globals.css` already covered `.ftextarea.opt` from Create Request, so only the mockups needed the selector added).

2026-08-07 — Seventh round: Add Dialog modal, dynamic Answer locking, which-Question picker, migration 006

Owner pasted a rough mockup of an “Add Dialog” modal (title + Cancel/Save on one row, a Question/Answer/Comment chip row, a Dialog Text box) meant for Create Request, Create ToDo, and Respond to Request, and later Request Detail. Resolved over several messages into the following.

Kind availability is per-screen, not one fixed rule. Asked the owner directly (two-part `AskUserQuestion`) rather than guess: on Create Request/Create ToDo, all three chips were first said to be “unlocked” — refined immediately after to Answer being *always* locked there specifically, not just unlocked-with-nothing-to-answer, since “the ‘answer’ could not be the 1st entry, but either a question or a comment could be” and a Request/ToDo’s thread is always empty at creation. On Respond to Request (and later Request Detail), Answer is dynamic: locked unless at least one Question in the thread is still open.

“Open” needed a real definition once more than one Question could be in play. A single flag (“has the most recent Question been answered”) only works for one open Question at a time; with two or three open at once it can’t say which is which. Added `dialog.replies_to_id` (migration 006, SQL in `docs/Week2 - SQL history.txt`, not yet run) — nullable, CHECK-constrained to answer-kind rows only, on delete set null. A Question now counts as open exactly when no Answer’s `replies_to_id` points at it.

Which-Question picker, scoped exactly as asked: “it does not make using the app harder for an end-user — however it only needs to be presented if there is more than one question in the dialog. And, if there is more than one question, the last question should default to the selection.” Implemented literally — one open Question links silently (no picker, no extra tap); two or more shows `.qpicker` (rows styled with `.lookup-item`, new `.selected` variant since this list stays visible after a pick rather than closing like a lookup dropdown), defaulting to the most recently created open Question. Read “more than one question” as “more than one *open* question” — the only reading that stays consistent with Answer’s own locking rule, since a fully-answered Question is never a candidate for a new Answer to point at in the first place; flagged here in case a different reading was meant.

Display order does not change. Confirmed with the owner before building anything (“That strategy would keep the ‘order of entry’ true...” — an Answer still renders wherever it was actually posted in the newest-first list (2026-08-05’s sort rule, untouched), not moved next to the Question it answers. The link is informational, not structural.

Answer entries get a “Re:” quote and a bolded body, per the owner’s own suggestion: “smaller font repetition of the question, as a ‘Re: ...’ with a different colored/bolded or sized answer text.” Built as `.dlgre` (11px italic Ink Soft, single-line ellipsis — matching `.attach`’s filename truncation, not the 2-line/3-char Description rule, since this is a compact inline quote) followed by `.dlgbody` (bold). **Bold chosen over color:** Brand Blue already carries dates, links, and (via `.dlgkind`’s weight) kind labels; Alert Red is status-only from v2.5 (§3.1). Adding a fourth meaning to an existing color risked diluting one of those; bold reuses emphasis language the system already has. Flagging the alternative in case color was actually wanted.

Modal gained a second header shape. `.modalhead` — title and Cancel/Save share the top row — sits alongside Add Category’s existing title-then-bottom-`.modalacts` layout as a second valid §6.12 variant, not a replacement; used

because that’s what the owner’s own Add Dialog mockup draws. Chosen per modal going forward rather than forcing one shape everywhere.

Create Request’s Dialog field changed shape, not just gained a modal: the old always-visible inline textarea (Add Dialog appended its text directly, 2026-08-06) is gone. Add Dialog now opens the modal, matching how Add Category already works; a saved entry appears in the `.dlgstaged` list below, now labeled with its Kind (`<b>Question:</b>` ...). `CreateRequestForm.tsx`’s `dialogEntries` changed from `string[]` to `{kind, body}[]`, and the Send handler now writes `kind` explicitly on each insert instead of relying on the table’s default `'comment'`. Create ToDo’s mockup got the identical change for consistency, even though it isn’t live yet.

Addendum, same day: the chip row’s label reads “Dialog Entry Type,” not “Kind” — owner’s correction after reviewing the built modal. Visible label only; the underlying `kind` column/variable name is unchanged (`dialogModalKind` in `CreateRequestForm.tsx`, `dlgKind` in the mockups’ JS, `dialog.kind` in the schema) since none of those are user-facing.

Respond to Request’s demo data was rewritten to actually exercise the feature — two open Questions by default (“Should I also include George?”, already answered; “Do you have enough time next week to accomplish this request?” — the owner’s own example wording, kept verbatim; “Should the meeting be in-person or a video call?”) — so a reviewer opening the mockup sees the which-Question picker’s multi-question branch immediately rather than only after clicking through several Adds. The whole flow (chip locking, picker, staged Answer, `.dlgre/.dlgbody`) is real vanilla-JS state in this mockup, not static markup, since a picker that only exists in a comment isn’t demonstrable.

2026-08-08 — Detail screens (Request Detail, Response Detail, Dialog Detail); Main Screen Tasks/How-to Videos chips; “Respond to Request” rename flagged, not applied

Owner’s presumption: clicking an entry to edit its detail opens screens shaped like the ones already built — Add Dialog → **Dialog Detail**, Create Request (Sent) → **Request Detail**, Respond to Request (Received) → **Response Detail**. Built all three as new mockups.

Request Detail (`screens/WYP_request_detail_palette1.html`) is Create Request’s mockup with the Recipient field switched to read-only (\$6.10 locked variant, ported from Your Account’s Email — dashed border, `--locked` fill, a `.lockrow` note explaining why, same pattern, not layered with `.req` since the locked treatment is already the stronger, more specific signal) and every other field pre-filled with demo data instead of blank. New notice band directly under the title band, Strip background — the owner’s own comparison (“like a chips background”): **“Note: The Request Recipient is notified of changes.”** Action buttons stay Send/Cancel, unchanged, per the owner’s explicit instruction — Send still means “commit this change (and notify),” not merely “save.” One thing worked out rather than asked: Request Detail’s Add Dialog uses the *dynamic* Kind-locking (open-Question rule, migration 006) from Respond to Request, not Create Request’s permanent Answer-lock — a Request already sent can already have an open Question waiting on the issuer, so Answer has to be reachable here, unlike at creation time when the thread is provably empty.

Response Detail (`screens/WYP_response_detail_palette1.html`) is Respond to Request’s mockup, retitled, for a signed-in user reached from inside the app rather than the anonymous `/r/[token]` link. Same fields, same dynamic Add Dialog. Dropped the “Create your own Free Account” `.promo` block — whoever’s looking at this screen already has an account, so the pitch has no one to persuade. Kept the subscription-upsell `.subbanner`, which isn’t specific to the anonymous flow. Shares its demo Dialog thread verbatim with Request Detail (same four entries, same ids) so the one demo Request reads identically from both the sender’s and recipient’s side.

Dialog Detail (`screens/WYP_dialog_detail_palette1.html`) is deliberately **read-only**, not just unfinished. The owner asked for something shaped like Add Dialog; the `dialog` table has no UPDATE or DELETE policy — append-only by design since migration 004, specifically so a past Question/Answer/Comment can’t be quietly rewritten. Editing an existing entry from a “Dialog Detail” screen isn’t something the schema supports today, so this was built as a viewer: the same Dialog Entry Type chip row and Dialog Text box as Add Dialog, but chips are plain `<span>s` (not `<button>s` — nothing here is clickable, and a disabled-looking button still invites a tap) with only the entry’s actual kind marked,

and Dialog Text renders in the same locked/dashed treatment as a readonly field. Single Close button, Primary (same “only button on the band” call as Storage Maintenance, 2026-08-07). **Flagging this rather than silently deciding it:** if Dialog entries should become editable, that’s a schema decision (an UPDATE policy — probably versioned/audited rather than a plain overwrite, given the whole point of append-only was an honest thread) that needs to be made on purpose, not backed into by building a form.

Main Screen: Housekeeping’s “Tasks” label became a two-chip toggle — “Tasks” (default-selected) and “How-to Videos” — reusing the screen’s own existing `.chips/.chip/.chip.sel` pattern already used for the Todos filter row (this screen predates the app-wide `.chip.selected` naming and wasn’t otherwise being touched, so `.sel` was kept rather than reconciled). Tasks’ content (My Contacts, Account Profile) is unchanged. How-to Videos is new and currently placeholder titles only (“Getting Started,” “Creating a Request,” “Responding to a Request”) with no real video content or links — out of scope for this pass, flagged in the row markup and here so it isn’t mistaken for finished content later.

Flagged, not applied: renaming “Respond to Request” to “Request Response.” The owner asked for this “for consistency with the new [Detail] screen names.” This is the exact rename tested and rejected on 2026-08-07: “*‘Respond to Request’ is unchanged — confirmed it would still wrap either way*” (the owner’s own on-device finding, Android, current 23px wordmark). Nothing about title width has changed since. Rather than silently reintroduce a known wrap or silently ignore the request, left “Respond to Request” as titled and surfaced the conflict for a decision — accept the wrap, explore the wordmark/title sizing further, or leave it as-is for naming-consistency’s sake despite the mismatch with the three new Detail-screen names.

2026-08-08 — Second round: **ToDo Detail; non-modifiable-as-text rule (§6.28) across four screens; Done Date/Time on Request Detail; Display Name wording; Change Email button; “Request Response” rename applied**

Same day as the first Detail-screens round, after the owner reviewed the results and uploaded reference screenshots.

“Request Response” — applied. The owner’s reason this time was explicit and unrelated to the earlier wrap finding: consistency with Request Detail / Response Detail / Dialog Detail’s naming, stated directly as “*unrelated to word-wrapping*.” That’s a different justification than the one already tested and rejected on 2026-08-07, so this isn’t a silent reversal — it’s a new decision on new grounds, superseding the “flagged, not applied” entry above. Band label and `<title>` changed in `WYP_respond_to_request_palette1.html`; filename unchanged, same precedent as “Create a Request” → “Create Request.” All README/decisions-log prose references updated to match.

Non-modifiable values render as text with a field title, never inside a boxed field — new rule, §6.28. Owner’s instruction, referencing Request Response’s own Date:/From:/Due: meta block as the existing proof this already worked: “*consistently show non-modifiable values as text with data field titles... as is now done for Respond to Request*.” This retires the `.finput[readonly]` + dashed-border-and--locked-fill variant introduced for Your Account’s Email a few sessions back and copied to Create Free Account and (a day earlier) Request Detail’s Recipient — all three now read as `.metarow/.mlabel/.mval`, reusing the exact component already sitting on Request Response rather than inventing a label:value component under a new name. The icon that used to justify Your Account’s boxed-and-locked Email (a small padlock SVG next to the explanatory sentence) is dropped along with the box — an icon earns its place explaining *why a field looks disabled*; plain text next to plain text needs no such explanation. Request Detail’s Recipient also drops its own explanatory `.lockrow` note (“The Recipient of a sent Request can’t be changed”) as redundant once there’s no box to explain and the notice band above already says why an edit here matters.

Dialog Detail rebuilt same day, not just re-skinned: the owner reviewed the chip-row version (built read-only that morning, `<span>s` not `<button>s`) and called it “*very readable as originally designed*” but a break from the new text-only rule, since a chip row is still a field-shaped control regardless of what it’s built from. Replaced with the value doubling as its own label — “Answer:” in bold Brand Blue — over a horizontal rule. The owner’s own reference sketch used an underline under the word and offered the rule as an alternative explicitly: “*the underline... would probably be a horizontal rule*.” Built as a full-width `.dlghr` line in the neutral Rule color, not Brand Blue — `.dlgtype` already

carries the emphasis, and a second blue element felt like the rule repeating the label’s job. Date/From converted to the same .metarow pattern used everywhere else in this round. Dialog Text’s own label was dropped entirely — once the value sits directly under a rule with nothing beside it, a second “Dialog Text” caption above the actual text was reporting the obvious.

Request Detail gains an editable Done Date/Done Time row. Owner’s reasoning: *“the requestor may want to update the request as completed.”* Given the same box treatment as Due Date/Due Time immediately above it — both are genuinely settable here, unlike Recipient, so neither belongs in the new text-only pattern. Row-Tinted while empty like any other optional field; both labels carry “(optional)” for consistency with the rest of the screen’s convention, even though the owner’s own reference sketch showed “Done Date” without the suffix — a small, flagged deviation rather than a silent one.

ToDo Detail built as a byte-for-byte duplicate of Create ToDo, retitled. Owner’s instruction was explicit: *“the only difference with the Create ToDo screen is the screen title.”* Kept as its own file rather than any kind of shared template, matching how every other screen pair in this app (Create Request / Request Detail, Request Response / Response Detail) is two self-contained files, not one parameterized one.

Wording fixes, applied verbatim from the owner’s message: “Recipients see this name on every request you send.” → “Recipients see your Display Name on every request you send.” (Create Free Account and Your Account, both — the caption under Display Name). Email’s explanatory note also changed, on both screens, from the older two-sentence version (“This is where your sign-in links are sent. Changing it requires confirming from both addresses.”) to the shorter “Your email address is also your sign-in ID.” — matched to the owner’s own uploaded reference images, which showed this exact wording on both screens; not separately called out in the numbered list, so flagging the match here in case it wasn’t an intended change.

Your Account gains a “Change my email address” button, .btn-quiet, next to “Sign out on this device” — same low-emphasis treatment, same row. Not wired to anything: the owner was explicit that the actual change-email flow (“the related dialog/screen with appropriate explanations”) is still to be designed. The detail that used to live in Email’s old two-sentence note (confirming from both addresses) is exactly the kind of explanation that flow will need to carry once it exists — noted here so it isn’t lost now that the sentence itself is gone.

Main Screen: Housekeeping’s “Account Profile” task row renamed to “Your Account” — matches the screen it now points at.

2026-08-08 — Third round: Main Screen converted to React, the first Live landing screen

Owner’s request: *“I think we are now ready to begin the REACT addition to the main screen... I would like to see the WYP app retain the device-login validation and be able to test it in a more normal way than needing to each time fill-in the URL after starting up and logging in each time.”* app/page.tsx had been rendering a placeholder (“Logged in [done]”) since the auth work was built — everything downstream of sign-in (magic link, /auth/callback, RequireAuth, “Keep me signed in”) already worked, but there was nowhere real to land, so testing the login loop meant hand-typing a route every time.

Received surfaced a real architectural gap, not a build task. requests RLS (migration 003) is owner-only — owner_id = auth.uid() — and no column links a row to its recipient’s own account, so there is currently no query a signed-in recipient could run to see “Requests sent to me.” Raised this directly rather than faking it or silently building around it. Resolved by scoping this pass to Sent and ToDos only; Received renders a plain explanatory .subempty note (new, §6.29) instead of either fabricated demo rows or an empty space that would read as a loading bug. The actual fix — some combination of a recipient-side query path or a SECURITY DEFINER function, matching the pattern already used for the anonymous /r/[token] link — stays an open, unscheduled decision.

Scope for this pass, settled over two rounds of questions rather than assumed: - Asked whether demo data would be seeded or the screen would start empty; the owner’s answer doubled as the deciding vote for the next question — *“If there will be Requests and ToDos, then the sort pills would be helpful”* — so Sent/ToDos needed to be genuinely live,

not just visually converted. - Asked whether the search bar and the All/Open/Overdue/Done (Todos: All/Open/Done) filter chips should also go live now. Owner picked **“Stay visual-only for now”** — they render but do nothing this pass.

Sort pills are real, not decorative. Sent’s Due v pill reflects an actual `order('due_date', { ascending: false })` query; Todos’ Priority v reflects `order('priority', { ascending: true })`. No click-to-resort was requested or built — the owner asked only that the pills’ claim match the data, which the default query order now does.

Three controls are inert on purpose, not by oversight, because they have nowhere to go yet: the Todos band’s Create ToDo button (`/todos/new` doesn’t exist — Create ToDo is still Mockup-only per design/README.md), and Housekeeping’s My Contacts / Your Account rows (no contacts list view and no `/account` route exist). Left visually normal rather than `.is-locked` — that treatment specifically means “unavailable because of the request’s tier” (§6.22), which isn’t the reason here, and using it would misstate why the control does nothing.

Log Out was built live, not left inert like the rest of Housekeeping. It’s the one control that directly serves the reason this screen exists: without a working sign-out, there’s no way to re-run the login loop the owner asked to be able to test “in a more normal way.” `supabase.auth.signOut()` followed by a redirect to `/login`.

Icons converted to inline SVG, matching every other screen — the mockup’s base64 PNG data-URIs were never going to carry over. Shapes came from the canonical `wyp_icon_*.svg` source (Dialog, Expand, Search, Voice Search — recolored to `currentColor` so `.iconbtn.ii`’s existing hover/rest colors drive them, rather than the asset source’s hardcoded brand-blue/grey), except Print, which reuses the exact icon already built for Create Request / Request Response rather than the asset source’s own printer glyph — one printer icon in the app, not two slightly different ones. Attachment’s info icon was not built: Attachments stays deferred app-wide (Scope discipline in CLAUDE.md), so there is nothing yet for that icon to indicate.

Seed script, not fixture data baked into the component. Wrote to `docs/Week2 - SQL history.txt` (Contacts, Sent Requests, Todos, under `jimgillon@gmail.com` specifically, looked up by email inside the script) for the owner to run himself, matching how every schema migration in this project has been handled — nothing runs against the live database except by his own hand. Not a numbered migration, since nothing in it alters a table. Guarded to be safely re-runnable (existence-checked before each insert, since `contacts/requests` have no unique constraint to lean on the way `categories` does) and dated relative to `CURRENT_DATE` rather than hardcoded, so the Open/Overdue/Done mix stays meaningful whenever it’s actually run.

Alternative considered and rejected: snapshotting a fake “Received” section from the same seed data, styled as if it were live, to make the screen look more finished. Rejected — it would have misrepresented a genuine schema gap as a solved one, and the first time a real second account tried to use it, it would silently show nothing.

2026-08-09 — Fourth round: Create ToDo, Request/ToDo Detail, live filter/search, Contact Detail, My Contacts

Owner’s ask, in order: finish the three items proposed at the end of the previous round (Create ToDo live, Sent/ToDo rows wired to their Detail screens, Main Screen’s filter chips and search made functional), plus a new ask surfaced in the same message — a Contact Detail screen (“exactly the same as Create Contact except for the screen title”), a redesigned My Contacts list (“should show the name, notify method and related value... upon click open up a Contact Details screen for editing”), and a Housekeeping wording fix: “*‘Your Account’ — ‘Account’ seems a bit impersonal for this app. So, I think we should change ‘Your Account - view and edit’ to ‘My Account - view and edit’.*” The related “My Account” screen itself was explicitly deferred: “*should await further product evolution before being designed.*”

ToDo Detail’s Done Date/Time — asked, not assumed. Wiring ToDo Detail live as originally specified (a byte-for-byte duplicate of Create ToDo, no Done fields) would have meant a real screen with no way to mark a ToDo complete through the UI at all — the same gap Request Detail’s Done Date/Time row was added to close a day earlier. Rather than silently repeat that fix or silently leave the gap, asked directly via `AskUserQuestion`; owner picked adding the row. Applied to both the mockup and `ToDoDetailForm.tsx`.

ToDo Detail’s Dialog panel changed to the existing-thread pattern without a separate ask. Once Done Date/Time was settled, converting the Dialog section from Create ToDo’s blank staged list to Request Detail’s real-thread-plus-dynamic-Answer pattern followed from the same underlying reasoning already accepted for Request Detail (a Detail screen views something that already exists, so its Dialog should too) — not something a reasonable reading of “add Done Date/Time” would leave out, but flagged clearly in both the mockup’s header comment and this entry rather than folded in silently.

Request Detail and ToDo Detail’s Dialog panel: write-through, not staged. Unlike Create Request/ToDo, where a Dialog entry can’t be written until the parent row has an id, a Detail screen’s Request or ToDo already exists — so Add Dialog’s Save writes straight to the `dialog` table and the panel re-fetches, rather than waiting for the screen’s own Send/Save. Chosen so a Dialog entry survives even if the user backs out of the rest of the edit with Cancel; the alternative (staging Dialog alongside the Detail form fields) would have made a real, already-visible conversation entry vanish on Cancel, which reads as data loss, not a discard.

Create ToDo’s missing Due Date — flagged, not fixed. While building this out live, confirmed the Create ToDo mockup has never had a Due Date field, despite the PRD’s core-objects table listing ToDos as having an optional due date and `requests.due_date` being a real, already-populated-by-seed-data nullable column. Left it out of `CreateToDoForm.tsx` to match the approved mockup — inventing a field the mockup doesn’t draw would be the same mistake in the other direction — and surfaced the gap in `design/README.md` and `CLAUDE.md` instead of picking a side unasked.

Contact Detail: Save + Close, not Save + Cancel. The owner’s instruction was “exactly the same as Create Contact except for the screen title,” but Cancel’s actual meaning throughout this app is “discard what I was about to create” — Storage Maintenance and Dialog Detail already use Close for “leave an existing/read screen,” and Contact Detail is editing something that already exists. Applied Close instead of copying Cancel verbatim, since a literal reading here would have reused a button whose established meaning doesn’t fit; flagged as an interpretive call rather than assumed to be exactly what “the same except the title” meant.

Contact Detail surfaced a pre-existing, unrelated gap: Add Contact’s mockup has drawn a Time Zone field since 2026-08-05, but it was never wired into `AddContactForm.tsx` (no `contacts.time_zone` column exists in any migration) — a gap from a prior session, not this one. Rather than propagate it silently to a second mockup, `ContactDetailForm.tsx` mirrors `AddContactForm.tsx`’s actual live fields (Name/Email/Phone/Notes) and the gap is now flagged explicitly in both docs files, since it’s more visible with two mockups showing a field that does nothing than one.

My Contacts reuses the Housekeeping row component (§6.23) rather than inventing a new list style. The owner’s spec — name, notify method, the matching email-or-phone value, tap to open Detail — is structurally identical to what `.hkrow/.hktitle/.hknote` already does for Housekeeping’s own navigational rows. Built as `<b>Name</b> - Email: address (or Text: phone)`, sorted alphabetically by `display_name` like every other lookup in the app.

Filter chips and search: client-side over already-fetched rows, no new queries. Same reasoning as the Recipient/Category lookups elsewhere — both lists are personal-scale, so filtering and searching in memory avoids a round trip per click or keystroke. ToDos’ filter defaults to Open selected (not All), matching the mockup’s own resting state, which had been noted but not re-verified when the chips were first built inert.

Scope button (“All v”) stays visual-only. It has never had a designed picker (see the 2026-08-07 “Main Screen’s search bar kept exempt on purpose” entry), and since search now runs across both Sent and ToDos at once, there’s nothing yet for a scope to actually narrow — making it interactive would mean designing a feature nobody asked for.

“My Account” — wording only, screen still undesigned. Owner’s reasoning, verbatim: “*‘Account’ seems a bit impersonal for this app.*” Applied to the Main Screen mockup and `MainScreen.tsx`; the row stays inert, since the owner explicitly said the Account screen itself should wait for further product evolution rather than be designed now on the back of a label change.

2026-08-09 — Fifth round: Contacts list Close button + rename, Add Contact’s return destination, scroll-position restoration on Detail screens

Close button on the Contacts list, plus a same-day rename. Owner asked for a Close (secondary) button next to Add Contact, then reasoned through the consequence himself before I could: *“That will probably cause an undesired line break on my Android, so the screen title would need to drop the ‘My’ to be sure the line break does not occur.”* Applied — title band now reads “Contacts”, not “My Contacts”. He then generalized the reasoning into a standing rule: *“I like to have a navigation always repeat exactly what the choice was named when selected.”* Since the destination screen is now titled “Contacts,” the Housekeeping row that opens it has to say “Contacts” too, not “My Contacts” — applied to WYP_main_screen_palette1.html and MainScreen.tsx. **And, for consistency, “My Account” → “Account”** — the owner’s own words: *“from a consistency perspective, the Task ‘My Account - view and edit’ will also have to be changed to ‘Account - view and edit.’ ”* This reverses the “Your Account” → “My Account” rename from earlier the same day, on entirely different grounds (nav-label-matches-destination-title, not “impersonal wording”) — not a silent flip-flop, a second decision superseding the first for a different reason, both recorded here rather than only the latest one kept.

Add Contact’s post-Save destination. Owner: *“When a contact is added from the Contacts list and saved, the process now returns to the main screen - it seems it should return to the Contacts list (as is done for an edit of a Contact from that list).”* Changed both Save and Cancel to return to the Contacts list. Reasoned past the literal ask to also fix Cancel, not just Save: the only current entry point to /contacts/new is the Contacts list’s own Add Contact button, so Save landing on the list while Cancel landed on the Main Screen would have been a new inconsistency of its own. Flagged in code that this assumption breaks if a second entry point (Create Request’s not-yet-built no-contact interception) starts reaching Add Contact — that path will need its own return address.

Scroll position after editing a Request or ToDo — a genuinely new question, not implicit in anything asked before.

Owner: *“When I edited a ToDo I was returned to the top of the main screen instead of to where I have edited the ToDo.”* Investigated two paths before choosing:

1. **Next 16’s Cache Components / <Activity> feature** (node_modules/next/dist/docs/01-app/02-guides/preserving-u — opt-in via cacheComponents: true, keeps a route’s whole component tree mounted (hidden, not unmounted) across navigation, preserving both DOM and React state automatically. Rejected: it’s an experimental, app-wide config flip with a broad blast radius for a single-screen complaint, and it would have made things worse here — a hidden, still-mounted Main Screen wouldn’t re-run its data-fetching Effect on return, so the just-edited row would show its pre-edit values until some other refresh happened.
2. **router.back() instead of router.push('/...')** on Request Detail, ToDo Detail, and (by the same reasoning, applied without a separate ask) Contact Detail. Every one of these screens is reached exactly one way — clicking a row on its parent list — so back() always returns to the correct history entry. Next’s browser-history-based scroll restoration is standard, always-on App Router behavior, unrelated to Cache Components, and the parent screen still fully remounts and refetches on the way back (no Cache Components enabled), so the edited data shows correctly rather than stale. Chosen over option 1 for being smaller, standard, and actually more correct for this specific “did the data change” scenario.

Trade-off flagged, not silently absorbed: router.back() restores scroll position but not Main Screen’s filter-chip/search useState, since the component does remount (just not by reloading the URL from scratch) — those reset to their defaults each time. Not raised as a problem to solve now, since the owner’s complaint was specifically about scroll position, but noted in CLAUDE.md in case it matters later (fix would be lifting that state into the URL as search params, which Cache Components’ Activity model would also have handled automatically had rejecting it not cost the data-freshness property above).

2026-08-09 — Sixth round: Main Screen chip state persists across the round trip

Owner, following directly from the scroll-position fix above: *“It would be appropriate to return to the same chip state on the main screen.”* This was the trade-off already flagged (not yet solved) in the Fifth round entry below —

`router.back()` restores scroll position, but Main Screen still remounts, so its filter-chip `useState` was resetting to defaults on every trip to a Detail screen and back.

sessionStorage over the two alternatives considered. Lifting the state into the URL as search params (the fix floated when the trade-off was first flagged) would have worked but meant every chip click rewriting the URL/history — more moving parts than the ask required. Cache Components/Activity was re-considered and re-rejected for the same reason as the Fifth round entry: it would preserve chip state for free, but at the cost of Main Screen no longer re-fetching on return, which would show stale post-edit data — a worse bug than the one being fixed. Landed on `sessionStorage`, matching the existing precedent in `supabaseClient.ts` (`REMEMBER_KEY/hybridStorage`, which already routes the Supabase session between `sessionStorage` and `localStorage` depending on “Keep me signed in”). Used `sessionStorage` specifically, not `localStorage`: this is a within-session view preference, not a durable account setting, so resetting when the tab actually closes is correct behavior, not a compromise.

Scope: the three chips, not the search box. Persisted the Sent filter, Todos filter, and Housekeeping’s Tasks/How-to Videos tab — matching the owner’s own wording, “chip state.” The search text field is a separate control and was left unpersisted. This is a scoping call on my part, not something the owner confirmed either way — flagged in the code comment and in `CLAUDE.md` in case it turns out search should persist too.

Implementation shape. Three `sessionStorage` keys (`wyp.mainSentFilter`, `wyp.mainTodoFilter`, `wyp.mainHkTab`), read via a small `readStoredChip` helper used as each `useState`’s lazy initializer (guards `typeof window === 'undefined'` and falls back to the mockup’s resting default — All-open-Todos, All Sent, Tasks tab — for any missing or invalid stored value), and written back with a `useEffect` per chip whenever it changes.

2026-08-09 — Migration 006 confirmed run; Week 3 plan scoped and two design questions settled

Migration 006 (`dialog.replies_to_id`) **confirmed run by the owner.** This closes out the one unverified risk flagged repeatedly across Request Detail, Todo Detail, and `CLAUDE.md`’s Known gaps — both screens’ Dialog panels were a hard, unconfirmed dependency on it. Language updated in `CLAUDE.md` and `design/README.md` from “hard dependency, not yet run” to confirmed.

Week 3 recommendation: the secure recipient link (`/r/[token]`), on the grounds that `CLAUDE.md`’s own Scope discipline already gated it on “the stack proven on Add Contact” — now true several times over — and the Database section already describes the intended shape in prose. Full plan in `docs/WYP_Week3_Plan.md`. Two of that plan’s open questions were settled the same day it was written:

- **No name collected from the recipient before responding.** Owner: “*the response needs to be as frictionless as possible.*” The anonymous responder’s Dialog who will show the Request’s own Contact name (already known server-side) rather than asking the visitor anything.
- **Token storage stays the simple shape — columns on requests, not a separate request_links table** — after talking through when a separate table would actually earn its keep: link *history* (a UI showing every link ever issued for a Request) or distinguishing, in the access log, which specific link-version a given open used. Neither is a stated requirement anywhere in the PRD, and an old, regenerated token already fails verification on its own (its hash no longer matches), which is the security property that actually matters. Revisit only if a link-history feature gets asked for later.

2026-08-09 — Time Zone: migration 007 drafted, wired into Add Contact/Contact Detail, mockups upgraded

Closes the Week 2 open item flagged earlier the same day: “*profiles.time_zone and contacts.time_zone columns need to be added to their respective tables. AddContactForm.tsx/ContactDetailForm.tsx should render*”

the Time Zone field (which is shown as required) with all available Time Zone names as a pull-down option, but default the selection to the time zone of the User (profiles.time_zone)."

Migration 007 (drafted, not yet run) — docs/Week3 - SQL history.txt. Both columns nullable, no DB-level constraint: matches Category's existing precedent (§2.3's "must be selected from the list" rule is client-side, not a check constraint), and there's no data to backfill from the way contacts.display_name had (migration 005).

Time Zone list: app/src/lib/timeZones.ts, backed by Intl.supportedValuesOf('timeZone') — every IANA zone name, displayed and stored as the raw id ("America/Chicago"), not a friendly label. No standard-library mapping to a friendly label exists without hand-maintaining one for 400+ entries, and raw ids are what the mockup's own pre-existing comments already pointed at (profiles.time_zone / contacts.time_zone). This does diverge from the Create Free Account mockup's old decorative value ("Central Time (Chicago)") — flagged in that file's own comment, not silently changed.

Field wired as a required \$6.16 lookup, same shape as Category — filtered text input with a selectedTimeZone guard, not a native <select>, for visual consistency with the Recipient/Category lookups already in the app. Always type-to-search (the list is far past the 12-item "browsable on focus" threshold used elsewhere).

Defaulting chain, and the write-back decision. Add Contact: the owner's own profiles.time_zone if set, else the browser's detected zone. Contact Detail: the contact's own stored zone takes priority (it may already have one from a previous Save); if not, same fallback chain as Add Contact. In both cases, if the chain had to fall all the way to browser detection, that value is also written back to profiles.time_zone — a decision made and flagged, not silently added: profiles.time_zone has no other live path to a value right now (see below), so without this write-back it would never settle into a stored value at all. Revisit if this feels like the wrong screen to own that side effect once a real Account screen exists.

Create Free Account and the no-contact-dialog mockups got the matching demo-JS pull-down for consistency, even though neither is a live React component. Flagged plainly, not silently discovered later: **Create Free Account is not reachable in the live app.** /login handles both sign-in and first-time account creation with no separate signup step — its own copy tells the user as much — so /account/new has no wiring path today. This means profiles.time_zone currently only ever gets a value via Add Contact/Contact Detail's fallback-and-write-back, never from a screen actually about the user's own profile, until Create Free Account is converted and wired into a real first-run step or the (explicitly deferred) Account screen is built.

2026-08-09 — PRD v12.8: §9.5 Archive Requests and Todos added to the Future Features Roadmap

Owner: *"This should be added to a list of things yet to do: A capability not discussed yet... the ability to 'remove' completed Requests and Todos from the list of items shown in the main screen, but keep them available when Searches are done..."* — followed by a fairly complete first-pass design (Archive screen, select-by-type-and-prior-to-date, pre-checked list, Detail-screen editability affecting eligibility, Archive Now / Remove Archive Status chips, and the schema need).

Landed in the PRD, not a new backlog file. §9 "Future Features Roadmap" already exists for exactly this purpose ("defined for future phases... not in scope... but architecturally considered to avoid rework") and already holds four sibling entries (§9.1–§9.4) in the same short-paragraph format. Adding §9.5 there uses the mechanism the project already has rather than starting a second, competing list.

Marked "Not yet phased" rather than guessing Phase 2 (the label most of §9.1–§9.3 carry) — nothing here was discussed against the phased roadmap in §10, so assigning a phase would be inventing a scope decision, not recording one.

Version bumped to v12.8 — title page, both footers, and a new Schedule A revision-history line. TOC on page 1 still shows the pre-edit page numbers/entries (a cached field — Word regenerates it on Update Field / F9, not something worth hand-editing in the XML). docs/WouldYouPlease_PRD_v12_7.docx deleted (via the file-delete permission

flow) now that v12.8 replaces it, matching the project’s one-file-per-version convention; v12.7 was already committed to git, so nothing was lost. **The Project’s own “Canonical sources” list still says v12.7 — that line lives in Claude.ai Project settings, not a repo file, so it needs updating by hand along with re-uploading the new docx to project knowledge**, per the Maintenance rule.

2026-08-09 — Time Zone browse-on-focus bug fixed

Owner: *“The time zones on Add Contact and Contact Detail are correctly selecting my time zone. However, if I wanted to change it for a contact, there are no other values shown in the pull-down except the selected one.”*

Root cause: the field always arrives pre-filled with a real, non-empty value (the defaulted zone), unlike Category — the field this pattern was copied from — which always starts blank. The dropdown’s filter, “show everything only when the query is empty,” never saw an empty query in practice, so focusing the field filtered the full zone list against the already-selected zone’s own name and showed only that one match.

Fixed the same way in all three affected places — `AddContactForm.tsx`, `ContactDetailForm.tsx`, and the Create Free Account mockup’s demo script (the no-contact-dialog mockup was unaffected; its field starts genuinely blank) — with a browsing flag: true from focus (regardless of what’s currently in the box) until the user types a character, at which point it drops and normal substring filtering takes over. Also select the field’s text on focus, so the first keystroke replaces the prefilled value instead of appending to it and immediately filtering to nothing.

2026-08-09 — Migration 008 drafted: secure recipient link token infrastructure (Week 3, Day 1)

Owner: *“Please work on the Day 1 — Migration 007: token infrastructure for the Week3 plan.”*

Renumbered 007 → 008. The plan’s Day 1 heading was written before the Time Zone work claimed migration 007 later the same day. Corrected in `WYP_Week3_Plan.md` and drafted under the right number in `docs/Week3 - SQL history.txt`, rather than either colliding with Time Zone or silently renumbering Time Zone instead (Time Zone was drafted first, chronologically, and already referenced as 007 in `CLAUDE.md/decisions log` — leaving it alone was the smaller change).

Implemented exactly the shape already agreed: three columns on requests (not a separate `request_links` table — see the “Settled” section of the Week 3 plan), events’ missing read policy, and `issue_request_link/get_request_by_token` as SECURITY DEFINER functions following `CLAUDE.md`’s Database section pattern (hashed token, generic failure message, multi-use logged via events rather than consumed).

One addition beyond the plan: `revoke_request_link`. The plan named two functions; a third followed from a gap noticed while writing the first two — `link_revoked_at` is a column `get_request_by_token` already checks, but nothing anywhere would ever set it without this. Added rather than left as a column with no path to a value. Day 4’s actual UI can still decide later whether to expose a Revoke control to the owner; this only makes the capability exist.

events grant, not just the policy. Migration 002 fully revoked table-level privileges on events from every client role (“Supabase grants table privileges to anon/authenticated by default, so this must be explicit”). An RLS policy alone doesn’t undo that — added `grant select on events to authenticated` alongside the new policy, or the policy would have been silently inert.

30-day link expiry — flagged, not discovered. Nothing in the PRD specifies a lifetime for this link (distinct from the 1-hour sign-in magic link). Chose 30 days because Dialog “continues over days or weeks” (`CLAUDE.md`’s own wording) and a Request’s Due Date is usually inside that window — a reasonable default, not a requirement anyone stated. Called out in both the migration’s own header comment and `WYP_Week3_Plan.md` for confirmation before running.

Scope held at Day 1. No screen was touched — `get_request_by_token/issue_request_link` have no caller yet. `/r/[token]` (Days 2–3) and the Request Detail “get a link” affordance (Day 4) are still ahead, along with `submit_request_response`, which needs the response screen’s actual fields decided first rather than being guessed at now.

2026-08-10 — Request Response converted to live `/r/[token]` (Week 3, Days 2–3); migration 009 drafted

Owner: *“The Week3 plan for days 2-3 ‘Convert Request Response... is for the Respond to Request screen as accessed by the anon end-user recipient of the Request. This same screen (or a version of it) should also be presented to a free or paid subscriber when they click on an item in the Requests Received list on the main screen. The screen design is ‘completed’, please work on the anon access as described in the plan.”*

Built `app/components/RequestResponseForm.tsx` and `app/r/[token]/page.tsx` — the one route in the app with no `RequireAuth` wrapper, since this is the anonymous recipient’s own entry point. The signed-in-subscriber reuse of this screen (clicking a Received row) is explicitly not built now — Received itself is still non-functional (no schema/RLS path exists for a signed-in user to query “Requests sent to me”), so there’s nothing yet to click through to it from. Noted for whenever Received gets its own schema pass.

Migration 009 drafted, catching a real bug in my own migration 008 draft before it was ever run: writing this screen’s data-loading code surfaced that `get_request_by_token` (migration 008) selected and returned `category_name` — a direct violation of PRD §2.3, which states plainly that Category is a sender-side-only organizing label never shown to the recipient, “not on... the Respond to Request screen, the recipient’s Detailed Item view, the non-registered web response view...” Fixed via `create` or `replace` function rather than silently editing migration 008’s already-drafted text: migration 009 removes the `categories` join/field entirely and adds `owner_name/created_at` (needed for the meta block’s Date/From: rows, which migration 008 hadn’t included either). Two new functions in the same migration: `set_response_done_by_token` (writes Done Date/Done Time) and `add_dialog_by_token` (writes a Dialog entry, who resolved server-side from the Request’s own Contact — no name collection, matching the plan’s already-settled “frictionless” decision). Both anon+authenticated callable, same hash/expiry/revocation-check and generic-error pattern as migration 008.

Two deliberate divergences from the mockup, both flagged in code comments and in design/README.md: - Done Date/Done Time render as real editable pickers (Request Detail’s `.fgroup.frow+.ffloat.picker.native` markup), not the mockup’s boxed `.duo/.fieldval` static-text preview. The mockup’s own `.panel.req` comment already flagged its border rule as unresolved (“should be conditional on Done Date equaling Due Date and a Due Time being part of the Request... not permanent”) — rather than inventing that comparison now, both fields are treated as ordinary optional `.opt` fields, matching how Request Detail already treats the same two fields. `.panel.req` was not carried into the live screen; the mockup file itself is unchanged. - Add Dialog’s Save calls `add_dialog_by_token` and appends the RPC’s returned `{id, created_at, who}` straight into local state, rather than re-running `get_request_by_token` — a re-fetch would log a second, semantically wrong ‘viewed’ event for what was actually a write.

Send/Cancel don’t behave like every other Detail screen’s. Send shows an inline `.noticeband` confirmation (“Response saved.”) instead of navigating anywhere, and Cancel resets the two editable fields to their last-saved values instead of calling `router.back()` — both because an anonymous visitor arriving from a mailed/texted link typically has no prior in-app history entry to return to, unlike every other screen’s Cancel/Close, which is only ever reached by clicking a row on its own parent list.

CSS: `.meta/.metatop/.metacol/.seclabel/.respdesc/.grabber/.promo*` ported into `globals.css` in an earlier pass this session; `.panelact/.panelfull` (the stacked Add Dialog/Add Attachment action row, already named in the mockup and in the §6.26 spec entry) added this batch. `.duo/.fieldhead/.fieldval` were **not** ported — they belong to the static-text display this screen no longer uses live, and stay mockup-only.

`npx tsc --noEmit` and `npm run lint` both pass clean.

2026-08-10 — Response Link band added to Request Detail (Week 3, Day 4)

Owner: *“Please provide the testing link as described in: Day 4 — Surface the link somewhere a sender can reach it.”*

Added a “Get Response Link” button to `RequestDetailForm.tsx`, directly under the existing recipient-notification notice band — calls `issue_request_link` (migration 008), then shows the resulting `/r/[token]` URL with Copy and Regenerate. New PROPOSED component, \$6.30 (`.linkband/.linkval`) — Day 4 was scoped in the plan as “surface the link somewhere,” with no mockup drawn for it, so this was built plain against the existing `Strip/.btn-secondary` vocabulary rather than left unbuilt for want of a design pass.

Could not hand the owner an actual working link directly, and said so rather than fabricating one. Two independent reasons: (1) migrations 008 and 009 are both still DRAFTED, not run — `issue_request_link` doesn’t exist in the live database yet, so the button will error until the owner runs both. (2) Even once run, `issue_request_link` is owner-only (`auth.uid()` checked against the Request’s `owner_id`) and its raw token is returned exactly once, never persisted anywhere — by design, only the salted hash is stored. There is no query, migration, or admin path that could produce a real token from outside the owner’s own authenticated browser session; the button has to actually be clicked by the owner. Told the owner this plainly, with the two-step path to get one (run 008/009, then click Get Response Link on an existing Sent Request).

`npx tsc --noEmit` and `npm run lint` both pass clean.

2026-08-10 — Migrations 008 and 009 confirmed run

Owner: *“Migrations 008 and 009 have been run.”*

Marked confirmed in `docs/Week3 - SQL history.txt`, `CLAUDE.md`, and `WYP_Week3_Plan.md`. The full recipient-link loop is now live and testable end to end: Request Detail’s “Get Response Link” issues a real token, `/r/[token]` reads and responds through it. Migration 007 (Time Zone) is unaffected by this confirmation and stays flagged DRAFTED, not yet run — the owner’s message named 008/009 specifically, and nothing here implies 007 was included.

2026-08-10 — Migration 010 drafted: fixes `add_dialog_by_token`’s events insert (bug found by the owner testing Day 4’s link)

Owner: *“I tried to add a Dialog Question to a Request Response and saw an error”* — screenshot showed column `"subject_id"` is of type `uuid` but expression is of type `bigint`.

Real bug in migration 009, not a data issue: `add_dialog_by_token` logged its events row as `('dialog', v_new_id, ...)`, using the new Dialog entry’s own id as `subject_id` — but `dialog.id` is a plain `bigint` identity column (the same type `dialog.replies_to_id` already points at) and `events.subject_id` has been `uuid` since migration 002. No `bigint` value can be cast into it; the insert was always going to fail the first time this path actually ran. `get_request_by_token` and `set_response_done_by_token` both happened to log with the Request’s own `uuid` as `subject_id` already, so neither ever exercised this bug — Add Dialog on `/r/[token]` was the first call to actually hit it, which is exactly what the owner’s testing found.

Fix (migration 010, drafted, not yet run): log with `subject_id = v_request_id` instead, matching the pattern the other two functions already use (and the precedent `subject_type = 'link'` already set — its “subject” is conceptually the link, but `subject_id` is still the Request’s own `uuid`, since there’s no separate link-row id to point to either). The new dialog entry’s own id moves into `detail->>'dialog_id'` instead, preserved for later auditing without needing a `uuid`.

2026-08-10 — Which-Question picker: show who asked, default to Answer, and show even for a single open Question

Owner, first message: *“On the Request Response Add Dialog with two questions... 1) the two questions are shown but it would be helpful to show who asked the question [a person may choose to answer their own question so only showing questions by the other party would not be appropriate], 2) the default response for the chip in this instance is Question even though the only prior dialog entries are questions [it seems more appropriate to show the Answer chip as selected if there are any questions in the dialog which have not been answered yet].”*

Owner, follow-up: *“when I answered one of the two questions and subsequently selected the Answer chip, the remaining unanswered question was not shown.”*

Three related fixes to the Add Dialog modal’s Kind-selection and which-Question-picker logic, applied identically everywhere the pattern exists — `RequestResponseForm.tsx`, `RequestDetailForm.tsx`, `TodoDetailForm.tsx`, and their three source mockups (kept in sync per the project’s standing convention):

1. **Each picker row now shows who asked**, not just the question text (`.qpicker-who`, bold, prefixed before the truncated body). Deliberately not filtered to “the other party” — the owner’s own point was that a person can answer their own Question, so the asker’s identity is informational, not a filter.
2. **Add Dialog now defaults to the Answer chip whenever at least one Question is open**, instead of always starting on Question. `openDialogModal()` now calls `selectKind(openQuestions.length > 0 ? 'answer' : 'question')` — reuses the existing default-question-selection logic inside `selectKind('answer')` unchanged, just changes which chip starts selected.
3. **The which-Question picker itself now renders for any open Question, not only for more than one**. This reverses the 2026-08-07 scoping decision recorded in `design/README.md`’s §6.27 entry (“it only needs to be presented if there is more than one question”) — the owner’s second message is direct field evidence that “links silently, no picker” left no visual confirmation of what a lone remaining Question even was once the picker disappeared. Superseded, not silently overwritten: the original entry stays on record, this entry is the correction. The underlying single-Question auto-select logic (`dlgSelectedQuestionId = open[open.length - 1].id`) needed no change — only the render condition (`openQuestions.length > 1 → > 0`) did.

`npx tsc --noEmit` and `npx eslint` on the three changed components both pass clean.

2026-08-10 — Request Response: quick-Done band, tier-gated Attachments, “(name)” consistency in the which-Question picker; display_name data gap diagnosed

Owner’s message covered several things at once, testing the live Request Response screen:

1. **Attachments segment shouldn’t show for a free-tier issuer**. *“It seems that a Request Response does not need to display the non-usable Attachment segment unless the requestor is a subscriber and if the requestor is a subscriber... it should be shown because it is usable by the recipient... only when the recipient explores the free account features does the ability to subscribe... become of interest to them.”* Added `owner_tier` to `get_request_by_token`’s payload (migration 011, drafted) and gated the whole Attachments `panelact+panelfull` block on `data.owner_tier === 'subscriber'`. **Flagged, not fully resolved**: real attachment storage/upload doesn’t exist anywhere in this app yet — not on this screen, not on the issuer’s own Request Detail, nowhere (still deferred per `CLAUDE.md`’s Scope discipline). So even for a subscriber-issued Request, there’s nothing actually usable behind the Add Attachment button today; I kept it locked and changed the copy from “A subscription feature” (wrong once the issuer already is one) to a plain “No attachments yet” / “not available in this preview yet,” rather than making it look functional when it isn’t. Real attachment infrastructure is still a separate, unscoped piece of work.

2. **From: was blank; Dialog entries by the signed-in owner show a raw email instead of a name**. Both trace to the same root cause: `profiles.display_name` has no live path to a value — Create Free Account isn’t wired into the app (no separate first-run step exists; see `CLAUDE.md` Known gaps) and Account is intentionally undesigned. `RequestDetailForm.tsx`/`TodoDetailForm.tsx`’s own Dialog-save fallback is `ownerName ??`

`userData.user.email ?? 'Unknown'`, so a null `ownerName` falls all the way to a raw email. Not a code bug — a data gap, already flagged, now actually biting real usage rather than just Time Zone. One-time fix recorded in `docs/Week3 - SQL history.txt` (not a migration): `update profiles set display_name = 'Jim Gillon' where id = (select id from auth.users where email = 'jimgillon@gmail.com')`.

3. Which-Question picker: show “(name)” not “name:”. *“In a Dialog Answer presentation, the person asking the question is shown with a following colon, it seems that repeating the use of parenthesis for the name would be more consistent with the full dialog list.”* Reverted the bold `Name:` prefix added earlier today (`.qpicker-who`) in favor of the main Dialog list’s existing `.dlgwho` convention — `(name)`, Ink-Soft, not bold — reused verbatim rather than keeping two ways of labeling who said something. `.qpicker-who` removed from `globals.css` and all three mockups.

4. New: quick-Done band. *“The Request Response recipient should be able to mark a Request as complete in as few keystrokes as possible.”* Owner walked through and rejected two alternatives himself before landing on the shipped design: a Done/Add-Dialog/Add-Attachment chip picker (rejected — not mutually exclusive, a recipient may want more than one), and auto-filling Done Date on page load (rejected — forces anyone who only wants to add Dialog to clear it first). Shipped: a Strip band (`$6.31`, PROPOSED, `.donerow/.donenote`) above the Done Date/Done Time row, with a “Done” button that fills Done Date with today’s date only (Done Time stays untouched — same optional-refinement role it has everywhere else). The band’s own text and the button’s disabled state are both purely reactive to whether Done Date currently holds a value, however it got there — clicking Done or typing directly into the field land in the identical state, so there’s no separate “did they click Done” flag that could drift out of sync. This was the owner’s own proposed resolution to the ambiguity he raised about whether clicking Done is required after a manual date entry — implemented as suggested, not modified.

Scope note: the quick-Done band and the tier-gated Attachments logic were added to the live component only; reconciling `WYP_respond_to_request_palette1.html`’s own demo (which still shows a full attachments file list and static Done Date/Time text, both already flagged as diverging from the live screen — see `design/README.md`) was judged lower value than answering the rest of this message and was skipped this round. Flagged, not silently dropped.

`npx tsc --noEmit` and `npx eslint` on the three changed components both pass clean.

2026-08-10 — Add Dialog: focus returns to Dialog Text on every chip click, not just the default

Owner: *“When another chip is clicked in the Add Dialog dialog, the same focus to the Dialog Text box is needed for UI consistency and is not provided.”*

Root cause: the `textarea`’s `autoFocus` attribute only fires once, on mount — it doesn’t re-fire on a later re-render, so switching chips after the modal was already open left focus wherever it last was (typically the clicked chip itself), with the placeholder-sized empty `textarea` visible but unfocused. Fixed everywhere the Add Dialog modal exists — `RequestResponseForm.tsx`, `RequestDetailForm.tsx`, `TodoDetailForm.tsx`, `CreateRequestForm.tsx`, `CreateTodoForm.tsx`, and all five mockups — by adding an explicit `.focus()` call to each screen’s kind-selection function (`selectKind`, or `selectDialogKind` where none existed yet, in the two Create screens’ React components, whose chip `onClick`s previously called `setDialogModalKind` directly). The call is a safe no-op the first time it runs — during `openDialogModal`, before the `textarea` has mounted (React) or the modal is shown (mockups) — so the existing `autoFocus/value=` initial-open behavior is unaffected; it only does something on a genuine later chip click, which is exactly the case that needed it.

`npx tsc --noEmit` and `npx eslint` on all five changed components pass clean.

2026-08-10 — Add Dialog: clicking a locked chip is now a true no-op

Owner, testing the fix above: *“The change works, unless a non-available chip is clicked... if I click the ‘not-available’ Answer chip, the dialog text displays the full-sized placeholder text and there is no focus. A preferred response to clicking a non-available chip would be none - which would then not deselect the last selected chip.”*

Root cause was different from what the symptom suggested: `selectKind('answer')`'s existing early-return guard (`if (kind === 'answer' && openQuestions.length === 0) return`) already prevented any state change or the newly-added `.focus()` call from running for the locked chip — so nothing in the fix above was wrong. But clicking *any* real, focusable `<button>` — including one carrying `aria-disabled="true"` rather than the native `disabled` attribute — still triggers the browser's own default click-to-focus behavior, which blurs whatever was previously focused (the Dialog Text textarea) before the `onClick` handler ever runs. `aria-disabled` over `disabled` is deliberate app-wide (§6.22, so screen readers still reach the explanation), so the chip stays a real, natively-focusable element; the guard inside `selectKind` was never going to be able to prevent that native side effect from the outside.

Fixed at the source: added `onMouseDown={e => e.preventDefault()}` (mockups: `onmousedown="event.preventDefault()"`) to every Kind chip button, locked or not, in every Add Dialog modal. Preventing the default action of mousedown — not click — stops the browser from shifting focus onto the button at all, so a click on any chip, available or locked, never moves focus away from wherever it already was; the click event (and each chip's own `onClick`) still fires normally afterward, since `preventDefault` on mousedown only cancels the browser's default focus-shift, not event propagation. This also makes the explicit `.focus()` calls added in the entry above redundant for the two always-available chips (focus never leaves in the first place now) — left in place rather than removed, since focusing an already-focused element is a harmless no-op and the calls still matter for restoring focus after a genuine kind change.

Applied to all five live components with an Add Dialog modal (`RequestResponseForm.tsx`, `RequestDetailForm.tsx`, `TodoDetailForm.tsx`, `CreateRequestForm.tsx`, `CreateTodoForm.tsx`) and all five mockups, including the two Create screens' permanently-locked Answer chip (no dynamic unlocking logic, but the identical native-focus-steal problem applies to any real button regardless of whether it has an `onClick` at all).

`npx tsc --noEmit` and `npx eslint` on all five changed components pass clean.

2026-08-10 — Request Response: Add to Calendar moved above Date/From/Due to fix Android word-wrap

Owner, testing live on a narrow Android phone: *“The date presentation on the Request Response causes more word-wraps than desired... ‘Date: Monday, August 10, [wrap] 2026’... There is a lot of visually-unused space under the Add to Calendar button and to the right of both the From and the Due text values.”* He proposed two options: (1) shorten the date string to an `mdy-plus-weekday` format and keep the side-by-side layout, vertically centering the button with the three rows, or (2) move Add to Calendar to its own row above, matching the Add Dialog/Add Attachment pattern already on this screen, with date formatting untouched — flagging a concern that this would push Done Date/Done Time further down, resolved (his own suggestion) by scrolling Done Date into view when the Done button is clicked.

Recommended and shipped option 2. Root cause of the wrap was the side-by-side layout squeezing the Date/From/Due column to make room for the button, not the date format itself — freeing that column to full width removes the constraint directly. Checked whether the current verbose weekday format (“Monday, August 10, 2026”) was safe to change in isolation before ruling out option 1: it's the identical format used in the Request Detail, To Do Detail, Response Detail, and Dialog Detail mockups' own `label:value` date displays, so reformatting it here alone would put this screen out of step with its siblings for a problem the layout change already solves on its own — a legitimate idea, but a separate, larger decision than fixing this wrap.

Add to Calendar now sits in its own `.panelact` row above `.meta` (reusing the row already used for Add Dialog/Add Attachment on this screen, rather than a new component) — `.metatop/.metacol` (the side-by-side wrapper) removed from `globals.css` and both mockups. `handleQuickDone()` now scrolls the Done Date field into view (`scrollIntoView({ behavior: 'smooth', block: 'center' })`) after filling it, addressing the push-down concern the owner flagged and pre-approved a fix for.

Owner confirmed the same change should apply to Response Detail's mockup (the signed-in-subscriber variant of this screen, not live) for consistency — applied there too, alongside `RequestResponseForm.tsx` and `WYP_respond_to_request_palette1.html`.

`npx tsc --noEmit` and `npx eslint` on the changed component pass clean.

2026-08-10 — Request Response: Add to Calendar top spacing; promo block trimmed and reordered

Two small follow-ups from the owner after the word-wrap fix above.

Add to Calendar top spacing. *“Please add a matching amount of vertical space above the Add to Calendar button as there is below the button. It looks a bit crowded/adjacent to the screen title row.”* Add to Calendar’s `.panelact` row is the first thing in `.scroll` on this screen, so it had no preceding sibling to supply a natural gap above it, unlike every other `.panelact` on the screen (which follows a `.grabber` or `.panel`) — it kept its usual 6px margin-bottom but had nothing matching above. Added a new `.panelact-top` modifier (`margin-top: 6px`) scoped to just this one row, rather than changing `.panelact` itself everywhere it’s used.

Promo block. *“Please drop the ‘Free Account Features’ text line, accordingly shorten the vertical size of the ‘sales pitch’ area, move the ‘Create your own Free Account’ [button] above the ‘The simple way to ask...’ sentence. The Free Account Features is not needed because the button says Free Account, and having the sentence precede the button incorrectly implies it should be read before clicking the button.”* Removed `.promo-kicker` entirely (CSS rule and markup) rather than just hiding it — it was purely redundant with the button’s own label. Reordered to heading → button → sentence; `.promo-h`’s margin dropped its now-unneeded top offset (it’s the first child now) and `.promo-p`’s margin flipped from bottom-only to top-only, since it’s the last child now and `.promo`’s own bottom padding supplies the box’s trailing space. Net effect is exactly what was asked: one fewer line, and the button no longer sits behind a sentence that reads like a precondition.

Applied to `RequestResponseForm.tsx`, `globals.css`, and `WYP_respond_to_request_palette1.html` (the promo block doesn’t exist on Response Detail’s mockup — signed-in users don’t see the free-account pitch, per the existing screen-map note — so nothing to port there for that piece; the Add to Calendar top-spacing fix is scoped to Request Response only, since that’s the screen the owner tested and reported on).

`npx tsc --noEmit` and `npx eslint` on the changed component pass clean.

2026-08-10 — Add to Calendar: real .ics generation, client-side

Owner posted a mockup of what the calendar entry’s own description text should read like: *“A Would You Please Request from Jonathan Jackson: **Create and send the Sacramento district sales projection report...** To mark it completed, click: ”* — bolded text is the Request’s own Description, the rest is fixed wrapper text around it. Confirmed: *“The bolded text is the Request Description, the ‘click on link..’ text would be a standard prefix to the link.”*

Built entirely client-side, on click of the previously-inert Add to Calendar button on Request Response. Every field the file needs — Description, owner name, Due Date/Time, and the response link (this page’s own URL) — is already loaded by `get_request_by_token`, so there’s no new migration or endpoint; `buildIcsContent()` assembles a standard RFC 5545 VCALENDAR/VEVENT block (proper TEXT escaping, 75-octet line folding, DTSTAMP in UTC, DTSTART/DTEND as floating local time since the Request has no stored time zone of its own) and `handleAddToCalendar()` triggers the download via a Blob + temporary `<a download>`, no server round trip. Considered and rejected building this server-side (a new endpoint reformatting data already in the page’s own state buys nothing) and as a live “webcal://” subscription feed (materially bigger — needs a stable public URL and its own token story, and doesn’t match the single-button/single-download shape already established) — see the earlier design-proposal exchange for the fuller reasoning.

No stored Due Time defaults to 9:00 AM. Owner: *“If there is no time, we can use 9am as a standard — probably later offer an Account profile for default time of day.”* `ICS_DEFAULT_DUE_TIME` is a top-level constant for now; per-account default time is flagged, not built — profiles has no such column, and Account itself is still intentionally undesigned (see CLAUDE.md Known gaps).

Boilerplate text is hardcoded, flagged for a future admin surface. Owner: *“There will need to be a Would You Please administrative interface where such standard text can be modified. That can just be a ‘will be done’ item at this point.”* The “A Would You Please Request from <name>:” opener and “To mark it completed, click:” closer live in `buildIcsDescription()` as a plain template literal — no admin UI, no schema for editable boilerplate strings, anywhere in the app yet.

Scope, clarified via AskUserQuestion: the owner’s ask to also build this “into the Response Detail” was ambiguous between porting the feature into that screen’s mockup demo (matching how every other live fix has been ported into mockups this session) and building Response Detail out as an actual live route — the latter would reopen the explicitly-deferred “Received Requests have no live data path” gap (owner-only RLS, no column linking a row to its recipient). Owner picked the mockup-only port. Applied to `RequestResponseForm.tsx` (live), `WYP_respond_to_request_palette1.html`, and `WYP_response_detail_palette1.html` (demo data matching each mockup’s own visible Date/From/Due/Description text, not read back out of the DOM).

`npx tsc --noEmit` and `npx eslint` on `RequestResponseForm.tsx` pass clean; both mockups’ `<script>` blocks pass `node --check`.

2026-08-10 — Three small fixes from live testing, and Week 3’s last open item closed out

.ics “From Would You Please”. Owner, testing live: *“The message body starts with ‘A Would You Please Request from Would You Please.’ I see that the underlying Request does not have a From value — which once the app is fully implemented could not happen.”* Correctly diagnosed as a test-data gap (`profiles.display_name` still unpopulated for the test account that issued this particular Request), not a real bug — but the fallback that produced it was worth tightening anyway: `buildIcsDescription()` now omits the “from <name>:” clause entirely when `owner_name` is null, instead of falling back to the app’s own name and producing a sentence that reads as nonsensical regardless of whose data is missing. Ported into both mockups’ `addToCalendar()` for consistency, even though their own demo `ownerName` is always set.

Add Dialog empty-body validation — focus and wording. Owner: *“If it is saved without text there is an appropriate error message in red, but instead of returning to a focus on the Dialog Text field with a floated label it shows the placeholder... and does not have a focus.”* Same root cause as the chip-switch focus fix two entries below, on a different trigger — `handleDialogModalSave()`’s empty-body guard set the error and returned without ever calling `dialogTextRef.current?.focus()`. Fixed across all five live components with an Add Dialog modal and their five mockups. Also reworded the error itself per the owner’s ask — *“I think it would be better to say ‘Enter Dialog Text or Cancel.’ ”* — from “Enter Dialog Text.” to “Enter Dialog Text or Cancel.” everywhere it appears.

Create ToDo’s Due Date — Week 3’s last open item. Owner: *“please add the Due Date to the Create ToDo, however it can hopefully be optional... perhaps for database integrity, when not entered store a future date such as 01-01-2099 and then have a SQL function that returns a blank... or some other universal way to allow such an inconsistency.”* Checked the schema before proposing a workaround: `requests.due_date` is already a plain nullable date column with no not null constraint (docs/Week2 - SQL history.txt), and every screen that already reads it (Main Screen’s sort/format, Request Detail, ToDo Detail) already handles a null value correctly — a sentinel date plus a SQL function to mask it back to blank isn’t needed here; the database already supports “no Due Date” natively. Added as a real, optional field: `TodoFormState.dueDate`, written as null when empty (not omitted from the insert, not a sentinel), using the same .opt Row-Tint-while-empty treatment as Done Date rather than Create Request’s required .req Ink border. Also added to ToDo Detail (component and mockup) beyond the literal ask — a Due Date set at creation needs somewhere to stay visible and editable afterward, same as every other field on that screen, so leaving it Create-only would’ve been a dead end. Ported to `WYP_create_todo_palette1.html` and `WYP_todo_detail_palette1.html`.

`npx tsc --noEmit` and `npx eslint` on all changed components pass clean; changed mockups’ `<script>` blocks pass `node --check`.

2026-08-10 — ToDo Due Date/Done Date: one combined row, Done Time dropped, Create ToDo gets Done Date

Owner, reviewing the just-added Due Date field with a pasted rough draft: *“The ToDos do not need Done Time. The ToDo Detail and Create ToDo should both show the Due Date and Done Date as optional with grey backgrounds since they are optional. I pasted-in a rough draft of the top of the screen layout. The reason a Create ToDo should allow a Done Date is to allow completed ToDos to be entered if desired.”*

Checked before applying the grey-background note, since it could have read either as restating the app’s existing rule or as a new one: asked whether Due Date/Done Date should use the standard §6.25 convention (grey while empty, white once filled — same as Due Time, Category, everywhere else) or stay grey permanently regardless of content. Owner picked the existing convention, so no new exception to §6.25 was introduced — just applied consistently to the newly-combined row.

Three changes, all owner-directed: 1. **Done Time removed from ToDo Detail.** ToDos never get a Done Time field anywhere, unlike a Request’s Done Date/Time pair, which keeps its Time field — `requests.done_time` stays in the schema (shared with Requests) but ToDo screens stop reading, writing, or displaying it. 2. **Due Date and Done Date combined into one side-by-side row**, matching the owner’s own pasted draft, on both Create ToDo and ToDo Detail — replacing Due Date’s own row plus (on ToDo Detail) the old Done Date/Done Time row. 3. **Done Date added to Create ToDo** (optional, `.opt`, no time) — *“to allow completed ToDos to be entered if desired.”* Written as `null` when empty, same pattern as Due Date.

Applied to `CreateToDoForm.tsx`, `ToDoDetailForm.tsx`, `WYP_create_todo_palette1.html`, `WYP_todo_detail_palette1.html`. `npx tsc --noEmit` and `npx eslint` on both components pass clean; both mockups’ `<script>` blocks pass `node --check` (neither script referenced the removed Done Time field).

2026-08-10 — Type-ahead lookups: exact-match click now shows the full list, not just the match

Owner, from live testing: *“When a value from the list is selected and filled-in to the field, a subsequent click on the field shows a pull-down with just that value. That is technically correct, but is not providing what the end-user needs at that point... My recommendation is to add a rule to how pull-down type-ahead lists are presented. If the in-field value is an exact match for a pull-down value, then when clicked — instead of showing just that value in a pull-down list, show all values in the pull-down list (preferably with the exact match displayed as selected).”*

Root cause: every lookup’s filtering logic ran a plain substring match against whatever the field currently held, with no distinction between “just focused, about to type” and “already has a value.” An exact match filtered the dropdown down to exactly one row, so reopening a filled field gave the user nothing to pick from except what was already there.

Generalized the `timeZoneBrowsing` pattern (first built 2026-08-09 for Time Zone’s own browse-on-focus bug) to every other lookup in the app: a `xBrowsing` flag set `true` on focus (alongside `e.target.select()`) and `false` on the first keystroke, which forces the filtered list to show everything, unfiltered, until the user actually types. Applied to:

- **Category** — `CreateRequestForm.tsx`, `CreateToDoForm.tsx`, `ToDoDetailForm.tsx`, `RequestDetailForm.tsx`
- **Contact/Recipient** — `CreateRequestForm.tsx`
- **Time Zone** — already had the browsing flag from the earlier fix; only missing the visual highlight (below)

The `LOOKUP_BROWSE_THRESHOLD` size gate (currently 12) is unchanged and still governs the *empty*-field case only — showing everything on focus of an already-filled field is a different scenario from dumping a large list on an empty one, and Time Zone’s own ~400-entry list has been showing in full on focus since 2026-08-09 without issue (the `.lookup-results` panel already scrolls).

Also added the **visual “selected” highlight** the owner asked for (*“preferably with the exact match displayed as selected”*) — reused the pre-existing `.lookup-item.selected` CSS class (originally built for the *which-Question*

picker) rather than adding a new one, applied via a conditional className to the currently-matching row in every lookup above, plus Time Zone’s own dropdown in `AddContactForm.tsx` and `ContactDetailForm.tsx`, which had the browsing fix already but never got the visual treatment.

Mockup scope, checked file-by-file rather than assumed: Create Request, Create ToDo, Request Detail, and ToDo Detail’s mockups have no interactive Category/Recipient lookup JS at all — those fields are static demo markup with no `onfocus/oninput` handlers — so none of the four needed any change for this fix. Time Zone lookup JS (the only lookup with real mockup JS) exists in two files, both updated with the same exact-match `.selected` treatment: `WYP_create_free_account_palette1.html` and `WYP_add_contact_no_contact_dialog_palette1.html`. `WYP_add_contact_palette1_floating.html`’s Time Zone field has no `<script>` behind it at all (confirmed — the file has zero `<script>` tags), and `WYP_contact_detail_palette1.html`’s Time Zone field is explicitly flagged in its own file header as carried over for visual consistency only, never wired — neither needed a change.

`npx tsc --noEmit` and `npx eslint` pass clean on all six changed components; both changed mockups’ `<script>` blocks pass `node --check`.

2026-08-10 — Create ToDo gets its own quick-Done band

Owner, with a pasted rough draft of Create ToDo’s own top layout: *“I realized that we can save the end-user a keystroke for completing a ToDo by adding a Done button and message similar to the Request Response... The wording with the active button could be ‘Note: To quickly complete this ToDo, click Done and Save.’ and the inactive Done button could have the text ‘This ToDo is now marked as Done, just click Save.’”*

Mirrors Request Response’s `.donerow/.donenote` pattern (§6.31) exactly: a Strip-background band with a “Done” button that fills Done Date with today, purely reactive to whether Done Date already holds a value however it got there (clicking Done, or typing directly into the field) — no separate “was Done clicked” flag to drift out of sync. Sets Done Date only; unlike Request Response, there’s no Done Time to leave untouched, since ToDos don’t have one (removed from ToDo Detail the same day as an earlier fix). Owner’s exact wording used verbatim for both states.

Added to `CreateToDoForm.tsx` (`handleQuickDone()`, `doneDateRef` for the scroll-into-view behavior, same as Request Response’s) and to `WYP_create_todo_palette1.html`’s own demo JS (`quickDone()`) — worth noting this mockup got real interactive JS for the feature even though Request Response’s own mockup never did (flagged as not-yet-ported in an earlier entry): Create ToDo’s Due Date/Done Date fields already exist as visible targets on this screen for the demo to fill, and the owner’s rough draft was drawn specifically against this screen.

`npx tsc --noEmit` passes clean; the mockup’s `<script>` block passes `node --check`.

2026-08-11 — Received Requests: design confirmed and built (migration 012)

Asked directly, “Are we ready for Week 4?” Answer surfaced two loose ends first: migrations 007 and 011 both still said “DRAFTED — NOT YET RUN” in their own file headers with no confirmation anywhere else (unlike migration 004’s case, which had a stale header but a real confirmation elsewhere). Owner confirmed 2026-08-11 that all provided migrations have been run, along with the Week 2 demo seed script — `CLAUDE.md` updated accordingly, and Week 4 opened with Received Requests as the priority: *“Received Requests seems like the best next effort. Once that is working, the main screen sorting can be completed and expanded screens can be designed.”*

Design proposed (recommendation / alternatives rejected / open questions) and confirmed the same day. No new columns — `contacts.email` (already required) matches against the signed-in recipient’s own session email (`auth.jwt() --> 'email'`), because a recipient almost never has an account yet when a Request is sent (magic-link auth creates the account on first sign-in), so a `recipient_user_id` column resolved at send time would sit null for nearly every real case — the same snapshot-vs-live argument the Entitlements section already settled once for tier. Four new SECURITY DEFINER functions (migration 012), parallel to the existing `/r/[token]` set but keyed by session identity: `get_received_requests()`, `get_received_request(p_request_id)`,

`set_response_done_as_recipient(...), add_dialog_as_recipient(...)`. Functions rather than RLS policies, for the same reason CLAUDE.md's Database section already gives for the anonymous link case: RLS is row-level, not column-level, so a policy scoped by a contact-email subquery couldn't hide Category from an otherwise-visible row — the exact bug class migration 009 already fixed once on the token path.

Self-sent Requests are not excluded. Owner: *“I can imagine circumstances where a person might choose to send themselves requests instead of using Todos — so, I would not exclude it — unless there are reasons to do so.”* No `owner_id <> auth.uid()` filter anywhere in migration 012.

Response Detail's route — the owner's response clarified this wasn't actually an open decision: *“It sounds like the `/requests/[id]/respond` is part of the designed strategy — I am not sure what decision is needed.”* Correct; proceeded as proposed, no further confirmation needed.

“Main screen sorting” clarified — not about Received specifically. Owner: *“Main screen sorting was referring to the various column headings and the ascending and descending sort options with the yellow background for the selected column title.”* This is the `.colbar/.pill` pattern already in the mockup (the `--sort` yellow token, §3.1): today only Due (Sent/Received) and Priority (Todos) render as a pill, and it's a static default, not a clickable, direction-toggling control — every other column header (To/From, Date, Done; Category — Description) is plain text. Logged as Priority 2 in `WYP_Week4_Plan.md`, not started this batch.

Built: `docs/Week4 - SQL history.txt` (migration 012, DRAFTED); `MainScreen.tsx` extended with a `ReceivedRow` type, `get_received_requests()` fetch, `receivedStatus/shared statusFor` helper, `filteredReceived`, and real chips/rows replacing the old `.subempty` placeholder — row click routes to `/requests/[id]/respond`, not `/requests/[id]` (that's the sender's own edit screen). `ResponseDetailForm.tsx` (new) and `app/requests/[id]/respond/page.tsx` (new, wrapped in `RequireAuth` — unlike `/r/[token]`, this route requires a real session) — a close adaptation of `RequestResponseForm.tsx` using the four new functions, with the “Create your own Free Account” promo dropped (this visitor already has an account) and `Cancel` using `router.back()` instead of a local-state reset (this screen, unlike an anonymous mailed link, is always reached from a real Main Screen history entry — same convention as Request Detail/ToDo Detail/Contact Detail).

Refactor alongside this: the `.ics` builder (`buildIcsContent` and its ~90 lines of RFC 5545 helpers, previously local to `RequestResponseForm.tsx`) moved to `app/src/lib/ics.ts` so `ResponseDetailForm.tsx` could reuse it verbatim, rather than duplicating a second copy of non-trivial, stateless RFC logic — an intentional, flagged exception to this codebase's usual per-component-duplication convention (short helpers like `todayISODate/formatMDY` and per-component hook logic like the lookup “browsing” pattern stay duplicated on purpose; this one didn't meet that bar). `todayISODate` and `truncate` moved with it, since both call sites needed them too.

`npm tsc --noEmit` and `npm eslint` pass clean on all changed/new files.

2026-08-11 — Simplified empty-state Dialog/Attachments row (§6.32)

Owner, with a pasted-in reference mockup of Create Request's top layout: *“The Create Request, Request Detail, Request Response, Create ToDo, and ToDo Detail screens can be simplified in respect to the presentation of the Dialog when there are no entries and for Attachments when there are no entries... Currently there is some inconsistency with the Add Dialog button and related field presentations across screens... The suggested text adjacent to the Add Dialog and the Add Attachments buttons would not be needed when there are either Dialog or Attachment entries to display.”* Response Detail (built earlier this session, not yet named in the owner's list since it postdates his last full read of the screen map) added to scope via `AskUserQuestion` — recommended and confirmed, since it carries the identical Dialog/Attachments blocks copied from Request Response.

The inconsistency, precisely: Create Request/Create ToDo (staged-entry screens) already showed nothing but a bare button when Dialog was empty — no panel, no heading. Request Detail/Request Response/ToDo Detail/Response Detail (existing-thread screens), by contrast, always rendered the full `.panelfull/.panel`, heading included (“Dialog

(Questions, Answers, Comments)), even at zero entries, showing “No Dialog entries yet.” as a placeholder. Two different empty-state treatments for the same underlying situation.

New component, §6.32 PROPOSED: `.actlabel` — a single `.frow` pairing a descriptive label with the Add button, replacing whichever heavier empty-state markup a screen used to show. Two variants, matching the owner’s own pasted reference exactly: Dialog gets `.actlabel` (a bordered box reading “Questions, Answers, Comments,” echoing the Recipient/Category field boxes it sits among — Dialog is a normal, available feature); Attachments gets `.actlabel.locked` (plain muted text reading “Subscription feature,” no box — Attachments is unavailable, so it stays in the existing `.is-locked` visual language instead of looking like an active field). Deliberately **not** a revival of the superseded §6.10 read-only-boxed-field variant (a fake input standing in for a value) — `.actlabel` is a static section descriptor next to a button, not a value display.

Behavior: once Dialog holds entries (`dialogEntries.length > 0` on the staged-entry screens, `sortedDialog.length > 0` on the existing-thread screens), the row reverts to whatever populated-state markup that screen already had — `.fieldact+.dlgstaged` or `.panelact/.fieldact+.panelfull/.panel+.panelhead` — unchanged from before this batch. Attachments has no code path to a populated state yet (no attachment storage anywhere in the app), so it stays the compact row unconditionally on every screen until that’s built; Request Response/Response Detail’s existing `owner_tier === 'subscriber'` gating is unchanged, only what’s rendered inside it is simplified.

Mockup scope, checked per screen: Create Request and Create ToDo’s mockups got a full functional swap (`dlg-empty-row/dlg-fieldact` toggled by `saveDialogEntry()`/a new `removeDialogRow()`, since these demos genuinely go from empty to populated as entries are staged and removed). Request Detail, Request Response, Response Detail, and ToDo Detail’s mockups are all seeded with sample Dialog data on load and have no remove-entry control, so their Dialog thread is never actually empty in the demo — documented with a comment rather than built as unreachable toggle JS. Request Response/Response Detail’s Attachments block is a separate, pre-existing, already-flagged divergence (a deliberately aspirational demo file list, “beyond what’s actually built” per design/README) — left untouched; only the true empty/locked case (matching every live screen) uses the new `.actlabel.locked` row.

Applied to `CreateRequestForm.tsx`, `RequestDetailForm.tsx`, `RequestResponseForm.tsx`, `ResponseDetailForm.tsx`, `CreateToDoForm.tsx`, `ToDoDetailForm.tsx`, and all six corresponding mockups. `npx tsc --noEmit` passes clean; every touched mockup’s `<script>` block passes `node --check`.

2026-08-11 — ToDo Detail’s missing quick-Done band; date/time fields click-anywhere-opens-picker on desktop

Two bugs the owner found testing the live app on his phone and on Windows.

ToDo Detail was missing the quick-Done band. Create ToDo gained the §6.31 `.donerow/.donenote` band on 2026-08-10, but that batch was scoped to Create ToDo only — it was never ported to ToDo Detail, unlike the parallel case of Request Response’s quick-Done band, which *did* get carried over to Response Detail the same day it went live there. A real gap, not a design choice: without it, ToDo Detail had no fast path to marking an existing ToDo Done, only the manual Done Date field. Fixed by porting Create ToDo’s `todayISODate/handleQuickDone/doneDateRef/.donerow` JSX into `ToDoDetailForm.tsx` verbatim (same owner wording, same reactive-to-whether-Done-Date-holds-a-value behavior). **Not ported into the ToDo Detail mockup** — same precedent as Response Detail’s still-unported quick-Done band (see that screen’s design/README entry): the live app is what’s being tested, and mockup fidelity wasn’t part of the report.

Date/time fields only opened their native picker via the calendar/clock icon on desktop. Owner: on a phone, tapping anywhere in a Date or Time field opens the picker; on Windows, only clicking the icon does — clicking elsewhere in the field does *not* open it, “which could be technically correct if hand-editing the date or time was needed or allowed - which it is not.” Root cause: mobile browsers treat any click on a date/time input as the trigger for their native picker UI; desktop browsers (Chromium, at least) only do that for a click on the input’s own calendar/clock icon, leaving the rest of the field’s click area doing nothing. Since typing a date or time by hand was never a supported input

method here (§6.16’s label-affordance glyph exists specifically to signal “focus opens a picker,” not “type here”), a click anywhere in the field should open the picker on desktop too.

Fix: a small `openPicker` handler — `el.showPicker()`, feature-detected (`typeof el.showPicker === 'function'`) and wrapped in `try/catch`, since pre-16.4 Safari has no `showPicker()` and the call can throw in some states — wired to `onClick` on every native `type="date"/type="time"` `.finput` across the live app: `CreateRequestForm.tsx` (Due Date, Due Time), `RequestDetailForm.tsx` (Due Date, Due Time, Done Date, Done Time), `RequestResponseForm.tsx` (Done Date, Done Time), `ResponseDetailForm.tsx` (Done Date, Done Time), `CreateToDoForm.tsx` (Due Date, Done Date), `ToDoDetailForm.tsx` (Due Date, Done Date) — 14 fields, 6 files. Duplicated per component rather than extracted to a shared lib file, matching this codebase’s own convention for short, stateless helpers (`todayISODate/formatMDY`) — `openPicker` is 6 lines with no component-specific parameterization.

Mockups needed no change. Checked all six Dialog/Attachments-batch screens’ date/time markup: none of them use real `<input type="date"/>/type="time">` elements — Create Request, Request Detail, Create ToDo, and ToDo Detail render these fields as `type="text" inputmode="numeric"` styled to *look* like pickers via the `.ffloat.picker` CSS wrapper and calendar/clock SVG glyph (no click handler, no JS picker library); Request Response and Response Detail don’t even have `<input>` elements for these fields, only static `.fieldval` display text (that screen’s own source comment already flags Done Date/Done Time as “should be conditional... not permanent” once they *do* become real pickers). `showPicker()` only exists on real native date/time inputs, so this fix is live-app-only by nature, not a mockup gap.

2026-08-11 — Due/Done Date-Time row width imbalance; Clear affordance for Time fields (§6.33)

Two more owner-reported bugs, found continuing to test the live app on his phone.

Due Date rendered wider than the field beside it, squeezing the second field’s label text off the edge — on Request Detail and ToDo Detail specifically, not Create Request or Create ToDo. Owner: “the Create ToDo scales down horizontally for the Due Date and Done Date fields which display as the same size. But the ToDo Detail screen shows a longer Due Date which results in the Done Date placeholder text bleeding out on the right side of that field.” He later confirmed this wasn’t a stale-screen artifact and still reproduces.

Root cause: `.frow .ffloat` was `flex: 1 1 auto`. With `flex-basis:auto`, a flex item’s hypothetical size falls back to its own content’s intrinsic width whenever its child can’t resolve a definite size against it — and `.finput` is `width: 100%`, a percentage, which can’t resolve until the flex-basis itself is known, so the browser falls back to the input’s own intrinsic content width. For a native `type="date"/type="time"` control, that intrinsic width isn’t fixed the way a plain text input’s is — it can depend on whether the field currently holds a value, most visibly on mobile, where the OS picker draws the actual formatted date/time as inline content. Request Detail and ToDo Detail load an *existing* record, so Due Date typically arrives pre-filled while its row-mate (Due Time, or Done Date) is still empty — different content, different intrinsic width, unequal flex-grow shares. Create Request and Create ToDo start every field empty, so both sides of the row have identical (zero) content and split evenly — which is exactly why the bug never showed up there.

Fix: `.frow .ffloat`’s flex-basis changed from `auto` to `0%` in `app/globals.css`. With a zero basis, both fields grow from nothing by equal flex-grow, so they always end up the same width regardless of which one happens to hold a value — content is removed from the sizing calculation entirely. This is the standard fix for “N equal-width flex columns regardless of content.” Checked every `.frow` with two `.ffloat` children in the app (Due Date/Due Time, Done Date/Done Time, Due Date/Done Date — Create Request, Request Detail, Request Response, Response Detail, Create ToDo, ToDo Detail) — that’s exactly the set of rows this was already meant to keep visually even, so one CSS rule fixes all of them at once. `.frow` rows with a single `.ffloat` beside a `.btn` (Recipient, Category lookups) are unaffected, since that lone growable item already claims all the leftover space either way.

Due/Done Time fields had no way to clear a set value; Due/Done Date fields did. Owner: “The Due and Done Time fields do not currently have a way to Clear the value once it is set - as is needed and as is available for the Due

and Done Date fields.” Some browsers show a native clear “x” on a populated `type="date"` field but not reliably on `type="time"` — a platform inconsistency, not something WYP controls, and since hand-typing into these fields was never a supported input method either (see `openPicker`’s own reasoning), a Time field that got set had no way back to empty at all.

New component, §6.33 PROPOSED: `.fclear` — a small “x” button, styled like the existing `.attremove` (plain text button, Ink-Soft, Alert-Red on press), absolutely positioned at the field’s right edge. Rendered only once the field holds a value. Placed in markup *after* `.flabel`, never between `.finput` and `.flabel` — CLAUDE.md flags that ordering as load-bearing for the floating-label adjacent-sibling selector, and inserting a button there would have silently broken it. Wired to every Due Time/Done Time field: `CreateRequestForm.tsx` (Due Time), `RequestDetailForm.tsx` (Due Time, Done Time), `RequestResponseForm.tsx` (Done Time), `ResponseDetailForm.tsx` (Done Time) — 5 fields, 4 files. Date fields were left untouched; the owner’s own report frames Date as already working, and duplicating a redundant clear control there would be pure churn.

Accepted tradeoff, not fixed: on desktop, `.fclear`’s absolute position at the field’s right edge visually overlaps wherever the browser draws its own native calendar/clock icon inside the input box. Left as-is — `openPicker` (2026-08-11, same day) already makes a click anywhere in the field open the picker, so nothing is functionally lost by the icon being partly covered.

Mockups needed no change, same reasoning as `openPicker`: none of the affected screens use real `type="date"/type="time"` inputs in their static HTML (styled `type="text"` fields, or no `<input>` at all on Request Response/Response Detail), so there’s no native picker whose width varies by content, and no native field to attach a clear button to.

2026-08-11 — Main Screen To/From column width; Add Contact’s return path from Create Request

Two more owner-reported items, testing the live app on his phone.

Main Screen’s To/From column truncated long names too aggressively. Owner, testing with a contact named “Maximillan”: it displayed as “Maximilla...”. His working theory was that the date columns (Date/Due/Done) used a fixed-width font to keep their digits vertically aligned, and that switching to a variable-width font with percentage-based column widths would free up room for the name — or, failing that, trimming the gap between the three date columns by roughly a character each, which he estimated would hand the name column about two more characters.

Checked both premises directly in `app/globals.css`. The font was never the issue — `.dt/.due/.dn` (the Date/Due/Done cells) inherit the app’s one font, Inter, same as everything else; there’s no separate monospace declaration anywhere in this row. What actually constrained the name column was `.r1`’s grid: `grid-template-columns: 1fr 58px 58px 58px` with `column-gap: 16px` — three 16px gaps between four tracks, 48px spent purely on spacing, on top of 174px of fixed date-column width. Reduced `column-gap` to 10px in both `.r1` (the data rows) and `.colbar.sr` (the header row above them, which has to stay in lockstep or the To/From/Date/Due/Done labels stop lining up with their columns). CSS Grid hands freed gap space straight to the sole 1fr track, so all 18px goes to the name column — close to the two characters the owner estimated, with no risk of the three numeric columns reading as run-together (10px is still a clear gap, just tighter than 16px). Left the 58px date-column widths themselves untouched — at 11px Inter, “MM-DD-YY” already fits them tightly, and touching a value that’s already close to its content’s minimum risked the exact overflow/wrap problem the owner was explicit about avoiding. Ported the same two values into `WYP_main_screen_palette1.html`’s embedded `<style>` block, matching this repo’s tokens-are-copied convention.

Add Contact, opened from Create Request, returned to the Contacts list instead of back to the Request with the new contact selected. This was a known, already-flagged gap — `AddContactForm.tsx`’s own header comment (2026-08-09) called it out directly: *“Revisit if a second entry point (e.g. Create Request’s no-contact interception, §6.24, not yet built) starts reaching this screen — that path will want its own return destination.”* Create Request’s own Add

Contact button reached this screen first, via the plain button that already existed rather than through the not-yet-built §9.9.5 interception dialog — so the gap arrived before the fuller feature that was expected to surface it.

Fix, scoped to the specific complaint rather than building the full §9.9.5 in-place interception dialog: Create Request’s Add Contact button now links to `/contacts/new?from=create-request` instead of a bare `/contacts/new`. `AddContactForm.tsx` reads that from param (via `window.location.search` inside the Save/Cancel handlers, not the `useSearchParams()` hook — those handlers already only run client-side in response to a click, so there’s no SSR timing concern to guard against, and using the hook instead would have forced a `Suspense-boundary` change to `/requests/new`’s page just to satisfy Next’s static-render analysis for a value this component never needs during any server pass). On Save, if `from === 'create-request'`, it now redirects to `/requests/new?newContactId=<id>` instead of `/contacts` — the contact insert gained a `.select('id').single()` it didn’t have before, since the redirect needs the new row’s own id. `CreateRequestForm.tsx`’s mount effect (already fetching the contacts list) checks for `newContactId` the same way, selects the matching contact once the list has loaded, and calls `router.replace('/requests/new')` to strip the query string so a refresh or back-navigation doesn’t re-select it. Cancel gets the same origin-aware treatment, minus the new-contact id — nothing was added, so there’s nothing to select, but the owner still lands back on Create Request rather than a Contacts list they never asked to see.

Explicitly not fixed, and flagged rather than silently left as a partial fix: every other field already typed on Create Request before clicking Add Contact — Due Date, Description, staged Dialog entries, Category — is still lost on the round trip, exactly as `CreateRequestForm.tsx`’s own comment already said. A full fix needs the real §6.24/§9.9.5 in-place interception (a dialog that never navigates away at all, so nothing is ever at risk of being lost) — this fix only corrects where the round trip lands and makes sure the one field that matters most (Recipient) comes back filled in, since that’s the specific complaint. Recommended next step if the owner wants the rest solved too: build §9.9.5 properly rather than growing this query-param approach to also round-trip the whole draft form.

Mockup needed no change — Create Request’s own mockup has no interactive Recipient/Category JS at all (a static demo field, per its own screen-map entry), so its Add Contact button was already inert with nothing to wire.

2026-08-11 — Done-band wording after Send (Request Response, Response Detail)

Owner, after successfully testing a self-sent Request end to end: once Send succeeds, the explanatory text next to the Done button itself should confirm that too, not just the `.noticeband` confirmation banner at the top of the screen. The donerow already had two reactive states — empty Done Date (“For a quick response, click Done and Send.”) and filled-but-not-yet-sent (“This Request is now marked as Done, just click Send.”) — both purely reactive to whether Done Date holds a value. Added a third: once `sendConfirmed` is also true, the text becomes “This Request is now marked as Done and has been Sent.” Same reactive pattern as the other two states — no new flag, so it can’t drift out of sync with what actually happened (there’s already exactly one source of truth for “did Send succeed,” the existing `sendConfirmed` state driving the `.noticeband`). Applied to both `RequestResponseForm.tsx` and `ResponseDetailForm.tsx` — identical donerow markup, identical `sendConfirmed` state, same fix in both. Neither mockup needed changes — the quick-Done band itself has never been ported into either screen’s static HTML (already flagged in `design/README.md`), so there was no third state to add there.

2026-08-11 — Main Screen column-header sorting

Owner: “Please work on the column heading sorting for the three sections of the Main screen” — the last of three items in the same message, and a callback to much earlier scoping (“the various column headings and the ascending and descending sort options with the yellow background for the selected column title”). Until now only Due (Sent/Received) and Priority (ToDos) ever rendered inside the `.pill` yellow-background treatment, and it was a static default — not a clickable, direction-toggling control. To/From, Date, Done on Sent/Received, and Category — Description on ToDos were plain text with no sort behavior at all.

Recommendation, implemented: every `.colbar` cell is now a real `<button>`. Clicking an inactive column makes it the active sort (in that column’s own sensible default direction — descending for Date/Due/Done, matching the existing Due-descending default; ascending for To/From, Category, Priority); clicking the already-active column reverses direction. The active column shows the existing `.pill` component with its label plus a $\wedge/v$ arrow — no new visual language, just extending the treatment Due/Priority already had to every column and making it live. Sort state persists per-section to `sessionStorage` (`wyp.mainSentSort / wyp.mainReceivedSort / wyp.mainTodoSort`) as a single `"key:dir"` string, read back via a lazy `useState` initializer — the same pattern and the same storage tier (session, not `localStorage`) already established 2026-08-09 for the filter chips, for the same reason: a within-session view preference, not a durable account setting. Nulls (empty Due Date, empty Category, etc.) always sort last regardless of direction, via a shared `compareNullable` wrapper, rather than drifting to the front on descending sorts the way a naive comparator would.

Sorting is client-side, over the already-fetched `filteredSent/filteredReceived/filteredTodos` arrays via `useMemo` — the same “fetch once, filter/sort/search client-side” precedent this file (`MainScreen.tsx`) already used for the filter chips and search box, appropriate at the personal/tens-of-rows scale this app targets. A generic `toggleSort(state, key, defaultDirTable)` helper and nulls-last comparators (`compareStrings`, `compareNumbers`) are shared across all three sections’ handlers, even though the three sections’ underlying row shapes differ, to avoid three copies of the same toggle/compare logic.

Alternative rejected: giving each column a fixed, single sort direction (no toggle) — simpler, but throws away half of what the owner explicitly asked for (“ascending and descending sort options”), and breaks the existing Due/Priority precedent, which already implied direction was meaningful even before it was interactive.

Accessibility fix along the way: the first pass set `aria-sort` directly on each `<button>`, which is only valid on an element with `role="columnheader"/role="rowheader"` (or a native `<th>`) — flagged by `eslint’s jsx-a11y/role-supports-aria-props` rule, since a `<button>`’s implicit role is `"button"`. Rebuilding these cells as a real `<table>/grid-role` structure just to host `aria-sort` correctly was judged out of scope for a CSS-Grid-based row layout that isn’t a table anywhere else in the app. Used `aria-label` instead (e.g. “Sort by Due, currently sorted descending”) — conveys the same state to assistive tech without the role mismatch, consistent with this codebase’s existing use of `aria-label` for interactive-element context (e.g. `.fcClear’s` “Clear Due Time”).

Mockup unchanged: `WYP_main_screen_palette1.html’s .colbar` cells are static `<span>s` with no click handlers (confirmed by inspection) — same situation as several other recent Main Screen features (filter chips, search) that only ever went live, per the screen-map’s existing “static demo” scoping for this mockup. Not ported.

Open question, not yet raised by the owner: whether a future multi-column sort (e.g. Priority then Due) is wanted. Not built — nothing in the request implied it, and the single-column model matches every comparable pattern already in this app.

2026-08-11 — PRD v12.9: Notification Email Templates (§7.3)

Owner: “It seems that to further test the Request process that email formats are needed,” followed by literal `to:/from:/subject:/body:` templates for an Initial Request email and a day-before Reminder email. Actually sending either requires infrastructure this project doesn’t have yet — no Resend/SMTP integration, no `RESEND_API_KEY`, no scheduled job for the reminder (confirmed by inspection: `package.json` has no mail dependency, `.env.local` has no mail-provider key). Offered the owner a choice of scope via `AskUserQuestion` rather than assuming; he chose spec-only for now — capture the templates as a precise PRD addition, no sending code yet. Matches this file’s own existing “wire Resend in when you first need the app to send a real email... not before” stance (`docs/WYP_Week1_Setup.md`).

PRD §7.3 updated, new file `docs/WouldYouPlease_PRD_v12_9.docx` (previous canonical, `WouldYouPlease_PRD_v12_8.docx`, kept alongside it — same precedent as v12.7 staying in `docs/` after v12.8 superseded it). Added a “Notification Email Templates (New in v12.9)” block directly after the existing “Request Email Format (New in v7.0)” bullets, rather than a new numbered subsection — §7.4 onward isn’t renumbered, avoiding a ripple through every existing cross-reference in the doc. **Flagged three points against existing PRD**

content before writing anything, per the owner’s own standing rule (“flag the conflict before making the change — don’t silently overwrite”), all three now resolved by the owner directly:

1. **From address.** The owner’s literal template wrote from: `<account user Email>` — flagged that most mail providers reject or spam-flag transactional mail claiming to be From a domain the sending service isn’t authorized for (a personal Gmail/Outlook address can’t be a legitimate From on service-sent mail). Proposed instead: From on Would You Please’s own sending domain, display name “`<sender Display Name> via Would You Please`”, Reply-To set to the sender’s real account Email — same “feels like it’s from `<sender>`” effect, actually deliverable. **Owner confirmed this is correct.**
2. **.ics delivery method.** The owner’s template only pointed the recipient at the in-app Add to Calendar button via the response link, dropping the existing §7.3 requirement that each email carries an .ics attachment directly. Flagged as a real behavior change, not just added wording. **Owner: attachment is the better solution — fewer clicks for the recipient — and he’d simply forgotten it was already specified.** PRD text keeps the attachment requirement; the new template cites it by reference rather than duplicating it.
3. **Notifications Matrix “approaching due date” row.** Currently Push, both parties, configurable — conflicts with a fixed, recipient-only, unconditional Reminder email. **Owner: leave the row as-is** — the Reminder email is additive, not a replacement for the configurable Push notification. No matrix edit made.

New in this round, not in the original three flagged points — owner-introduced: if a Request’s Due Date leaves less than 24 hours before Send, there’s no time for a day-before Reminder to fire at all. Owner’s own resolution: advise the Request creator of this at Send time (not the recipient), and adjust the Initial Request email’s body wording to drop the reminder-promise sentence in that case, rather than silently sending a promise that won’t be kept. Written into the PRD as a “Tight-window rule” bullet. **24 hours is a proposed default of mine, not something the owner specified a number for** — flagged in the PRD text itself, same treatment as the existing unconfirmed 30-day link-expiry default elsewhere in this project. The creator-facing advisory itself (where it appears — Create Request? Request Detail’s existing notice band?) is not designed yet; this PRD entry only establishes that the behavior is required, not its UI.

Owner also flagged a related, not-yet-built dependency: Create Free Account. The From/Reply-To design above depends on `profiles.display_name` and the sender’s real account Email being populated — and per this file’s own existing Known-gaps entries, `profiles.display_name` has no live path to a value today (Create Free Account is mockup-only; Account is intentionally undesigned), the same gap already noted for Time Zone. Not resolved here — flagged in CLAUDE.md’s Known gaps as a dependency of the new email work, not built.

Not yet done, flagged rather than silently skipped: the imported knowledge project’s own “Canonical sources” listing still names `WouldYouPlease_PRD_v12_8.docx` — that’s the Claude.ai project’s own knowledge-base instructions, outside this repo, and I have no write access to it from here. The owner will need to update that listing (and re-upload the new file to project knowledge) himself if he wants v12.9 treated as canonical there too, per this file’s own Maintenance rule.

2026-08-11 — Create Free Account goes live as the mandatory first-run step

Owner: “Can we work on the Create Free Account screen (I am thinking of using it for limited testing), or are other activities needed first?” — genuinely open, not a go-ahead, so this started as an assessment rather than straight into building.

Checked for a conflict with CLAUDE.md’s Auth section first, since “no separate signup step... do not add one” is an explicit, capitalized rule. Reading the mockup itself resolved it before it became a real conflict: Email renders `.metarow`, read-only, sourced from the session, and the mockup’s own footer copy already says “No password on this account — you have already signed in with the one-time link emailed to the address above.” This screen was always designed as post-magic-link profile completion, never a signup form — nothing to flag, nothing to reconcile.

Found a genuine prerequisite while checking the schema: the original Week 1 `profiles` migration revoked table-wide UPDATE and replaced it with a column list (`display_name`, `first_name`, `last_name`, `phone`, `notify_by`). `time_zone` didn’t exist yet — it was added later in migration 007 — and nothing since extended that

grant to include it. A column-specific GRANT is a snapshot; ALTER TABLE ADD COLUMN doesn't retroactively extend one already issued. Net effect: every write to `profiles.time_zone` — including Add Contact/Contact Detail's own browser-detected fallback write-back, described in their own file comments as quietly keeping the owner's zone current — has been failing with a permission error since migration 007, silently, because neither file checks that update's returned error. SELECT was unaffected (table-wide, set up before the column carve-out), which is exactly why nothing had surfaced this yet: reads worked, so the feature looked like it was working. Wrote migration 013 (`docs/Week4 - SQL history.txt`, drafted, not yet run) to fix it — a one-line grant, plus it retroactively explains why `profiles.time_zone` may never have actually taken a value despite that feature apparently shipping in Week 3.

Also found, not something new I built: the original Week 1 schema comment on `profiles.display_name` already says “NULL means account setup is incomplete, which is how /auth/callback decides whether to route to Create my Free Account.” /auth/callback never actually did this — it's always redirected everyone straight to /. Wiring Create Free Account now isn't new scope invented for this request; it's finishing a design that was already committed to the schema and just never implemented.

Built: `app/components/CreateFreeAccountForm.tsx` + `/account/new`, converting the mockup field-for-field (First/Last/Display Name and Time Zone required, same lookup pattern as Add Contact; Phone optional; Notify Me By locked to Email, Text subscription-gated) — Save-only, no Cancel, matching the mockup's own reasoning that an exit here would strand an authenticated account with an incomplete profile. Writes via UPDATE, since the `handle_new_user` trigger already creates the profile row the moment the auth account is created.

Enforcement — asked rather than assumed, since it changes what gets built: offered the owner a choice between a mandatory redirect (any account with no Display Name gets sent here automatically) and building the screen without wiring the redirect yet (reachable only by navigating to `/account/new` directly). Recommended mandatory, since it completes the already-documented design intent and — because the owner's own seeded profile already has a Display Name via the earlier one-time SQL fix (`docs/Week3 - SQL history.txt`) — poses no risk to his own existing testing, only to genuinely new accounts, which is exactly what “limited testing” describes. **Owner chose mandatory.** `app/auth/callback/page.tsx` now queries `profiles.display_name` after `getSession()` succeeds and routes to `/account/new` when it's null, / otherwise.

Scoped deliberately narrow: enforcement lives only in `/auth/callback`, not in `RequireAuth.tsx` itself. A `RequireAuth`-level check would also catch a direct/bookmarked/back-button visit to / with an incomplete profile, but that would add a `profiles` fetch to every single page load app-wide for a case that isn't the one the owner described (testing via fresh sign-ins). Flagged as a possible harder-enforcement follow-up, not built.

2026-08-12 — .actlabel empty-state Dialog row: (optional) + Row Tint background

Owner, with a pasted-in screenshot of the live Request Detail screen: “To be consistent with optional data elements on screens, the text ‘Questions, Answers, Comments’ adjacent to the Dialog button when there are no dialog entries... should be presented with the word ‘(optional)’ appended to the text and should have a standard grey background.”

Both pieces were already-established rules (§6.25: Row Tint background at rest, “(optional)” via `.subnote` in the label) that simply hadn't been applied to `.actlabel` when §6.32 introduced it 2026-08-11 — not a new design decision, a consistency fix. `.actlabel` shipped with `background: #fff` and bare text; changed the background to `var(--row-tint)` and the six live JSX usages (`CreateRequestForm.tsx`, `CreateTodoForm.tsx`, `RequestDetailForm.tsx`, `RequestResponseForm.tsx`, `ResponseDetailForm.tsx`, `TodoDetailForm.tsx`) to Questions, Answers, Comments `<span className="subnote">(optional)</span>`, matching the exact pattern already used for Due Time/Category/Phone.

Scoped to the unlocked .actlabel only — `.actlabel.locked` (Attachments' “Subscription feature”) is untouched on purpose. That one isn't an optional-but-available field; it's an unavailable one, deliberately kept in the `.is-locked` visual language (no box, muted text) rather than looking like a normal field at rest. Dialog is genuinely optional (§6.25 already lists it as `.opt` on Create Request/Create ToDo); Attachments isn't offered at all without a subscription, so the two states aren't the same case and shouldn't read the same.

Not ported to the mockups — the six affected screens’ static HTML link a shared `components.css`, which lives in the Would You Please Claude.ai project’s own knowledge base, not this git repo, and is mounted read-only in this session. Flagged rather than silently skipped: two of the six mockups (Create Request, Create ToDo) have working empty/populated JS toggles that would visibly render the stale white-background, no-“(optional)” version if opened right now. The owner would need to either apply the equivalent `components.css` change himself, or ask for it in a context where that project knowledge file is writable.

Superseded 2026-08-12, same day, a few messages later — the “mockups link a shared, read-only `components.css`” claim above turned out to be wrong; see the entry at the top of this log for the correction and the actual port into the four mockups it affects.

2026-08-12 — Missing gap bug; Attachments’ locked note becomes a `.donerow` band; `tokens.css/components.css` found stale

Three more items from the same owner message, continuing straight off the `.actlabel` fix above.

The “(optional)” space bug had a real cause, not a typo. Owner: the space between “Comments” and “(optional)” was missing on screen. It wasn’t missing from the JSX — `Questions, Answers, Comments <span className="subnote">(optional)</span>` has it, verified in all six files. The actual cause: `.actlabel` is `display:flex` with no gap set. Its two children are an anonymous flex item (the bare text run “Questions, Answers, Comments”) and the `.subnote <span>`. A flex item establishes its own inline formatting context, and CSS trims trailing white space at that boundary the same way it trims it at a soft line-wrap — so the text run’s own trailing space silently never rendered, independent of whether the box was wide enough to fit everything on one visual line. `.ffloat .flabel` has the identical shape (glyph + text as separate flex children) and never had this problem only because it already carries an explicit `gap:6px` — spacing that doesn’t depend on a raw text node’s own whitespace surviving. Fixed by adding `gap:4px` to `.actlabel`, the same category of fix, not a new one.

Attachments’ “Subscription feature” text moves onto `.donerow/.donenote`, off `.actlabel.locked` entirely. Owner: centered, box-less muted text next to a button “is not immediately visually related to the button on the right” — asked for the same “Note: ...” treatment (bolded “Note:”, left-aligned) already sitting one field above it on these same screens, the quick-Done band’s “Note: To quickly complete this ToDo, click Done and Save.” Rather than adding a third visual language for this row, reused `.donerow/.donenote` verbatim — it already does exactly this job (note text + adjacent button, tinted strip, left-aligned), and this project has a standing precedent for reusing one generic component across unrelated meanings rather than minting a new class per screen (`.metarow` alone already covers Email, Recipient, the Date/From/Due block, and Dialog Entry Type). New copy: “**Note:** Attachments are a Subscription feature.” Applied to all six screens (`CreateRequestForm.tsx`, `CreateToDoForm.tsx`, `RequestDetailForm.tsx`, `RequestResponseForm.tsx`, `ResponseDetailForm.tsx`, `ToDoDetailForm.tsx`) — the two Request Response/Response Detail instances keep their existing `owner_tier === 'subscriber'` gate around the row, just with the row’s own markup swapped. `.actlabel.locked` is now dead code and removed from `app/globals.css`; `.actlabel` (unlocked) survives for Dialog only.

Owner asked directly: is “the Would You Please project’s Claude.ai knowledge base” the Project’s documents, are `tokens.css/components.css` those files, and shouldn’t they be in `C:\Project\wyp` by now? Yes to both identification questions. Checked before answering the third: found both files at `.../docs/tokens.css` and `.../docs/components.css` in that project’s own knowledge base — which is mounted read-only in this session, so nothing here could have written to or synced from them regardless. Opened them: both are headed “Draft v0.1 · July 2026,” and neither contains anything built since — no `.actlabel`, `.donerow`, `.metarow`, `.subnote`, none of `.is-locked`’s three `--locked*` tokens, nothing from this entire multi-week session’s worth of component work. CLAUDE.md’s own maintenance rule states these files are supposed to be regenerated after every mockup-affecting change; that plainly stopped happening early on, while `app/globals.css` (in this repo, touched and kept current in every batch) became the real single source of truth without the docs ever being updated to say so. **Not fixed** — copying the stale files into the repo as-is would just import broken `<link>` targets with outdated content, not a working setup; a real fix needs either regenerating both files from `app/globals.css`’s current state, or converting the mockups that still

link them over to the self-contained inline-`<style>` pattern `WYP_create_free_account_palette1.html` already uses. Documented in `design/README.md`'s Notes section; left as an open decision for the owner rather than started unprompted, given the scope.

Superseded 2026-08-12, same day, a few messages later — “the mockups that still link them” turned out to be none: re-verified while organizing the stylesheets and found every apparent `<link>` reference was a docstring inside a mockup's own `<style>` comment, not a real tag. See the entry at the top of this log for the correction, the regenerated `design/screens/tokens.css/components.css`, and the mockup fixes that followed.

2026-08-12 — Stylesheets organized and realigned; corrects the previous entry's mockup-CSS claim

Owner: “please organize and realign the style sheets as needed,” following straight on from the previous entry below, where he'd asked whether `tokens.css/components.css` should be living in `C:\Project\wyp` by now.

The previous entry's answer was wrong, and needs correcting here rather than left standing. It said the six affected mockups “link a shared `components.css`... read-only from this session,” so the day's `.actlabel/.donerow` fixes could only land in the live app. Re-verified while scoping this task, since acting on “organize the stylesheets” meant actually testing that claim rather than continuing to assume it: none of the 17 `design/screens/*.html` files has a real, working `<link rel="stylesheet" href="tokens.css">` or `href="components.css">` tag. Checking with a regex that only counts `<link>` tags appearing *before* a file's first `<style>` tag (rather than anywhere in the file) found zero matches everywhere. What the earlier, looser check had actually matched was a docstring pasted inside each mockup's own embedded `<style>` comment block — literal text like “ORDER OF STYLESHEETS on every screen: ... 2. `tokens.css` ... 3. `components.css` ...” describing a hypothetical future build convention, not an active `<link>` tag. **All 17 mockups are, and apparently always have been, fully self-contained** — each embeds its own complete `:root` token block plus every component rule it needs directly in its own `<style>` tag(s), the same pattern `WYP_create_free_account_palette1.html` was already known to follow. Told Jim directly in the next reply rather than letting the wrong claim stand uncorrected.

Regenerated `tokens.css/components.css` as `design/screens/tokens.css` (49 lines) and `design/screens/components.css` (2,100+ lines), extracted from `app/globals.css` — which turns out to be the file that's actually been kept current every session, even though every mockup's own header comment still claims the opposite sync direction (“Source of truth: ... Keep in sync with the mockups”). These two new files exist purely as a reference for keeping each mockup's embedded copy in sync by hand; no mockup was converted to link them, since doing so would be a separate architectural decision (and would cost each mockup its “opens directly from disk, no missing assets” property, which `design/README.md`'s own Conventions section calls out as intentional).

Audited all 17 mockups' embedded CSS against the new canonical copy. Token values matched everywhere they existed — zero conflicts, only expected gaps for tokens newer than a given mockup (`--locked`, `--focus-halo`, etc.). Class coverage found five mockups with real, pre-existing gaps between markup and CSS — not all created by this session's own `.actlabel` work:

- `WYP_create_request_palette1.html` — `.actlabel/.dlgstaged/.ferror` referenced in markup but never defined. Separately, and not caught by the first coverage pass (a naive per-class-token check saw `.chip.is-locked` present and wrongly concluded bare `.chip` was covered too): `.chip/.chip.selected/.chip:focus-visible` — the Add Dialog Kind picker's base button styling — were missing entirely, so Question/Comment have been rendering as unstyled native `<button>`s in this one mockup the whole time. Found by manually diffing against the five sibling mockups with an Add Dialog modal, all of which had it.
- `WYP_create_todo_palette1.html` — `.actlabel/.ferror` undefined. More notably: `.donerow/.donenote` are used by the quick-Done band (shipped 2026-08-10) but were never defined anywhere in this file — that band has been rendering completely unstyled since the day it was added.

- `WYP_request_detail_palette1.html`, `WYP_todo_detail_palette1.html` — `.donerow/.donenote` added for today's Attachments conversion (see below); neither had a quick-Done band of its own to also fix.
- `WYP_respond_to_request_palette1.html` — `.subnote` (used by the which-Question picker's own note) was undefined.

Added each missing rule verbatim from the new canonical `components.css`, with a comment explaining what was missing and why.

Ported the same day's .actlabel(optional)/Row-Tint and .donerow/.donenote Attachments fix into the four mockups using the current .actlabel pattern (`WYP_create_request_palette1.html`, `WYP_create_todo_palette1.html`, `WYP_request_detail_palette1.html`, `WYP_todo_detail_palette1.html`) — Dialog's empty-state row now reads "Questions, Answers, Comments (optional)"; Attachments' locked row is now `.donerow/.donenote` ("Note: Attachments are a Subscription feature.") in place of the retired `.actlabel.locked`. **Deliberately not** ported into `WYP_respond_to_request_palette1.html/WYP_response_detail_palette1.html` — both still use the older `.panel`-based Dialog/Attachments markup (already flagged, separately, as not yet converted to `.actlabel` at all); bringing those two forward is a markup-pattern port, a bigger and separately-scoped piece of work, not a stylesheet-alignment fix.

Checked, not fixed: `WYP_main_screen_palette1.html`'s `c-cat/c-nm/c-pri`. The initial coverage pass flagged these as missing too. Confirmed not a bug — `app/globals.css` itself only ever styles `.colbar .c-dt/.c-due/.c-dn` (the date columns, which need centering); the `name/priority/category` columns were never meant to carry a rule of their own, relying on the grid's default alignment instead. The mockup matches the live app exactly.

Verification: re-ran the class-coverage audit after all edits — zero remaining gaps across all 17 mockups (aside from the confirmed-not-a-bug Main Screen case above). Confirmed every edited file's `<style>` tags stay balanced via a small `HTMLParser`-based check (regex alone had already produced one false alarm earlier in this session, when it miscounted `<style>` inside a comment as a real tag).

2026-08-12 — Todos gain an Overdue chip; Expand icon dropped app-wide

Two related, unrelated-in-cause requests from the owner, same message.

Todos gain a fourth filter chip, Overdue, matching Sent/Received's order. Owner: "Now that Todos have Due and Done Dates, we need to add the Overdue chip for Todos to match the chips order for Requests Sent and Received on the main screen. Todos without a Due Date would be ignored for the Overdue chip." Implementation turned out to need no new logic — `statusFor(due_date, done_date)`, already shared by `sentStatus()/receivedStatus()`, already treats a null `due_date` as never-overdue (the `due_date && due_date < today` check short-circuits on null), which is exactly the owner's own stated rule. Added `todoStatus()` as a third thin wrapper around the same function, added `due_date` to `ToDoRow` and to the `Todos select()` (neither had ever fetched it — Main Screen's Todos list has never displayed Due/Done Date as its own column, only used them via other screens, so this was a genuine gap, not a redundant fetch), and inserted the Overdue chip between Open and Done in `todoFilter`'s state, storage key, and JSX — same chip `over/sel` class pattern `Sent/Received` already use.

Row-level red-text treatment needed its own CSS, not just the filter. Owner, in a follow-up message once he saw the first pass: "The overdue items in the Todo list should follow the red-display of text to match the Requests." The `.row.overdue/.row.done` rules already in `globals.css` only ever targeted `Sent/Received`'s own child classes (`.nm/.dt/.due/.dn/.desc`) — a `ToDo` row uses an entirely different set (`.pri/.cat/.tdd`, since it's laid out as one flowing line, not `Sent/Received`'s four-column grid), so applying `overdue/done` to a `ToDo` row's own `className` alone did nothing visually. Added `.row.overdue .pri/.cat/.tdd { color: alert-red }` and `.row.done .pri/.cat/.tdd { color: ink-soft; font-weight: 500 }`, mirroring the existing `Sent/Received` rules exactly — `overdue` only recolors (keeps each class's own font-weight), `done` normalizes weight to 500 the same way `Sent/Received`'s own `done` rule overrides `.nm`'s 600 and `.due`'s 700.

Not flagged as a conflict requiring a pause, but worth recording: both the PRD (“Overdue is not applicable to Todos since due date is optional”) and UI spec §6.2 (“Todos uses three chips (All / Open / Done) because the due date is optional and Overdue does not apply”) document the old three-chip rule explicitly — this change reverses it. Not treated as a stop-and-ask conflict, since the owner’s own message shows he’s already reasoned through the exact edge case those documents’ rule was protecting (“Todos without a Due Date would be ignored for the Overdue chip” is, functionally, the same case “the due date is optional” describes) and is deliberately choosing a different resolution now that Todos actually have a Due Date column to test against — the PRD/spec’s own rule predates that column entirely. Left as a flagged gap for the owner to decide whether/when to formally revise those two documents (see the entry below, which raises the same kind of gap for the Expand icon and asks about both together).

The Expand icon is gone from the app, app-wide. Owner, in the same message, explaining the reasoning rather than just issuing the instruction: the icon’s whole premise was that each Main Screen section had its own limited “elevator” view, with Expand opening a full-screen page to see everything. That’s not how Sent/Received/Todos actually got built — every selected item already renders under its own section on the Main Screen, which the owner also judged the practically correct behavior for normal use, not an implementation shortfall to fix. He considered an expanded view’s only remaining possible value (showing more lines of Request Description) and judged it minimal utility on its own — not worth building a screen for. Conclusion: drop Expand/Contract’s use in the app rather than design the “Priority 3: Expanded screens” phase WYP_Week4_Plan.md had reserved for it (that plan doc’s own entry updated to record the reversal, not just deleted).

Removed from mainScreen.tsx: all three .subicons clusters (Sent, Received, Todos) drop their Expand `<span>`, leaving Print alone in each; the now-unused `ExpandIcon()` function removed entirely. Removed from WYP_main_screen_palette1.html the same way — three `<span class="iconbtn" aria-label="Expand ...">` elements (each wrapping a base64 PNG, not an inline SVG like the live component’s version — this mockup predates the inline-SVG icon conversion) — plus its own now-stale header comment (“Expand/Print on the right”) corrected to say Print only. Contract was never actually built anywhere in this app — no mockup, no live component — so there was nothing to remove for it; it only ever existed as a documented, not-yet-reached full-page-view control in the PRD/UI spec.

Same open-documentation-gap pattern as the Overdue chip above, and the two are being flagged to the owner together: the PRD’s §3.7 “Expanded Full-Page View” and the UI spec’s §8.7 “Expand to full page”, §9.7 “Expanded Full-Page List”, and §5.1/§11.1 icon inventory (which lists Expand/Contract among the app’s eight functional icons) all still describe this feature in detail. Not edited here — a multi-section revision touching TOC page references, the icon count/inventory table, and possibly figures, in two formal documents, is a bigger and more consequential change than either doc edit this session has made unprompted so far. Flagged as an open question for the owner: revise both documents now (dropping §3.7/§8.7/§9.7, updating the icon inventory to seven functional icons, and reconciling the PRD/spec’s “Overdue does not apply to Todos” wording with the change above), or leave both flagged as stale pending a later, dedicated documentation pass.

`npx tsc --noEmit` and `npm run lint` both clean after this batch.

2026-08-12 — Migrations 012 and 013 verified directly; Week 4 closed out

Owner asked whether Week 4 was done so he could start Week 5. Both of Week 4’s own migrations (012, Received Requests’ four functions; 013, the `profiles.time_zone` UPDATE grant) were already marked confirmed-or-drafted in the docs, but the owner reported logging into Supabase and seeing SQL Editor History entries only through migration 011 — worth taking seriously rather than trusting the docs’ own prior claims, since the History view is the owner’s direct window into what actually ran.

Resolved by direct verification, not by re-trusting either side’s memory. Rather than asking the owner to re-run migrations that might already be applied (safe, since both are idempotent — create or replace function and a plain grant — but wasteful if unnecessary) or leaving the discrepancy unresolved, gave the owner a single paste-and-run script: a `pg_proc` lookup for migration 012’s four function names, and an

`information_schema.column_privileges` lookup for migration 013's grant. Both are authoritative — they read the database's actual current state, not a query log that may not capture everything (e.g., statements run via a different tool, or simply scrolled out of a paginated History view).

Results: migration 012 — all four functions (`get_received_requests`, `get_received_request`, `set_response_done_as_recipient`, `add_dialog_as_recipient`) exist, confirming Received Requests' backend is real, matching what its own file header already claimed. Migration 013 — authenticated holds UPDATE (plus SELECT/REFERENCES/INSERT, all from table-wide grants that already covered `time_zone` automatically once the column existed — only the column-specific UPDATE carve-out from Week 1 had ever excluded it) on `profiles.time_zone`, confirming the grant is in place and Create Free Account's Save now writes Time Zone successfully. The owner's own History-view observation and the earlier "confirmed run" claims were **not actually in conflict** — most likely the History view just doesn't show every migration the owner has run, or displays a limited window; not investigated further, since the direct query settles the substantive question either way.

Updated migration 013's own header in `docs/Week4 - SQL history.txt` from "DRAFTED — NOT YET RUN" to "CONFIRMED RUN," and every doc that still said otherwise: CLAUDE.md's Known gaps (two separate bullets), `design/README.md`'s Create Free Account entry, and `CreateFreeAccountForm.tsx`'s own header/inline comments.

Week 4 is now fully closed. All three of its own priorities are done: Received Requests (migration 012, live), Main Screen column-header sorting (live), and Expanded screens (deliberately dropped rather than built — see the entry above). The additional work that came up mid-week — Create Free Account going live (migration 013), the PRD §7.3 email-template spec batch, the stylesheet-organization pass, and the Todos Overdue chip / Expand-icon-removal batch above — is also closed out. Two items remain explicitly flagged rather than resolved, both by the owner's own choice to leave them for later: the PRD/UI-spec formal revision for the dropped Expand/Contract feature and the now-inaccurate "Overdue doesn't apply to Todos" wording (owner: "We can leave the document updates flagged for later"), and Week 5's own priority, not yet chosen as of this entry.

2026-08-12 — Week 5 scoped: email sending, then Account screen, then real Attachments

Owner: "It seems like the order of these items should be email sending, account screen, real attachments (based on an account screen status of subscribed)." Drafted `docs/WYP_Week5_Plan.md` capturing this order and its reasoning — Attachments' subscriber gate already reads `profiles.tier` live, but nothing in the app has ever displayed or changed that value, so testing the gate meaningfully needs the Account screen (or some way to set tier) first.

Real question surfaced while scoping Priority 2/3's link: `profiles.tier` is deliberately not writable by a signed-in user — migration 002 never grants authenticated column-level UPDATE on it, reserving that for a future `service_role` billing webhook. So "an account screen status of subscribed" has no live path today without either building real payment processing or a lighter stand-in. Asked the owner directly rather than assume either direction, since they're very different scopes (a full Stripe-or-similar integration vs. a read-only display plus a manual toggle). **Owner chose the lighter path:** the Account screen will display tier read-only; he'll flip his own profile to 'subscriber' manually via the Supabase SQL editor to test Attachments — the exact statement is already sketched in migration 002's own comments. Real payment processing stays explicitly deferred, not part of this week's scope; CLAUDE.md's Scope discipline section already listed "payments" as deliberately deferred, and this keeps it that way rather than quietly expanding scope while answering a different question.

Priority 1 (email) has its own external dependency, flagged in the plan doc rather than blocking on it here: a live send path needs a Resend account, API key, and a sending domain with SPF/DKIM/DMARC configured — none of which can be created from inside this session. The template-rendering module, the send-on-create call site, and the Tight-window advisory UI can all be built without it; only the actual `send()` call needs the key to test end-to-end. Two smaller open decisions noted for when that priority starts: Vercel Cron vs. `pg_cron` for the day-before Reminder's scheduled job, and confirming PRD §7.3's 24-hour Tight-window threshold, which its own text already flags as an unconfirmed proposed default.

2026-08-12 — Priority 1 built: email template module, send-request route, Tight-window advisory

Owner: “I have a site wouldyouplease.com. I will set up the email capability. Please start on the part you suggested and await my real MX information to proceed.” Confirms wouldyouplease.com is the owner’s real, owned domain (he’ll handle the Resend account and DNS himself) and authorizes building everything that doesn’t need the API key/verified domain yet.

Extracted PRD §7.3’s literal to:/from:/subject:/body: wording directly from docs/WouldYouPlease_PRD_v12_9.docx (the zipfile+regex technique this session already uses for docx reads) rather than working from memory or the plan doc’s own paraphrase, to make sure the templates match word-for-word.

Built, in order: - app/src/lib/email.ts — pure, isomorphic template rendering: isTightWindow (24-hour threshold, missing Due Time falls back to ICS_DEFAULT_DUE_TIME from ics.ts rather than inventing a second “unspecified time” convention), buildRequestEmailSubject (both Initial and Reminder — Reminder is the identical subject prefixed “REMINDER:”, exactly as specified, not a separate template), buildRequestEmailBody (the four required parts in order, reminder sentence omitted under Tight-window), and buildRequestEmailFromName/EMAIL_FROM_ADDRESS. No env var access anywhere in this file — safe to import from a client component or a server route. - app/api/email/send-request/route.ts — the one place RESEND_API_KEY is read. **Re-derives description, Due Date/Time, and the recipient’s email from Supabase itself, scoped by the caller’s own forwarded JWT** (anon key + Authorization header passthrough, no service_role), rather than trusting whatever the client posts — the client only supplies which Request and the already-minted response link. Every failure path returns 200 with sent: false and a reason, including the current no-RESEND_API_KEY case (reason: 'not_configured') — this must never look like an error to someone who just successfully saved a Request. Sends via a plain fetch against Resend’s REST API rather than adding the resend npm package, since this is the only call site so far. - Wired into CreateRequestForm.tsx’s handleSubmit: after the Request (and any staged Dialog entries) save, mints a response-link token via the existing owner-only issue_request_link RPC (migration 008 — the same call RequestDetailForm.tsx’s “Get Response Link” band already makes, just automatic here) and POSTs to the new route — fire-and-forget, wrapped so a failure anywhere in this block can never undo or surface as an error against the already-saved Request. - Tight-window advisory note added to Create Request only (not Request Detail) — Request Detail’s own “Send” button re-triggering the Initial email on an edit is a genuinely open question (resend every edit? only when Due Date changes? never?), not silently decided either way; flagged in docs/WYP_Week5_Plan.md rather than built.

Destination for the closing “set up a Free Account” link: /login, not a marketing product page — the PRD’s own text already says that destination “not yet built,” and CLAUDE.md’s Auth section is explicit that /login is the only account-creation entry point that exists (“there is no sign-up screen”). Chose this over inventing a new destination.

Not done this batch, explicitly deferred: the Reminder email’s own scheduled job (needs Vercel Cron vs. pg_cron decided first — still open) and the send path actually working end-to-end (needs the owner’s Resend API key and wouldyouplease.com’s SPF/DKIM/DMARC DNS, per his own “await my real MX information” instruction). npx tsc --noEmit and npm run lint both pass clean.

2026-08-12 — Landing page drafted (mobile-first), new design/marketing/ category

Owner: “Could you draft for me would be a landing page for Would You Please for opening the URL without any parameters - as a sales pitch to set up a Free Account. The existing sales literature could be used for reference... I have also attached a placeholder image which was created by google ai for a Facebook would you please account (I am not fond of the lettering colors - but the image on the right is along the right lines of presentation). The action item from the page would be the login/create account screen.”

Uploaded reference material: WYP onepager.html (the owner’s own letter-sized sales one-pager — headline, feature copy, \$17.95/yr subscription panel, “Coming soon” roadmap panel, footer, all reused near-verbatim) and a Facebook-

cover-sized AI-generated placeholder image (blue-to-teal gradient, “WOULD YOU PLEASE” in gold lettering, a stock photo of a person at a laptop, a device mockup with floating “SEND IT”/“TRACK IT”/“GET IT DONE!” badges).

Built design/marketing/WYP_landing_page.html — a new category in design/ (public-facing, full-width, responsive; not an app screen, so none of the palette-1 mobile-frame component classes apply) rather than dropping it into screens/ alongside the 480px app mockups. design/README.md gained a marketing/ row in the Folders table and its own “Marketing pages” status table.

Hero illustration — hand-built SVG, not the placeholder photo. The placeholder’s own “dashboard” content is AI-garbled nonsense text (“DeeFlines,” “Gcatifnes,” etc.) and there is no image-generation tool available in this session to produce a real replacement photo. Built a simplified, legible SVG rendering of the app’s own real Main Screen instead — rows, due dates, an Overdue row in red, a Done row muted, a search strip — with two floating callout badges (“Send It,” “Get it Done!”) echoing the placeholder’s own composition idea, which the owner did say he liked (“the image on the right is along the right lines of presentation”). Colors are strictly the canonical palette-1 tokens (brand blue, ink, ink-soft, alert-red) rather than the placeholder’s teal/gold, which also sidesteps rebuilding “gold lettering the owner already said he doesn’t like” from a different source. :root values are pulled from design/screens/tokens.css (the canonical, app/globals.css-derived copy), not the one-pager’s own hardcoded hex, so this file can’t quietly drift from the live app’s palette.

Mobile-first — a follow-up correction, not part of the original ask. The owner’s next message: “I forgot to mention that the primary people going to this landing page would be mobile phone users - so, the content/layout needs to keep that in consideration.” The page was rebuilt around that from the ground up rather than patched: every CSS rule outside the two @media (min-width: ...) blocks (600px, 900px) targets a narrow viewport by default, and those two blocks are what widens things for tablet/desktop, not the reverse. The hero specifically stacks the illustration below the headline/CTA on a phone, so the sales pitch and the button are what a mobile visitor sees first, and only moves the illustration beside the copy once there’s room (900px+) — matching the placeholder’s own desktop-oriented two-column composition at that width, but not before.

Action item, per the owner’s own instruction: both “Start Free Account” (primary button) and “Sign In” (secondary link) point to /login — the one live account-creation entry point that exists anywhere in the app (CLAUDE.md’s Auth section: “there is no sign-up screen”), same destination the Initial Request email’s own closing link already uses (see the entry above). This also finally gives the “sales page” a real file — design/README.md’s existing “Sign-up flow” diagram already showed one leading here, before this batch, with nothing built yet.

Verification: no headless browser is reachable from this session’s sandbox (playwright install chromium failed — “Connection blocked by network allowlist” — and no system Chromium/wkhtmltoimage is installed), so this could not be screenshotted for a visual check. Verified instead by parsing the file with Python’s html.parser to confirm every tag closes and pairs correctly (clean, no mismatches) and a brace-count check on the embedded stylesheet (78 open, 78 close). Flagged to the owner rather than silently skipped — worth an eyes-on pass in an actual browser before this is reviewed further or wired to a route.

Not done this batch, explicitly out of scope for a draft: wiring this as the actual / route for an unauthenticated, no-parameter visit (today / always renders RequireAuth → MainScreen, regardless of session state — making this page live would mean branching that route on auth state, a real architectural change not part of “draft... a landing page”); and porting it into the standalone/self-contained mockup convention some other screens use, since this file is already fully self-contained by construction (no external stylesheet dependency to begin with).

2026-08-12 — Email send path switched from Resend to Hostinger SMTP; credentials wired

Owner signed up for Hostinger’s mailbox hosting (their “Agentic Email” add-on) instead of Resend — the provider docs/WYP_Week5_Plan.md had assumed as a placeholder, never confirmed as a hard requirement. Asked what local-part to use for the sending address; recommended notifications@wouldyouplease.com (a system/transactional address rather than a personal one, matching the PRD’s own From display-name convention of putting the human’s

name in "<name> via Would You Please" rather than the address itself) over noreply@ (misleading here specifically, since Reply-To is set to the real sender's address) or requests@ (narrower than "notifications," which stays accurate as more automated email types get added). Owner registered exactly that address; no code change needed for EMAIL_FROM_ADDRESS in app/src/lib/email.ts, which already used it.

Owner then pasted Hostinger's full IMAP/SMTP connection details, including the mailbox password, directly into chat. **That password is not repeated in this log, in CLAUDE.md, or in any other committed file** — it went only into .env.local (git-ignored — .env* in .gitignore, confirmed via git check-ignore -v), which is the same file the app's existing Supabase keys already live in. Flagging this rather than treating it as routine: pasting a live credential into a chat transcript is generally worth avoiding where practical, though nothing further was done about it here since the owner had already sent it and rotating it wasn't asked for.

Rewrote app/api/email/send-request/route.ts: swapped the Resend fetch('https://api.resend.com/emails', ...) call for nodemailer over SMTP (smtp.hostinger.com:465, implicit TLS). Added export const runtime = 'nodejs' explicitly — nodemailer needs Node's net/tls modules, unavailable on Next's Edge runtime, and there was no code signal before this that the route depended on that. New env vars: EMAIL_SMTP_HOST, EMAIL_SMTP_PORT, EMAIL_SMTP_USER, EMAIL_SMTP_PASSWORD (replacing RESEND_API_KEY) — secure is derived from port === 465 rather than a fifth env var, since the two are never independent in practice (587 would be STARTTLS instead). Installed nodemailer/@types/nodemailer (npm install, 12 packages, npx tsc --noEmit and npm run lint both clean afterward) — npm audit flags 6 high-severity advisories, all pre-existing in next/eslint/postcss/sharp's own transitive deps, none introduced by this change.

Verification attempted, blocked by the sandbox's own network allowlist, not a credentials problem: ran nodemailer's transporter.verify() (an auth-only handshake, no email sent) directly against Hostinger from this session's sandbox — failed with getaddrinfo EAI_AGAIN smtp.hostinger.com, a DNS resolution failure, meaning this sandbox's outbound network doesn't reach that host at all, before authentication is ever attempted. This is an environment limitation specific to this cloud sandbox, not evidence the credentials or code are wrong — .env.local was written directly into C:\Project\wyp, the owner's own real project folder, so npm run dev on his own machine (unrestricted network) should be able to verify this for real; the deployed Vercel app can too, once the same 4 values are added there under Settings -> Environment Variables (Production + Preview) — not done from this session, since writing account/deployment settings wasn't explicitly asked for and the owner needs to do that step himself regardless (secrets shouldn't be typed by an agent into a dashboard on his behalf).

2026-08-13 — Live email test: all-day .ics for no Due Time; Subject's missing name is a data gap, not a bug

Owner tested the live send end-to-end (Gmail, localhost) and reported two things from the actual received email. First: Google's own inbox preview showed a 9:00–9:30 AM timed calendar event with an "Invite Others" button, on both Gmail and Outlook — "which for WYP purposes is 'noise'... Let's avoid that and set the due date with no time as a whole day event."

Fixed in app/src/lib/ics.ts's buildIcsContent — the only place any .ics gets built (Request Response, Response Detail, and the new send-request route all call this one function). Previously, a missing Due Time fell back to ICS_DEFAULT_DUE_TIME (9:00 AM) and always built a 30-minute timed VEVENT; that fabricated time is exactly what made Google/Outlook treat it as a meeting and offer to invite attendees. Now: no Due Time renders DTSTART;VALUE=DATE/DTEND;VALUE=DATE (RFC 5545 §3.3.4 all-day form, DTEND one calendar day past DTSTART per §3.6.1's exclusive-end-date rule) instead. A Due Time the sender did set is unaffected — still a timed event, unchanged. ICS_DEFAULT_DUE_TIME itself stays exported and in use elsewhere (app/src/lib/email.ts's isTightWindow, an unrelated "hours until due" calculation, not calendar rendering) — only buildIcsContent's own use of it as a calendar fallback time is gone.

Second: the email Subject read "A Would You Please Request, Due: 08-13-26" with no "from <name>" clause, though the owner is the sender. Checked the code before assuming a bug: buildRequestEmailSubject

(app/src/lib/email.ts) omits the “from <name>” clause specifically when ownerName is null — the exact behavior PRD §7.3 specifies (“matching the existing .ics description fallback”), and app/api/email/send-request/route.ts reads ownerName from profiles.display_name for the signed-in caller, same query CreateRequestForm.tsx already uses for Dialog’s own who field. So this reads as profiles.display_name being empty for whichever account sent this test, not a route bug — asked the owner to confirm directly (select id, email, display_name from profiles where email = '<test account email>';) rather than changing code against an unconfirmed cause. Not fixed pending that answer.

npx tsc --noEmit and npm run lint both clean after the .ics change.

2026-08-13 — Real bug found: profiles table completely empty; Create Free Account was silently no-oping

Follow-up to the missing-Subject-name investigation directly above (read that entry first — this one supersedes its tentative conclusion). Asked the owner to check profiles.display_name for his own account; the join query against auth.users on email returned zero rows. Had him run two simpler checks instead: select id, email from auth.users; (found exactly one row, his own — confirming the earlier join’s WHERE clause wasn’t the problem) and select id, display_name, tier from public.profiles; (returned **zero rows, full stop** — not just missing his row, the table has never held a single row).

Root cause, found by reading the code rather than guessing: handle_new_user() (migration 002) is supposed to insert a stub profiles row the moment auth.users gets a new row, via an AFTER INSERT trigger. Either that trigger never actually ran for this account or was never created in this database — not confirmed which, see below. Separately, and this is the part that actually mattered: CreateFreeAccountForm.tsx’s handleSubmit only ever ran supabase.from('profiles').update({...}).eq('id', user.id) — a plain UPDATE ... WHERE id = X. Postgres does **not** treat “the WHERE clause matched zero rows” as an error; it just returns success with nothing changed. So every time the owner completed Create Free Account, the screen said “Saving...” → redirected to / as if it worked, while nothing was ever actually written — for as long as the account has existed. This is why display_name (and every other profile field) has stayed empty through the whole session, not a one-off gap.

Fixed in app/components/CreateFreeAccountForm.tsx: the UPDATE now chains .select('id'), which forces Postgres to return the row(s) actually matched — an empty result is now distinguishable from success. If the UPDATE affected nothing, the code now falls back to a plain INSERT with the same id and field set, safe under the existing "profiles: insert own" RLS policy (with check (id = auth.uid()), migration 002) and not dependent on tier, which was never in this screen’s payload anyway. This makes the screen self-healing regardless of whether the trigger is ever fixed — worth doing either way, since relying on a trigger having fired correctly, with no verification and no fallback, is exactly the failure mode that produced this bug in the first place.

Not yet confirmed: whether handle_new_user’s trigger currently exists in the database at all (a migration a client-visible bug like this can hide behind for a long time, same category of risk as the migration-history-view gaps found in Week 4). Gave the owner a direct pg_trigger/information_schema check and a one-time backfill INSERT for his own row, to unblock him immediately without waiting on that answer. npx tsc --noEmit and npm run lint both clean.

Confirmed fixed, live: the owner went through Create Free Account again — not the manual backfill INSERT — and reported his name now shows correctly on Received Requests. That’s the handleSubmit INSERT-fallback path working end to end, the best possible confirmation of the code fix. Of the two trigger-check queries, only the function-existence one came back with a result (handle_new_user, one row); the trigger-existence query wasn’t reported, which reads as it having returned nothing — not confirmed either way, so **migration 014** (docs/Week5 - SQL history.txt) recreates on_auth_user_created idempotently (drop trigger if exists + create trigger, the same statement migration 002 itself used) rather than leaving the question open. **Confirmed run by the owner, 2026-08-13.** Fixes new sign-ups going forward if the trigger really was missing; doesn’t retroactively fix any other account that

might have signed up during the gap — for this app, at this stage, that’s only the owner’s own account, already fixed via the app fix above.

2026-08-13 — Live send confirmed on Vercel; Outlook .ics rejection fixed; landing page badge fixes

Owner confirmed the Vercel-deployed app now sends real email (env vars added there per the earlier ask) and reported two further issues from testing.

Outlook rejected the .ics attachment — “Invalid ICAL element: Inbound Mime method and ICAL method mismatch. Invalid ICAL element: VCALENDAR” — on both mobile and web Outlook, while Gmail accepted the identical file without complaint. Root cause: `app/api/email/send-request/route.ts` declared the attachment as `text/calendar; method=REQUEST`, but the VCALENDAR body itself (`buildIcsContent`, `app/src/lib/ics.ts`) carried no `METHOD` property at all — Outlook cross-checks the MIME-declared method against the body and rejects on a mismatch; Gmail apparently doesn’t check. `REQUEST` was also the wrong iTIP method on its own terms regardless of the mismatch: it means “meeting invitation, expects an ORGANIZER/ATTENDEE who can accept or decline,” and this event has neither — a WYP Request’s due date was never a meeting. Fixed both sides to agree: `ics.ts` now writes `METHOD:PUBLISH` into the VCALENDAR body (the correct iTIP method for a one-way informational entry), and the route’s attachment `contentType` changed to `method=PUBLISH` to match. The two direct-download call sites (`RequestResponseForm.tsx`, `ResponseDetailForm.tsx`) never declared a MIME method in the first place (plain Blob download, not an email attachment), so they were never affected and needed no change. Owner separately found he could add the event to Outlook manually by opening the attachment — useful confirmation the file itself wasn’t corrupt, just the method declaration.

Landing page hero badges — two issues from a visual review of the drafted SVG illustration (`design/marketing/WYP_landing_page.html`). (1) The “Get it Done!” badge’s second line, “Confirmed & closed out,” overflowed its 128px-wide rect to the right — widened to 200px and shifted left (translate 288,268 → 216,264) to stay within the 440-wide `viewBox` with margin to spare; text/icon positions shifted to match. (2) Owner: the Track It badge’s teal fill (`#1C8FA0`) is the same color as the hero gradient’s own right-hand end, so it barely registered against the background, and the white Get it Done badge “visually runs together with the gray and white image behind it” (the dashboard illustration it floats over). Fixed by adding a 2px `#123B7A` border to both — the same navy the Send It badge already uses as its own fill, which is why Send It didn’t have this problem and needed no change. Verified structurally (tag-balance check) rather than visually — no headless browser reachable from this session’s sandbox, same limitation noted when this file was first drafted.

`npx tsc --noEmit` and `npm run lint` both clean after the `.ics/route` changes.

2026-08-13 — Add to Calendar hidden when the visitor arrived via the calendar’s own link

Owner: “Is there a way to add information in the .ics link so the app knows the Request Response came from a calendar click so that the ‘Add to Calendar’ button would not be shown to the end-user.” Yes — a query-string marker on the link embedded inside the .ics itself, read back once the recipient’s click lands them on the response page.

`app/src/lib/ics.ts`: `buildIcsContent` now runs every link it embeds (both the VEVENT’s own URL property and the inline link inside `DESCRIPTION`) through a new `calendarLinkFor()`, which appends `?src=calendar` (idempotent — checks first, won’t double-append). A matching `cameFromCalendarLink(search)` reads it back from a `location.search`-shaped string. Applied unconditionally inside `buildIcsContent`, so this covers both the emailed .ics (server-side, `app/api/email/send-request/route.ts`) and the client-side “Add to Calendar” button’s own manually-downloaded .ics (`handleAddToCalendar` in both response forms, which passes `window.location.href` as the link) — a manually re-downloaded file deserves the same marker on its own embedded link, so a future visit via *that* link also hides the button.

`RequestResponseForm.tsx` (anonymous `/r/[token]`) and `ResponseDetailForm.tsx` (signed-in `/requests/[id]/respond`): both read `cameFromCalendarLink(window.location.search)` once via a lazy `useState` initializer, guarded `typeof window === 'undefined'` the same way `MainScreen.tsx`'s existing `sessionStorage` lazy initializers already are (`readStoredChip`) — consistent with an established pattern rather than a new one. The `.panelact.panelact-top` Add to Calendar row is hidden entirely (not just disabled) when `true`, rather than left as dead space.

Explicitly a per-click signal, not a persistent “already added” flag — Request links are multi-use (CLAUDE.md, Database section), so the same recipient can still reach this page via the original, unmarked email link after already adding the event once via the calendar link, and the button will show again. Judged the right trade-off: a false “not yet added” just means the button reappears (harmless); a false “already added” would hide a button someone genuinely still needed, with no way back to it short of re-requesting the email. `npx tsc --noEmit` and `npm run lint` both clean.

Confirmed working, live: the owner verified the button drops when the link is clicked from inside an actual `.ics` file.

2026-08-13 — Landing page becomes the live, unauthenticated / route

Owner: “When I open the `https://wyp-three.vercel.app` URL in a browser, I am directed to `https://wyp-three.vercel.app/login` and presented with the sign-in page... I want to have people come to the website and see the `WYP_landing_page.html`... If logging-in was an every-time activity, it would be a nuisance... but my understanding is that a login is only needed on a per-device basis. So, can we make the `WYP_landing_page.html` the default 1st page?” His understanding is correct — “Keep Me Signed In” persists the session via `localStorage` (`supabaseClient.ts`'s existing `REMEMBER_KEY/hybridStorage`), so a returning user on the same device stays signed in and lands on `MainScreen` exactly as before; only a new or signed-out visitor now sees the landing page instead of a forced bounce to `/login`.

`app/page.tsx` no longer wraps `/` in `RequireAuth`, whose `router.replace('/login')` on no-session is unconditional (see `app/RequireAuth.tsx`) — that's precisely the behavior being changed. Instead `page.tsx` calls `supabase.auth.getUser()` itself and renders `MainScreen` when a session exists, `LandingPage` otherwise, with a brief `Loading...` state in between. Every other authenticated route in the app (`/requests/[id]`, `/todos/[id]`, `/contacts`, etc.) keeps using `RequireAuth` unchanged — this carve-out is specific to `/`.

`app/components/LandingPage.tsx` and `app/components/landing.css` (new) convert `design/marketing/WYP_landing_page.html` into a live component. Content, copy, and the hand-built hero SVG are carried over verbatim — only mechanical JSX conversions were made: `class`→`className`; kebab-case SVG presentation attributes→camelCase (`stroke-width`→`strokeWidth`, `text-anchor`→`textAnchor`, `font-family`→`fontFamily`, etc.); `<a href="/login">`→`next/link`'s `<Link>` (already the established pattern in `RequestResponseForm.tsx`, `MainScreen.tsx`, `ContactsList.tsx`); no re-added Google Fonts `<link>`, since `app/layout.tsx` already self-hosts Inter via `next/font` (`--font-inter`) app-wide. Two JSX-text characters needed a decision the mockup's raw HTML didn't: the apostrophe in “your phone's browser” and the straight double quotes around “did you get to it?” in the Formal Responses card both trip `react/no-unescaped-entities`. Checked for an existing curly-quote convention elsewhere in `app/components` first (none found) rather than guessing — resolved by using real typographic characters directly in the JSX source (`'` and `“`) rather than HTML entities (`'` and `"`), which reads more like the actual marketing copy this is and needs no special-casing if the same question comes up again in future copy.

CSS scoping: `app/globals.css` is imported once, globally, in `app/layout.tsx`, so a plain CSS import in `LandingPage.tsx` would otherwise apply everywhere. Checked every landing-page class name against `globals.css` (`grep`) and found exactly one real collision: `.panel` (the app's own `Dialog/Attachments` panel, `globals.css` line 1877) versus the landing page's own `subscription/roadmap` panels. Fixed two ways at once, deliberately belt-and-suspenders: every selector in `landing.css` is scoped under a `.wyp-landing` root wrapper (specificity alone would still let `globals.css`'s unscoped `.panel` rule contribute any property `landing.css` doesn't itself override, since

class-name matching doesn't care about ancestry), and the landing page's own panels are additionally renamed `.lpanel` so there's no collision to rely on specificity for in the first place. `:root` custom properties (`--brand-blue`, `--ink`, etc.) are not redeclared — `globals.css`'s own `:root` already defines every one this page uses, and duplicating them risked a second copy drifting from the canonical set; same reasoning for not re-reporting the mockup's own global `*/body/html` resets, which are scoped onto the `.wyp-landing` container instead (`font-family/color/line-height`) or dropped (`html { scroll-behavior: smooth }`), judged not worth a site-wide side effect for one page).

`npx tsc --noEmit` and `npm run lint` both clean. Not yet visually verified against a real browser — no headless browser reachable in this session's sandbox (same limitation as the landing page's own earlier badge fixes); structurally sound (typechecks, lints, and is a close mechanical port of an already-reviewed mockup), but worth the owner's own look once deployed.

2026-08-13 — Landing page header/hero redesigned for phone; “Sign In for Free Account” title variant

Owner tested the live landing page on his phone and reported several related problems in one message. The header's single row (logo + wordmark + both CTA buttons) broke down on a narrow viewport: the logo failed to render at all, “Start Free Account” truncated mid-word (readable only by scrolling the page sideways), and “Sign In” rendered as plain text easily misread as a caption under “Start Free Account” rather than its own button. Separately, on tablet/desktop the same logo+wordmark read too small once stretched across a full-width page (they'd been sized to the mockup's own original small header, never scaled up at the existing 600px/900px breakpoints). He also flagged that landing on a plain “Sign In” screen right after clicking “Start Free Account” is a little jarring, since it's the same one screen either way (no separate sign-up form exists).

Fix, matching the owner's own described redesign directly: the header now shows logo + wordmark + a “Tracking Requests and Todos” tagline (the same tagline `WypHeader.tsx` already uses elsewhere in the app) and nothing else — no CTA buttons competing for width, which is what let the logo get squeezed to invisible and the button text clip in the first place. `Start Free Account` and `Sign In` moved into the hero itself, stacked in a column (`.hero-btns`) beside three explicit headline lines (“Send it.”, “Track it.”, “Get it Done.” — each its own line, not one string relying on the browser to wrap it well) rather than below the lede paragraph. Owner's own reasoning, quoted: “The width of the ‘Start Free Account’ button when placed adjacent to the ‘Get it Done’ line works on a vertical columnar basis as long as there is either no gap or a very little gap between those columns” — `.hero-top` uses a 4px gap between the two columns for exactly this reason.

Button colors went through a few iterations in the owner's own testing before landing on: `Start Free Account` (`hero-top`, small) is light-blue (`.btn-tint`, `var(--strip)` background, `brand-blue` text) and `Sign In` (`hero-top`, small) is white (`.btn-white`, reused, just smaller here) — his own words: “With the light blue for the Start Free Account it looks better when on the same page as the larger white background version deeper in the text,” referring to the final CTA band's own, larger, white “Start Free Account” button further down the same page — two identical white buttons at different sizes on one page would have read as redundant rather than as a hierarchy. The headline lines themselves stay white (`.hero-line`), same as the hero always was — the owner tried black and a dark-blue variant while tinkering but settled back on white as most integrated with the gradient background.

Logo sizing: `.brandmark svg` gained explicit `width/height` attributes (42/40) in addition to the CSS-driven height, as a defensive fix alongside removing the buttons — `app/components/WypHeader.tsx`'s identical inline SVG has never needed this elsewhere in the app, so the header row's own overflow (not a rendering bug in the SVG itself) was almost certainly the real cause; the explicit attributes cost nothing and remove any residual risk. Header and hero-line sizes now scale up at both existing breakpoints (600px logo 34→40px, hero-line 24→30px; 900px logo →46px, hero-line →38px), rather than staying fixed at the original small mobile-only values the owner found too small once stretched wide.

`/login?intent=signup`: `LandingPage.tsx`'s `Start Free Account` links (`hero-top` and the final CTA band) now carry `?intent=signup`; `Request Response`'s existing “Create your own Free Account” link

(`RequestResponseForm.tsx`) got the same param for consistency, since it's the same signup entry point. `app/login/page.tsx` reads it via `window.location.search` in a lazy `useState` initializer — same precedent as `AddContactForm.tsx`'s `?from=create-request` param, avoiding a `useSearchParams()` Suspense-boundary requirement this page doesn't otherwise need — and swaps the band label to “Sign In for Free Account” when present, “Sign In” otherwise. Deliberately not the older mockup's proposed “Start Free Account” band label (`design/screens/WYP_signin_palette1_floating.html`'s dual-mode demo, never wired live) — the owner's own suggested wording is more accurate here, since this screen only ever collects an email address and was never a real sign-up form.

Ported into `design/marketing/WYP_landing_page.html` (header, hero-top markup and CSS, both breakpoints, both Start Free Account hrefs) so the mockup and the live component stay in sync, per this project's usual convention. `npx tsc --noEmit` and `npm run lint` both clean. Not yet visually verified against a real phone — no headless browser reachable in this session's sandbox, same limitation as every other landing-page visual change; worth the owner's own look once deployed.

2026-08-13 — Open chip was excluding Overdue rows on Sent/Received/Todos

Owner: “Open chip displayed items do not include Overdue items - which should be shown - because they are open.” Root cause: `statusFor()` (added 2026-08-12 for Todos, already governing Sent/Received) returns a three-way exclusive status — `open / overdue / done` — and the three filter predicates in `MainScreen.tsx` compared the Open chip against that status with plain equality, so an Overdue row (status `'overdue'`) never matched the `'open'` filter. Overdue is a stricter subset of “not done,” not a sibling category disjoint from Open — the Overdue chip's own job is narrowing to just that subset, which already worked correctly and is unchanged.

Fixed with one shared `matchesStatusFilter(status, filter)` helper, used by all three `filteredSent/filteredReceived/filteredTodos` predicates: `filter === 'open'` now matches both `'open'` and `'overdue'`; `'overdue'/'done'/'all'` behave as before. `npx tsc --noEmit` and `npm run lint` both clean.

2026-08-13 — Private-testing signup gate (migration 015, drafted)

Owner: “could there be a way to set up test users [maybe with SQL] as ‘allowed to create a Free Account’? That way, during testing with a small group of users, the app testing group will not [be at] risk [of] an unexpected expansion.” Confirmed the scope in a follow-up: only brand-new signups should be gated (a returning user, including his own account, is never affected), the blocked-message strategy is preferred over a silent failure, and the exact wording to show: “This app is currently in a private testing mode with a limited number of users. If you would like to participate in this testing process, let us know how you heard about Would You Please and introduce yourself in an email to `notifications@wouldyouplease.com`” — with the email address as a clickable `mailto:` link. He also asked for the gate itself to be toggleable with a plain SQL statement against a control table, for when testing moves to public launch.

Migration 015 (`docs/Week5 - SQL history.txt`, drafted, not yet confirmed run) adds two new tables and one function. `app_settings` is a plain key/value control table — `update app_settings set value = false where key = 'signup_gate_enabled'`; turns the gate off with no redeploy, matching the owner's own ask verbatim. `beta_allowlist` is one row per invited email, managed the same way with a plain `insert`. `can_create_account(p_email)` is the only thing the client ever calls: it always returns `true` if the email already exists in `auth.users` — a returning user is never gated, regardless of the allowlist or the gate setting, which is the “only brand new signups” scoping the owner confirmed — and otherwise returns `true` only if the gate is off or the email is on the allowlist. `SECURITY DEFINER, revoke all ... from public then explicit grant execute to anon/authenticated`, same posture as every other function in this file; `anon` never gets direct access to `auth.users`, `app_settings`, or `beta_allowlist` — only a yes/no boolean back through this one function.

`app/login/page.tsx`: `handleSubmit` now calls `can_create_account` before ever calling `signInWithOtp` — gating has to happen before that call, not after, since `signInWithOtp` is what actually creates the `auth.users` row and sends a real email. A blocked email shows a new gated screen state (parallel to the existing sent state) with the owner’s exact wording and a `mailto:notifications@wouldyouplease.com` link (a prefilled subject was added as a small, low-risk nicety — “Would You Please — Testing Access” — the owner asked only for the link to be clickable, not for a specific subject, so this can be dropped or changed easily). The band label shows “Private Testing” while gated, overriding the existing `intent=signup` wording. A “Try a different email” link resets back to the form, reusing the existing `startOver()` handler (now also clearing gated).

Considered and rejected: gating at the Auth-Hook layer instead of in the client — more airtight against a direct API call bypassing the UI, but a real setup lift (a Postgres function wired into Supabase’s hook config, or an external HTTPS endpoint) that this short testing phase doesn’t need; worth revisiting if the gate needs to live longer-term or needs to be bypass-proof rather than just UI-level. Gating at Create Free Account instead of at sign-in was rejected earlier in the same conversation — by then the `auth.users` row and a real emailed link already exist.

Migration 015 has not been run yet — flagged, not assumed. `npx tsc --noEmit` and `npm run lint` both clean.

2026-08-13 — `/login`’s intent param going stale on client-side navigation; Private Testing copy revised twice

Owner-reported, with a screenshot: the address bar showed `/login?intent=signup` but the band still read plain “Sign In.” Root cause: the `isSignupIntent` fix from earlier the same day read `window.location.search` once via a lazy `useState` initializer, which only runs on that component instance’s original mount — a real gap once it’s possible to reach `/login` via a same-route, params-only client-side navigation (Next’s router can reuse an already-mounted `/login` instance rather than remounting it fresh for every link click). Fixed properly with `useSearchParams()` from `next/navigation`, which subscribes to the router’s own search-params state and re-renders on every change regardless of mount history — this requires a `Suspense` boundary around anything that calls it, so `app/login/page.tsx` now default-exports a thin wrapper (`<Suspense><LoginScreen /></Suspense>`) with the actual screen moved into `LoginScreen.AddContactForm.tsx`’s own `?from=create-request` read (still `window.location.search`, not `useSearchParams()`) doesn’t share this bug — that value is read live inside a click handler at the moment of the click, never cached in state across renders, so it can’t go stale the way a mount-only initializer can.

Private Testing paragraph revised twice, same day. First addition, owner’s own wording verbatim: “If your participation is approved, you will receive an email explaining the Private Testing process, related limitations, the expected testing duration, and a ‘Start a Free Account’ link to click” — added as a new paragraph directly after the participation-instructions paragraph. Second, the participation-instructions paragraph itself was rewritten, again the owner’s own wording verbatim: “If you would like to participate in this testing process, let us know in an email to `notifications@wouldyouplease.com` the following information: your first name, how you heard about Would You Please, and a short introduction” — replacing the earlier phrasing (“let us know how you heard about Would You Please and introduce yourself”), now itemizing three specific pieces of information rather than two general asks. The `mailto:` link and its prefilled subject are unchanged.

`npx tsc --noEmit` and `npm run lint` both clean.

2026-08-13 — Supabase Auth Custom SMTP configured (Hostinger), confirmed working

Owner was hitting Supabase’s built-in-mailer limits testing sign-up/login: 2 magic-link emails per hour, and — more restrictively — delivery refused entirely to any address that isn’t a member of the Supabase project’s own team, which would have blocked every outside tester regardless of rate. Fixed by enabling Custom SMTP in the Supabase dashboard (Authentication → SMTP Settings) using the same Hostinger mailbox (`notifications@wouldyouplease.com`) already used for WYP’s own Request-notification emails — a separate configuration from that one, since Supabase Auth’s own mailer and `app/api/email/send-request/route.ts`’s `nodemailer` transport are two independent systems

that happen to share the same mailbox. Confirmed working: the owner had typo'd the sender name as "Woudl You Please" when first entering the SMTP settings and saw that exact typo in a received magic-link email, which he then corrected — solid proof the emails are actually flowing through the new Hostinger connection rather than the old built-in mailer. No code change in this repo — this is entirely a Supabase project setting, not an env var or file WYP itself controls.

2026-08-13 — Three small live-testing fixes: dead print icons, session-check flakiness, per-account chip persistence

Three separate owner reports in one round of testing.

Main Screen's Print Sent/Received/ToDos icons did nothing. Root cause, found by reading the code: they were `<span role="button" tabIndex={0}>` elements with no `onClick` at all — apparently never wired when Main Screen was converted, unlike Create Request's own Print icon (`<button onClick={() => window.print()}>`), which the owner correctly remembered working. Fixed by matching that exact working pattern on all three (`MainScreen.tsx`).

Closing the browser while signed in, then reopening it later, sometimes showed the landing page instead of Main Screen — then correctly showed Main Screen again on a later visit with no action taken in between. Root cause: `app/page.tsx`'s anonymous-vs-signed-in check (added 2026-08-13, the landing-page routing batch) used `supabase.auth.getUser()`, which makes a live round-trip to Supabase's Auth server to validate the current token — and the code treated *any* failure of that call, including a transient network hiccup right after the browser reopens, as "not signed in." `RequireAuth.tsx` had the identical bug on every other protected route in the app, just never reported because it fails toward `/login` rather than a distinct landing page. Both switched to `supabase.auth.getSession()`, which reads the already-initialized/refreshed local session instead of making its own network call — this is also already the pattern `app/login/page.tsx`'s own already-signed-in check uses, so both files now match an existing convention rather than diverging from it. Not a security change: this only decides which screen a client renders, and every actual data read or write still goes through Supabase's RLS/JWT verification server-side regardless of which check picked the screen.

Main Screen's filter chips (Sent/Received/ToDo, Housekeeping's Tasks/How-to Videos tab) reset to their hardcoded defaults on every visit. This was deliberate as of 2026-08-09 ("a within-session view preference, fine to reset once the tab closes") — the owner is now asking for more: "keep track of the chip settings last-used for an account user... these defaults should only be used the first time an account user sees the main screen," explicitly flagged as new, not a correction of that earlier decision. Added migration 016 (`docs/Week5 - SQL history.txt`, drafted, not yet confirmed run): one `profiles.main_chip_prefs jsonb not null default '{}'::jsonb` column plus a column-level `grant update (main_chip_prefs) to authenticated`, same pattern as migration 013's `time_zone` grant — the existing `"profiles: update own"` RLS policy (migration 002) already scopes the row, the grant is what allows the column at all. One `jsonb` column rather than four separate ones, since this is UI view state that may grow more fields later, not a modeled business fact.

`MainScreen.tsx` keeps the existing `sessionStorage` read/write exactly as it was — it's still valuable as a synchronous fast path so a quick round trip to a Detail screen and back shows the right chips instantly, with no flash of the default state while the slower DB read is in flight. A new one-time effect fetches `profiles.main_chip_prefs` on mount and applies any saved values on top of whatever already rendered; a second effect writes the current four values back to the same column on every change, but only once the initial load has actually resolved (`prefsLoaded`), so a save can never fire with default values before the real saved prefs have been read and stomp them. An empty `{}` — a brand-new account's starting value — is the one and only condition under which the hardcoded defaults (`all`, `all`, `open`, `tasks`) apply; once any value is ever saved for an account, it's used from then on, on any device or session.

`npx tsc --noEmit` and `npm run lint` both clean. Migration 016 has not been run yet.

2026-08-13 — Fix Main Screen scroll position not surviving Add/Edit ToDo

Owner: “the ‘back’ does not work as expected, e.g., when I add or edit a ToDo and cancel or save, it returns to the top of the screen and shows Requests Sent.”

Two distinct bugs, both on the same symptom.

Root cause, the real one: `.scroll` (`app/globals.css`) is an internally-scrolling `overflow-y: auto` div wrapping all three Main Screen sections — it is not the browser window. `router.back()`’s scroll restoration (the fix applied to Request/ToDo/Contact Detail on 2026-08-09, and believed at the time to be sufficient — see that date’s own entry) only ever restores `window.scrollToY` across a client-side navigation; browsers do not track or restore an arbitrary element’s own `scrollTop`. Since Main Screen remounts fresh on the way back regardless of `back()` vs `push()`, that inner div’s `scrollTop` was always reset to 0 — “top of the screen, Requests Sent” is just literally the top of that div’s content. Fixed by adding explicit persistence: `MainScreen.tsx` now writes the `.scroll` div’s `scrollTop` to `sessionStorage` (`wyp.mainScrollTop`) on every scroll event, and restores it once — after loading first turns false on a fresh mount, so the restore lands after real row heights are in place rather than against the shorter “Loading...” placeholder. Same tier choice as the existing chip-filter/sort persistence: `sessionStorage`, not the database — this is moment-to-moment scroll position, not a preference worth carrying across devices.

Second, smaller bug, Create ToDo only: `CreateToDoForm.tsx`’s Save and Cancel both used `router.push('/')`, not `router.back()` — the one create-a-new-item screen that never got the 2026-08-09 Detail-screen convention applied (Request Detail/ToDo Detail/Contact Detail are all *edit* screens reached by clicking an existing row, and already used `back()`; Create ToDo is reached by a button, and was written independently). `push('/')` creates a brand-new history entry rather than returning to one, which also meant clicking the browser’s own Back button afterward didn’t behave as expected either. Fixed to match: `router.back()` on both. This explains why the owner’s report named “add or edit” together — add was double-broken (wrong navigation call *and* no scroll restore), edit was singly broken (right call, but nothing to restore).

2026-08-13 — Main Screen Print Reports (§6.34 PROPOSED)

Owner uploaded `Main Screen sections.xlsx`, a mockup of what the printed report for each Main Screen section (Requests Sent, Requests Received, ToDos) should look like, reporting that the current Print output “only shows what can fit onto a page.” Asked whether an HTML-based approach sent to a selected printer is feasible, since browsers can’t natively render spreadsheets.

Root cause of the truncation complaint: Main Screen’s row lists sit inside `.scroll`, an internally-scrolling `overflow-y: auto` div — printing the live page can only ever capture whatever currently fits that scrollable viewport, not its full scrollable content. Confirmed the HTML + native `window.print()` approach is entirely sufficient (no platform-specific work needed on Windows/Mac × Edge/Chrome/Safari; printer selection happens in the OS’s own print dialog, already triggered by `window.print()`) — the actual fix is a dedicated print-only layout, not a different technology.

Design, from inspecting the `xlsx` cell-by-cell (`openpyxl`): a masthead (“Would You Please” left, generation timestamp right, thin rule beneath), a bold section title naming the current filter chip (“Requests Sent — Open”), a shaded/bordered column-header row with a real $\wedge/v$ arrow on the active sort column (the mockup’s own literal “#” was a placeholder — its own in-sheet comment said so directly), and per-item rows with fully unclipped wrap-text descriptions (no 2-line truncation, unlike the on-screen `.desc`), the Dialog icon reused at the start of the description when present, a red font for an example Overdue row (confirming the existing red-overdue convention should carry into print), and an example row with an 8:30 AM sub-line under Due (confirming Due Time support is wanted). The ToDos section’s own header and rows show only Description, Due, and Done — no Priority or Category column at all, a deliberate divergence from the on-screen ToDos row (which shows Priority/Category and no dates at all) rather than an oversight, followed literally.

Chip + sort behavior, confirmed directly: “The print should follow the chip and sort set for the section by the

user.” Implemented by sourcing each report from `sortedSent/sortedReceived/sortedTodos` — the same already-filtered-and-sorted arrays the live rows themselves render from — so no separate filtering/sorting logic was needed; the report is always exactly what’s currently on screen (chip and sort-wise), just laid out for paper instead of a phone viewport.

Attachments icon: owner asked whether the Attachments icon could appear alongside Dialog’s at the start of the description, matching the screen. Dialog: yes, trivial, reused directly (`dialogCount(r.dialog) > 0/r.dialog_count > 0`). Attachments: not yet possible — no attachment data model exists anywhere in this app (Priority 3, still pending) — so there is no icon slot to add for it yet, only the Dialog one.

Mechanism: a `printSection` state (`'sent' | 'received' | 'todos' | null`) plus `printGeneratedAt` (captured once per click, so the masthead timestamp reflects the click moment rather than whatever instant a later re-render happens to land on). Clicking a Print button calls `startPrint(section)`, which sets both; an effect on `printSection` calls `window.print()` once the resulting `.print-report` JSX has actually committed to the DOM (effects run post-paint, so this can’t race a stale render), and an `afterprint` listener resets both back to null once the browser’s print dialog closes. `.no-print` (added to the existing `.app` wrapper) and `.print-report` are mutually exclusive via `@media print in globals.css`, so exactly one of “the live app” or “the current report” is ever visible, on screen or on paper, never both.

Due Time: added to `SentRow/ReceivedRow` (not `TodoRow` — `Todos` have never had a Due Time field anywhere in this app) and to the `Sent` query’s plain `.select()` string directly (the column already existed on requests). `Received` goes through `get_received_requests()` (migration 012) instead, whose return table had to be extended explicitly — migration 017.

Migration 017’s first draft failed when the owner ran it: ERROR: 42P13: cannot change return type of existing function / DETAIL: Row type defined by OUT parameters is different. / HINT: Use DROP FUNCTION `get_received_requests()` first. The draft used `create or replace` function, on the mistaken assumption — read from migration 011’s own `owner_tier` addition, which happened to work — that Postgres allows a RETURNS TABLE function’s row shape to change freely under CREATE OR REPLACE. It doesn’t: RETURNS TABLE is implemented as a set of OUT parameters, and CREATE OR REPLACE can never alter that list once the function exists. Migration 011 only worked because `owner_tier` was appended last; migration 017 inserted `due_time` in the middle (between `due_date` and `done_date`), which is what triggered it — though per Postgres’s own behavior here, even an append-only change to the OUT list can require a drop in some cases, so DROP FUNCTION first is the reliable fix regardless of column position. Corrected: drop function if exists followed by a plain create function, with the `revoke/grant execute` pair re-run afterward, since dropping a function also drops its own grants. **Re-run by the owner 2026-08-13 — confirmed working.** Print Received (and the Received subcard generally) can now show a Due Time sub-line for a Request that has one.

Not yet ported into any mockup — this is a print-only layout with no natural on-screen equivalent to draw in `design/screens/`; flagged in `design/README.md`’s status table (§6.34 PROPOSED) rather than silently left undocumented.

2026-08-13 — Landing page lede: avoid opening a sentence with the app name

Owner: starting the second sentence of the lede with “Would You Please turns every ask...” reads, on first pass, like the sentence itself begins with a request (“Would...”) before the reader gets far enough to recognize it as the app’s name — awkward. Reworded to “With Would You Please, every ask is turned into a structured, trackable request...” (owner’s own wording, one small edit: fixed a doubled space). Applied to both `LandingPage.tsx` and `design/marketing/WYP_landing_page.html`, kept in sync per this project’s usual convention.

2026-08-13 — Private Category becomes an opt-in account preference

Owner: “In the interest of keeping this app as simple as possible, I think the Private Category should be an account option, not a standard presented data element... available for Free Accounts, but only if they turn it on... A single option could control its availability for both Requests and Todos.”

Not a tier gate. Any account, free or subscriber, may turn Private Category on — this is a decluttering preference, not an entitlement, unlike Request Texting or Attachments elsewhere in this app. Off by default: a brand-new account never sees Category anywhere until it’s explicitly turned on, at which point the field behaves exactly as it always has, including the existing “no Categories yet — use Add Category” empty-state copy, which needed no changes at all.

Where the toggle lives, and why that needed asking. Account has been left deliberately unbuilt every time it’s come up — “intentionally undesigned pending further product evolution,” most recently reaffirmed in the Create Free Account batch. Offered a straight choice — build a minimal sliver of Account now (just this one control) versus add the toggle to Housekeeping instead and leave Account untouched — the owner picked building Account. `AccountForm.tsx` (new) is not a conversion of the existing `WYP_your_account_palette1_floating.html` mockup; it’s a new, deliberately minimal screen with exactly one control, reusing the app’s existing `.checkrow` component (first used by “Keep me signed in” on Sign In) rather than inventing a new one. Auto-saves on toggle (no separate Save step — nothing to stage for a single boolean), with an optimistic-then-reverted UI update on a failed write. Housekeeping’s “Account” row (Main Screen) now navigates to `/account`, ending its long run as an intentionally inert row.

Migration 018 — `profiles.private_category_enabled` boolean not null default false, plus the column-level grant to authenticated (same pattern as migrations 013/016). Confirmed run by the owner 2026-08-13 — the toggle at `/account` is live end to end.

Gated everywhere Category currently appears, reading the new flag off the same profiles round trip each screen already made for `display_name/main_chip_prefs` rather than a separate query: - Create Request, Create ToDo, Request Detail, ToDo Detail: the entire Category `.fgroup` (lookup field + Add Category button + dropdown) is not rendered at all when off — genuinely simpler, not a locked/upsell control, matching the owner’s own stated goal. - Main Screen’s Todos section: the colbar’s “Category — Description” column becomes plain “Description” text (no longer clickable — sorting by a hidden field doesn’t make sense with nothing to click into it), and each ToDo row drops its `<span className="cat">` segment, keeping the “Priority — Description” dash for readability either way. - Sent/Received (Main Screen and both print reports) never showed Category to begin with — nothing to gate there.

Edge cases flagged, not specially handled: (1) a Request/ToDo that already has a `category_id` set from before the account turned Category off keeps that value in the database untouched — it’s simply not shown or editable until Category is turned back on, same as any other hidden field. (2) If a ToDo’s persisted sort state (`sessionStorage`) was category from before the toggle was turned off, `sortedTodos` still technically sorts by the now-hidden category name — a minor, low-priority inconsistency (the column to re-select a different sort is still there, just relabeled), not worth the added complexity of forcing a sort-key reset on toggle-off for a single-user testing app at this stage.

Not touched: none of the five mockups these screens were converted from (Create Request, Create ToDo, Request Detail, ToDo Detail, Main Screen) were updated — Category has always been drawn as always-present in each one, and porting a conditional toggle state into static demo HTML wasn’t asked for. Flagged in `design/README.md`, not silently skipped.

2026-08-13 — Due/Done Time becomes an opt-in account preference (Requests)

Owner, immediately after Private Category shipped: “A similar modification for simplification can be done for the Due Time and Done Time for Requests without much of a complication (I think). As another account option, when turned off the four-value two-line presentation of Due Date Due Time Done Date Done Time on Requests would become like a ToDo one-line two-value presentation of Due Date and Done Date.”

More complex than the owner’s own estimate — flagged before starting, not silently over- or under-built. Unlike Category (which only ever appears on screens the *issuer* controls — Create Request, Request Detail), Due Time

and Done Time are also shown on the *recipient's* own screens for the same Request: Request Response (the anonymous `/r/[token]` link) and Response Detail (a signed-in recipient's equivalent). This app's own standing rule — CLAUDE.md's Entitlements section, "rights on a request come from its issuer, never from whoever is reading it" — already settled this exact question for Attachments (`owner_tier`, migration 011): a display setting like this has to be read from the *sender's* account, not the viewer's own, or a subscriber recipient would see a different presentation than a free recipient for the identical Request, which is backwards. Offered a straight choice — extend the RPC functions to carry the issuer's own setting through to recipient screens (more migrations, fully consistent) versus scope this to the two owner-only screens only (simpler, but Request Response/Response Detail would keep showing Due/Done Time regardless of what the issuer chose) — the owner picked the full, recipient-inclusive scope.

Default true, not false like Category. Category was a brand-new field nobody had ever seen — safe to default off. Due Time/Done Time is the opposite: pre-existing, already-relied-upon behavior (Print Reports' Due Time sub-line, the `.ics` calendar builder's own Due Time handling) that many already-created Requests already have real data in. Defaulting this off would have silently hidden already-set data for every existing account the moment the migration ran, with no owner action taken. On by default, so nothing changes unless an account explicitly opts out — documented directly in migration 019's own header comment for future reference, since the two features' opposite defaults could otherwise look like an inconsistency rather than a deliberate distinction.

Migration 019 — `profiles.request_time_enabled` boolean not null default true, plus the column-level grant to authenticated, same pattern as migration 018. **Migration 020** extends `get_request_by_token` and `get_received_request` (both returns `jsonb`) with `owner_request_time_enabled`, safe as a plain `create` or `replace` function — no OUT-parameter constraint applies to a `jsonb`-returning function, unlike a `RETURNS TABLE` one. **Migration 021** extends `get_received_requests()` the same way, but that function *is* `RETURNS TABLE`, so — applying the migration-017 lesson proactively this time, rather than waiting for a second runtime failure — it's `drop` function if exists followed by a fresh `create` function, re-granting `execute` afterward. All three **confirmed run by the owner 2026-08-13**.

Correction to earlier reasoning, written into migration 020's own header comment: an earlier note in this project's own history (the migration-017 fix) claimed migration 011's `owner_tier` addition "only worked because it was appended last." That's not the real reason — `get_request_by_token` returns `jsonb`, so it was never subject to the OUT-parameter restriction at all, regardless of where a new field is added. `RETURNS TABLE` functions are the ones that need `drop-then-create`; `jsonb`-returning functions can always use `create` or `replace` freely. This distinction is now documented as reusable knowledge for any future RPC-return-shape change.

Second AccountForm.tsx toggle, alongside Private Category's own: "Show Due/Done Time (Requests)," same auto-save-on-change pattern, same optimistic-update-reverted-on-failure behavior. The two toggles now share one generalized `handleToggle(field, next, setLocal)` helper rather than two near-duplicate functions.

Gated everywhere Due Time/Done Time currently appears, reading `request_time_enabled` off the same profiles round trip each screen already makes for `display_name/main_chip_prefs/private_category_enabled`: - Create Request: the Due row's Due Time field is simply not rendered when off, leaving Due Date alone (Create Request has no Done Date/Time fields at all — those only exist once a Request has been sent). - Request Detail: the two separate two-value rows (Due Date/Due Time, Done Date/Done Time) collapse into one combined row — Due Date, Done Date — reusing ToDo Detail's own combined-row markup exactly, rather than inventing a new layout. - Request Response and Response Detail: driven by the *issuer's* `owner_request_time_enabled` from the RPC payload, not the viewer's own account (there usually isn't one, on Request Response). The Due: `.metarow` drops its time suffix; the editable Done Date/Done Time row collapses to Done Date alone, matching Request Detail's own collapse. - Print Reports: Sent is gated by the signed-in owner's own flag (every Sent row belongs to this account). Received is gated per-row by each row's own `owner_request_time_enabled` (migration 021) — a different sender may have this on or off, so one flag can't govern the whole report.

Existing value stays in the database, just hidden — same convention as Category. A Request's own `due_time/done_time`, if already set from before the account turned this off, is untouched: the form state that loads it is never cleared, only the input that would edit it is unrendered, so an unrelated Save still round-trips the same value

back unchanged.

Not touched: no mockup was updated — none of the affected screens’ static HTML has a toggle-driven collapsed state to demonstrate, unlike Create ToDo/Create Request’s own Dialog empty/populated demo JS. Flagged for design/README.md, not silently skipped.

2026-08-13 — Cap a lone native date field’s width to keep Safari’s short date format

Owner, on Response Detail: a solo Done Date field (Done Time turned off for that Request’s issuer) rendered its value as “Wednesday, September 30, 2026” — full weekday-spelled-out format — instead of the app’s usual compact date, asking for it to match “the Due Date presentation above” (the Due: metarow’s own short-form label:value line just above it).

Root cause: `.frow .ffloat`’s existing `flex: 1 1 0%` rule (2026-08-11, the equal-width fix for paired Due/Done rows) grows a lone field to the full row width — roughly 400px+ — when nothing else shares the row. Safari’s native `<input type="date">` renders its own selected value using a width-dependent format: narrow (~200px, the width two paired fields split evenly) gets the abbreviated mm/dd/yyyy; wide enough (~400px+) gets the full spelled-out date. This is the browser’s own rendering, not something the page’s CSS or JS controls directly — Chrome doesn’t do this, which is presumably why it wasn’t caught earlier.

Where this can happen: any `.frow` whose native date/time field lost its sibling to the new Due/Done Time toggle (migration 019) and is now alone. That’s three places, not just Response Detail: Create Request’s Due row (Due Time off), and Request Response’s/Response Detail’s Done row (issuer’s `owner_request_time_enabled` off) — the same collapse logic built for that feature. Flagged to the owner as broader than the one screen he tested, and fixed at the CSS source rather than per-screen, since it’s the identical browser behavior in all three places.

Fix: `.frow > .ffloat.picker.native:only-child { flex: 0 1 220px; }` in `app/globals.css`, right after the existing `.frow .ffloat` rule — higher specificity (an extra class plus `:only-child`) wins over it without touching the paired-field case, which still needs its own equal-width behavior untouched. 220px matches roughly what a paired field already gets, so the lone field now renders at the same width (and therefore the same short format) it always had before the Due/Done Time toggle existed, rather than stretching to fill the empty row. No mockup changes — confirmed none of the three affected screens’ static HTML use real `type="date"` inputs (Create Request’s is a styled `type="text"`; Request Response/Response Detail have no `<input>` for these fields at all, just static `.fieldval` text), so there’s nothing there to trigger the same browser rendering.

2026-08-14 — Logged for later, not implemented this batch

Four items from the same message, each explicitly deferred by the owner or clearly a design note rather than a build instruction:

- **Scroll-restore confirmed working.** Owner: “the return to the main screen after creating or viewing a Request, creating or viewing a ToDo, and opening a Housekeeping task now comes back to the main screen vertical location as desired.” Confirms the 2026-08-13 `wyp.mainScrollTop` fix — no further action.
- **Incremental report printing deferred.** Owner: “I want to defer implementing incremental report printing until I can design a wider print format with a larger font. I originally did a full-sized portrait print format and then got hung up on seeing it on a screen (which is not needed) — and did not keep my original work.” No code change; the existing Print Reports feature (§6.34) stands as-is until the owner brings a new design.
- **Attachments design notes, captured for when Attachments (task Priority 3, still not started) is actually built** — not implemented now, since no attachment storage model exists yet to apply them to: (a) only the person who added an attachment should be able to delete it; (b) a duplicate attachment name should get “(1)” appended automatically; (c) if attachment types end up limited, that limitation needs to be explained to the end user, not just silently enforced; (d) **Add Attachment’s interaction model (2026-08-14, owner asked directly)** — first step is a native `<input type="file">` picker, the same “click a control, let the browser handle it” pattern

already used everywhere else in this app, not a custom-built chooser. Paste should be supported as a layered enhancement once the core upload path exists, not a replacement for the picker: pasting an image (screenshot, or an image copied from a webpage) via the paste event and `clipboardData.items (kind === 'file')` is reliable across Chrome/Edge/Firefox/Safari; pasting an actual file copied from the OS file manager works well in Chromium browsers via `clipboardData.files` but is spottier in Safari/Firefox, so it can't be the *only* path to attach an arbitrary file (a PDF from Downloads, say). Drag-and-drop shares nearly the same code path as paste (both hand you a `FileList`) and is worth building alongside it rather than as a separate later effort, if the fuller experience is wanted. Recommendation: file-picker button as the always-available primary path; paste and drag-and-drop layered on afterward as enhancements to the Attachments panel, not blockers on shipping it.

- **Archive UI.** Owner has designed one and will present it for discussion after Attachments is complete — no action yet, noted so it isn't lost track of.

2026-08-14 — “Keep It as Simple as Possible,” round three: **ToDo Status**, simpler defaults, and a testing-only **Subscribed? toggle**

Continuing the path started by migrations 018 (Private Category) and 019 (Due/Done Time), all in one request.

ToDo Status (migration 022, `profiles.todo_dates_enabled` boolean not null default false). Owner, with a pasted ToDo Detail mockup: “please add another Account option related to Todos... showing the Status element as an Open or Done chip and an accompanying Note... In this format, when the Done chip is selected and the ToDo is saved, the Done Date can be set as the current date... The Open chip status is based on the ToDo Done Date being empty — so, it does not seem that any database changes are needed [for Status itself].” Correct, and implemented exactly that way: Status is a UI-only reinterpretation of the existing `done_date` column, not a new fact to store — the toggle itself is the only new column. Off collapses `CreateToDoForm.tsx`'s and `ToDoDetailForm.tsx`'s quick-Done band *and* Due Date/Done Date row into one Open/Done chip pair (§6.35 PROPOSED) reusing `.sendrow+.chippair+.gatenote` verbatim — the identical combo `AddContactForm.tsx` already uses for its Send Requests By picker, not new markup. On Save: flipping to Done sets `done_date` to today *only if it wasn't already set* (an existing `done_date` — e.g. one set while the toggle was on — is preserved, not clobbered); flipping to Open clears it. `due_date` is simply never shown or touched in this state, same “hidden, not cleared” convention already used for Category and Due Time. Create ToDo always starts Status at Open (a brand-new ToDo has no existing `done_date` to derive from); ToDo Detail derives its initial Status once, on load, from whatever `done_date` the record already has.

Simpler defaults (migration 023, `alter column request_time_enabled set default false`). Owner, closing the same message: “With these 2 changes to the Account screen, all 3 currently available options are defaulted to the simpler use of the WYP app.” Migration 019 had deliberately defaulted `request_time_enabled` to `true`, reasoning that this app might already have real accounts with real Due Time data a `false` default would silently hide — flipped here, deliberately, because that concern doesn't actually apply: this app has no real users yet besides the owner. `ALTER COLUMN ... SET DEFAULT` only changes what a brand-new signup gets; it does not retroactively touch any already-set row, including the owner's own (already toggled off, live, before this migration was even drafted). All three account preferences (Private Category, Due/Done Time, ToDo Status) are now off/simple by default for a new account, matching the stated goal exactly. Flagged directly, not silently reversed, since it contradicts migration 019's own stated reasoning at the time — see that migration's header and CLAUDE.md.

Subscribed? (migration 024, TESTING ONLY — `grant update(tier) on profiles to authenticated`). Owner: “For the development and Attachments testing, perhaps an Account ‘Subscribed?’ option is appropriate. Later, this option would present differently and only able to be set by opening a subscription page with appropriate eCommerce links... In the full implementation, a ‘Subscription Details’ Task could replace the Account option.” A real conflict, flagged rather than silently overwritten: migration 002 deliberately excluded `tier` from the owner's own column grant specifically so no free user could self-upgrade — “tier is therefore writable only by `service_role` (the billing webhook, later).” This migration reopens that path, on purpose, because the owner's own request already anticipates and accepts it as a temporary stand-in — he is the only account that exists to use it. Must be revoked (or replaced by the real billing-webhook-only path he describes) before this app has any real second user or actual payment processing; tracked in

CLAUDE.md's Known gaps, not left implicit. `AccountForm.tsx` gained a fourth `.checkrow`, "Subscribed? (testing only)," writing `profiles.tier` directly via its own `handleTierToggle` (text-valued, so it doesn't join the shared boolean `handleToggle` the other three toggles use).

Migrations 022 and 023 are **DRAFTED, not yet confirmed run** as of this writing. Migration 024 (Subscribed?) is **confirmed run by the owner, 2026-08-14** — the `AccountForm.tsx` wiring for all three is live.

2026-08-14 — Get Response Link removed from Request Detail

Owner: "Since it is no longer needed for testing, drop the 'Get Response Link' section from the Request Detail screen." Removed the `.linkband` UI (Get Response Link/Copy/Regenerate) along with its own `linkUrl/linkLoading/linkCopied/linkError` state and `handleGetLink/handleCopyLink` handlers from `RequestDetailForm.tsx` — it was always a manual testing stand-in for real email delivery (§6.30 PROPOSED, Week 3 Day 4), never drawn in any mockup. **Not removed:** `issue_request_link` (migration 008) itself, and its use inside `CreateRequestForm.tsx`'s own automatic Initial Request email flow (Week 5 Priority 1) — that's a separate, still-needed call site. `.linkband/.linkval` CSS stays in `globals.css`, unused for now rather than deleted, on the chance a future admin/debug surface wants the same pattern.

2026-08-14 — Create ToDo defaults to SOON, not ASAP

Owner: "I realized the Create a ToDo should default the the 'Soon' Priority. I think that would be the appropriate Priority for a 'normal' ToDo — which can be changed by the end-user to either ASAP or Later as they desire." `CreateToDoForm.tsx`'s `initialState.priority` changed from 1 (ASAP) to 2 (SOON); ported into `WYP_create_todo_palette1.html`'s static Priority chip row for consistency. ToDo Detail is unaffected — it always loads the record's own real priority, never a hardcoded default.

2026-08-14 — Attachments plan: all seven open questions resolved

Owner answered every open question `docs/WYP_Attachments_Plan.md` raised, in one message, plus a follow-up refinement on the ToDo "Locations" UI. All folded into that doc directly rather than re-summarized here in full; key points:

Delete permission — the Request/ToDo owner has full delete control over any attachment on their own item, including ones a recipient added; a non-owner (a Request's recipient) can only delete their own. Resolves the asymmetry the plan doc had flagged as unusual and worth confirming deliberately.

File size limit / file types / virus scanning — owner asked whether virus-scanning tools exist or whether that's the recipient's own responsibility, and said file types don't matter "unless you indicate there are types we should not support." Recommended and written into the plan: a 10 MB default per-file size cap (adjustable constant, not a hard number from the owner), a blocklist of executable/installer/script extensions rather than an allowlist (Request Response accepts uploads from anonymous recipients, so this is a real surface), and no built-in virus scanning for v1 — Supabase Storage doesn't provide it, and wiring a third-party scanner is materially more infrastructure than this pass is scoped for. Recipient-side caution remains the practical mitigation, consistent with how most products this size operate before real scale.

Max attachments per item — owner offered 10 or 15 ("this app isn't intended to be a 'file-transfer service'"); picked 10.

Lapse-and-auto-delete job — owner: "later would be a good strategy," as long as it's tracked. Deferred, logged in the plan doc's build order as its own follow-up priority, not lost.

TDos: Attachment References ("Locations"), not real storage. Owner's own proposal, since a ToDo has no recipient to share a file with: "instead of directly storing 'Attachments'... we offer 'Attachment References' and an 'Add

Locations’ button for file paths or URL links instead of actually storing the attachment. Is that practical?” Answered yes, and designed to reuse the same `attachments` table via a new `kind` column (`'file'` vs `'reference'`) rather than a second table — Requests only ever produce `kind = 'file'` rows, Todos only ever produce `kind = 'reference'` rows, but the delete rule, the 10-item cap, and the panel/empty-state markup stay identical either way. Rejected alternative: a separate `todo_references` table, which would duplicate nearly every column and component for no real benefit and would foreclose ever giving a Request its own lightweight reference-style entry later.

Owner’s follow-up, same day: “If Locations are used for Todo attachments, the UI could prompt for an optional description — which would precede the path or URL in the list.” Folded in: Add Location captures an optional **Description** field before the **Location** (path or URL) field; when present, Description renders above the location value in the list, the same relationship Dialog’s `.dlgre/.dlgbody` pairing already uses for a reply’s context line above its body. Also flagged in the plan doc: a typed local file path is inert text the app can never open or verify (no filesystem access to the user’s device) — only a well-formed `http(s)://` value should render as an actual clickable link; anything else displays as plain text.

Print Reports icon slot — add the Attachments icon now, alongside Dialog’s existing one, rather than deferring.

Nothing has been built yet — this is the plan doc’s open questions closing out, not an implementation batch. See `docs/WYP_Attachments_Plan.md` for the full resolved design and build order.

2026-08-14 — Real Attachments built (Week 5 Priority 3)

Owner: “OK, please start.” Built from `docs/WYP_Attachments_Plan.md`’s resolved design in full, across migrations, three new API routes, a shared panel component, and all six affected screens. Migrations 025 (`public.attachments`), 026 (private Storage bucket), and 027 (`attachment_count` on `get_received_requests()`) are all **confirmed run by the owner, 2026-08-14**. `SUPABASE_SERVICE_ROLE_KEY` is likewise **confirmed set in both `.env.local` and Vercel, 2026-08-14** — the owner used Supabase’s newer secret-key system (`sb_secret_...`, named `default`) rather than the legacy `service_role` JWT; both work identically as the second argument to `createClient()`, which is all these routes ever do with it. Feature is live pending the owner’s own end-to-end test.

Architecture. One `attachments` table serves both a Request’s real uploaded files (`kind = 'file'`) and a Todo’s “Location” references (`kind = 'reference'`), matching the plan doc’s own recommendation. A real file’s bytes and its Storage bucket are only ever touched by three new server-only routes — `app/api/attachments/upload`, `list`, `delete` — using `SUPABASE_SERVICE_ROLE_KEY`, because an anonymous Request Response visitor has no session at all for Storage’s own RLS to scope to, and unlike a plain database write, Storage’s upload/download API isn’t reachable from a SQL SECURITY DEFINER function the way every other anonymous-write problem in this app has been solved so far. This is a deliberate, narrow exception to CLAUDE.md’s “`service_role` never goes near the browser” rule — the key stays server-side in these three files only, and every route still independently verifies who’s calling and what they’re allowed to do (via a forwarded-JWT client for the owner/signed-in recipient, or the existing `get_request_by_token/get_received_request` functions for the anonymous/signed-in recipient cases) before it ever touches Storage or the table — `service_role` only ever carries out an already-approved write, it never decides permission on its own. `kind = 'reference'` rows (Todo Locations) skip all of this — no file bytes, so a direct RLS-scoped client insert/delete is enough, and migration 025’s own insert policy refuses a client-inserted `kind = 'file'` row on purpose, so the upload route stays the only path a real file can ever be created through.

Resolved-question fidelity. Delete permission: the Request/Todo owner can always delete any attachment on their own item; a non-owner uploader only their own — implemented as an RLS policy plus the same check re-run in `delete/route.ts`, never trusted from a Delete button’s own visibility. Size/type: a 10 MB cap and an executable/installer/script extension blacklist, both enforced server-side (`app/src/lib/attachments.ts`), client-side checks are a courtesy only. Cap: 10 per Request/Todo. Lapse-and-auto-delete: not built, tracked as its own later priority. Todo Locations: implemented via the `kind` column exactly as recommended, with the owner’s own follow-up refinement (an optional Description rendered above the Location value in the list, mirroring Dialog’s `.dlgre/.dlgbody` reply-context pairing) built in from the start. Print Reports icon: added immediately, for Sent (a plain PostgREST

`attachments(count) embed`) and `Received` (migration 027, since `get_received_requests()` is a `RETURNS TABLE` function needing the drop-then-create fix migrations 017/021 already established).

One design call made without an explicit owner instruction, flagged rather than silently assumed: `ToDo` Locations are gated behind the same subscriber tier as a `Request`’s real `Attachments`, for one consistent “Attachments is a paid feature” story across both object types. This is weaker reasoning than for real files, though — a `Location` has no storage cost at all — so it’s worth the owner’s own call on whether `Locations` should just be free for everyone.

Delete semantics. A manual `Delete` (the button a user actually clicks) hard-deletes — removes the `Storage` object and the row outright. `attachments.deleted_at` exists but is deliberately never set by that path; it’s reserved for the still-unbuilt lapse-and-auto-delete job, which needs a way to tell “this was reclaimed because the owner’s subscription lapsed” apart from “the user simply removed it” — a hard delete already fully expresses the second case on its own.

Not ported into any mockup — none of the six screens’ static HTML has real upload/list JavaScript to convert; flagged in `design/README.md`, not silently skipped. `npx tsc --noEmit/npm run lint` both clean throughout.

2026-08-14 — Locations UX: note text, clickable URLs, Copy-path for file paths

Owner, testing `Locations` live: (1) wanted a note to the left of `Add Location` explaining what a `Location` is for (“Locations can be used for URLs or File paths”), pasted a reference screenshot of `Create ToDo` showing the exact wording and placement — a `.donerow/.donenote` strip (bold “Note:” prefix, button on the right), the same pattern already used for the `Attachments` empty-state note and the quick-Done bands, rather than the bordered `.actlabel` treatment `Locations` had been using. (2) wanted a saved URL to render as a clickable, underlined link. (3) asked whether a saved file path could open the browser’s own file explorer, the way an `Attachment` does.

Note text — done as asked. `AttachmentsPanel.tsx` (mode = ‘reference’, used by `ToDo Detail`) and `CreateToDoForm.tsx`’s own staged `Locations` both now show the `.donerow` note persistently — before any `Location` exists and after, matching the owner’s screenshot exactly, rather than switching between an empty-state row and a bare populated-state button the way `Attachments` (mode = ‘file’) still does. `.donerow` doesn’t collapse for a gated (non-subscriber) account with existing `Locations` from before the gate — same “viewing is never gated, only adding” rule the `Entitlements` section already states; the note+button simply doesn’t render at all in that case, only the read-only list.

Clickable URLs — already partly built, extended for consistency. `AttachmentsPanel.tsx` already rendered `isHttpUrl(reference_url)` as a real `<a>`, added the same day `Real Attachments` shipped — but `Tailwind`’s preflight resets `<a>` to `color: inherit; text-decoration: inherit`, so it was rendering as plain unstyled text, not visibly a link. Added an explicit `.attname` a rule (`var(--brand-blue)`, underlined) to `globals.css`. Also extended the same `isHttpUrl` check to `CreateToDoForm.tsx`’s own staged list (before `Save`), which had never had it — it showed every `Location` as plain text regardless.

File-path “open a file explorer” — not built, recommended against, with an alternative built instead. A browser has no filesystem access beyond a file the user explicitly picked via `<input type="file">` (already how real `Attachments` work) — a typed string like `C:\Project\wyp\docs` isn’t something a web page can hand to the OS’s own file explorer. A `file://` link was considered and rejected: modern browsers largely block or silently no-op navigation from an `https://` page to a `file://` URI, and even where it technically works it would only ever succeed if the exact path exists on whatever device is currently viewing the `ToDo` — meaningless the moment a `Location` is checked from a different device than the one that typed it. This constraint was already documented in `app/src/lib/attachments.ts`’s own `isHttpUrl` comment before this batch (“a typed local file path is inert text the app can never open or verify”), not a new finding. **Built instead:** a small “Copy” button (reusing the existing `.linkbtn` style) next to any non-URL `Location`, both in `AttachmentsPanel.tsx` and `CreateToDoForm.tsx`’s staged list — copies the path to the clipboard via `navigator.clipboard.writeText`, with the button label flipping to “Copied” for 1.5 seconds as feedback, so the owner can paste it into his own OS file explorer’s address bar rather than retyping it. Silently no-ops if the Clipboard API is unavailable (permissions, non-secure context) — the path is still fully visible in the row to select and copy by hand either way.

Not ported into any mockup — none of the affected screens’ static HTML has a real Locations list to update (Create ToDo’s own mockup demo doesn’t render attitem rows with a Copy affordance or link detection). `npx tsc --noEmit/npm run lint clean`.

2026-08-14 — Location URL detection: bare-domain heuristic, live-verification rejected

Follow-up to the same day’s Locations UX batch. Owner-reported testing: `ft.com` and `www.ft.com` weren’t recognized as URLs — only `https://www.ft.com/` was — since `isHttpUrl` required a working `http://https:` scheme via `new URL()`, and a bare domain has no scheme to parse. Owner also asked whether a network-based check (HTTP HEAD, a ranged GET, or a DNS lookup) would give “a more accurate this-is-a-URL judgement.”

Recommendation, implemented: keep the check purely syntactic, but widen it to recognize a bare domain shape (`ft.com`, `www.ft.com`, with an optional port/path/query) and treat it as a link — prepending `https://` for the actual href while still displaying what the owner typed. Replaced `isHttpUrl` with `urlLocationHref()` in `app/src/lib/attachments.ts`, wired into both `AttachmentsPanel.tsx` and `CreateToDoForm.tsx`’s staged list. Guards against the obvious false positive — a bare filename that happens to look domain-shaped (“report.pdf”) — two ways: reject anything shaped like a file path first (drive letter, UNC `\\server\share`, a leading `//~/.`, any backslash or whitespace), then reject a matched “domain” whose last label is a common file extension (`FILE_EXTENSION_BLOCKLIST` — pdf, docx, jpg, mp4, etc.) rather than a plausible TLD. Not exhaustive — an obscure extension not on the list, or a real but unusual TLD, can still misfire in either direction — but covers every case actually seen in testing.

Live verification (HEAD / ranged GET / DNS lookup) — considered, not built. Rejected for three reasons, not just the two adjacent to the recommendation above: (1) a browser can’t run any of these against an arbitrary third-party origin itself — blocked by CORS for any site that hasn’t opted in with `Access-Control-Allow-Origin`, which is most sites — so it would have to be proxied through our own server, the same service-role-API-route shape Attachments already uses. (2) That proxy is a standing SSRF surface: a saved Location is user-supplied text, and a server making outbound requests to whatever a user types needs real hardening (blocking private/internal IP ranges, redirect-chasing limits, timeouts) that doesn’t exist anywhere else in this app yet — a disproportionate amount of new attack surface for what’s ultimately a rendering decision. (3) It answers a different question than the one being asked: “is this text shaped like a URL” (a syntax question, decided once, instantly, client-side) is not the same as “is this URL currently reachable” (a liveness question that can change hour to hour) — a real site that’s briefly down would wrongly stop rendering as a link under a liveness check, which is a worse outcome than the syntactic heuristic’s rare false positive/negative. If the owner wants reachability checked later (e.g., to warn about a dead link), that’s a separate, bigger feature — a dedicated server route with its own SSRF guardrails — not a refinement of this one.

`npx tsc --noEmit/npm run lint clean`. No mockup changes — same reasoning as the note-text/Copy-button batch above.

2026-08-14 — Archive screen designed (mockup only); PRD §9.5 conflicts flagged

Owner drafted a full UI strategy for the Archive feature PRD §9.5 (v12.9) already names but leaves unbuilt, with pasted-in reference screenshots for the Sent Requests and Todos states. Built as `design/screens/WYP_archive_palette1.html` (one file, three Record-Type states) plus a new Housekeeping “Archive” row in the Main Screen mockup — see that screen’s own design/README.md entry for the full build write-up (selection-persistence rule, the Overdue-row correction, the real `type= "date"` exception, §6.36). **Mockup only, not converted to live** — no archives table, no `/archive` route, and the Main Screen row wasn’t added to `MainScreen.tsx` — following this project’s own established two-phase convention (design first, approved, then converted), and because the owner’s message described a “strategy” and a “mockup,” not an instruction to build the live feature yet.

Three real divergences from the existing PRD §9.5 text, flagged per CLAUDE.md’s own rule (“when I ask for something that conflicts with an existing PRD decision, flag the conflict before making the change”) rather than silently built over:

1. **No Archive Now / Remove Archive Status mode-chip pair.** PRD §9.5 (added 2026-08-09) describes two chips switching the screen between archiving and un-archiving. The owner’s own words this time: “I did not tackle an ‘Un-Archive’ feature - that can be done later.” The mockup has only the Record Type chip row and a single “Archive Selected” action button — there’s nothing to switch between yet. Un-Archive stays a real, deferred piece of §9.5, not dropped.
2. **A Recipient/Requestor filter, not in the PRD at all.** §9.5 only describes filtering “by type... and a prior-to date.” The owner’s mockup adds a Recipient (Sent) / Requestor (Received) lookup field, combining with Before Done Date as two narrowing criteria rather than one.
3. **The list starts empty, not pre-populated.** §9.5 reads as though every matching Done item appears as soon as a Record Type is chosen (“each matching item appears with a pre-checked checkbox”). The owner’s own description is explicit that the screen “would open with no Sent Requests displayed” until a filter is entered — arguably always the intended reading of “selects candidate records by type... and a prior-to date” (a date is itself a filter), but different enough from a literal reading of the existing text to call out rather than assume.

Proposed replacement §9.5 text, for the owner to confirm before it’s merged into the actual .docx (not done in this pass — this is a draft for approval, same pattern as the §7.3 email-template proposal):

9.5 Archive Requests and Todos Not yet phased. Allows completed Requests and Todos to be removed from the Main Screen’s lists while remaining available through Search. A new Archive screen selects among Requests Sent, Requests Received, and Todos via a Record Type chip row (Requests Sent is the default); the list below starts empty and shows only Done items matching whatever the user has entered into Recipient (Requests Sent) / Requestor (Requests Received) and/or a Before Done Date field — Todos has no Recipient/Requestor field, so Before Done Date alone gates it. Each matching item shows with a pre-checked checkbox and is clickable to its own Detail screen for viewing or editing; unchecking a record excludes it from that Archive action, and a deselection persists even if the user subsequently narrows or widens the filter criteria further, as long as the record keeps matching. Removing an item’s Done Date on its Detail screen removes it from the eligible list, since only Done items qualify. An “Archive Selected” button archives every currently-checked, currently-matching item. Un-Archive (reversing archived status for a previously archived item) is deferred to a later phase — not in this screen yet. Requires, at minimum, an Archive Status flag and an Archived Date column on the `requests` table.

Not yet phased in the roadmap, same as the existing text — this proposal only revises the behavior description, not the phasing.

2026-08-14 — Archive mockup: rendering fixes + row-click-to-Detail

Owner tested the freshly-built `WYP_archive_palette1.html`, screenshots attached, and reported four things. All fixed the same day.

1. **Before Done Date’s label overlapped the native “mm/dd/yyyy” placeholder**, reading as garbled text. Traced to a mismatch between this file and the live app: the live app’s `.ffloat.picker.native` pattern (label permanently risen and absolutely positioned over the input) relies on Tailwind’s preflight reset being in effect, which keeps a native `type="date"` control’s internal placeholder confined to `.input`’s own padded content box. This mockup is fully self-contained (see the file’s own header) with no such reset, so the browser’s own UA styling for the date control doesn’t reliably respect the asymmetric top padding that trick depends on. Fixed by not using that pattern here at all — a plain static label sits above a normally-padded input (`.plaingroup/ .input.plain`), the same shape as any ordinary non-floating label elsewhere in the app; the two elements can never overlap because they’re just stacked blocks, regardless of how any given browser renders the native control’s internals.

2. **“Archive Selected” should be colored as primary.** It was — but only when at least one record was selected; at zero selected (the state every one of the owner’s own screenshots happened to show) it rendered in the locked/grey treatment, an addition of mine, not something the owner’s own reference ever depicted. Removed the disabled/grey state entirely — the button now stays plain primary blue always; clicking it with nothing selected is a quiet no-op instead.
3. **“A blue background is missing for the chips row.”** The Record Type chip row (`.archtyperow`) was sitting directly on the white/`.scroll` background with no wrapper of its own — every other chip row in the app (Main Screen’s `.subhead`, `.sendrow`, `.chiprow`) sits on the Strip tint specifically so a resting chip’s own semi-transparent white background has something to contrast against. Missed when this screen was first built. Added `background: var(--strip)` to `.archtyperow`, matching `.subhead`’s own treatment.
4. **“Can we let the user see the related Detail screen from clicking the respective item?”** Yes — matches the row-opens-Detail convention every other list in the app already uses (Main Screen’s Sent/Received/ToDo rows). Added `role="button"/hover/focus` styling to each row’s body (`.archbody`, outside the checkbox so the two never conflict) and a click/ Enter handler; since this file is a standalone mockup with no router, it surfaces what a live conversion would do (an inline note naming the target Detail screen — Request Detail for Sent, Response Detail for Received, ToDo Detail for ToDos) rather than silently doing nothing or faking a real navigation.

`npx tsc --noEmit/npm run lint` not applicable — this file has no build step; checked instead with a Node `--check` pass on the extracted `<script>` body and a tag-balance check (no headless browser reachable in this session’s sandbox to screenshot it, same limitation noted for the landing page and Archive’s own first build).

2026-08-14 — Archive mockup: Recipient/Requestor overlap, date-box explanation, icon clickability, calendar-picker-only

Owner tested `WYP_archive_palette1.html` again after the prior round’s fixes and reported four more things.

1. **Recipient/Requestor floating label overlapped the typed text.** Not the classic bug this project already has tribal knowledge about — the field’s markup order (`.finput` then `.flabel`) and `placeholder=" "` both matched the known-working Create Request Recipient field exactly. The remaining difference is that this file loads Inter from Google Fonts over the network rather than the live app’s self-hosted `next/font` copy; an unset `line-height` computes to “normal,” which is derived from whichever font is actually active at that moment, so a fallback font’s different internal metrics can shift text within the same fixed padding box before (or if) the network font finishes loading. Fixed by adding an explicit `line-height: 1.2` to both `.finput` and `.ffloat .flabel`, removing the font-dependent variability regardless of which font is active. **Flagged as a best-diagnosis fix, not a confirmed one** — no headless browser is reachable in this sandbox to render and screenshot the file (same limitation as every other visual fix in this screen’s history), so this hasn’t been visually confirmed against the reported symptom.
2. **“Why does the date entry box differ from elsewhere in the app?”** By design, and already flagged in the prior round’s own log entry (item 1 above it) — the live app’s `.ffloat.picker.native` pattern depends on Tailwind’s preflight reset, which this self-contained mockup doesn’t have, so it was deliberately replaced with a plain stacked label-above- input layout here. This is a real, acknowledged divergence specific to the standalone-mockup file format, not an oversight; converting this screen to a live React component would use the exact same `.ffloat.picker.native` component every other date field in the app already uses, and the difference would disappear.
3. **Calendar icon on Before Done Date “looks clickable, which it is not.”** Correct catch against this project’s own §6.16 rule (the label-affordance glyph is a passive signifier, never an independent clickable icon) — the new stacked layout put the icon visually apart from the input box, unlike the live app’s convention where it sits inside the field. Rather than restyling the icon to look inert on its own, made the entire label-plus-input region read and behave as one uniform unit: `cursor: pointer` across `.plaingroup`, its label, and the input, plus a focus listener that calls `showPicker()` (same try/caught pattern as the live app’s existing `openPicker()` convention, for pre-16.4 Safari). A native `<label for="beforeDone">` already delegates any click within it to the input,

so clicking the icon, the label text, or the input all now open the picker identically — nothing about the icon reads as separately or specially clickable anymore.

4. **Manual m/d/y entry allowed invalid dates** (uncontrolled month/day rollover, no Feb 29 validation, a year of “276750” accepted). Owner asked this field match a “calendar-picker-only strategy” used elsewhere in the app. Considered readonly on the input and rejected it — its effect on a native date picker’s own open-on-click/focus interaction is inconsistent across browsers, so it risked blocking the picker itself, not just manual typing. Used a keydown listener instead: every key except Tab is prevented, so the field can never receive typed characters at all; the only way to set a value is through the native picker, which structurally cannot represent an invalid calendar date. This resolves the Feb 29 and out-of-range-year concerns by construction, not by adding validation logic — there is no longer any code path that accepts typed input to validate.

Positive: the selected-record count in the “Archive Selected” button label (from the prior round) was called out as “a nice touch” — kept as-is, no change.

Verified the same way as the prior round: Node --check on the extracted `<script>` body (clean) and a `<div>`/`</div>` tag-balance count (27/27, clean). No headless browser reachable in this sandbox for a real render — items 1 and 3 above are reasoned fixes, not screenshot-confirmed ones.

2026-08-14 — Archive converted to live (/archive, ArchiveForm.tsx); per-viewer scope; migration 028

Owner: “Let’s converted to a live screen next - along with the additional Housekeeping task.” Converts design/screens/WYP_archive_palette1.html to app/components/ArchiveForm.tsx / /archive, and wires Main Screen’s Housekeeping “Archive” row (already in the mockup) into mainScreen.tsx for real.

Per-viewer archive scope — asked, not assumed. A Request is one database row but is viewed by two different accounts (sender and recipient). Before writing the migration, asked via AskUserQuestion whether archiving a Received item should hide it only from the recipient’s own Received list (leaving the sender’s Sent view of the same row untouched) or be one shared flag that hides it from both sides at once. Owner picked **per-viewer**. This matches the reasoning CLAUDE.md’s own Entitlements section already gives for reading tier live per-viewer rather than snapshotting it — two accounts can each have their own current relationship to the same row.

Migration 028 (docs/Week5 - SQL history.txt, drafted 2026-08-14 — **confirmed run by the owner the same day**): - requests.archived_at — the row’s own owner archiving it from Sent (a Request) or their only list (a ToDo, contact_id null — there is no recipient, so this is a ToDo’s only archive state). Plain-RLS-writable via the existing “requests: owners update own” policy (migration 002) — no new function needed; ArchiveForm.tsx does a normal .update().in('id', ids). - requests.received_archived_at — the recipient archiving their own Received copy. RLS on requests is owner-only, so this needed a new SECURITY DEFINER function, archive_received_request(p_request_id) — same email-match-through-contacts pattern as set_response_done_as_recipient/add_dialog_as_recipient (migration 012), single-row, called once per id. - get_received_requests() gains received_archived_at in its own row shape (drop-then-create, the RETURNS TABLE lesson migrations 017/021/027 already learned). get_received_request() (singular, Response Detail’s own RPC) was deliberately left untouched — nothing reads received_archived_at there yet. - Un-Archive stays out of scope, per the owner’s own words from the mockup round (“that can be done later”) — no reverse function.

“Still findable through Search” is a real behavior, not just PRD copy. The owner’s own drafted §9.5 replacement text (this file, earlier 2026-08-14 entry) says an archived record is “no longer displayed” on the Main Screen “while remaining available through Search.” Implemented literally: neither archive column is ever excluded from a query anywhere — mainScreen.tsx’s filteredSent/filteredReceived/filteredTodos hide an archived row only while the search box is empty, and re-include it (subject to the usual status-chip filter and text match) the moment a query is typed. This is why migration 028 adds the columns to get_received_requests()’s row shape rather than filtering by them in the function’s own WHERE clause — the show/hide split happens entirely client-side.

`ArchiveForm.tsx` ports the mockup’s own logic faithfully: per-Record- Type deselected id sets (React `useState<Record<RecordType, Set<string>>>`, same persistence rule as the mockup — a deselection survives further narrowing/widening of the filters, resets only on a Record Type switch), the empty-until-a-filter-is-entered gate, and the Recipient/Requestor type-ahead sourced from the currently-loaded candidate data itself (not the Contacts table — a Received row’s “Requestor” is a sender’s `profiles.display_name`, which isn’t a Contact at all). Row click opens the record’s real Detail screen now (`/requests/[id]`, `/requests/[id]/respond`, `/todos/[id]`) — the mockup’s own inert “would navigate there” note was exactly this, promised in the first build’s own log entry. Archive Selected calls a plain batched `.update()` for Sent/Todos or loops `archive_received_request()` via `Promise.all` for Received, then updates local state so the just-archived rows drop out of the eligible list immediately, matching the mockup’s own `archived[currentType].add(...)` behavior.

Before Done Date uses the app’s real `.ffloat.picker.native` component here, not the mockup’s own `.plaingroup/.finput.plain` substitution — that substitution existed only to route around the standalone mockup file’s missing Tailwind preflight reset (see this file’s two prior 2026-08-14 entries), and this live component has the real reset, so the normal pattern needs no modification. The calendar-picker-only keydown block (from the second mockup fix round) is carried over as-is — this remains the one date field in the app with that behavior; the other 14 date/time fields keep their existing typing-plus-click-to-open behavior, not retroactively changed.

Main Screen row types gained `archived_at/received_archived_at` (both still selected/fetched, per the “findable through Search” reasoning above), and the Housekeeping band gained a real “Archive” row (– remove completed items from these lists) navigating to `/archive`, following the exact markup/handler pattern of the existing Contacts/Account rows.

PRD §9.5’s replacement text is still only drafted in the decisions log, not merged into the actual `.docx`. Attempted this batch; the title-page and footer version-number text in `WouldYouPlease_PRD_v12_9.docx` turned out to be split across `<w:r>` runs in a way `merge_runs.py` didn’t fully coalesce (a raw-text search for “12.9” found zero contiguous matches in `document.xml` despite pandoc’s own rendered markdown showing it plainly), making a safe, verifiable edit slower than the rest of this batch — deferred rather than risk a bad edit to the canonical PRD file under time pressure. The live behavior already matches the drafted text; only the document itself still needs the version bump (title page, both footers, Schedule A entry, §9.5 body) whenever the owner confirms he wants that done now.

`npx tsc --noEmit/npm run lint clean.`

2026-08-14 — Archive live: field alignment, chip highlighting, state lost on Detail round trip

Owner tested `/archive` after migration 028 was confirmed run, two screenshots attached, and reported three things.

1. **Recipient and Before Done Date fields: “placeholder/labels are ‘in the box’, but not vertically aligned as they should be.”** Root cause: `ArchiveForm.tsx` wrapped each field in a bare `.fgroup` — every other field of this shape in the app (Create Request’s Recipient, its Due Date) wraps in `.fgroup > .frow > .ffloat`, not `.fgroup > .ffloat` directly. `.frow` is `display: flex`, and `.frow .ffloat { flex: 1 1 0%; min-width: 0; }` is what actually stretches `.ffloat` (a `<span>`, inline by default) to the field’s full width. Without `.frow`, `.ffloat` stayed an unstretched inline box, so `.finput`’s own `width: 100%` had nothing correctly sized to resolve against and the absolutely-positioned `.flabel` (top: 15px, meant to read as a caption near the top of a 50px-tall input) ended up positioned against the wrong box entirely — this is a different, live-app-specific bug from the two prior mockup rounds’ Recipient/Before-Done-Date issues (those were a missing line-height and a Tailwind-reset gap in the *standalone mockup file*; this is a missing wrapper element in the *live component*). Fixed by adding the `.frow` wrapper to both fields, matching `CreateRequestForm.tsx`’s own markup exactly.
2. **“The selected chip is not highlighted.”** The Record Type chips toggle a `sel` class (ported straight from the mockup’s own demo JS, which matches Main Screen’s own filter-chip convention) — but Main Screen’s `.sel` styling is deliberately scoped to `.chips .chip.sel`, per that rule’s own existing comment (“this screen’s own local `.sel...` convention, not the app-wide `.chip.selected`”), and Archive’s chip row uses `.archtyperow`, not

.chips, so the rule never matched. Fixed by adding an .archtyperow .chip.sel rule to globals.css with the identical declarations, rather than switching the JSX to the generic .selected class name purely to dodge a CSS scoping issue.

3. **“I was able to see the detail for a Request, but when I returned to the Archive screen, all of the entries and the search logic was cleared — the same was true for Todos.”** Expected, given how the screen was built: a row click does router.push to the Detail screen, which returns via router.back() (Request/ToDo/Response Detail’s own established convention) — ArchiveForm.tsx fully remounts on that return trip (no Cache Components/<Activity> enabled, same reasoning already documented on mainScreen.tsx), so every piece of useState — Record Type, Recipient/Requestor query, Before Done Date, and the per-type deselected sets — was resetting to its default every time. Fixed with the same pattern Main Screen’s own 2026-08-09 chip-persistence fix used: sessionStorage (not localStorage — a within-session view/selection state, not a durable account setting), read via lazy useState initializers and written back with one useEffect per piece of state. The deselected Sets aren’t JSON-serializable directly, so they’re stored as plain arrays and rehydrated into Sets on read.

```
npx tsc --noEmit/npm run lint clean.
```

2026-08-14 — Archive column-header sorting

Owner: “the sorting does not work for column headings in the displayed search results. I’m not sure this was or was not intentionally deferred - but, it would be helpful to have it react to users as they have learned to expect.” It wasn’t deferred as such — the mockup’s static “Due v”/ “Priority ^” pill labels were ported as pure decoration when ArchiveForm.tsx was first built and never wired to real behavior, unlike Main Screen’s own column headers, which gained real sorting back on 2026-08-11.

Fixed by duplicating Main Screen’s own ColSort/toggleSort/compareNullable/compareStrings/compareNumbers/readStored/writeStoredSort into ArchiveForm.tsx (this app’s established convention for small stateless helpers — see openPicker, formatMDY) rather than importing them, since mainScreen.tsx doesn’t export any of them. Sent and Received share one sort state (name/date/due/done, defaulting to Due descending, matching Main Screen’s own default); Todos here only has Priority to sort by (defaulting ascending) — this screen never shows Category at all, unlike Main Screen’s own Todos list, so there’s no second sortable column. Each Record Type’s sort persists to sessionStorage independently (wyp.archiveSentSort/wyp.archiveReceivedSort/wyp.archiveToDoSort), same pattern already used for currentType/recipientQuery/beforeDone/deselected.

Required extending the shared Row type: due/date were already-formatted MM-DD-YY display strings (via formatMDY), which don’t sort correctly as plain strings. Added raw dueISO/dateISO (and priority as a number) alongside the existing display fields, sourced straight from the underlying due_date/created_at/priority columns. Also changed name from a '-'-baked-into-the-map fallback to a real string | null, so compareNullable can push a missing Recipient/Requestor name to the end of the list regardless of sort direction (mirroring Main Screen’s own contacts?.display_name ?? null convention) — the '-' fallback moved to render time in the row JSX instead. nameOptions’s own filter simplified to match (no longer needs to exclude the literal string '-').

```
npx tsc --noEmit/npm run lint clean.
```

No mockup change — the mockup’s column headers were always static and this is a live-only behavior fix.

2026-08-14 — Archive filter persistence, Attachments alignment fix, Todos Overdue chip gating

Three more owner-reported items, same day as column-header sorting above.

1. **Archive Recipient/Requestor query and Before Done Date now persist across Record Type chip switches within a session** — the follow-up to item 5 from the previous batch, actually implemented this round.

`selectType()` no longer resets `recipientQuery/beforeDone`; both keep whatever the user last typed regardless of which chip is active, only clearing when the user edits or clears them directly. `confirmMessage/archiveError` still reset on a chip switch — those are action feedback tied to whatever was just archived, not a filter, and go stale the moment the underlying list changes. Owner confirmed working live (“I now see the criteria persistence in the Archive session!”).

2. **Add Attachment button/box alignment fix, second pass.** The first pass (this same day, standalone prop on `AttachmentsPanel.tsx`) carried over only the horizontal padding from Dialog’s own matching empty-state row on Request Response/Response Detail, not its `marginBottom: 12`. Split into two style objects: `rowPad` (horizontal padding only, still used by the populated-state `.fieldact` Add Attachment row, which already has its own margin-bottom baked into that CSS class) and a new `emptyRowStyle` (`padding: '0 var(--pad)'`, `marginBottom: 12`, byte-for-byte matching Dialog’s own inline style) applied only to the zero-entries `.actlabel + button` row. The locked/subscription-note branch (`.donerow`) needed no change — that class already carries its own `margin: 4px var(--pad) 14px`, baked in regardless of standalone.
3. **ToDo’s Overdue chip now hidden on Main Screen when Show Due/Done Dates (ToDo’s) is off** (migration 022, `profiles.todo_dates_enabled`, default false) — owner: “the Overdue chip for ToDo’s should not be shown for either the Main or the Archive screens” when that preference is off, since Create ToDo/ToDo Detail collapse to the simple Open/Done Status chip pair in that mode and never touch `due_date` at all, making an Overdue reading meaningless. `MainScreen.tsx` didn’t read `todo_dates_enabled` anywhere before this — added to the existing profiles round trip (alongside `private_category_enabled/request_time_enabled`) and used to conditionally render the Overdue chip only. `statusFor()/todoStatus()` are unchanged — they still compute a real status off whatever `due_date` already exists in the database (toggling the preference off never clears it), so a ToDo switched back to All/Open/Done keeps sorting/filtering correctly either way; only the Overdue chip’s own visibility is gated. Also resets `todoFilter` away from 'overdue' on load if a stale `sessionStorage/main_chip_prefs` value had it selected from before the preference was turned off, so the ToDo’s list can’t get stuck showing an empty list with no visible way back to All. **Archive needed no change** — `ArchiveForm.tsx` only ever lists Done records and has no Overdue chip or Overdue row styling anywhere in it to begin with, so the owner’s ask was already satisfied there; noted rather than silently skipped.

`npm run tsc --noEmit` `npm run lint` clean for all three fixes. No mockup changes — none of the affected behavior has a drawn precedent to update (the Archive mockup never had an Overdue chip either).

2026-08-14 — Create ToDo’s Locations empty/populated split (the piece left undone earlier)

Owner re-reported, with a Create ToDo screenshot: “the Create ToDo and the ToDo Detail should have the Add Location behave like the Add Dialog to include erasing the ‘placeholder’ box and explanation... when a Location is added” — the exact ask from earlier the same day. `AttachmentsPanel.tsx` (used by ToDo Detail, `mode='reference'`) already got this split at the time; `CreateToDoForm.tsx`’s own separate, bespoke Locations markup — staged client-side since Create ToDo has no real id yet, so it never went through `AttachmentsPanel.tsx` at all — was the piece that got missed, still always rendering the `.donerow` note+button regardless of `stagedLocations.length`.

Fixed by splitting `CreateToDoForm.tsx`’s Locations block the same way: the `.donerow` note+button now renders only when `stagedLocations.length === 0 && !locationFormOpen`; once a Location is staged, only a bare `.fieldact` Add Location button remains, matching `AttachmentsPanel.tsx`’s own `mode='reference'` branches exactly. `ToDoDetailForm.tsx`’s own `<AttachmentsPanel mode="reference" .../>` call was re-checked and is already correct — no code change needed there; if it still isn’t showing the fix live, the most likely explanation is a deploy that hasn’t picked up today’s earlier `AttachmentsPanel.tsx` change yet, not a code gap.

Mockup not updated — `WYP_create_todo_palette1.html`’s Attachments segment still shows the pre-Locations locked “Subscription feature” state from before real Attachments/Locations existed; converting it to a real subscriber-tier demo (staged form, staged list, empty/populated toggle) is a larger lift than this fix and wasn’t asked for — same already-flagged gap as the rest of the Locations/Attachments build (`design/README.md`).

```
npx tsc --noEmit/npm run lint clean.
```

2026-08-14 — Locations empty-state box unified with Attachments/Dialog (dropped the “Note:” band)

Owner, two annotated screenshots (ToDo Detail marked “What I now see,” Create ToDo marked “Preferred method”): wanted Locations’ empty state to look like the plain bordered box Add Dialog uses, not the tinted “Note:” band.

Tracing it: `AttachmentsPanel.tsx`’s `mode = 'file'` (real Attachments) empty state has always used a `.frow + .actlabel` bordered box — the exact same component Dialog’s own empty state uses. `mode = 'reference'` (Locations) was routed through a separate `showReferenceNote` branch instead, rendering `.donerow/.donenote` (a tinted strip with a bold “Note:” prefix) — a leftover from earlier the same day, when the note was still meant to stay permanently visible even once populated; the later empty/populated split (also earlier today) kept that `donerow` *styling* even after making it conditional, so Locations’ unlocked empty state ended up looking different from Attachments’ own, with no one having asked for that difference.

Fixed by deleting `showReferenceNote` and its `donerow` branch entirely — `mode = 'reference'` now falls through the exact same `.frow/.actlabel` branch `mode = 'file'` already used, just with different text inside the box: `file` mode keeps its short “Attachments (optional)” label, `reference` mode shows the fuller descriptive `referenceNote` (“Locations are URLs or File paths.”) un-bolded, matching Dialog’s own plain-descriptive-copy register rather than a “Note:”-prefixed callout. `CreateToDoForm.tsx`’s own separate, bespoke Locations markup (staged client-side, no real id yet) got the identical treatment for consistency — its empty-state `.donerow` swapped for `.frow/.actlabel` with the same text. The locked/free-tier state is unchanged in both files — `.donerow/.donenote` stays exactly right there, this was only ever about the unlocked empty state.

`npx tsc --noEmit/npm run lint clean`. No mockup change — same already- flagged gap as the rest of the Locations/Attachments build.

2026-08-15 — Print Reports rebuilt: full Dialog/Attachments per record, Category prefix, single-item print, Archive print

Owner provided three `xlsx` mockups (“Main Screen sections - Requests Sent/ Received/Todos”) with real font sizes for a portrait print layout, plus comments embedded in the sheets themselves: sort-arrow header placeholder (“#”), drop the `ToDo Due Date` column when the account’s own `todo_dates_enabled` is off, and prefix the description with `[Category]` when `Private Category` is on (“as it does in the Main Screen”). Colors were explicitly left flexible (“designed to follow the app standards and can be adjusted accordingly”), so the build maps to existing app tokens/weights (Overdue red, Done grey, bold names) rather than the spreadsheet’s own literal font-weight/color choices.

Supersedes the 2026-08-13 Print Reports feature — that version showed a Dialog/Attachments *icon* only (count > 0). The new layout expands each printed record in place: full Dialog thread (Kind body, e.g. “Answer I can do it.”), and a full Attachments (Sent/Received) or Locations (Todos) list (`.pdlg/.patt`, `app/globals.css`). None of the three Main Screen sections’ own list queries ever loaded full Dialog/Attachment *content* before, only `dialog(count)/attachment(count)` embeds or their RPC-computed equivalents (Received) — switching the always-loaded queries to full content would add real payload weight to every ordinary Main Screen load for content the on-screen row never shows. Fetched instead only at the moment Print is clicked, for just the currently visible (filtered+sorted) ids — `loadOwnedPrintDetail()` (Sent/Todos, plain owner-scoped RLS select) and `loadReceivedPrintDetail()` (Received, migration 029’s new `get_received_print_detail()` RPC — Received rows belong to a different owner, so a recipient can’t query `dialog/attachments` directly the same way, parallel to how `get_received_requests()` already handles the list-level case). `startPrint()` is now `async`, awaiting the detail fetch before flipping `printSection` (and thus firing `window.print()`), so the report never renders against stale/empty detail.

Category prefix — implemented per the owner’s literal instruction for Sent and Todos, both gated on `categoriesEnabled`. **Flagged, not silently followed everywhere:** the same comment appeared verbatim in all three sheets, but (1) Main Screen’s own Sent row has never shown Category anywhere on screen (only Todos has a `.cat` column) — added a `categories(name)` embed to Sent’s query specifically for this, so Sent print now shows Category for the first time anywhere in the app, on the owner’s own written instruction, even without an on-screen precedent to match; (2) Received print does **not** get a Category prefix — PRD §2.3 withholds Category from the recipient entirely (already enforced by `get_received_requests()/get_received_print_detail()` themselves returning no category field), so honoring the comment there as literally written would leak the sender’s private Category to the person it’s private from.

Todos’ Due/Done columns now drop entirely (not just Due, as the owner’s comment said) when `todoDatesEnabled` is off — both fields are governed by the same single toggle (migration 022), so there’s nothing for either column to show in that state; `.pcolbar.ptdc-nodates` (single-column grid) replaces `.pcolbar.ptdc` when off.

Single-item print — Request Detail’s and Todo Detail’s own Print icons now render the same `.print-report/.prow` layout for one record instead of calling `window.print()` on the live screen directly. **No sort-arrow header row** — owner, same session: “Obviously the up/down arrow for a selected sort would not be shown for a detail print of a single item.” Each screen already had its Dialog thread loaded (`dialogList`); Attachments/ Locations needed a small dedicated fetch each (AttachmentsPanel keeps its own list private). Helpers (`formatTime12h/formatPrintTimestamp/categoryPrefix/PrintDialogList/PrintAttachmentList`) are duplicated per file, matching this codebase’s established convention for small stateless helpers rather than a shared lib.

Archive print — a Print icon in WypHeader’s action slot (matching every other screen’s placement), reusing the identical record layout plus a narrow checkbox column (`.archprow/.pr0/.pbody` in `globals.css`) — “the same formats would work along with the insertion of a checkbox in its own narrow column as is done with the on-screen view” (owner). Prints `sortedMatches` — the Record Type’s currently filtered-and-sorted visible set, same “prints what you see” principle as every other Print button in the app. **Deliberately does not show the filter criteria (Recipient/Requestor, Before Done Date) anywhere in the printed header** — the owner flagged this as its own follow-up mid-session (“the Archive report needs to show selection criteria - I will work on that next”), so it’s left out here rather than guessed at; worth a return pass once he has a design for it.

Migration 029 (`get_received_print_detail(p_ids uuid[])`, `docs/Week6 - SQL history.txt`) — DRAFTED, NOT YET CONFIRMED RUN. Same `contacts.email` match against the caller’s session email as `get_received_requests()`; returns one row per matched id with `dialog` and `attachments` each as a `jsonb` array (a plain `create` or `replace` function is safe here — new function, not altering an existing `RETURNS TABLE` row shape, so migration 017’s drop-first lesson doesn’t apply).

A real lint regression, caught and fixed before shipping: Main Screen’s `startPrint()` originally sat beside `printSection`’s own state, *before* `sortedSent/sortedReceived/sortedTodos`’s `useMemo` calls in source order — syntactically fine (the closure only runs on a later click, well after render), but the React Compiler couldn’t preserve those three `useMemo`s’ memoization with a function defined earlier in the component body already referencing them, and failed lint with `react-hooks/preserve-manual-memoization`. Fixed by moving `startPrint`’s definition to just after `sortedTodos`, no behavior change — a real constraint of this codebase’s React Compiler setup worth remembering for any future closure that reads multiple `useMemo` values from a Main-Screen- sized component.

`npx tsc --noEmit/npm run lint clean`. **No mockups updated** — none of the affected screens’ static HTML has real print JS to convert; flagged in `design/README.md`, not silently skipped.

2026-08-15 — Archive print gains Selection Criteria line; fixed print reports rendering at half page width

Two follow-ups to the print-report batch below, same day.

Selection Criteria line — owner supplied three more `xlsx` mockups (“Archive - Requests Received/Recipient/Todos”)

for the Archive print report specifically, fulfilling his own stated follow-up from the batch below (“the Archive report needs to show selection criteria - I will work on that next”). Format taken verbatim: Selection Criteria: <Noun> <value or (blank)> Before Done Date <value or (blank)> for Sent/Received, Selection Criteria: Before Done Date <value or (blank)> alone for Todos (no name field to filter by). New .pcriteria (app/globals.css) sits between .ptitle and .prows; criteriaText (ArchiveForm.tsx) is built from the same noun/query/beforeDone state already driving the on-screen filter, so the printed line always matches what actually produced the list.

Flagged and resolved via AskUserQuestion: the “Requests Received” and “Requests Recipient” xlsx uploads were byte-identical — both titled “Archive Requests Sent” and both headed “Recipient.” The live Archive screen already labels this field “Requestor” for Received, not “Recipient” (NOUN, existing convention). Rather than guess whether the owner wanted “Recipient” on both regardless, or hit a duplicate-upload mistake, asked directly — owner confirmed “Requestor” for Received, matching the existing convention (no code change needed there; NOUN already resolves correctly, only the new criteria line needed to read it).

Print reports rendering at roughly half the page’s width — owner, comparing his own printed Excel mockup (full page width) against a live Sent Requests printout: “it still is centered on the page and only takes up approximately half of the available portrait width... is it a font-size change? — it seems to be...” Not a font-size issue: .print-report renders as a descendant of .frame-none (\$6.8’s 480px mobile-first app shell, max-width: 480px), and nothing had ever overridden that for print, so the printed report inherited the same 480px cap the on-screen app uses and printed centered in roughly the left half of a Letter-width page. Fixed with an @media print override on .frame-none itself (max-width: none, margin: 0, height: auto, box-shadow: none) — safe to apply unconditionally since .no-print already hides .app during print, so nothing on-screen is affected; the rule only ever takes effect while printing. Applies to every screen’s print report (Main Screen’s three sections, Request/ToDo Detail’s single-item print, and Archive), not just Sent, since all of them render inside the same .frame-none shell. Not visually confirmed in this sandbox (no headless browser reachable, same limitation noted elsewhere in this project) — worth a quick owner confirmation on the next printout.

Font sizes normalized to the owner’s exact spec, same day: “font sizes of 15 for the report title and 11 for everything else... 14, not 15.” .ptitle (the one large heading per report — “Sent Requests (Done)”, “Request Detail,” etc.) is now 14px; every other print-only class that wasn’t already 11px (.pmast-brand, .pnm, .ptime, .pempty, .pdsc) dropped to match. .pmast-time, .pcolbar, .pdt/.pdn/.pdue, .pcriteria, .pdlghead/.pdlgitem/.patthead/.pattitem were already 11px.

npx tsc --noEmit/npm run lint clean.

2026-08-15 — Print reports: dropped the duplicate masthead, bumped and differentiated font sizes

Owner tested a real printout (Sent Requests) after the width fix below landed and flagged three more issues from one screenshot of Chrome’s print preview, plus a fourth (Archive) reported separately. Fixed the first three; the fourth needs the owner’s own follow-up (see below).

Duplicate masthead removed. The screenshot showed two “Would You Please” + date/time lines stacked at the top of the page: Chrome’s own default print header (date/time top-left, page title top-center — “Headers and footers” in More settings, on by default in his test) and this app’s own .pmast row underneath it. Owner: “the masthead of ‘Would You Please’ and the as of date/time are repeated at the top of the report [in reverse order - the order does not matter].” Since the browser’s own header isn’t something this app can reliably suppress (it’s a per-user print-dialog setting, not CSS-controllable), the fix removes our own .pmast — the duplicate under our control — from all four print reports (Main Screen’s three sections, Request Detail, ToDo Detail, Archive). printGeneratedAt state and the formatPrintTimestamp helper are now dead in every one of those four files and were removed along with the JSX, not left orphaned.

Font sizes bumped and differentiated. Two more issues from the same screenshot: “the font-size for the report title is not differentiated from the rest of the report” and “all font sizes are too small.” This revises the initial pass from earlier the same day (title 14px, body 11px, per the owner’s own stated numbers at the time) — seeing a real printout changed his assessment. New scale: `.ptitle` 18px, every other print-only class 13px (was 11px: `.pmast-brand` is gone entirely; `.pnm`, `.ptime`, `.pempty`, `.pdesc` bumped along with the ones already sharing the 11px value). Date/Due/Done columns (`.pcolbar.psr/.ptdc`, `.pr1/.pr1.ptd`) widened 70px → 78px so “MM-DD-YY” still fits cleanly at the larger size.

A real mistake caught before it shipped: the first attempt at the font bump used a file-wide `replace_all` on `font-size: 11px;`, which silently changed 14 unrelated live-app rules (`.flabel`, `.subnote`, `.attachpanel`, `.plabel`, `.attitem`, `.minreq`, `.subbanner`, `.adbox`, `.colbar`, `.dt/.due/.dn`, `.field`, `.dlgre`) to 13px along with the 10 actual print-report rules — none of those on-screen classes were supposed to change. Caught immediately by grepping the diff for every remaining 11px/13px occurrence and cross-checking each one’s selector; all 14 non-print rules were reverted to 11px, leaving only the 10 print rules at 13px. Flagged here as a caution for any future file-wide find/replace on a value this common in a shared stylesheet — a scoped edit (or per-selector edits, as used for the revert) is safer than `replace_all` whenever the search string isn’t unique to the feature being changed.

Archive still printing a near-blank page, reported separately, not yet resolved. Owner: “for the Archive report, it is only printing a single blank page for each of Sent and Received Requests, and the ToDo... I tested all Archive reports with the same result.” Most likely explanation, not yet confirmed with the owner: Archive’s list (and therefore its print) is designed to stay empty until at least one filter — Recipient/Requestor and/or Before Done Date — is entered (existing behavior, unchanged today); printing with no filter set would show the title, an empty Selection Criteria line, and “No records match.” — closer to “a nearly empty page” than a truly blank one, but plausibly read that way in a quick check. Flagged rather than guessed at further — needs the owner to confirm whether a filter was applied before printing, or share another screenshot, before changing anything.

Update, same day: Archive’s print turned out fine once a filter was applied (owner’s next screenshot showed real Received rows) — no bug, just the empty-until-filtered design doing exactly what it’s supposed to. That same screenshot flagged the 18px/13px pass as still too small. The owner looked up the actual conversion he needed: he’d been thinking in Excel font points, not CSS pixels — Excel 14pt (his title size) is 18.67px, Excel 11pt (his body size) is 14.67px, both round differently than a naive 1:1 pt-to-pixel reading would suggest. Retuned to `.ptitle` 19px / everything else 15px to match his exact numbers; `.pcolbar.psr/.ptdc` and `.pr1/.pr1.ptd`’s fixed date-column widths widened again, 78px → 92px, to keep “MM-DD-YY” comfortable at the larger size.

```
npx tsc --noEmit/npm run lint clean.
```

2026-08-15 — Print reports: pt units for font size, white page background, Due/Done Time inline

Fourth round of print-report feedback the same day, following the 19px/15px retuning below. Owner supplied a side-by-side screenshot (“WYP App printed” vs. “Excel printed”) showing WYP’s text still visibly, dramatically smaller than Excel’s own 11pt Arial, “even with the 1.333 conversion multiplier” — past what any rounding difference could explain — plus three new xlsx mockups with an Arial-based design (previously Aptos Display) and a fuller formatting spec: white page background, Brand Blue title/column-header text, alternating Row Tint record shading, and rule lines above/below the column header. He explicitly sequenced the work: get the font size right on one report first, hold off on the color/banding/rule-line treatment.

Switched every print font-size from px to pt. Rather than chase another round of px conversion, `.ptitle` and the ten body-text print classes now specify 11pt/14pt directly — the same physical unit Excel’s own font-size picker uses, removing any px/DPI/rounding step for a browser or printer to get wrong. If this still prints small, the cause isn’t a unit conversion and needs a different diagnosis (a stale deploy is the leading suspect — this batch hasn’t been tested against a live printout yet). Verified no unrelated rule was touched: grepped `font-size: 15px;` before editing,

confirmed the only non-print hit was `.attremove` (untouched), edited the ten print classes individually rather than with `replace_all`, then re-grepped to confirm a clean split.

Page background forced white in print. `html, body`’s own `@media (min-width: 520px) { background: #eef1f5 }` rule (the on-screen desktop-frame letterboxing color) was never scoped away from print, and a printed page is wide enough to match that breakpoint — so a browser with “Background graphics” enabled would tint the whole page grey. Added `html, body { background: #fff !important; }` inside the existing `@media print` block, alongside the `.frame-none` width override from the batch below.

Due/Done Time now renders inline, same line as the date, not stacked beneath it. Owner, mid-turn: “In my examples, the date/time is shown as 7/15/2026 8:30:00 AM” — then self-corrected twice, to “7/15/2026 8:30 AM” (no seconds) and finally “7/15/26 8:30 AM” (2-digit year). New `formatMDYSlash` helper (duplicated in `MainScreen.tsx` and `RequestDetailForm.tsx`, this codebase’s usual convention for small formatters) produces exactly that shape — no zero-padding, slash-separated, 2-digit year — **scoped to just the Due/Done columns that can carry a Time**, not applied to the plain Date (`created_at`) column or to Archive/ToDo Detail’s date-only columns, which keep the existing `formatMDY` dash convention. Flagging this asymmetry rather than silently generalizing it: a row’s Date and Due columns now use two different punctuation styles (08-15-26 vs. 8/15/26) side by side. `.ptime` dropped its `display: block` (the CLAUDE.md on-screen convention, “the time renders beneath the date,” is untouched — this only affects print) and a two-space-prefixed `<span>` supplies the gap between date and time. Touched: Main Screen’s Sent and Received print rows, and Request Detail’s single-item print (its Done Date can also carry a Time; Main Screen’s own Done column never does, but shares the same CSS).

Due and Done columns widened, not just Due. Owner separately noted (same turn) that he’d widened the date columns in his own mockup “because of the option to also show a time” — his three new `xlsx` files didn’t show an explicit width for those columns to copy exactly, so this widens `.pr1/ .pcolbar.psr`’s Due and Done columns from 92px to 150px each (Date stays 92px, it never carries a time) rather than guess a precise figure from the mockup. Worth re-checking against a real printout once deployed.

Explicitly not done this batch, per the owner’s own sequencing request: Brand Blue title/column-header color, alternating Row Tint record shading, and the above/below rule lines on the column header. `--brand-blue (#2A5FC8)` and `--row-tint (#F6F7F9)` already match his Excel colors exactly, and `--strip (#E5ECF7)`, already used for `.pcolbar`’s background) matches his column-header fill too — so implementing the color/banding pass should be mechanical once the font-size fix is confirmed working, no new tokens needed. Also unresolved: since all four print reports share one CSS class system, this change (and the pending color/banding pass) necessarily applies to every report at once — there’s no way to test “just Archive Requests Sent” in isolation without duplicating classes under new names, which hasn’t been requested and wasn’t done here.

```
npx tsc --noEmit/npm run lint clean.
```

2026-08-15 — Print reports: real Archive blank-page bug found, stuck-print-state bug fixed everywhere, missing column headers added to Request Detail/ToDo Detail

Fifth round of print-report feedback the same day. Font size is still unresolved (see the end of this entry) but three concrete, reproducible bugs were found and fixed.

Archive’s actual blank-page bug, found while investigating a second report (“the second attempt to print anything with a specific print icon does not respond... If I print with a different icon and come back to the original icon, it works once again”). `ArchiveForm.tsx` had two Print icons, not one: the real one in `WypHeader`’s action slot (wired to `startPrint()`, the whole print-report pipeline), and a second, leftover one in the record-type band, calling a bare `onClick={() => window.print() }` — never wired to `showPrint/printDetail` at all. Clicking it opened the print dialog while `.print-report` was never mounted (`showPrint` stayed false) and everything else was hidden by `.no-print`, producing a genuinely blank page. This is almost certainly the exact bug from the owner’s very first Archive print report, well before the font-size investigation started — not the “empty until filtered” explanation offered

at the time. Removed the stray button entirely; WypHeader’s own Print Archive icon is the only Print control this screen needs.

Stuck-print-state bug, all four print reports. Each screen’s print trigger followed the same pattern: a piece of state (`showPrint` boolean, or `printSection: 'sent' | 'received' | 'todos' | null`) flips on click, a `useEffect` keyed on that state calls `window.print()`, and the browser’s `afterprint` event resets it back to hide the report. Clicking the *same* print icon twice in a row set the state to the value it already held — no real change, so React never re-ran the effect, and the second click did nothing. `afterprint` doesn’t fire reliably in every browser/print flow (a known cross-browser quirk, more common with “Save as PDF” or a cancelled dialog than a real printer), so the state could get stuck in its “already printing” value indefinitely — explaining why navigating to a different screen’s Print icon and back “fixed” it: that navigation remounts the component, resetting the state fresh. Fixed with a new `printTick` counter in all four files (`MainScreen.tsx`, `ArchiveForm.tsx`, `RequestDetailForm.tsx`, `TodoDetailForm.tsx`) that strictly increments on every `startPrint()` call and is what the effect is actually keyed on now — guarantees a real dependency change every time, regardless of whether the previous print’s `afterprint` ever fired.

Column headers added to Request Detail and Todo Detail’s own single-item prints, not just Archive. Owner: “The Request Detail print does not have a column heading... please check to make sure they all have column headings.” The original 2026-08-15 design call (“no sort-arrow header row — nothing to sort with one record”) had been implemented as *no header row at all*, not just no arrow — overreaching the owner’s own stated reasoning. Both screens now show plain, static (arrow-less) labels: Request Detail gets “To / Due / Done”, Todo Detail reuses Main Screen’s own `.pcolbar.ptdc/.ptdc-nodates` verbatim (“Description” alone, or “Description / Due / Done” when `todoDatesEnabled`). Fixing Request Detail’s header surfaced a real, separate layout bug: its print row has only three fields (To/Due/Done, no separate Date/created_at column) but was reusing `.pr1`’s shared 4-column template built for Main Screen’s Sent/Received rows (Name/Date/Due/Done) — the three spans were silently shifting one column left, squeezing Due into Date’s 92px slot and Done into Due’s 150px slot, with Date’s own 150px slot going unused. New `.pcolbar.detail3/.pr1.detail3 (1fr 150px 150px)` fixes both the header and the body row.

Font size: still unresolved, and the owner’s newest screenshot doesn’t settle it either way. Confirmed via the Vercel MCP that the commit with the pt-unit fix (a5e4241) was READY in production roughly 13 minutes before the owner’s “still unchanged” report — not a stale-deploy or caching issue. The owner then asked a sharp diagnostic question: does WYP specify a print width, and could the printer be scaling to fit margins? Answer given: no `size`: is set on `@page` (only `margin: 0.5in`), so page dimensions come from whatever the print dialog’s own paper size is; no CSS `zoom/transform` is applied anywhere. That leaves the print dialog’s own **Scale** control — separate from both CSS and paper size — as the most likely remaining explanation, especially since some browsers/drivers persist a per-printer Scale setting across print jobs, and this printer’s Scale may have gotten set below 100% back when the report was printing at half-width (before that bug was fixed) and never reset. **Not yet confirmed either way** — the owner has not reported back on the actual Scale field. Separately worth noting for next time: Chrome’s Print Preview pane auto-fits the whole page to the preview window, so relative font size *looks* similar across screenshots taken at different times regardless of the real point value — a screenshot comparison alone can’t confirm or rule out a genuine size change; only an actual printed page, or a saved PDF opened and measured/zoomed outside the preview pane, can.

Resolved, same day: it was the Scale field, exactly as suspected. Owner checked the print dialog directly and found Scale set to “Custom, 75%” — independent of anything WYP controls, and unrelated to his normal printer use. Setting it to 100% fixed both the preview and the actual printed output immediately, no code change needed. Notably, after signing out and back in, the next print’s Scale was already 100% — confirming this is a setting the browser/print driver (Chrome or the OS’s HP driver) persists per-printer across sessions, not a WYP or per-account setting, and most likely got set to 75% during the earlier half-page-width bug (2026-08-15, below) as an automatic or manual attempt to make oversized content fit the page. The three rounds of px/pt font-size changes earlier the same day were real, correct fixes to genuine (if much smaller) sizing issues, but were never going to be visible against a 75%-scaled print regardless of the CSS value used — which is also why none of those three passes ever “worked.”

```
npx tsc --noEmit/npm run lint clean.
```


2026-08-15 — Request Response / Response Detail: donerow wording for a Request already Done on load

Owner-reported: opening either screen for a Request that was **already** marked Done before this visit showed “This Request is now marked as Done, just click Send.” — worded as if the visitor had just taken an action that still needs sending, when in fact nothing had changed yet in this session.

Fixed by capturing a new `alreadyDoneOnLoad` flag, set once from the RPC payload’s own `done_date` at load time and never touched again (not by quick-Done, not by a manual edit, not by Send). The donerow’s existing three-way reactive message (empty / filled-not-sent / sent-this-session) gains a fourth branch, checked between “sent this session” and “filled, not yet sent”:

- Done Date empty — “Note: For a quick response, click Done and Send.”
- Filled, `sendConfirmed` this session — “This Request is now marked as Done and has been Sent.” (unchanged, 2026-08-11)
- Filled, not sent this session, **and already Done on load** — “This Request is reported as completed.” (new)
- Filled, not sent this session, not already Done on load (i.e. quick-Done or a manual edit just now) — “This Request is now marked as Done, just click Send.” (unchanged)

Applied identically to `RequestResponseForm.tsx` (anonymous `/r/[token]`) and `ResponseDetailForm.tsx` (signed-in `/requests/[id]/respond`) — same donerow, same fix, matching every other instance of this pair being kept in lockstep. No mockup change — neither mockup’s static HTML demonstrates this reactive state. `npx tsc --noEmit/npm run lint clean`.

2026-08-15 — Contacts print report (icon, migration 030, .pcon- CSS)

Owner uploaded his own “Contacts list.xlsx” mockup and asked for a matching Print icon on the Contacts screen. Styling pulled from the `xlsx` via `openpyxl` inspection rather than his prose description, where the two differed: his message said the body font was reduced from 11pt to 9pt: the `xlsx` itself measured body text at 10pt, with only the Sent/Rec’d column headers and their values actually at 9pt, italicized. Went with the measured file.

Sent/Rec’d definition, clarified via `AskUserQuestion` (three options offered: “Sent = requests I sent them, Rec’d = requests they sent me”; “Sent = requests I sent them, Rec’d = how many of those they’ve completed”; free text) — owner picked the first, matching Main Screen’s own existing Sent/Received vocabulary.

Migration 030 — `get_contact_request_counts()`, confirmed run by the owner 2026-08-15. Sent is trivial (owner-scoped count of requests grouped by `contact_id`, no new privilege). Rec’d needed one join beyond `get_received_requests()`’s (migration 012) existing pattern: that function matches a Request’s own Contact email against the caller’s session email, but has only ever needed the sender’s display name for output, never their real account email. Attributing a Received Request back to *which contact row* sent it means knowing the sender’s actual login email — added via a join to `auth.users` inside this new SECURITY DEFINER function, reading `auth.users` directly, the same precedent already established by `can_create_account()` (migration 015): only the function’s owner role can see that table; a caller only ever gets integer counts back.

New .pcon- CSS namespace, not a reuse of `.pr1/.pcolbar` — this report has six columns (Name/Email/Phone/Time Zone/Sent/Rec’d) where every other print report has three or four, and the two count columns need their own smaller italic treatment the others don’t. Print title reads “Contacts”, not the `xlsx`’s own “My Contacts” — flagged rather than silently decided, since this screen dropped “My” app-wide on 2026-08-09 and the print should match the screen’s current name.

Built with the `printTick`-counter pattern from the start (see the “stuck-print-state” entry below), so this report never had the bug the other four did. `npx tsc --noEmit/npm run lint clean`. No mockup change — `WYP_contacts_list_palette1.html` has no print JS to convert.

2026-08-15 — Reminder checkbox on Create Request/Request Detail, migration 031, replaces the Tight-window advisory

Owner mocked up a user-facing Reminder opt-in checkbox (“Reminder - send on the morning before unless it is marked Done.”) on Create Request and asked for a review before building it. Reviewed against the existing PRD §7.3 Tight-window infrastructure (`isTightWindow`, the passive advisory paragraph on Create Request, and the still-unbuilt day-before scheduled job); five open items were flagged and resolved by the owner directly:

1. The old passive advisory paragraph (“This Due Date is less than 24 hours away...”) is removed, not kept alongside the new checkbox.
2. The actual day-before send remains unbuilt — this batch ships the checkbox and its persisted preference ahead of that job, same sequencing this app already used for Attachments and the ToDo Status chip.
3. The checkbox appears on both Create Request and Request Detail, not Create Request alone.
4. Not a subscription gate — plain disabled `.checkrow`, not `.is-locked`. Default checked.
5. “Whose morning” (for the eventual day-before job) is the Request recipient’s own stored Contact Time Zone, not the sender’s. Owner added a refinement on top: the checkbox itself can’t be usefully set until a Contact and Due Date both exist (Create Request only — Request Detail’s Recipient is already fixed), and separately greys out again if the Due Date is too soon for a Reminder to have a real day to send on. Each state gets its own native `title` tooltip: “Please select Contact and Due Date before modifying the Reminder.” and “A Reminder is not available due to the short lead time.” (owner’s exact wording, corrected once from an earlier draft — “A Reminder is...” not “Reminders are...”).

Rule change, not just a UI addition: the owner explicitly rejected trying to hit a precise 24-hour window (“a day-before would suffice... if a Request is set for the next day, no reminder is needed”), extended one step further to grey the checkbox out through the day after tomorrow too, not just tomorrow. `isTightWindow/TIGHT_WINDOW_HOURS` (clock-precise, PRD’s own “proposed default, not yet confirmed” 24-hour figure) is removed outright and replaced with `isReminderEligible()/MIN_DAYS_FOR_REMINDER` (`app/src/lib/email.ts`) — pure calendar-day arithmetic, Due Date must be more than two calendar days out (i.e. at least 3 days away). This also retires Due Time from the calculation entirely, matching the owner’s own “the wording... could be used for requests either without a Due Time or if there was a Due Time set.”

Migration 031 — `requests.reminder_enabled` boolean not null default `true`, confirmed run by the owner 2026-08-15. No column-level grant needed, same as `archived_at` (migration 028) — “requests: owners update own” (migration 002) is a full row-level UPDATE policy with no column restriction. Nothing reads this column’s value to gate an actual send yet (the day-before job doesn’t exist); `app/api/email/send-request/route.ts`’s own `reminderPromised` combines `isReminderEligible()` with this column to decide whether the Initial Request email’s “a reminder will arrive” sentence is honest, same purpose the old `tightWindow` param served, now correctly named.

Placement diverges between the two screens, flagged rather than forced to match: Create Request can place the checkbox beside a lone Due Date field when Due Time is off (`.checkrow-inline`, a new `.frow` modifier, and a paired `.ffloat.picker.native.due-with-reminder` class reproducing the existing §6.33 220px Safari-format cap that `:only-child` alone can no longer supply once the checkbox joins as a sibling) — or as its own standalone row after Attachments when Due Time is on. Request Detail always uses the standalone placement regardless of Due Time, since its Due Date row is never alone — Done Date is always paired with it, on or off. New CSS section in `app/globals.css` (`.checkrow-disabled`, `.checkrow-inline`, `.due-with-reminder`) documents the reasoning inline. New component identifier §6.37 PROPOSED — not drawn in any mockup yet, flagged in `design/README.md`.
`npx tsc --noEmit/npm run lint clean`.

Confirmed working, live-tested by the owner 2026-08-15 — presentation and interaction on both screens, including the too-soon-Due-Date disabled state, behave as designed.

2026-08-15 — Sign-in session persistence investigated; remembered-email fallback shipped

Owner: “I need to Sign In to the App more often than I expect. After I have logged-in and close the browser and later re-open it, I go to `wyp-three.vercel.app`. Instead of still being logged-in on the same device as I expect, I am provided the landing page and need to go through the login and click the link in an email.” Asked whether that’s expected, and, if so, for at least a remembered-email fallback.

Investigation found this app’s own session-persistence code is already correct, not the cause. Read through the full chain again end to end: `supabaseClient.ts`’s `hybridStorage` routes the actual Supabase session token to `localStorage` (survives a full browser close) whenever `shouldRemember()` is true — the default, and “Keep me signed in on this device” is checked by default on `/login` — with `persistSession: true/autoRefreshToken: true` on the client config; `app/page.tsx` and `RequireAuth.tsx` both already use `getSession()`, not `getUser()` (the 2026-08-13 fix for a *related but different* bug — a live round-trip to Supabase’s Auth server misread as “signed out” on a transient network hiccup right after reopening); `/auth/callback/page.tsx` establishes the session the same way. No bug found in any of these — the architecture is sound for the scenario as described.

Given that, the two most likely explanations are both outside this codebase, and neither is fixable from here:

1. A browser (or browser extension/privacy setting) configured to clear cookies and site data — which includes `localStorage` — when all windows close. Common in Chrome/Firefox privacy settings, Brave’s default shields, or if the tab in question was ever opened in a private/ incognito window.
2. A Supabase project-level Auth session setting (Authentication → Sessions in the Supabase dashboard — inactivity timeout / time-boxed sessions) forcing re-authentication on a schedule independent of anything this app’s own code controls.

Neither can be confirmed or ruled out from this session — there’s no Supabase dashboard access here, and no way to inspect the owner’s own browser settings. Flagged for the owner to check directly: Supabase dashboard → Authentication → Sessions, and the browser’s own cookie/ site-data-on-close setting for `wyp-three.vercel.app` specifically.

Shipped the owner’s own requested fallback regardless of root cause: `/login`’s Email field now remembers the last-used address (`wyp.lastEmail` in `localStorage`), pre-filling on return — tied to the *same* “Keep me signed in” checkbox rather than a separate toggle, since an unchecked box already promises “leave no trace on this device,” and remembering just the email while otherwise leaving nothing behind would quietly break that promise on a shared/public computer. Saved (or cleared, if unchecked) at the same point `setRememberMe()` already runs, in `sendLink()`. Doesn’t reduce how often a magic-link click is needed — only saves re-typing the address each time. `npx tsc --noEmit/npm run lint clean`.

Follow-up, same day — owner shared a screenshot of Supabase’s own Authentication → Sessions dashboard page, narrowing cause #2 above to a specific, likely culprit. Access token expiry is 3600s (1 hour) — this is the *recommended default*, not a misconfiguration, and isn’t the problem on its own: `autoRefreshToken: true` and the refresh token stored alongside the access token exist precisely so a 1-hour-old access token gets transparently replaced, including right after a long-closed browser reopens. The more likely culprit is the pair directly below it: **“Detect and revoke potentially compromised refresh tokens” is ON, with a 10-second reuse interval.** This is Supabase’s refresh-token *rotation* protection — every time a refresh token is used, it’s replaced with a new one, and the old one is only allowed to be reused within that 10-second grace window; a reuse after that window is treated as a compromised-token replay and the *entire session family* is revoked, not just the one stale request. The known failure mode this creates: a browser that reopens more than one previously-open tab at once (Chrome’s “Continue where you left off,” Firefox’s “Restore previous session,” or simply having WYP open in two tabs/windows across a close) can race two tabs’ independent `autoRefreshToken` timers against each other — one tab rotates the token first, and if the other tab then presents the now-superseded token more than 10 seconds later, Supabase reads that as a replay attack and kills the session outright, forcing a fresh sign-in. This fits the owner’s own description (works fine on a fresh single-tab open; breaks specifically after closing and reopening the browser) better than a plain 1-hour access-token expiry would. **Not a code fix — this is a Supabase dashboard setting, no app change involved.** Recommended for the owner to try: raise “Refresh token reuse interval” from 10s to something more forgiving of a multi-tab reopen (e.g. 30–60s), and/or check whether his browser is configured to restore multiple previous tabs on launch. Disabling the compromised-token detection entirely would also remove the failure mode but gives up real replay protection — not recommended as the

first move. Unconfirmed pending the owner trying the reuse-interval change and reporting back.

2026-08-15 — PRD v12.10: §9.6 My Phrases added to the Future Features Roadmap

Owner: *“Please add to the Phase 10 Development Roadmap (and where else it should be) a feature I call ‘My Phrases’... a subscription feature — probably most useful for business use. It would consist of up to 12 phrases (text of up to 150 characters each) with two elements for each phrase — an optional Description and Phrase Text (essentially like the Add Location feature). Housekeeping would have a Phrases Task, which would open a list of phrases and be able to Add Phrase or select/edit an existing phrase much like the Contacts Task. The phrases would be utilized by an Account optional button placed above and the right of the Request Description.”*

“Phase 10 Development Roadmap” resolved to §9, not §10. The PRD’s §10 is literally titled “10. Phased Development Roadmap” — almost certainly what got read back as “Phase 10.” But §10 itself has no per-feature entries; it’s a short narrative paragraph about the Phase 0/Phase 1 boundary, with the real work-breakdown detail in §11. Individual future features are catalogued in §9 “Future Features Roadmap” instead — §9.1 through §9.5 (Archive, added 2026-08-09, is the most recent sibling and the direct precedent for this batch). Landed as **§9.6 “My Phrases”**, following that section’s own established format (bold “Not yet phased (owner request, DATE).” lead sentence, then the description), not a new backlog file.

Phase assignment left “Not yet phased,” matching Archive’s own precedent rather than guessing “Future Phase 2” (the label §9.1–§9.3 carry) — the owner didn’t discuss this against the phased roadmap, so assigning a phase would invent a scope decision rather than record one.

Tier packaging left as an open question in the PRD text itself, rather than asserting it either way. The owner called this “a subscription feature,” but the existing §9 Monetization direction note draws a real distinction between the base \$17.95/yr subscription (which unlocks exactly two things: file attachments and Request Texting) and a further tier of optional paid add-ons layered on top of it for business-style features (Recurrence, Custom Data Fields, Contact Import, Shared Todos). My Phrases matches the “business-style usage” framing of that second bucket closely enough that I added it to that paragraph’s own parenthetical list of examples, but the PRD text for §9.6 itself says packaging is undecided rather than picking one silently.

Mid-batch correction, same message thread: the original description (owner-approved, first drafted into the docx) had the per-phrase button on Create Request insert the phrase’s text directly into the Description — at the end, or at the current selection/cursor position if available. Before this was finalized, the owner simplified it: *“Actually, the simplest and best way to offer this is just a copy like Add Location offers. That way the user can just place the text anywhere they want (I have developed rich text editors with email variable insertion/bolding/etc. in the past.)”* — deliberately choosing not to rebuild that complexity here. §9.6’s final text has each phrase’s button copy its Phrase Text to the clipboard — the same Copy affordance already shipped for a Todo’s own Locations (2026-08-14) — rather than attempting cursor-aware insertion, leaving placement entirely to the user’s own paste.

Mechanically: edited docs/WouldYouPlease_PRD_v12_9.docx directly (unzip → edit word/document.xml → rezip, per the docx skill’s edit workflow), producing docs/WouldYouPlease_PRD_v12_10.docx. Three edits: (1) the Monetization-direction parenthetical gained “, My Phrases”; (2) the new §9.6 heading + body paragraph inserted between §9.5 and the §10 heading, with a fresh bookmark id (58, _Toc_MyPhrases_96) since the existing max was 57; (3) a new Schedule A revision-history line, “v12.10 §9.6 My Phrases added...”. Title page (“Product Requirements Document (PRD) — v12.10”, “Prepared: August 15, 2026”) and both footer instances (“Would You Please — PRD v12.10 — Confidential”) were bumped to match — all three occurrences were contiguous single runs, not split across fragments the way task #296’s still-deferred §9.5 replacement text is, so this batch didn’t hit that known difficulty. Validated with scripts/office/validate.py --original (clean, +3 paragraphs) and visually confirmed via a rendered PDF (title page, §9.6’s own page, and the Monetization-direction page) before replacing the working file — old WouldYouPlease_PRD_v12_9.docx left in place alongside it, matching how v12.8 was kept when v12.9 superseded it.

Follow-up confirmed done by the owner, same day: the Project’s “Canonical sources” custom instructions now name `WouldYouPlease_PRD_v12_10.docx`.

2026-08-16 — Request Detail Date: line; Description/Dialog/Notes textareas drop floating labels; Dialog cap lowered to 500 with live character counts; Initial Request email and .ics redesigned

Owner, in one message: (1) Request Detail shows no issuance date at the top of the screen, unlike Request Response/Response Detail, which both show “Date: <long date>” above the Recipient — should be added; separately, a long (480-character) Description on a live screen showed the floating label overlapping the text once the box was scrolled, “an issue in all Description text boxes using floating labels.” (2) Dialog Text allowed 999 characters where it should be 500 (his own recollection was 150, but agreed 500 was fine); when a limit is hit, a paste silently truncates and typing silently stops accepting keystrokes, neither with any visible feedback — asked for both to show something. (3) A full rewrite of the Initial Request email body and its `.ics` DESCRIPTION, from a literal template: the response link as an anchor (“Click to respond or mark as completed”) immediately after the Description, a conditional Reminder-promise sentence, one combined attachments/Dialog note, and a closing “New to Would You Please? click to set up a free account” signup link — replacing the old layout that put the link at the very end, after several sentences of boilerplate. His own reasoning: the link used to appear last specifically so its visible URL would build trust that the recipient wasn’t being sent somewhere untrusted, but on a mobile calendar app’s own event-preview (which truncates a long description with an ellipsis), that placement could hide the link from view entirely before the recipient ever saw one was being offered.

Request Detail’s Date: line — added a `formatLongDateTime()` helper (already existed, verbatim, in `ResponseDetailForm.tsx`) and a `created_at` field pulled into the existing `.select()`, rendered as a new `.metarow` directly above the existing Recipient row, matching Request Response’s own “Date:” wording and position exactly.

Floating-label overlap — root cause and the owner’s own simplification. `.ffloat`’s floated `<label>` is a position: absolute sibling of the `<textarea>`, positioned relative to the `.ffloat` wrapper. That works for a `.finput` because the input’s own value never scrolls independently of the field’s box. A `<textarea>`’s *content* does — once typed text exceeds the visible box height, the textarea scrolls internally, but the label sibling has no way to track that scroll region (a browser doesn’t expose textarea content as DOM children a CSS selector could react to), so the label stays pinned at the top of the box and visually overlaps whatever text has scrolled up underneath it. While this was being investigated, the owner proposed the fix directly: “*Perhaps, the scrollable text boxes could just have disappearing placeholder text and not utilize a floating label.*” Adopted exactly as suggested, app-wide, for every `.ftextarea` field (12 usages across 8 files: Description ×4 — Create Request, Request Detail, Create ToDo, ToDo Detail; Dialog Text ×6 — the same four plus Request Response and Response Detail; Notes ×2 — Add Contact and Contact Detail). Each field drops its `.ffloat` wrapper and `<label>` element, gains `ftextarea-plain` (a new CSS class restoring normal 10px top padding — the base `.ftextarea`’s 20px existed only to leave room for the now-removed floated label) plus a real `placeholder=<Field Name>` and matching `aria-label`. The pre-existing `.ffloat .flabel` CSS combinators were left in place rather than deleted — they’re `.finput`-only in practice now, but harmless no-ops for any `.ftextarea` that no longer sits inside `.ffloat`.

Dialog cap and live character-count feedback. New per-file `DIALOG_MAX = 500` constant (was 999 everywhere it appeared) plus `DESCRIPTION_MAX = 500` and (Add Contact/Contact Detail only) `NOTES_MAX = 500`, keeping each field’s `maxLength` and its on-screen count always in sync. New global `.charcount` CSS class — a persistent small muted “N/MAX” line under every capped `.ftextarea`, turning `--alert-red` and bold once `N >= MAX`. This answers both halves of the owner’s ask with one mechanism: a user mid-paste sees the count jump straight to the capped value (visible truncation, where before there was none), and a user still typing at the cap sees the same red “500 / 500” the moment further keystrokes stop landing, rather than silently hitting a wall with no visible cause.

Initial Request email — reordered, now real HTML. `buildRequestEmailBody` (plain text, link-last) removed; replaced by `buildRequestEmailHtml/buildRequestEmailText` in `app/src/lib/email.ts`, both built

from a shared `RequestEmailBodyFields` shape and following the owner’s literal order — call-to-action link, Description, conditional Reminder sentence, combined attachments/Dialog note, closing signup link — with `buildRequestEmailHtml` rendering the link and the signup line as real `<a href>` anchors (the owner’s original show-the-URL rationale is now served by the link *text itself* being legible and by Reply-To already pointing at the sender’s own account email, rather than by the raw URL appearing in the body). A plain-text `buildRequestEmailText` companion was added for the email’s multipart/alternative part — not explicitly requested, standard practice alongside an HTML body, same content and order with bare URLs instead of anchors. New `escapeHtml()` helper protects the one piece of this email that’s real user text (the sender’s own Description) before it’s embedded in HTML, converting `\n` to `<br>` to preserve any line breaks the sender typed. The closing signup link now points at the bare site root (`siteUrl`, no path) rather than the old `/login` destination — `/login` predates the real marketing `LandingPage.tsx` (shipped 2026-08-13), which now serves a proper sales-first page at `/` with its own CTAs, so sending a new recipient there first is no longer a dead end. Subject line and From/Reply-To were explicitly left unchanged, per the owner’s own “[un-changed from presently formatted text]” instruction. `app/api/email/send-request/route.ts` updated to import and call both new builders and attach `html` alongside the existing `text` part of `transporter.sendMail(...)`.

Matching .ics DESCRIPTION rewrite, and a same-day follow-up correction on ordering. `buildIcsDescription` in `app/src/lib/ics.ts` rewritten to the same structure and reasoning — RFC 5545 TEXT has no markup, so the link renders as a bare URL (most calendar/mail clients auto-linkify one on their own) — and drops the old “A Would You Please Request from `<name>`:” opener entirely, matching the owner’s literal template, which has no name-attribution line (that already lives in the email’s own Subject/From). First draft put the Description before the link, matching the owner’s very first example; **the owner then corrected this same day** — *“the example format for the email should have the ... reversed so the WYP Request Link is the first item shown in the Body of the message”* — so both `buildIcsDescription` and `buildRequestEmailHtml/buildRequestEmailText` were re-ordered to put the call-to-action link first, ahead of the Description, in every representation (HTML, plain-text email, and the `.ics` body) for consistency. `reminderPromised` defaults to `false` — the two client-side “Add to Calendar” call sites (`RequestResponseForm.tsx`, `ResponseDetailForm.tsx`) only ever have a Request’s `due_date` in hand, never the sender’s own `reminder_enabled` preference, so they can’t honestly promise a reminder either way; only the emailed `.ics`, built server-side in `send-request/route.ts` (which already computes the real value for the email body), passes it through explicitly. `buildIcsContent` gained a new optional third parameter, `options?: { reminderPromised?: boolean }`, and now derives `siteUrl` internally from `new URL(link).origin` rather than requiring every caller to supply a separate site-root value it may not have. `send-request/route.ts` passes `{ reminderPromised }` through at its own call site; the two client-side call sites are unchanged, relying on the new default.

`npx tsc --noEmit/npm run lint` both clean for the full batch.

2026-08-16 — Archive: filters and checkbox selection now reset on a fresh visit, not just narrowed to the Detail round trip

Owner, testing: selected Sent Requests matching a Recipient + Before Done Date filter, hand-deselected a few, closed Archive back to Main Screen, and reopened Archive later in the same login session — the same filtered records reappeared, but none were checked anymore. His own read: “logically” they should still be selected, since nothing about them changed; failing that, his stated preference was that an Archive “session” last only as long as the screen stays open — a later reopen should reset the Recipient/Requestor and Before Done Date filters and “show nothing selected.”

Root cause: the 2026-08-14 fix for a different bug was too broad. `ARCHIVE_QUERY_KEY/ARCHIVE_BEFORE_KEY/ARCHIVE_DESELECT` were added to `sessionStorage` so a row click into a Request/ToDo/Response Detail screen (`router.push`, returning via that screen’s own `router.back()`) wouldn’t wipe out the filter and selection the owner had just built — Archive fully remounts on that round trip, same as every other list screen in the app. But `sessionStorage` survives *every* navigation in the tab, not just that one round trip — including a full Close back to Main Screen and a later, unrelated return to `/archive`, which the owner’s testing showed behaving identically to the Detail round trip even though he considers them different situations.

Fix — a marker distinguishes the two round trips. New `ARCHIVE_ROUNDTRIP_KEY` (`wyp.archiveDetailRoundTrip`) in `sessionStorage`: `openDetail()` sets it immediately before `router.push`-ing to a row's Detail screen. On mount, the three `useState` lazy initializers for `recipientQuery/beforeDone/deselected` check `isArchiveRoundTrip()` — if the marker is present, they restore from storage exactly as before; if absent (a fresh arrival at `/archive` from Main Screen, a reload, or a new tab), they start blank instead, matching the owner's stated preference exactly (no filter match, no records shown, nothing checked). A separate mount effect then clears the marker, so the *next* mount defaults to “fresh” again unless `openDetail()` sets it once more. Record Type (`ARCHIVE_TYPE_KEY`) and the three sort-order keys are untouched — the owner's report was specifically about the filter fields and selection, and an empty filter already shows nothing regardless of which Record Type chip or sort order is active.

Implementation note: the first draft reset the three pieces of state from inside a `useEffect` via `setState` calls, gated on the same marker check — this compiled but failed `npm run lint`'s `react-hooks/set-state-in-effect` rule (an unnecessary extra render for a value already known synchronously at mount). Moved the marker check into each `useState`'s own lazy initializer instead, which React guarantees runs exactly once per mount before first paint — no `setState` call needed, and the marker-clearing effect that remains does only a `sessionStorage.removeItem`, not a state update.

`npx tsc --noEmit/npm run lint clean.`

2026-08-17 — Landing page final CTA band revised; new “Who benefits” section added to landing page and sales one-pager

Owner uploaded a Word doc (“WYP Landing page and Sales literature change”) with two asks, plus a reference image. (1) The landing page's final CTA band should have “Start free today at wouldyouplease.com” removed, “since this is being read at the wouldyouplease.com site” — his own image showed the intended replacement wording and layout. (2) A new section, just before “Coming with a subscription,” matching existing sales literature and landing page fonts/colors: “WHO BENEFITS FROM WouldYouPlease? Busy people who assign tasks - including for a wide range of business, professional, community, religious, government, and political organizations, as well as individuals.” — he explicitly invited light copy improvements, and noted the “implied, not claimed” framing was deliberate (describing categories of people the product could serve, not asserting who currently uses it).

Final CTA band. `LandingPage.tsx` and `design/marketing/WYP_landing_page.html`'s `.ctaband` dropped the old `.big` headline (“Start free today at wouldyouplease.com”) and `.sub` line entirely, replaced with two equal-weight bold lines under a new `.lead` class (“No credit card. No download. No setup.” / “Send your first request in under a minute.”), matching the owner's reference image exactly. The price line's wording also changed to match the image, from “for advanced features, subscription \$17.95/yr” to “Advanced features with subscription \$17.95/yr.” Both `landing.css` and the mockup's own embedded `<style>` were updated in lockstep (`.ctaband .big/.ctaband .sub` rules replaced with `.ctaband .lead`), keeping the two files' styling identical per this project's own convention. The sales one-pager's own CTA band (`docs/WYP_onepager.html`, `.ctabar`) was deliberately left unchanged — the owner's stated reasoning (“since this is being read at the wouldyouplease.com site”) is specific to the live landing page; a printed/emailed one-pager isn't “read at the site,” so “Start free today at wouldyouplease.com” still does real work there.

Who benefits section. Added identically to both the live landing page (`LandingPage.tsx` + `landing.css`) and its mockup (`design/marketing/WYP_landing_page.html`), positioned directly before the `Subscription/Coming-soon .cols` block as instructed: a bare `.slabel` (uppercase label, no badge — CSS already handles a badge-less `.slabel` fine) reading “Who benefits from Would You Please?”, followed by a new `.benefits` paragraph class (14.5px, --ink, matching the feature-card body-copy scale) holding the body text. Also added to `docs/WYP_onepager.html`, in the same position, using a new but much more compact `.who` class (11.5px) — this file is a fixed one-page 8.5x11 letter layout with real space constraints, unlike the landing page's own unbounded scroll. Copy lightly tightened in all three places — “including for a wide range of” → “across” — to read more naturally; the “implied, not claimed” framing

the owner was careful about is unchanged, flagged as an edit rather than silently kept verbatim, per his own invitation to suggest improvements.

Not visually verified against a real print/render. No headless browser or Chrome extension was reachable in this session (attempted Claude in Chrome — not connected; attempted installing Puppeteer’s bundled Chromium — blocked by the sandbox’s network allowlist, `getaddrinfo EAI_AGAIN storage.googleapis.com`) to confirm the one-pager still fits a single printed page after the addition, or that the landing page’s new section renders as intended on a phone-width viewport. The new one-pager section was sized conservatively (11.5px, single short paragraph, no extra top/bottom margin) against the page’s existing apparent slack (the CTA band’s `margin-top:auto` only pushes to the bottom because there’s already room to spare), but this is a judgment call, not a confirmed measurement — flagged for the owner to check the actual printed/PDF output, or to connect the Chrome extension so a future session can verify directly.

`npx tsc --noEmit/npm run lint clean` (the two React/TSX files only — `docs/WYP onepager.html` and the design mockup aren’t part of the Next.js build, per CLAUDE.md’s repository-layout table).

2026-08-17 — Main Screen: Done column header no longer sortable/colorized on Sent/Received unless All or Done chip is selected

Owner: the Done column header was colorizing (the `.pill + ^/v` indicator) and toggling sort direction even while the Open or Overdue chip was selected — “which is not true - unless the All or Done chips are selected no Done items are in the list below the column heading.” Correct: `filteredSent/filteredReceived` under Open or Overdue never include a Done row, so a sort-by-Done indicator there was describing an ordering that couldn’t actually be observed in the visible list.

Fix: `ColSort` (the shared column-header button component) gained a `disabled` prop. When true, the cell renders as inert plain text — no `.pill`, no arrow, `aria-label` drops the “currently sorted” clause, and the underlying `<button>` gets the native `disabled` attribute (`onClick` becomes `undefined`) — regardless of whether Done is still the stored sort key underneath. Both Sent and Received Done columns now pass `disabled={<filter> !== 'all' && <filter> !== 'done'}`. New `.colbar` button:`disabled` CSS rule (`app/globals.css`) dims it (`opacity: .55`) and drops the pointer cursor, consistent with `.is-locked`’s general “still visible, visibly inert” pattern elsewhere in the app, though this uses the simpler native `disabled` attribute rather than the full `.is-locked` treatment since there’s no unlock explanation to show (CLAUDE.md’s `.is-locked` guidance is about a gated *feature*, not a temporarily-inapplicable sort).

Deliberately not reset: switching to Open/Overdue while Done is the active sort key does not change the stored `sentSort/receivedSort` state — it only suppresses the Done column’s own active indicator. The stored key resumes, unchanged, the instant All or Done is reselected, matching this app’s existing convention of preserving sort/filter state across chip changes rather than clearing it. Practically inert either way while disabled: `sortedSent/sortedReceived`’s case `'done'` comparison already degenerates to a stable no-op when every visible row’s `done_date` is null, so no rows are silently reordered by an invisible Done sort while the column is disabled.

`npx tsc --noEmit/npm run lint clean`.

2026-08-17 — Un-archive-on-clear: reopening a Done Request/ToDo from Archive returns it to active status

Prerequisite for the Chron notification work below — the owner’s own answer on notification scope (“archived items will not be subject to any notifications”) only holds if a Request/ToDo edited back open from Archive can actually leave the archived state; otherwise a corrected item would become permanently invisible and permanently notification-exempt. Owner, in the same message: “If a Request Detail is edited from the Archive list view and the Done Date is removed, the end-user should be advised that the item will be returned to active status and will appear in their lists.”

Applied identically across the three Detail-type screens — `RequestDetailForm.tsx`, `TodoDetailForm.tsx`, `ResponseDetailForm.tsx` — each now loads its own `archived_at/received_archived_at` (Request Detail and Todo Detail via a plain `.select()` column addition; Response Detail via a new field on `get_received_request`'s jsonb payload, migration 032) into local state, unedited by the user directly. `handleSubmit`'s update payload sets it to null whenever the Save is clearing Done Date to empty, and otherwise carries the loaded value through unchanged (a harmless no-op when it was already null). A `<p className="subnote">` advisory appears next to the relevant field whenever `archivedAt !== null` and Done Date is about to go empty this Save, worded “This <Request/Todo> will be returned to active status and will appear in your lists again once saved.”

Todo Detail's condition branches on `todoDatesEnabled` (§6.35, migration 022) rather than reading `form.doneDate` directly — when Due/Done Dates are off, `done_date` is driven by the Open/Done Status chip instead, and the existing `effectiveDoneDate` computed variable in `handleSubmit` already reduces both modes to the same null/non-null signal, reused here for both the advisory condition and the Save payload.

Response Detail needs no client-side write of its own — migration 032's `set_response_done_as_recipient` already clears `received_archived_at` server-side whenever `p_done_date` is null, so `receivedArchivedAt` state exists purely to drive the advisory note, not the Save call.

`npx tsc --noEmit/npm run lint clean.`

2026-08-17 — Chron notification system built: day-before Reminders, Overdue transition, recurring nudges, two Requestor digests

Owner's own original ask (verbatim, condensed): day-before Reminder emails to the Account holder of a Received Request or Todo, and to the Recipient of a Sent Request; an opt-in daily digest to the Requestor of which Reminders just went out to Recipients; a daily digest to the Requestor of Requests that just became Overdue, at “12:01am the morning after”; an individual Overdue email to each Recipient; and an hourly-then-daily nudge to the Recipient of a Due-Time Request specifically. Every open design question (digest scope, nag cadence, Todo gating, digest pricing tier, target local hour, and whether Archive exempts a Request from all of this) was answered directly in his next message — see this same date's earlier CLAUDE.md entry for the full Q&A.

Architecture: one hourly route, not three. `app/api/cron/tick/route.ts`, invoked every hour by Vercel Cron (`vercel.json`), runs all four phases (day-before Reminders, Overdue transition, recurring nudges, digests) in a single pass, gating each candidate row's own action on that row's own *local* hour (`app/src/lib/cronTime.ts`, pure `Intl.DateTimeFormat`-based helpers — no date library). Vercel Cron entries are each fixed to one UTC time, but “morning” and “the morning after” mean each *owner's* (or *recipient's*) own local hour, not one global clock time — an hourly tick checking local hour per row serves every time zone from one schedule. **This design requires Vercel Pro** (Hobby caps Cron Jobs at once-per-day) — discovered mid-build, raised with the owner via `AskUserQuestion`; he upgraded the same day.

Whose time zone governs what: the day-before Reminder to a Sent Request's Recipient uses the *Recipient's* own zone (`contacts.time_zone`, falling back to the owner's `profiles.time_zone`) — the one recipient-facing email in this batch timed to the reader's own morning, not the sender's. Every other timing decision (Todo Reminders, the Overdue transition, recurring nudges, both digests) uses the *owner's* own zone — “12:01am the morning after” is naturally about whose day just ended.

Idempotency: migration 032's `reminder_sent_at/overdue_notified_at/last_overdue_nudge_at` (drafted alongside this build, not yet confirmed run) — a send that fails or finds SMTP unconfigured is simply retried next hour, never marked sent. `overdue_notified_at` fires exactly once per Request; a Due-Date-only Request's first nudge is that same event (its own `last_overdue_nudge_at` is set in the same UPDATE), while a Due-Time Request's first nudge is a separate, later “hour after” event, left for the recurring-nudge phase to set.

Response links inside these emails are minted by a new `cron_issue_request_link()` (migration 033, drafted) — a service_role-only sibling of the existing owner-only `issue_request_link` (migration 008), since a cron run has no

session for that function's `auth.uid()` check to pass. Always mints fresh (no attempt to “reuse” a still-valid link) — the raw token is never persisted, only its hash, so an earlier link's raw value can never be recovered to re-embed; this is the same behavior `issue_request_link` itself already has on every call, not a new limitation introduced here.

Email templates (`app/src/lib/email.ts`): the Sent Request Reminder reuses the *existing* Initial-Request templates (`buildRequestEmailSubject('reminder', ...)`, `buildRequestEmailHtml/Text`) with `reminderPromised: false` — sending the Reminder now would make “a reminder will arrive” self-referential nonsense. New: `buildOverdueRecipientEmail*` (one template for both the first Overdue notice and every later nudge — the owner's own wording works unchanged for a repeat send), `buildTodoReminderEmail*` (to the owner's own account, no Recipient exists for a `ToDo`), and `buildReminderDigestEmail*/buildOverdueDigestEmail*` (share one `DigestItem` row shape and list-rendering helper, differ only in subject/intro sentence).

ToDo Reminders are gated purely on the owner's own `todo_dates_enabled` account toggle, per the owner's own answer — not a per-`ToDo` checkbox; `reminder_enabled` (migration 031) is a Request-only UI concept even though the column is physically shared with `ToDo` rows in the same `requests` table.

New Account toggle: “Notify Me When Reminders Are Sent” (`profiles.reminder_digest_enabled`, migration 032) — free feature, off by default, same `handleToggle` pattern as every other `AccountForm.tsx` checkbox.

service_role (`SUPABASE_SERVICE_ROLE_KEY`) is used throughout this route — a cron run has no user session for RLS to scope to at all. Same justified, narrow exception `CLAUDE.md` already carves out for the attachments API routes, extended here since a background job legitimately needs to read and act across every owner's account. Owner email lookups use `sb.auth.admin.getUserById()` (GoTrue Admin API), cached per run.

New `CRON_SECRET` env var (`.env.local`, git-ignored; also needs adding to Vercel's own Environment Variables) — randomly generated, checked as a bearer token against every request to `/api/cron/tick`, same mechanism Vercel Cron itself uses to authenticate its own scheduled calls.

`npx tsc --noEmit/npm run lint clean`. **Migrations 032 and 033 confirmed run by the owner, 2026-08-17.**

Deployment saga, same day — root cause was Vercel's Hobby-plan cron frequency cap, not a code bug.

The commit carrying this batch (090ce36) never produced a real deployment on Vercel at all — GitHub's own commit-status check showed 1/2 failing, and the Deployments list (even with every status filter enabled) never listed a build for it. Traced to `vercel.json`'s hourly cron schedule being rejected outright on Hobby — the same limitation flagged earlier in this same entry, now the actual, confirmed cause of the silent deployment failure, not just a theoretical blocker. The owner's earlier Vercel Pro upgrade (in response to the `AskUserQuestion` above) hadn't yet taken effect on a fresh deployment attempt; an empty commit (`git commit --allow-empty`) pushed after the upgrade triggered a clean build that went Ready/Production. **Live end-to-end test, from the owner's own PowerShell**: `Invoke-WebRequest -Uri ".../api/cron/tick" -Method POST -Headers @{Authorization="Bearer $CRON_SECRET"} -UseBasicParsing` returned `{"ok":true,"counts":{"requestReminders":0,"todoReminders":0,"overdueNotices":0,"overdueNudges":0,"reminders":0}}` — a clean run (all-zero counts are expected unless a row happens to be due at that exact local hour), confirming `CRON_SECRET` auth, the service-role DB queries, and the SMTP configuration are all wired correctly in production. The hourly `vercel.json` schedule is now live and will fire on its own from this point forward — the manual test above was the last verification step before letting it run unattended.

2026-08-17 — Main Screen: greyed “Description” column heading (all three sections) + Todos Date/Due/Done columns aligned with Requests

Owner, with two pasted mockup screenshots, in two parts. Part 1: treat the word “Description” the same way the Done column header already greys out when not selectable — add it to Sent/Received's colbar (right-aligned, To/From left-aligned) and apply the same muted treatment to Todos' existing Description heading. Part 2: when the Account screen's “Show Due/Done Dates (Todos)” toggle is on, Todos' colbar should show Priority on the left and Due Date/Done Date on the right, lined up with the same columns' position in the Requests presentation; off, only Date (created)

and Done show. Sorting should work for every shown date column. The Done column heading greys out (matching Sent/Received's existing rule) unless the All or Done chip is selected.

Apparent conflict, resolved by asking: Part 2's pasted mockups showed no "Description" text at all in the new Todos header, seemingly superseding Part 1's Todos instruction. Asked directly; owner's answer: "Removing the Description column heading for Todos was not intended — the text explanation was all I could create — I tried using a fill to color the word Description without success. All three sections should have the word Description greyed out as a non-selectable column heading / explanation." All three sections keep it.

New shared pattern — `.namecell/.c-desc` (`app/globals.css`): a flex row with `justify-content: space-between` pairing the existing sortable label (To/From/Priority) on the left with a plain, non-interactive, 55%-opacity bold "Description" span on the right — applied identically to Sent's, Received's, and Todos' colbars in `MainScreen.tsx`. Reuses the existing muted-disabled visual language already established for the Done column's own disabled `ColSort` state, rather than inventing a new one.

Todos' new column grid (`.colbar.dcols/.colbar.dcols.wide, .trd/.trd.wide`): deliberately matches Sent/Received's own `1fr 58px 58px(.58px) / 10px gap` grid exactly, so Date/Due/Done line up pixel-for-pixel across all three sections, per the owner's own "lined up with the position... above in the Requests presentation" instruction. `.wide` (4 columns: adds Due) is used when `todoDatesEnabled` is on; the plain 3-column form (Date, Done only) when off. Row markup changed from the old single-flowing-line `.t1/.tdc` shape to a two-line shape mirroring Sent/Received's own `.r1/.r2`: a new `.trd` grid line (Priority/Date/[Due]/Done) followed by a `.r2` description line (Dialog icon + optional Category + description) — reusing the *existing*, already-generic `.pri/.dt/.due/.dn/.r2/.desc/.cat` classes verbatim, so the existing `.row.overdue .dt/.due/.dn/.desc` and `.row.overdue .pri/.cat/.tdd(red)/.row.done` (grey) row-state coloring rules already apply with no new CSS needed for that part.

Not touched, on purpose: `.colbar.td/.t1/.tdc/.pri` (original block)/`.cat/.tdd` are left exactly as they were — `ArchiveForm.tsx` independently reuses these same class names for its own, differently-shaped Todos display (a single flowing line, not a grid), and redefining them in place would have silently broken Archive. All new CSS uses non-colliding names (`.colbar.dcols, .trd, .namecell, .c-desc`) instead. Archive's own Todos presentation is unchanged and out of scope for this batch — the owner's request was scoped to "the main screen."

TodoSortKey changed from 'priority' | 'category' to 'priority' | 'date' | 'due' | 'done' — Category sorting is retired from the column header (Category is no longer a header column at all in the new design; it still renders inline on the description line when the Private Category toggle is on, just not as a sortable header). 'date' sorts by `created_at` (new field added to `TodoRow`/the Supabase query — needed because the Date-created column is now always shown regardless of `todo_dates_enabled`, per the owner's own added sentence "Date created and Date Done are always captured and shown in the Todos list view").

Account toggle wording updated (`AccountForm.tsx`, "Show Due/Done Dates (Todos)"), to the owner's own exact replacement text: "Adds Due Date and Done Date for creating and editing Todos instead of just a Status of Open and Done. Off by default. Turn it on for more precise Todo tracking. Date created and Date Done are always captured and shown in the Todos list view."

No mockups updated — none of the five Todo/Request mockups (`WYP_main_screen_palette1.html` included) were ported to the new column layout; flagged in `design/README.md`, not silently skipped, same posture as every other recent live-only batch. `npx tsc --noEmit/npm run lint clean`.

2026-08-17 — Todos colbar black-text bug fixed; Archive Todos matched to Main Screen's new layout; print reports redesigned; Todo Detail Done-band wording

Owner testing the batch above found five issues, all from the same session. Fixed together.

Root cause of the black header text: the new colbar modifier class was named `.tdd` ("ToDo Dates"), which collides with a pre-existing, unrelated `.tdd` class (`font-weight: 400; color: var(--ink)`, Todo row description text,

still used by `ArchiveForm.tsx` and by `.row.overdue .tdd/.row.done .tdd`'s own row-state coloring). `.colbar` and `.tdd` both carry one-class specificity, so the later-declared `.tdd` rule won the cascade tie and silently painted the Todos header black instead of white. Renamed the modifier to `.dcols` (`.colbar.dcols/.colbar.dcols.wide`) — a name with no other meaning anywhere in the stylesheet — in `app/globals.css` and `MainScreen.tsx`, and updated every doc reference to match. A real lesson for this app's own class-naming convention: reusing an abbreviation like "tdd" without grepping for existing occurrences is exactly the kind of collision the `.pri/ArchiveForm.tsx` care taken earlier in this same batch was meant to prevent, and this one slipped through.

Archive's Todos view rebuilt to match Main Screen's new layout. Owner: "The column headings on the Archive Todos should match the main screen... Both the screen presentation and the report should follow the new Todos view and related changes based on the related Account option." `ArchiveForm.tsx` previously never read `profiles.todo_dates_enabled` at all (its Todos view only ever showed Priority — Description, Done Date always) — added a one-time `loadPrefs()` read on mount, matching `MainScreen.tsx`'s own pattern. `TodoCandidate` gained `due_date/created_at`; the query gained `due_date, created_at`; the shared `Row` type's `due/date/dueISO/dateISO` fields (already populated for Sent/Received) are now populated for Todos too, which let `TodoSortKey` extend from 'priority' alone to 'priority' | 'date' | 'due' | 'done', mirroring Main Screen's own four keys. On-screen `colbar/rows` now reuse `.colbar.dcols/.namecell/.c-desc/.trd` verbatim — the exact same classes Main Screen uses, so "match the main screen" is literal, not just visually similar. No disabled-Done-column gating needed here (unlike Main Screen) — Archive shows exclusively Done records, so there's no Open/Overdue chip state where Done can't be sorted.

Both Todos print reports (Main Screen and Archive) redesigned — owner: the old report was "missing the Priority value for each item," and asked for "the Priority value and the appropriate dates on a first line... to match the on-screen view." The old `.pcolbar.ptdc/.ptdc-nodates` shape (Description-only header, Due/Done as a second line, no Priority column at all) is superseded on both screens by new `.pcolbar.pdcols/.pri.pdcols` (`.wide` adds Due) — a first print line of Priority/Date/[Due]/Done mirroring `.trd`'s own on-screen grid, 92px columns (matching Sent/Received's own print date-column width; Todos have no time component to need Sent/Received's 150px) rather than the screen's 58px. `.namecell/.c-desc` are reused verbatim in the header for the same muted "Description" label. New `.ppri` print class (11pt/700/ink, matching `.pnm`'s own treatment) added to both `.prow.overdue/.prow.done`'s existing color rules so a Priority value participates in the same red/grey row-state coloring every other print field already gets. Archive's print row also dropped the old inline `r.priLabel ? \${r.priLabel} — ':'` prefix on the description line — Priority now lives in its own column, not smashed into the description text — which also fixes the specific complaint that the Archive report's column heading read "Description" while the thing actually being sorted was Priority (the header now correctly reads "Priority ^/v" with "Description" as its own separate, muted label). ****Not touched, flagged rather than silently skipped**:** `TodoDetailForm.tsx`'s own single-item `ToDo` print still uses the old `.pcolbar.ptdc/.ptdc-nodates` shape and has the identical "no Priority column" gap — out of scope for this batch (the owner's report named Main Screen and Archive specifically), but the same fix would apply there verbatim if raised.

ToDo Detail/Create ToDo Done-band wording — owner: "This ToDo is now marked as Done, just click Save." reads as if the user still needs to take a Save-related action for the Done status itself to register, when the field already holds the value. Changed to "This ToDo is now marked as Done." verbatim in both `TodoDetailForm.tsx` and `CreateToDoForm.tsx` (both screens had independently copied the identical string, per this app's small-stateless-duplication convention) — not the alternate "This ToDo is marked completed." wording also offered, chosen for consistency with the existing "This Request is now marked as Done..." family of `donerow` strings on Request Response/Response Detail.

`npx tsc --noEmit/npm run lint clean.`

2026-08-17 — ToDo Detail’s single-item print closes the same “missing Priority” gap

Owner, same day, immediately after confirming the batch below: “For consistency, please apply the same fix to the ToDo Detail print” — the gap that entry’s own “Not touched, flagged rather than silently skipped” note called out. `ToDoDetailForm.tsx`’s query gained `created_at` (this ToDo’s own creation date wasn’t fetched at all before); a new `createdAt` state holds it, kept separate from `form` since it’s never editable or saved, the same convention already used for `ownerName/tier`. No `PRIORITY_LABEL` map existed in this file — the Priority chip UI has always rendered “ASAP”/“SOON”/“LATER” as literal JSX text, never a lookup — so one was added locally. The print block now uses `.pcolbar.pdcols/.pr1.pdcols`, identical to Main Screen’s and Archive’s own redesigned reports, with no sort arrow (nothing to sort with a single record, matching `RequestDetailForm.tsx`’s own single-item header precedent).
`npx tsc --noEmit/npm run lint clean`.

2026-08-18 — Response Detail and Create Request converted from raw `window.print()` to the detailed print-report format

Owner: “Some report formats need to be brought up to the new format (they are doing a screen print instead)-as follows: 1. Response Detail - from main Request Received 2. archive Request Received screen (Request Detail uses the new format) 3. Create Request.” `RequestDetailForm.tsx` was confirmed as the reference — its own `.print-report/.pcolbar.detail3/.pr1.detail3/PrintDialogList/PrintAttachmentList` shape is the template every conversion below follows.

Item 2 needed no code change. `ArchiveForm.tsx`’s `openDetail()` already routes a clicked Received row to `/requests/[id]/respond` — the same `ResponseDetailForm.tsx` component Main Screen’s own Received rows use — so item 1’s fix covers it automatically. Separately, Archive’s own list-level print report for Received records (`loadReceivedPrintDetail()`, the shared non-ToDos print JSX branch) was re-checked and already uses the full `.pcolbar.psr/.pr1/PrintDialogList/PrintAttachmentList` format from the 2026-08-15 print-report redesign — never a raw `window.print()` to begin with.

Item 1 — `ResponseDetailForm.tsx`. Added the same print infrastructure `RequestDetailForm.tsx` already has: `formatMDYSlash`, `PrintAttachmentEntry` type, `PrintDialogList/PrintAttachmentList`, `printAttachments/showPrint/printTick` state, `startPrint()`, and the `printTick`-keyed effect. One real difference from the owner-side template: this screen is a signed-in RECIPIENT, and attachments RLS is owner-only (migration 025), so a plain `.from('attachments').select(...)` (`RequestDetailForm.tsx`’s own approach) would silently return nothing here. Used `get_received_print_detail(p_ids uuid[])` instead (migration 029, already granted to authenticated, already used by `ArchiveForm.tsx`’s own `loadReceivedPrintDetail()`) — fetched eagerly inside the existing `load()` effect, for a single id, alongside the `get_received_request` RPC call. `dialogList` needed no new fetch — it’s already populated from the same RPC’s own `dialog` field. Print JSX uses `.pcolbar.detail3/.pr1.detail3` with “From” instead of “To” (`data.owner_name`); Due/Done Time are gated by `data.owner_request_time_enabled` — the *issuer*’s own account setting, never this viewer’s — per CLAUDE.md’s Entitlements rule that rights on a Request come from who created it, not who’s looking at it.

Item 3 — `CreateRequestForm.tsx`. This is a genuinely different case from every other print conversion in the app: nothing has been saved yet. Recipient, Due Date, Category, Description live in local form state; Dialog entries and Attachments are staged client-side (`dialogEntries`, `stagedFiles`) and only written once Send succeeds. So `startPrint()` here is synchronous — no RPC, no fetch, everything needed is already in React state. `PrintDialogList/PrintAttachmentList` were rebuilt locally against this screen’s own staged shapes (`{ kind, body }` with no id/who/created_at; `File[]` with no upload yet), keyed by array index rather than a real id, matching the same convention the staged list already uses on-screen. New `.pcolbar.detail2/.pr1.detail2` CSS (`app/globals.css`, `1fr 150px`) — an unsaved Request has no `created_at` and no Done state yet, so neither Request Detail’s Date column nor its Done column has anything to show; the print preview is To/Due only. Titled “**Request Preview**”, not “Request Detail” — nothing exists to be a record of yet, this is a preview of what’s been filled in so far. Not itself a design instruction from the owner; flagged here rather than assumed uncontroversial.

`npx tsc --noEmit/npm run lint` clean for both files.

2026-08-18 — Fixed: live screen printing alongside the new print-report on Response Detail and Create Request

Owner, three pasted printout screenshots (Create Request, Response Detail via Archive, Response Detail via Main Screen), all showing the full live on-screen form — header, Send/Cancel buttons, input fields, staged Attachments row and all — printed directly above the new, otherwise-correct `.print-report` block. Root cause: the same-day print-format conversion (previous entry) added the `.print-report` sibling and its `startPrint()/printTick` machinery to both files, but never added the `no-print` class to their outer `<div className="app">` — every other working print screen (`RequestDetailForm.tsx`, `TodoDetailForm.tsx`, `MainScreen.tsx`, `ArchiveForm.tsx`, `ContactsList.tsx`) wraps its live content in `<div className="app no-print">`, which `globals.css`'s `@media print { .no-print { display: none !important } }` rule depends on to hide the live screen and show only `.print-report`. Missing that one class meant the browser had nothing telling it to hide the live view, so both rendered on the printed page. Fixed by adding `no-print` to the single live-render `<div className="app">` in each file (their loading/error-only early returns in `ResponseDetailForm.tsx` don't need it — same as `RequestDetailForm.tsx`'s own loading/error returns, which never render a `.print-report` sibling to hide anything from). `npx tsc --noEmit/npm run lint` clean.

2026-08-18 — Real web app manifest + service worker + cross-window auth sync fix

Traced from a live bug report while the owner was demoing WYP to a second business contact: clicking the magic-link email on Android Chrome started showing a system dialog — *“wyp-three.vercel.app wants to open the external app ‘Would You Please’”* — with no such native app or manifest anywhere in this codebase. Root cause: Android Chrome can “install” any site as a lightweight shortcut using just the page's `<title>` (which is literally “Would You Please,” matching the dialog exactly) even with zero manifest present, and once that happens the OS registers it as a link-handling target for that origin. The owner independently confirmed a home-screen icon existed.

Diagnosis, via live testing. Tapping that icon opened a standalone window showing the landing page (not signed in); clicking the magic link and cancelling the “open in app” dialog kept the flow in a regular Chrome tab, which did complete sign-in. The owner separately did his own research (attached doc, an AI-assistant transcript) that named the real mechanism correctly on its second pass, after an internally-contradictory first pass: **“Standalone Window Isolation.”** `localStorage` genuinely is shared between a standalone “installed” window and a regular tab of the same origin (both are the same browser, same origin-scoped storage) — `supabase-js` even broadcasts `SIGNED_IN/SIGNED_OUT` across tabs of the same origin via an internal `BroadcastChannel`, confirmed via Supabase's own GitHub issue tracker. What isn't shared is *awareness*: `app/page.tsx` and `RequireAuth.tsx` both only ever called `getSession()` once, at mount, so a window already sitting on the landing page before a sign-in completed elsewhere had nothing telling it to re-check.

Fix 1 — real web app manifest, replacing Chrome's ad hoc guess with a deliberate one. `app/manifest.ts` (Next.js's native manifest route convention — auto-generates `/manifest.webmanifest` and its own `<link rel="manifest">`, no manual `<head>` edit): `name/short_name` “Would You Please”, `start_url`: `'/'`, `display`: `'standalone'`, `theme_color`: `'#2A5FC8'`. Icons (`public/icons/icon-192.png/icon-512.png`) are rasterized (ImageMagick) from a new `public/icons/icon-source.svg` — not a new design, a recolored, recentered version of the existing “checked request” brandmark already used in `LandingPage.tsx`'s header (same shape, same brand blue), placed on a full-bleed rounded-square background with the glyph kept inside a ~66%-of-canvas safe zone so the same 512px file can serve both purpose: “any” and purpose: “maskable” manifest entries without a second image. `app/layout.tsx`'s `metadata.icons` also points at these for the plain favicon/apple-touch-icon case.

Fix 2 — minimal service worker. `public/sw.js`, registered from new `app/components/ServiceWorkerRegister.tsx` (a side-effect-only client component mounted once in `app/layout.tsx`, since the root layout itself is a server component). Android Chrome won't offer a real install prompt without an active service worker — this one deliberately does no offline caching (fetch handler just

passes every request straight through), since WYP has no offline story yet and a caching layer here would be a much bigger, separate feature with real risk of accidentally serving stale auth state.

Fix 3 — the actual cross-window desync bug. `app/page.tsx` and `RequireAuth.tsx` both gained a `supabase.auth.onAuthStateChange` subscription alongside their existing one-time `getSession()/getUser()` checks (not replacing them) — `page.tsx` updates `authed/anon` status live so an already-open window picks up a sign-in that happened elsewhere; `RequireAuth.tsx` redirects to `/login` immediately on a live sign-out instead of only at the next remount. This is the fix that actually closes the bug the icon/manifest work merely made more visible.

Explicitly not done, per the owner’s own conversation with the AI tool he consulted: switching from magic link to a 6-digit email OTP. That would sidestep cross-window issues entirely, but it’s a real UX/architecture change — CLAUDE.md documents magic-link-only as a deliberate decision — and wasn’t warranted once the actual gap (a missing subscription, not a storage-architecture problem) was identified. Revisit only if window-isolation symptoms persist after this batch.

```
npx tsc --noEmit/npm run lint clean.
```

2026-08-18 — Findable Install control; Archive wording fix; expired magic-link error surfaced

Same-day follow-up after the owner tried the new manifest/PWA batch for real.

“Install” Housekeeping row. Owner: he accepted the browser’s own install offer during a magic-link sign-in and then couldn’t find the resulting icon anywhere on his phone — Android’s “Install” typically adds the app to the app drawer like any other installed app, not directly to the home screen, and the two are easy to conflate; the browser’s own install banner is also a one-shot, opportunistic prompt with no way to bring it back. `ServiceWorkerRegister.tsx` (same-day, never pushed, so renamed rather than deprecated) became `PWAProvider.tsx`: still registers `public/sw.js`, and now also captures the `beforeinstallprompt` event and exposes it via a `usePWAInstall()` hook (`canInstall/promptInstall`), wrapping `{children}` in `app/layout.tsx` instead of sitting beside them so the captured event is reachable from any descendant. `MainScreen.tsx` gained a new Housekeeping row, “Install — add a Would You Please icon to your home screen,” only rendered when `canInstall` is true — never a dead control on a browser that doesn’t support installation or a device that already has it. Title/note split to match the existing `Contacts/Account/ Archive` row convention rather than using the owner’s longer sentence verbatim as a single string.

Archive row wording. Owner’s suggested replacement text (“remove completed items the above lists”) dropped the word “from” — restored it: “remove completed items from the above lists.” Flagged rather than applied silently verbatim, since the original clearly needs “from” grammatically.

Expired/invalid magic-link error surfaced. Owner-reported, with the exact URL: clicking Sign In sometimes landed him on the *landing page* (confirmed directly, not Main Screen) with no visible explanation, address bar reading `...vercel.app/#error=access_denied&error_code=otp_expired&error_description=Email+link+is+invalid+or+has+expired&sb=`. Root cause: Supabase’s own redirect-on-failure behavior sends a used/expired/invalid OTP link back to the project’s Site URL (this root route — not `app/auth/callback`, which only ever runs on a *successful* verification) with the failure encoded in the URL hash, not a query string or a page Supabase renders itself. Nothing here was reading that hash, so the failure was real but silent — indistinguishable from “nothing happened,” a materially worse experience for a soon-to-be second real user than an error he’d at least recognize as one. `app/page.tsx` now parses the hash via `parseAuthError()`, called from a lazy `useState` initializer (reads `window.location.hash` synchronously on first render, matching the existing `cameFromCalendarLink` pattern already used elsewhere in this codebase) rather than from inside a `useEffect` that calls `setState` — the latter tripped a real lint rule (`react-hooks/set-state-in-effect`, flagging an avoidable cascading-render pattern) on first pass; clearing the hash afterward via `history.replaceState` is a genuine side effect, so that part stayed in a small `useEffect`. `LandingPage.tsx` gained an optional `errorMessage` prop (no default required, per this app’s own no-required-props convention), rendered as a `.noticeband` near the top of the

hero — reusing the same banner component other screens already use for confirmation messages, since this app has no bright-red banner precedent anywhere and inline `.error` text doesn't fit a whole sentence. `npx tsc --noEmit/npm run lint clean`.

2026-08-18 — Description column heading added to every print report, centered

Owner: several printed reports were missing the “Description” column heading entirely, and where present it was right-aligned against the Date column instead of “centered between the ‘To’ or ‘From’ and the ‘Date’ column.” Two separate gaps, both fixed.

Missing entirely — added. `.namecell/.c-desc` (2026-08-17, built for Main Screen's on-screen ToDos colbar and later reused on its print version) had only ever been ported into 3 of the app's 8 print colbars — Main Screen's ToDos, Archive's ToDos, and Todo Detail's single-item print. The other 5 (Main Screen Sent/Received, Archive's combined Sent/Received, Create Request's `.detail2` preview, Request Detail's `.detail3`, Response Detail's `.detail3`) had no Description label at all. Added identically to each: wrap the existing `c-nm` (To/From) span in a `.namecell`, add a sibling `.c-desc` span reading “Description” — preserving each colbar's own existing sort-arrow logic where present (Main Screen, Archive) rather than dropping it.

Right-aligned where present — recentered. The existing `.namecell` CSS (`justify-content: space-between`) was written for the on-screen colbar, where right-aligning Description against the adjacent Due/Done columns is correct and unchanged here. A printed colbar's own To/From-to- Date/Due gap is a different, usually wider shape, and `space-between` there reads as glued to the Date column rather than centered in its own cell. New print-scoped override, `.pcolbar .namecell { position: relative; justify-content: flex-start } + .pcolbar .c-desc { position: absolute; left: 50%; transform: translateX(-50%) }` — left-aligns the sortable label as before, then absolutely centers Description within the `.namecell` cell's own width, which in every `.pcolbar` variant (`.psr`, `.pdcols`, `.detail2`, `.detail3`) already spans exactly the To/From-to- Date/Due region the owner described. Scoped under `.pcolbar` specifically so Main Screen's own on-screen colbars are untouched. `npx tsc --noEmit/npm run lint clean`.

2026-08-18 — contacts.phone_ext (migration 034, drafted, not yet confirmed run)

Owner: “unless the company provides direct phone numbers, the phone number by itself is not enough information to be useful” — a real, recurring frustration entering his own company's contact info elsewhere with no extension field. Raised while scoping Request Texting (see that same-day answer): E.164, the normalized digits-only format an SMS provider like Twilio requires, has no room for a post-connect extension, so phone itself was never a place an extension could live.

New `contacts.phone_ext` text column, plain free text (not numeric — org PBX extensions sometimes carry a leading zero or a short alpha prefix, same posture as phone itself). New field in `AddContactForm.tsx` and `ContactDetailForm.tsx`: a narrow `flex: 0 0 84px .ffloat` immediately after Phone inside the existing `.phone-row`, rather than `flex: 1 1 auto` like Phone itself — an extension is a handful of digits, not a full field's worth. Shares Phone's own `Row-Tint-while-Email-is-the-channel` rule (`sendBy === 'email' ? 'opt' : ''`) verbatim, same reasoning as Phone.

Owner's own instruction on display: “only as a space and the extension at the end of the phone number... would not need a separate column on the printed report.” New `phoneWithExt()` helper in `ContactsList.tsx` — `${phone}${ext}`, no “ext”/“x” label, appended literally as instructed — used both in the on-screen Contacts list row's existing `Text: <phone>` note and the print report's existing Phone column. Applying it to the on-screen row too (only the print report was explicitly requested) is a scoping call, flagged rather than silently assumed: an extension is exactly as useful on screen as on paper, and `phoneWithExt()` was going to exist regardless, so reusing it in both places rather than only print seemed like the obviously-intended reading of “not enough information to be useful” — revisit if the owner wants the on-screen note to stay phone-only. `npx tsc --noEmit/npm run lint clean`.

2026-08-18 — Fixed: raw “JWT issued at future” error rendered in place of Main Screen’s three lists

Owner-reported: periodically, on app start, all three Main Screen sections (Sent/Received/ToDos) showed the literal phrase “JWT issued at future” instead of their rows, and it stayed that way until he scrolled.

Root cause traced, not assumed. `MainScreen.tsx`’s one-shot `load()` effect ran all three queries once on mount and, on any error, rendered `error.message` — the raw string Supabase’s API returned — directly as each section’s entire content, with no retry and nothing else in the app that would ever re-trigger the fetch (confirmed: no visibility/focus listener anywhere touches this effect, so “scrolling fixed it” was very likely coincidental timing, not causal). “JWT issued at future” specifically is a known, previously-documented Supabase-infrastructure symptom: the access token’s own `iat` claim is validated against the clock of whichever edge/Postgres node happens to handle a given request, and those nodes aren’t perfectly synced — a request landing on a node a moment behind another can see a token’s `iat` as “not yet” valid, self-correcting on any later retry. Nothing in this app’s own code sets or checks `iat`; this isn’t a client device clock problem, and it isn’t fixable by changing when or how WYP mints or stores a session.

Fix: the load effect now retries up to two more times (600ms, then 1600ms after the first attempt) before giving up, absorbing the transient case invisibly in the vast majority of real occurrences. If every attempt still fails — a real outage, not clock skew — the raw error text is replaced with a generic “Could not load your Requests and ToDos. Check your connection and try again.” plus a Try Again button (new `reloadTick` state, bumped by the button, added to the effect’s own dependency array) — a raw backend error string should never have been user-facing regardless of what caused it, so this fixes the display problem even in the rare case the retries don’t. `npx tsc --noEmit/npm run lint clean`.

2026-08-18 — Default standalone-window size on desktop PWA launch

Owner: the installed desktop icon opens Would You Please in a very wide standalone window — most of it bare grey letterboxing around the app’s own 480px-capped content column — and asked whether the window width can be controlled. He’d also independently discovered that manually pulling the window narrower persists across future opens.

Confirmed via current documentation, not assumed (web.dev’s own PWA Window Management guide, fetched live): there is no manifest field for a preferred launch size — “There is no way to define your PWA’s preferred size and position within the manifest.” Chrome’s own default for a freshly-installed desktop PWA is “a percentage of the current screen, with a maximum resolution of 1920x1080,” which is exactly the oversized window the owner described. The documented, sanctioned mechanism is `window.resizeTo()`, called once on launch — the same page confirms Chrome remembers whatever size the window is left at afterward, matching what the owner had already found by hand.

Implementation: `PWAProvider.tsx` (already the root-level component handling install/service-worker setup) gained a second `useEffect`, `[]` deps, gated on `matchMedia('(display-mode: standalone)').matches` so it never touches a normal browser tab and is a no-op on mobile (where `resizeTo` has no effect at all, per the same documentation). Target size, 552x968, isn’t an arbitrary guess — it’s the exact dimensions of the pulled-in window the owner’s own screenshot demonstrated as comfortable. Since this now runs on every standalone launch rather than relying on the owner’s own manual resize, it also means a fresh install (or a second device) opens at the right size from the first launch, not just after someone notices and fixes it by hand. `npx tsc --noEmit/npm run lint clean`.

2026-08-19 — Auto-growing Description on Request Detail / ToDo Detail

Owner: “When entering [Description] in Create it is scrolling as typed, so that is not an issue. If it is being edited, it would be easier to see all of the text.” — scoped deliberately to just the two Detail (edit) screens, since Create Request/Create ToDo start every Description empty and never have this problem the way an existing, possibly long record does the moment it loads.

Both `RequestDetailForm.tsx` and `TodoDetailForm.tsx` gained a `descRef + useEffect` keyed on `form.description`: on every render where the `Description` value changed — the initial async load included, since that’s just another state update — the `textarea`’s own `.style.height` is reset to `'auto'` then set to its `scrollHeight`, so the box always grows to fit whatever it holds rather than internally scrolling. New CSS modifier `.ftextarea-autosize` (`app/globals.css`) turns off the ordinary fixed `min-height/scrollbar/resize:vertical` handle, which would otherwise fight a JS-managed height — added only alongside `.ftextarea-plain` on these two screens’ `Description` field, not the shared base `.ftextarea` rule, so `Create Request/Create Todo` and every other `textarea` in the app (`Dialog Text, Notes`) keep their existing fixed-height/manual-resize behavior unchanged. `npx tsc --noEmit/npm run lint clean`.

2026-08-19 — Subscribed toggle locked down, private-testing style —

migration 035 DRAFTED, NOT YET CONFIRMED RUN

Owner: “We should lock down the subscribe., but can we do it in a way which is similar to the Private Testing method in place for opening a Free Account?” — followed, mid-turn, by “And. let the user know that the status will only be in effect during the testing - afterward, they can ‘actually’ subscribe.”

Deliberately mirrors migration 015’s already-battle-tested shape rather than inventing a new one: an `app_settings` key/value gate (`tier_toggle_gate_enabled`), a dedicated allowlist table (`tier_toggle_allowlist` — a new table, not a reuse of `beta_allowlist`, since “may create an account” and “may self-grant Subscriber for testing” are different permissions that shouldn’t share one ambiguous list, seeded with Jim’s own email so this migration doesn’t lock him out of the Attachments testing he’s already using the toggle for), and two `SECURITY DEFINER` functions: `can_toggle_tier()` (returns whether the gate is off or the caller is allowlisted — same true-when-off, check-allowlist-when-on shape as `can_create_account()`) and `set_tier_for_testing(p_tier)` (the only permitted write path, re-checking `can_toggle_tier()` itself rather than trusting a prior client-side check, per `CLAUDE.md`’s own Entitlements principle: “the `SECURITY DEFINER` function must refuse the write regardless; assume the control was bypassed”). Migration 024’s direct grant update (`tier`) on profiles to `authenticated` — the actual security hole, not just a missing convenience — is revoked in the same migration, so the old unguarded path can’t be used alongside the new gated one.

`AccountForm.tsx`: `can_toggle_tier()` is called once on load and the whole Subscribed row is wrapped in `{canToggleTier && (...)}` — hidden entirely for anyone not allowed, rather than shown and left to fail on click, same posture as every other gated control in this app. `handleTierToggle` now calls the `set_tier_for_testing` RPC instead of a raw table update, keeping the existing optimistic-update-reverted-on-failure shape. Per the owner’s follow-up, the checknote copy was rewritten to say the Subscribed status “only lasts for the testing period” and that afterward the checkbox goes away and real subscription happens “through an actual Subscription Details page with eCommerence links” — the existing forward-reference to that not-yet-built page, now framed as what replaces this toggle rather than what merely “will replace this checkbox” someday. `npx tsc --noEmit/npm run lint clean`. **Migration 035 still needs to be run in Supabase before this is live** — until then, `can_toggle_tier()/set_tier_for_testing()` don’t exist and the toggle will fail to load/save.

2026-08-19 — Archive rows gain the Dialog/Attachments icons Main Screen’s own rows already have

Owner, while reviewing Archive: “I noticed that the description does not have the Dialog and Attachments icons shown.” True — `ArchiveForm.tsx`’s own `Sent/Received/Todos` queries never selected `dialog(count)/attachments(count)` (or, for `Received`, the RPC’s own `dialog_count/attachment_count`) at all, so its row JSX had nothing to check even though `DialogIcon/AttachmentIcon` were already established components elsewhere in the app.

Fixed by extending `SentCandidate/ReceivedCandidate/ToDoCandidate` with the same fields `MainScreen.tsx`'s own `SentRow/ReceivedRow/ToDoRow` already carry (`dialog(count)/attachments(count)` PostgREST embeds added to the `Sent` and `ToDos` queries; `Received`'s `get_received_requests()` RPC already returns `dialog_count/attachment_count` — migration 027 — so only the type needed the two fields, no query change), then threading them through the shared `Row` type used by `Archive`'s `sent/received/todos` `useMemo` (as plain `dialogCount/attachmentCount` numbers, resolved once at that mapping step rather than re-derived in `JSX`) and rendering the same `icon-if-count-greater-than-zero` pattern in both `row-JSX` branches. `ToDos` get `Dialog` only, `attachmentCount` hardcoded to 0 — matching `Main Screen`'s own `ToDoRow`, which has never had an `attachment/Locations` icon either. `DialogIcon/AttachmentIcon` are duplicated into `ArchiveForm.tsx` verbatim rather than imported, per this codebase's established per-file-duplication convention for small stateless helpers. `npx tsc --noEmit/npm run lint clean`.

2026-08-19 — Voice dictation for Description (subscriber-gated, Create Request + Create ToDo)

Owner, in the same conversation that dropped the `Search bar`'s own voice icon: “As to removal of the voice search icon - I see it as a good option for entry of the `Description` during a `Create`. And, it could be a subscription option to be an `Account` option.” Recommendation given (recommendation → alternatives rejected → open questions): build on the browser-native `Web Speech API` (`SpeechRecognition/webkitSpeechRecognition`) rather than a paid hosted transcription service — no vendor, no per-use cost, no server round trip, unlike the `Twilio/10DLC`-dependent path `Request Texting` would need; gate it the same way `Attachments/Locations` already are, off the signed-in owner's own `live profiles.tier`, not anything baked into the `Request/ToDo` itself; scope to just the `Description` field on `Create Request` and `Create ToDo` for this batch. Owner: “Let's test for browser-support later (post `Private Testing`)” — read as approval to build now, basic feature detection still in place, with thorough cross-browser QA deferred. Follow-up note on framing: “The feature could be described as ‘available if your browser supports it (and most do)’ ” — reflected in code comments rather than any user-facing copy, since the mic button simply doesn't render at all on an unsupported browser (there's nothing to word for that case).

Implementation, `CreateRequestForm.tsx/CreateToDoForm.tsx/globals.css`:

- **Duplicated per component**, per this codebase's established convention (`DialogIcon/openPicker/etc.`) rather than extracted to a shared lib file: minimal local structural types (`SpeechRecognitionEventLike`, `SpeechRecognitionLike`, `SpeechRecognitionConstructor`) stand in for the real, non-standard `Web Speech API` (not part of `TS`'s default `DOM` lib) rather than reaching for any; `getSpeechRecognition()` reads `window.SpeechRecognition ?? window.webkitSpeechRecognition` via a single `as unknown as {...}` cast; `MicIcon()` is a small inline `SVG`.
- **voiceSupported is computed once, client-only, via a mount effect**, never read directly during render — starts `false` on both the server render and the first client render, so there's no hydration mismatch, then flips `true` a tick later if the API is actually present. The `setVoiceSupported` call itself is deferred one microtask (`queueMicrotask`) rather than called synchronously in the effect body — satisfies `react-hooks/set-state-in-effect`, the same shape `PWAProvider.tsx`'s own `beforeinstallprompt` listener already satisfies via a real browser event; here there's no event to listen for, so a microtask stands in for one.
- **toggleDictation()** starts/stops a single `SpeechRecognition` instance (`continuous: true`, `interimResults: false` — only finalized phrases are appended, avoiding duplicate text from a browser that re-sends firming-up interim results); `onresult` appends only `event.resultIndex` onward to `form.description` via the existing `set()` helper; `onerror/onend` both reset dictating to `false`, since a real error and a natural stop (e.g. a silence timeout) look identical from the button's point of view. A cleanup effect stops any live recognition on unmount.
- **Mic button placement**: the `Description` `<textarea>` is now wrapped in a `.descwrap(position: relative)`, with a `.micbtn` — a small circular button, absolutely positioned in the `textarea`'s own bottom-right corner — rendered only when `tier === 'subscriber' && voiceSupported`. Rest state reuses `--ink-soft` (the same

muted-icon convention as `.fclear`); `.listening` switches it to `--alert-red`, background-filled — a deliberate reuse of the app’s one “something is happening, pay attention” color (Overdue rows, error text), not a new token.

- **Gating:** sender-side only, off the signed-in owner’s own tier — both forms already load it in their existing mount effect (`profiles.tier`), no new query needed. Not wired into Request Detail/ToDo Detail (editing an existing item) or any recipient-facing screen — scoped to Create Request/Create ToDo only, per the owner’s own framing (“during a Create”).

`npm tsc --noEmit/npm run lint clean`. New `.descwrap/.micbtn/.micbtn.listening` CSS, §6.4x PROPOSED — not drawn in any mockup (neither source mockup has interactive Description JS to add a mic button to); flagged in `design/README.md`.

2026-08-19 — Search bar relocated under Housekeeping, Housekeeping hidden while searching, voice-search icon dropped

Owner, testing the Search Mode redesign above on a phone: “I think the search will be an infrequently used feature. Search now requires two line large vertical lines (on my phone) and three lines when searching by date range. The result is to leave significantly less vertical space to scroll the app.” Several relocation/reduction options were floated (narrower scope select, text field that scales to width, mic icon moved into the text field’s own corner, floating labels on the date fields); recommendation given (recommendation → alternatives rejected → open questions, per the owner’s own working-style rule) was that the search bar’s real cost was its fixed position outside `.scroll`, not just its width — moving the whole control into the scroll area under Housekeeping was the highest-leverage fix on its own. Confirmed: “hold off on the floating labels, drop the voice-search icon now, having Clear Search in both places is useful, subscription banner and ad stays pinned and only Search itself relocates.”

Implementation, `MainScreen.tsx/app/globals.css`:

- **Search relocated into `.scroll`.** The `<div className="searchbar sb">` block (scope select, text-field-or-date-range fields, Clear Search, the remaining Search icon) moved from its old fixed position beside `.subbanner/.adslot` (outside `.scroll`) to a new `<div className="band"><span className="glabel">Search</span></div>` + `searchbar` pairing placed right after the Housekeeping block and before `.scroll-pad`, inside `.scroll`. `.subbanner` and `.adslot` are untouched — per the owner’s explicit instruction they stay pinned; only Search itself moved.
- **Housekeeping hidden while searching.** The entire Housekeeping band + subcard is now wrapped in `{!isSearching && (<...>)}` — the owner’s own reasoning (“while in a ‘search results state’, I don’t think we need to show the Housekeeping section”). Housekeeping reappears the instant `isSearching` goes false, same as everything else driven by that derived flag.
- **Voice-search icon removed outright**, not just from the search bar’s horizontal budget — `VoiceSearchIcon()` deleted entirely from `MainScreen.tsx`. It was always decorative (never wired to anything), and the owner separately redirected the actual idea toward Description dictation (see the entry below) rather than Search, where “the main field for using it would be for Description text” anyway.
- **Per-section Clear Search added alongside the band one.** Each section’s `isSearching` chip-row branch (`.chips`) gained a `.searchresultsrow` modifier (`justify-content: space-between`) so “Search Results” and a second `.clearsearch` button render at opposite ends of the same row — owner: “having Clear Search in both places is useful,” avoiding a trip back to the bottom of the screen to exit search from wherever a result is being read. Both Clear Search controls (band and per-section) call the same `clearSearch()`.
- **Floating labels on the Date Range fields — held off**, per the owner’s own instruction, not attempted this batch. Reasoning noted at proposal time: native `type="date"` rendering makes a floated label fiddly for little width gained, worth revisiting only if the relocation above turns out not to be enough on its own.

`npm tsc --noEmit/npm run lint clean`. No mockup updated — same reasoning as the Search Mode entry above.

2026-08-19 — Search Mode redesign: results shown within Main Screen, Date Range scope, Archived badge, “Search Results” notice

Owner had been designing a separate Search Results screen and reconsidered: “showing results within the main screen would be more logical.” Design discussion (recommendation → rejected alternatives → open questions, per the owner’s own working-style rule), then confirmed point by point: Sent and Received stay in their own separate sections/lists during a search (they already were — search was never blending them, each of `filteredSent/filteredReceived` has always filtered independently); Archived items are automatically included and badged, not opt-in, “at this point... maybe an ‘Advanced’ search option can be offered later”; status chips are removed in favor of a plain “Search Results” notice; and the exit mechanism (discussed separately, below) should auto-exit the instant the text field is cleared by hand.

Implementation, `MainScreen.tsx/app/globals.css`:

- **Scope becomes a real two-item picker** — a plain `<select>` styled to match the old visual-only `.scope` button (`selectSearchScope`), replacing it outright. “All” is the existing text search; “Date Range” swaps the text field for paired From/To Due Date fields (`.daterange-fields/ .drfield`, reusing the equal-width-paired-field spirit of the app’s existing `.frow/ .ffloat` convention without literally reusing those classes, since this is a compact search-bar context, not a form panel). Switching scope clears the other scope’s own fields (`selectSearchScope`), so there is never stale hidden criteria silently narrowing a result set the person can no longer see.
- **isSearching is derived, never a stored mode flag** — `searchScope === 'all' ? searchText.trim() !== '' : fromDate !== '' || toDate !== ''`. This is what makes the owner’s exact instruction (“the text box emptying by hand should auto-exit search mode immediately”) fall out for free: there is no separate flag to reset, so the moment the relevant field(s) go empty, every downstream read of `isSearching` — filtering, the chip-row swap, Archived inclusion — reverts on the very next render.
- **Filtering** (`filteredSent/filteredReceived/filteredTodos`): at rest, unchanged (status chip + hide-archived, as before). While `isSearching`, the status chip check is skipped entirely rather than still narrowing an already-narrowed result set, and matching uses either the text query or `matchesDateRange(dueDate, fromDate, toDate)` depending on scope — never both, since only one scope’s fields are ever populated at a time. `matchesDateRange` treats either side alone as valid (From alone = on/after, To alone = on/before, both = inclusive between, generalizing Archive’s own existing “Before Done Date” single-sided convention) and a row with no Due Date never matches. The existing archived-row-hidden-at-rest check now keys off `!isSearching` rather than `query === ''`, so Date Range searches surface Archived rows too, not just text ones.
- **“Search Results” notice**: each section’s `.chips` row conditionally renders a plain `.searchnotice` span instead of the status-chip buttons while `isSearching` — matches the owner’s own simplification (“maybe chip filters are removed and a ‘Search Results’ notice is shown instead”) rather than the heavier per-section band originally floated in the design discussion. Each section’s Done column header (`ColSort`’s disabled prop, 2026-08-17) is also un-gated while searching, since a Done row can legitimately appear in results regardless of which chip was last selected.
- **Archived badge**: a small `.archtag` (“Archived,” muted grey, Strip background) renders in each matched row’s `.r2` line, before the existing Dialog/Attachments `.ii` icons — keeps an archived row inside its normal Sent/Received/Todos section (never a separate fourth list, which was explicitly rejected in the design discussion) while still reading as more than merely Done.
- **Drop-out-of-search-results mechanism** — the owner’s own follow-up question, answered and confirmed separately: a `.clearsearch` “Clear Search ×” control appears next to the field(s) whenever `isSearching`, regardless of scope (the one reliable, always-visible exit); the text field additionally gets its own inline `.fclean ×` once it holds a value (reusing the existing Due/Done Time Clear-affordance convention, 2026-08-11), wrapped in a new `.fieldwrap` positioning context. Both call the same `clearSearch()`, which resets text, both date fields, and the scope back to “All” together.

`.searchbar` gained `flex-wrap`: wrap so the Date Range fields and Clear Search drop to their own line on a narrow phone rather than squeezing. `npx tsc --noEmit/npm run lint clean`. No mockup updated — none of the existing

mockups model Search at all; flagged in design/README.md.

2026-08-19 — Search bar: “Date Range” shortened to “Dates,” field/date-field flex-basis fixed to stop the search icon wrapping

Owner, still on a phone: “it still wraps the magnifying glass to the next line on my phone - it would look better if it did not do that... the phrase ‘Date Range’ can be shortened to ‘Dates’ without any loss of meaning - since the data fields say ‘From/To’, and the text entry field and respective date fields can still be a bit shorter.”

Root cause, not just width: `.field`, `.fieldwrap`, and `.daterange-fields` all had `flex-basis: auto` (or, for `.daterange-fields`, an explicit 100%) with no fixed basis. A wrapping flex container decides where to break lines using each item’s *hypothetical* main size — its flex-basis, or, when that’s auto, its max-content size — not its ability to shrink; flex-shrink only acts after an item has already been placed on a line. The text field’s max-content size (driven by its placeholder text) and the date-range group’s explicit 100% basis were both large enough that the search icon, last in DOM order, didn’t fit on the line and wrapped away, regardless of how much `.field/.drfield` could actually shrink once placed.

Fix: the select option’s label changed to “Dates” (MainScreen.tsx); `.field` and `.fieldwrap` given a small fixed `flex-basis: 60px` instead of auto, and `.daterange-fields` dropped from `flex: 1 1 100%` to `flex: 1 1 190px` — all three still `flex-grow: 1` and shrinkable, so they fill available space exactly as before once placed, but the line-breaking decision no longer overestimates their size. `.drfield` also tightened slightly (padding 4px 8px → 4px 6px, internal gap 6px → 4px) — the “date fields can still be a bit shorter” the owner flagged as acceptable, read as permission for this shrinkage rather than a separate ask. `npx tsc --noEmit/npm run lint clean`. No mockup updated — same reasoning as the two Search entries above.

2026-08-19 — Add Dialog’s locked Answer chip loses its padlock icon on Create Request/Create ToDo

Owner: “When I Create a Request or a ToDo and Add Dialog, the Comment chip wraps to a second line for chips. The same chip-wrap is not true if I am looking at other screens with an Add Dialog.” Root cause: Create Request’s and Create ToDo’s Add Dialog modal render the always-locked Answer chip (there’s never anything yet to answer on a brand-new Request/ToDo) with an inline `.lockglyph` padlock SVG plus the word “Answer”; Request Detail, Request Response, Response Detail, and ToDo Detail’s own *dynamically* locked Answer chip (locked only when no Question is currently open) has never carried that icon — plain text with `is-locked` styling and `aria-disabled` only. The icon’s own width (plus the JSX-implicit space before “Answer”) was enough extra horizontal room, on a phone, to push Comment onto its own line — a wrap the four other screens never had, since their chip was already narrower.

Fixed by removing the `.lockglyph` SVG from Create Request’s and Create ToDo’s Answer chip, matching the other four screens exactly — the app now has one consistent (icon-less) treatment for a locked Add Dialog chip everywhere, `aria-disabled="true"` still carries the same meaning for assistive tech. `npx tsc --noEmit/npm run lint clean`. No mockup updated — none of the six Dialog/Attachments-batch mockups have real, interactive Add Dialog chip JS.

2026-08-19 — Reminder-related email link text: “Open Request/ToDo to mark Done...” replaces generic “Request Detail”/“ToDo Detail”

Owner: “For reminders being sent out, the URL link text is ‘Request Detail’, maybe to avoid the risk of being reported as a spam source it should be ‘Open Request to mark Done or to turn off notifications’. The wording may need some tweaking.”

Applied to every remaining generic “Request Detail”/“ToDo Detail” anchor text in `app/src/lib/email.ts` (the day-before Reminder email to a Sent Request’s own Recipient was already excluded — it reuses

buildRequestEmailHtml/Text, the Initial Request template, whose link text was redesigned separately on 2026-08-16 to “Click to respond or mark as completed”):

- buildOverdueRecipientEmailHtml/Text and digestRowHtml/Text (the two Requestor-facing digests) all link to the recipient’s own /r/[token] response screen (confirmed by reading app/api/cron/tick/route.ts’s mintLink) — used Jim’s suggested wording verbatim, “**Open Request to mark Done or to turn off notifications**” (OVERDUE_LINK_TEXT constant), since that screen genuinely offers both actions as of the same day’s Reminder-checkbox batch above.
- buildTodoReminderEmailHtml/Text links to the owner’s own /todos/[id] ToDo Detail, which has **no** per-ToDo Reminder toggle (todo_dates_enabled gates the whole feature, not a per-item checkbox — confirmed no Reminder markup exists in TodoDetailForm.tsx), so used narrower wording that only promises what’s actually there: “**Open ToDo to mark Done**” (TODO_REMINDER_LINK_TEXT constant).

`npx tsc --noEmit/npm run lint clean.`

2026-08-19 — Reminder checkbox extended to Request Detail (relocated), Response Detail, and Request Response — migration 036

Owner pasted three screenshots of his own new design: Response Detail unchecked (“Turn off Reminders for this Request (Check and Send)”), Response Detail checked (“Turn on Reminders...”), and Request Detail showing a *second* Reminder checkbox in a new top band alongside the existing bottom-of-form one built 2026-08-15. Two real conflicts with what was already shipped, resolved via AskUserQuestion before building anything:

1. **Request Detail now has two Reminder checkboxes — replace or keep both?** Recommended and chosen: **replace** — the new top-band control supersedes the old standalone one rather than the two coexisting.
2. **The mockup’s checkbox is inverted** (unchecked = “Turn off Reminders... Check and Send”; checked = “Turn on Reminders...”), while every existing Reminder checkbox in the app (Create Request, the old Request Detail one) is a plain checked-means-on toggle with static wording. Recommended and chosen: **keep plain checked = on** — the new controls reuse the existing .checkrow/.checktext/.checknote component and wording verbatim, not the inverted semantics or copy from the mockup.

Migration 036 (docs/Week6 - SQL history.txt, **confirmed run by the owner 2026-08-19**) extends the two jsonb-returning read functions (get_request_by_token, get_received_request) to include reminder_enabled, and adds a new trailing p_reminder_enabled boolean default null parameter to both write functions (set_response_done_by_token, set_response_done_as_recipient), written with coalesce(p_reminder_enabled, reminder_enabled) so an unpassed value leaves the column untouched. Both are returns void/returns jsonb, never RETURNS TABLE, so a plain CREATE OR REPLACE FUNCTION with a new trailing default parameter is safe — no drop-then-recreate needed, unlike the migration-017/021 precedent for RETURNS TABLE functions.

Built, all sharing the identical reminderCheckbox().checkrow pattern and isReminderEligible eligibility rule (app/src/lib/email.ts):

- **RequestDetailForm.tsx** — the checkbox moved from its old standalone bottom-of-form row up to immediately after the Date/Recipient .metarow block. Deliberately its own full-width row below the metarow, not beside it — a side-by-side Date/Recipient-plus-control layout was tried and reverted here once already (.metatop/.metacol, 2026-08-10, word-wrapped “Wednesday, August 10,” on a narrow Android phone), so staying full-width avoids repeating that.
- **ResponseDetailForm.tsx** — new capability, not a relocation: the signed-in recipient can now opt out of the shared Reminder for a Received Request. Eligibility mirrors Request Detail’s own rule (isReminderEligible on the Due Date) but with no “select Contact/Due Date first” prerequisite — Due Date isn’t editable on this screen, it’s whatever the owner already set. Rendered as its own row right after the existing .meta (Date/From/Due) block, same wrap-avoidance reasoning as above.

- `RequestResponseForm.tsx` (anonymous `/r/[token]` path) — identical pattern, extending `ResponsePayload` with `reminder_enabled`, loading it in the token-fetch effect, and passing `p_reminder_enabled` on `Send` via `set_response_done_by_token`.

`npx tsc --noEmit/npm run lint` clean across all three files. No mockup updated — none of the three source mock-ups (Request Detail, Response Detail, Respond to Request) have this control drawn in; flagged in `design/README.md`.

2026-08-19 — Reminder checkbox greyed out for archived Requests (Request Detail + Response Detail)

Owner: “The Reminder checkboxes and text for the Sent and Received Request Detail screens when viewed from the Archive lists should be greyed out.”

Rather than plumbing an “opened from Archive” navigation flag through `ArchiveForm.tsx`’s `openDetail()` (which today does a bare `router.push` with no query param or state), gated the Reminder checkbox on the real persisted archived column each screen already fetches: `RequestDetailForm.tsx`’s own `archivedAt` (already read for the un-archive-on-clear feature) and `ResponseDetailForm.tsx`’s `receivedArchivedAt` (ditto, via `get_received_request`, which already returns it — no SQL change needed). This is simpler than a navigation flag and more correct: it covers every path that can land on an archived Request’s Detail screen (Archive’s own list, or a Main Screen search result showing the `.archtag` badge), not just a literal click from Archive, and it self-corrects the moment the item is un-archived (clearing `Done Date`, which un-archives per the existing feature) without any extra wiring.

Both screens’ `reminderCheckbox()` disabled/tooltip logic gained an archived check that takes priority over the existing `Due-Date-lead-time` checks: `'Reminders are not available for archived Requests.'` when archived, falling through to the existing tooltips otherwise. Uses the existing `.checkrow-disabled` styling (opacity 0.55, not-allowed cursor) already applied elsewhere for this same component — no new CSS. `npx tsc --noEmit/npm run lint` clean.

2026-08-19 — Description auto-grow fixed with native field-sizing: content (Request Detail + Todo Detail)

Owner reported, with a screenshot, that Request Detail’s Description box was still a fixed-height scrolling box despite the 2026-08-19 auto-grow feature (commit 53c41d2) — confirmed via the Vercel MCP that production was already serving that exact commit (b55ea01, READY), ruling out a stale-deploy explanation. Rather than chase the React-effect-timing theory blind, added `field-sizing: content` to `.ftextarea.ftextarea-autosize` (`app/globals.css`) as the primary mechanism — per spec this CSS property overrides any specified height, including one a script sets via `.style.height`, so it wins over the existing `descRef` effect regardless of whatever was preventing that effect from taking visible effect. Supported in Chrome/Edge 123+, which covers this app’s tested browsers. The existing JS effect (`RequestDetailForm.tsx/ToDoDetailForm.tsx`) is left in place as a fallback for an unsupported browser — harmless where the CSS property is supported, since it wins regardless. One CSS property, no component changes. Applies to both Request Detail and Todo Detail in one shot, since both already share the `.ftextarea-autosize` class — covers the owner’s follow-up (“The same treatment should be true for Todo Detail”) for free.

2026-08-20 — Request Response Cancel removed + banner wording; Close/Cancel dynamic label on Detail screens; voice dictation extended to Detail Descriptions and all Dialog Text fields; Archived badge/search-text visibility bump

Owner’s 5-item batch (item 5, `ToDo Attachments+Locations`, answered separately, informational only — see the decisions log’s companion entry or the chat response itself; no code change).

1. Request Response Cancel button removed; banner wording changed. Owner: on the anonymous `/r/[token]` screen, Cancel “has no useful purpose... all it does that I see is remove the banner... I tend to click it to try to close the browser tab - which of course does not and cannot work.” Removed the Cancel button and its `handleCancel()` handler entirely from `RequestResponseForm.tsx`, along with the now-dead `savedDoneDate/ savedDoneTime` state that only existed to support it. The post-Send banner changed from “Response saved. Your update has been recorded.” to “**Response sent.**” — more accurate once there’s no Cancel to imply “saved” was ever a reversible, draft-like state.

2. Close/Cancel dynamic label on Detail screens. Owner: the Cancel button on Request Detail, Response Detail, and Todo Detail should read “**Close**” at rest and switch to “**Cancel**” only once real form data has changed — explicitly excluding Dialog/Attachments/Locations, since those already write immediately and independently of Save/Cancel (“additional Dialog or Attachments currently are kept even if the Cancel button is clicked or the web page is closed”). Implemented with a `useRef` snapshot of each screen’s own form fields, captured once inside `load()` right alongside the existing `setForm(...)` calls, compared against current state directly in the render body (`hasChanges`, plain field equality, no `useMemo` needed — cheap comparisons). Request Detail’s snapshot covers Due/Done Date/Time, Category, Description, and the Reminder checkbox; Response Detail’s covers Done Date/Time and Reminder; Todo Detail’s covers Priority, Due/Done Date, Category, Description, and the Status chip (only meaningfully variable when `todo_dates_enabled` is off — harmless to include unconditionally, since nothing else touches it when dates are on).

3. Voice dictation extended from Create-only to every Detail screen’s Description and every screen’s Dialog Text. Owner: “the voice typing feature for the Create screens works really well. I would like to add it to all Detail screen Descriptions and to Dialog text.” Ported the existing Web-Speech-API pattern (`SpeechRecognition/webkitSpeechRecognition`, duplicated per-file per this codebase’s convention) to: - **Description** on Request Detail and Todo Detail (Response Detail and Request Response have no editable Description — issuer content, always read-only there — so nothing to add). - **Dialog Text** everywhere it appears — all six screens (Create Request, Create Todo, Request Detail, Todo Detail, Response Detail, Request Response) — a genuinely new capability; no screen had dictation on Dialog Text before this batch.

Each field that can dictate gets its own independent `dictating/ recognitionRef` state pair (Description and Dialog Text are different inputs that could theoretically both want to listen, though never simultaneously in practice) sharing one `voiceSupported` browser-capability flag per screen. Gating follows the Entitlements rule (CLAUDE.md: rights on a Request come from its issuer, never from whoever is reading it) — the four owner-side screens (Create Request, Create Todo, Request Detail, Todo Detail) gate on the signed-in owner’s own tier; the two recipient-facing screens (Response Detail, Request Response) gate on `data.owner_tier === 'subscriber'` (the Request’s own issuer’s tier, already fetched by `get_received_request/get_request_by_token`), never the viewer’s own tier — an anonymous Request Response visitor has no tier of their own to gate on in the first place. Mic button sits in a new `.descwrap` wrapper alongside each textarea, reusing the existing `.micbtn/.listening` CSS from the Create-screen build (2026-08-19) — no new CSS needed.

4. Archived badge and Search Results text visibility bumped. Owner: “I thought some type of ‘Badge’ was going to distinguish Archive items in the Search Results and do not see that” (pasted screenshot of a Sent row with no visible badge), plus “The font-size should be increased a bit for the ‘SEARCH RESULTS Clear Search x’ text. The first time I used the Search, I did not notice it.” Traced the badge report to a visibility problem, not a logic bug — read `MainScreen.tsx`’s full `filteredSent/sortedSent/row-` render chain end-to-end and confirmed `.archtag` was already rendering correctly on every archived matched row; it was just too subtle (10px, `--ink-soft`) to notice against the row text around it. Bumped `.archtag` to 11px/`--ink/tighter` padding, and `.clearsearch/.searchnotice` from 11px to 13px — pure CSS, no logic changes, in `app/globals.css`.

`npx tsc --noEmit` and `npm run lint` both clean across all files touched in this batch (`RequestResponseForm.tsx`, `RequestDetailForm.tsx`, `ResponseDetailForm.tsx`, `TodoDetailForm.tsx`, `CreateRequestForm.tsx`, `CreateTodoForm.tsx`, `app/globals.css`).

2026-08-20 — “Reminders until Done” banner: two-checkbox Reminder control replacing the single Reminder checkbox, plus a new Overdue-notification opt-out — migration 037

Owner pasted two mockup screenshots (Create Request and Response Detail) showing a boxed “Reminders until Done” banner with two checkboxes: “Morning before” (the existing day-before Reminder) and a new “Daily thereafter.” Explicit requirement, stated up front: **“if a wrap were to occur, it would be important to wrap both the checkbox and the associated text together”** — a checkbox must never visually separate from its own label across a line wrap, even though the banner as a whole may wrap.

The real design question wasn’t the checkbox — it was what “Daily thereafter” controls. The existing automatic Overdue-notification cron system (built 2026-08-17, migrations 032/033) already sends the Recipient an unconditional one-time “just became overdue” notice plus recurring nudges, with no existing per-Request opt-out. Flagged this before building anything and proposed treating “Daily thereafter” as an opt-out for that system via a new column. Owner then described the cadence he wanted in detail (hour-after-Due-Time first nudge, not hourly all day — spam-risk concern; a Due-Date-only Request gets no notice during its own Due Date but still gets the daily cadence) — read through `app/api/cron/tick/route.ts` in full and confirmed the *existing* shipped cadence already matched this exactly; reported that back rather than building anything for the cadence itself. One open question remained: when “Daily thereafter” is unchecked, should the Recipient’s Overdue emails stop entirely (including the first notice) or just the recurring nudges after it. Asked via `AskUserQuestion`; owner chose **“Stop entirely (Recommended).”**

Migration 037 (`docs/Week6 - SQL history.txt`, DRAFTED, NOT YET CONFIRMED RUN) adds `requests.overdue_reminder_enabled` boolean not null default true — default true deliberately, because this is a behavior-*preserving* default (the Overdue system has been unconditional since 2026-08-17), not a neutral opt-in default; false would silently regress every existing Request’s Overdue notifications the instant the migration runs. Extends the two jsonb-returning read functions (`get_request_by_token`, `get_received_request`) with both `overdue_reminder_enabled` and `reminder_sent_at` (the latter needed for the new grey-out rule below, not previously exposed to either recipient path), and adds a 5th parameter (`p_overdue_reminder_enabled` boolean default null) to both write functions (`set_response_done_by_token`, `set_response_done_as_recipient`), coalesced against the existing column so an unpassed value leaves it untouched — `revoke/grant execute` updated to each function’s new 5-arg signature. Deliberately does **not** touch `get_received_requests()` (RETURNS TABLE, would need a drop/create cycle) — confirmed via code reading that Main Screen’s Received list has no per-row Reminder control to read this value for.

A second, independent grey-out condition for “Morning before.” Owner, same message: **“This should be recipient-facing, but if the ‘morning before’ date is prior to today or has been sent already for today - that part of the option should be greyed out.”** The existing `isReminderEligible` threshold (Due Date must be more than two calendar days out) already subsumes “the morning-before date has passed” as a *stricter* condition, so the only genuinely new check needed is `reminder_sent_at !== null` — layered on top of the existing eligibility/archived checks, applying to “Morning before” only; “Daily thereafter” has no eligibility gate of its own on any of the four screens.

Markup: one flexible shape, not two. `.reminderbanner/ .reminderbanner-items/.reminderitem` (§6.41 PROPOSED, `app/globals.css`) — a single `.reminderbanner-items` flex-wrap container holding two `.reminderitem` units, each `display: inline-flex` with `white-space: nowrap`. This one markup shape naturally produces both of the owner’s pasted layouts (stacked in the narrow Due-Date-adjacent inline slot, side-by-side in the full-width placement) purely through CSS wrapping behavior — `white-space: nowrap` is scoped to each individual `.reminderitem`, never the banner as a whole, which is what satisfies the “wrap checkbox and label together” requirement: wrapping can only ever happen *between* the two items, never *inside* one. `.reminderitem-disabled` is a per-item, not per-banner, disabled state — “Morning before” can be disabled while “Daily thereafter” stays enabled. The inline placement beside Due Date (Due Time off) reuses the same `.frow > .checkrow-inline` override pattern already established for the old single checkbox (§6.37), reapplied to `.reminderbanner.reminderbanner-inline`.

`app/api/cron/tick/route.ts` — Phase B (the one-time Overdue-transition notice) gates on `row.overdue_reminder_enabled`: when false, the state machine still advances

(`overdue_notified_at/last_overdue_nudge_at` set, matching the existing “sent” branch’s own update shape) but the actual `sendMail` call and Requestor-digest push are skipped, so counts. `overdueNotices` isn’t incremented either — the Recipient truly gets nothing, not a a silently-swallowed send. Phase C (recurring nudges) gets a one-line early exit on the same flag. No other phase touches this column — the day-before Reminder (`reminder_enabled`) and the Overdue system (`overdue_reminder_enabled`) remain two independently-controllable columns on the same banner, by design.

Built across all four Request-facing screens, each following the same pattern: `RequestFormState/payload` type gains `overdueReminderEnabled` (and, on the three existing-record screens, `reminderSentAt`); the old `reminderCheckbox()/single-.checkrow` function becomes `reminderBanner()` rendering the two-item component; the Save/Send RPC or `.update()` call gains the new field; Request Detail’s and Response Detail’s existing `hasChanges/initialFormRef` dirty-check snapshots (Close/Cancel dynamic label, 2026-08-20 batch above) are extended to include `overdueReminderEnabled` so toggling “Daily thereafter” alone correctly flips the button label:

- **Create Request** — no existing-record concerns (no `reminder_sent_at`, no archived state); `overdueReminderEnabled` defaults true in `initialState`, written on insert alongside `reminder_enabled`.
- **Request Detail** — full grey-out logic (archived, prereqs-missing, ineligible, already-sent, in that tooltip priority order), banner moved into the same top-band placement the old single checkbox already occupied (2026-08-19 relocation), `reminder_sent_at` newly selected in the record fetch.
- **Response Detail** — recipient-facing via `get_received_request`; `reminder_sent_at/overdue_reminder_enabled` read from the RPC payload, written back through `set_response_done_as_recipient`’s new 5th argument.
- **Request Response** — the anonymous `/r/[token]` path; same shape via `get_request_by_token/set_response_done_by_token`, no `hasChanges/dirty-check` snapshot on this screen (Cancel was removed from it entirely in the earlier 2026-08-20 batch, so there’s no label to keep in sync).

Alternatives considered and rejected

- *Two separate markup shapes (a narrow stacked variant and a wide side-by-side variant).* Rejected — the single flex-wrap shape produces both from identical JSX with no duplicated component logic.
- *A single combined “Reminders” boolean instead of two independent columns.* Rejected — the owner’s own mockups show two checkboxes with clearly separable meanings (one governs the day-before Reminder, the other the ongoing Overdue cadence), and the existing `reminder_enabled` column already has its own eligibility rules that don’t apply to the Overdue system.
- *“Daily thereafter” unchecked silencing only the recurring nudges, not the first Overdue notice.* This was the actual open question put to the owner; he chose “stop entirely,” so the one-time transition notice in Phase B is gated identically to the Phase C nudges, not treated as a separate, always-sent event.
- *Reusing `isReminderEligible`’s existing threshold as the only grey-out condition for “Morning before,” on the reasoning that it’s “close enough.”* Rejected once the owner’s own wording (“has been sent already for today”) named a condition the existing threshold literally cannot express (a Request can be well past the 3-day threshold and still have its Reminder already sent) — added `reminder_sent_at` as its own, independent check rather than stretching the existing one to cover both cases.

Not done in this batch: no mockup was updated — none of the five source mockups involved (Create Request, Create ToDo — N/A, Request Detail, Response Detail, Respond to Request) draw the old single Reminder checkbox either, so there was no existing markup to convert; flagged in `design/README.md`’s §6.41 row. **Migration 037 confirmed run by the owner, 2026-08-20** — the “Reminders until Done” banner is now live end to end on all four Request-facing screens.